

インテル・プロセッサ・ノート

LdLabo

1998 年 8 月 15 日

目次

第 I 部	アーキテクチャ	1
第 1 章	基本アーキテクチャ	3
1.1	汎用レジスタ	3
1.1.1	構成	3
1.1.2	リアル・モードの制限	3
1.1.3	アドレッシング・モードの拡張	3
1.2	セグメント・レジスタ	4
1.2.1	構成	4
1.2.2	セグメント・レジスタとリアル・モード	4
1.2.3	セグメント・レジスタとプロテクト・モード	4
1.3	インストラクション・ポインタ	4
1.4	フラグ・レジスタ	4
1.4.1	CF キャリーフラグ	5
1.4.2	PF パリティフラグ	5
1.4.3	AF 補助キャリーフラグ	5
1.4.4	ZF ゼロフラグ	5
1.4.5	SF サインフラグ	5
1.4.6	TF トラップ・フラグ	5
1.4.7	IF 割り込み許可フラグ	5
1.4.8	DF ダイレクション・フラグ	5
1.4.9	OF オーバーフローフラグ	5
1.4.10	IOPL I/O 特権レベル	5
1.4.11	NT ネスト・タスク	5
1.4.12	RF リジュームフラグ	5
1.4.13	VM 仮想 8086 モード	5
1.4.14	AC アラインメント・チェック	5
1.4.15	VIF 仮想割り込みフラグ	6
1.4.16	VIP 仮想割り込み保留フラグ	6
1.4.17	ID 識別フラグ	6
1.5	浮動小数点演算ユニット	6
1.5.1	歴史	6

	8087 以前	6
	8087 数値演算プロセッサ (NPX)	6
	80287 数値演算プロセッサ	6
	80387 数値演算プロセッサ	6
	80486CPU 内蔵の FPU	6
	Pentium 内蔵の FPU	6
	PentiumPro 内蔵の FPU	6
1.5.2	浮動小数点レジスタ	6
	数値データ型	7
1.5.3	10 進 浮動小数点数の 2 進数変換	7
1.5.4	12.34 を 2 進数に変換する	7
1.5.5	単精度実数	8
1.5.6	倍精度実数	8
1.5.7	拡張実数	9
	浮動小数点の用語について	9
1.6	ステータス・レジスタ	10
1.6.1	IE 無効演算例外	10
1.6.2	DE デノーマル・オペランド例外	10
1.6.3	ZE ゼロ除算例外	10
1.6.4	OE オーバーフロー例外	10
1.6.5	UE アンダーフロー例外	10
1.6.6	PE 精度例外	10
1.6.7	SF スタック・フォールト	10
1.6.8	ES 例外要約ステータス・フラグ	10
1.6.9	C3-0 コード・フラグ	10
1.6.10	TOP スタック・トップ	10
1.6.11	B ビジー	10
1.7	コントロール・レジスタ	12
1.7.1	IM 無効演算マスク	12
1.7.2	DM デノーマル・オペランド・マスク	12
1.7.3	ZM ゼロ除算マスク	12
1.7.4	OM オーバーフローマスク	12
1.7.5	UM アンダーフローマスク	12
1.7.6	PM 精度マスク	12
1.7.7	PC 精度制御	12
1.7.8	RC 丸め制御	12
1.8	命令ポインタとデータポインタ	12
1.9	FPU の命令	12
1.9.1	ロード命令	12
1.9.2	ストア命令	12
1.9.3	定数命令	12
1.9.4	交換命令	12

1.9.5	比較命令	13
1.9.6	算術演算命令	13
	減算命令	13
	加算命令	13
	乗算命令	13
	除算命令	13
	その他	13
1.9.7	超越関数	13
1.9.8	プロセッサ制御命令	14
1.9.9	分岐命令	14
1.10	FPU の使い方	14
1.10.1	FPU の初期化	14
第 2 章	リアル・モードとプロテクト・モード	21
2.1	リアル・モード	21
2.2	プロテクト・モード	21
2.2.1	リニア・アドレス	21
2.2.2	物理アドレス	22
2.3	セグメント・セクタ	22
2.3.1	インデックス	22
2.3.2	テーブル・インジケータ	22
2.3.3	RPL	22
2.4	ディスクリプタ・テーブル	22
2.5	ディスクリプタ・テーブル・レジスタ	23
2.5.1	GDTR と IDTR	23
2.5.2	LDTR	23
2.6	ディスクリプタ	23
2.6.1	セグメント・ディスクリプタ	23
2.6.2	システム・セグメント・ディスクリプタ	23
2.6.3	G (GRANULARITY)	24
2.6.4	D (DEFAULT OPERATION SIZE)	24
	コード・セグメント	24
	スタック・セグメント	24
2.6.5	AVL (AVAILABLE FOR USE BY SYSTEM SOFTWARE)	24
2.6.6	リミット	24
2.6.7	P (SEGMENT PRESENT)	24
2.6.8	DPL (DESCRIPTOR PRIVILEGE LEVEL)	24
2.6.9	S (SEGMENT DESCRIPTOR)	24
2.6.10	ベース・アドレス	24
2.6.11	タイプ	24
	データ・セグメント	24
	コード・セグメント	26

	コンフォーミング・セグメント	26
2.6.12	A (ACCESSED)	26
	カウンタ	27
2.6.13	ディスクリプタ・テーブルとディスクリプタ	27
	GDT グローバル・ディスクリプタ・テーブル	27
	LDT ローカル・ディスクリプタ・テーブル	27
	IDT 割り込みディスクリプタ・テーブル	27
2.6.14	セクタ・レジスタとディスクリプタ	27
	ディスクリプタの属性に対するプロセッサの保護	27
	コード・セグメント・レジスタに対するディスクリプタのロード	29
	CS コード・セグメント・レジスタ	29
	DS データ・セグメント・レジスタ	29
	SS スタック・セグメント・レジスタ	29
	ES,FS,GS その他のセグメント・レジスタ	29
2.6.15	特権レベル	29
	その条件は	30
	この時に	30
2.6.16	コード・セグメント	30
	その条件としては	30
	そのコール・ゲート	30
2.6.17	プログラムの実行レベル	30
	TSS タスク・ステート・セグメント	31
	データ・セグメント	31
	コール・ゲート	31
	タスク・ゲート	31
	割り込みゲート／トラップ・ゲート	32
	インテルのマニュアル	32
2.7	割り込みと例外	32
2.8	タスク	34
	2.8.1 TSS (タスク・ステート・セグメント)	34
	2.8.2 I/O ビット・マップ・オフセット	35
	2.8.3 セグメント・レジスタ	36
	2.8.4 EIP	36
	2.8.5 EFLAGS	36
	2.8.6 T ビット	36
2.9	タスク・スイッチの準備	36
	2.9.1 タスク・スイッチ	36
	JMP	37
	CALL	37
	2.9.2 タスクのネスティング	37
	2.9.3 タスクのネスティングからの復帰	37
	通常の復帰	37

	ネスト・タスクに割り込みがあった場合	37
2.10	仮想 8086 モード	37
2.10.1	仮想 8086 モードにあってリアル・モードに無いもの	38
2.10.2	リアル・モードにあって仮想 8086 モードに無いもの	38
2.10.3	注意点	38
	IOPL	38
	INT n	38
2.10.4	割り込み	38
2.10.5	仮想 8086 モードへの移行	38
2.10.6	仮想 8086 モードの拡張機能	38
	拡張機能の許可・禁止	38
	ソフトウェア割り込み	38
	マスク可能なハードウェア割り込み	39
2.10.7	仮想 8086 モードとページング	39
2.10.8	保護モード仮想割り込み	39
	拡張機能の許可・禁止	39
2.11	特権レベルと命令	39
第 3 章	ページングとその他	41
3.1	ページング	41
3.1.1	メモリ空間	41
3.1.2	仮想記憶	41
3.1.3	ページング方式	41
	テーブル	41
	エントリ	41
	テーブルの構成	41
	テーブルの大きさ	42
	ページの保護	42
	ページのキャッシング	42
	仮想記憶のサポート	42
	ページのチェック	42
	エントリによるシステム提供ビット	44
	ページング使用中のページ・テーブルの変更	44
3.2	Pentium 以降のページングの拡張	44
3.2.1	コントロール・レジスタ	45
	PAE (物理アドレス拡張)	45
	PSE (ページ・サイズ拡張)	45
3.2.2	ページ・サイズの拡張	45
3.2.3	物理アドレス空間の拡張	45
	テーブル	45
	CR3	45
	PDPT	45

	PDE,PTE	45
	PTE(PDE) の個数	45
	論理アドレスから物理アドレスへの変換	45
3.3	コントロール・レジスタ	46
3.3.1	CR0	46
	PE (プロテクション・イネーブル)	46
	MP (モニタ・コプロセッサ)	48
	EM (エミュレート・コプロセッサ)	49
	TS (タスク・スイッチ)	49
	ET (拡張タイプ)	49
	NE (数値演算エラー)	49
	WP (書き込み保護)	49
	AM (整列マスク)	49
	NW (ライト・スルー・ビット)	49
	CD (キャッシュ・ディスエーブル)	50
	PG (ページング・イネーブル)	50
3.3.2	CR1	50
3.3.3	CR2	50
3.3.4	CR3	50
	PWT (ページ・ライト・スルー)	50
	PCD (ページ・キャッシュ・ディスエーブル)	50
3.3.5	CR4	50
	VME 仮想 8086 モード拡張	50
	PVI 保護モード仮想割り込み	50
	TSD タイム・スタンプ・ディスエーブル	50
	DE デバッグ拡張	50
	PSE ページ・サイズ拡張	50
	PAE 物理アドレス拡張	50
	MCE マシン・チェック・イネーブル	50
	PGE ページ・グローバル・イネーブル	50
	PCE パフォーマンス・モニタリング・カウンタ・イネーブル	51
3.4	デバッグ	51
3.4.1	8086 のデバッグ機能	51
3.4.2	80286 のデバッグ機能	51
3.4.3	80386/80486 のデバッグ機能	51
3.4.4	Pentium のデバッグ機能	51
3.4.5	デバッグ・レジスタ	51
3.4.6	DR7	51
	LEN0~3	51
	R/W0~3	52
	GD 一般検出イネーブル	52
	LE,GE	52

	G0~3	52
	L0~3	52
3.4.7	DR6	52
	BT	52
	BS	53
	BD	53
	B0~3	53
3.4.8	DR4,5	53
3.5	プロテクト・モードのプログラム	53
3.5.1	ディスクリプタ・テーブル・レジスタ	53
3.5.2	コントロール・レジスタ/デバッグ・レジスタ	53
3.5.3	プロテクト・モードへの移行	53
3.5.4	プロテクト・モードからリアル・モードに移行	53
3.5.5	ページングのイネーブル	54
3.5.6	ページングのディセーブル	54
3.5.7	キャッシュのイネーブル	54
	ページングを使わない場合	54
	ページングを使う場合	54
3.5.8	キャッシュのディセーブル	54

第 II 部 ハードウェア 63

第 4 章 80486 のハードウェア 65

4.1	80486 の概要	65
4.1.1	クロック	65
4.1.2	リセット	65
4.1.3	アドレス・バス	66
4.1.4	データ・バス	66
	バイトイネーブル	66
	パリティ・チェック	66
4.1.5	バスサイクルの定義	66
	プロセッサ側	67
	ユーザ側	67
4.1.6	キャッシュ	67
	キャッシュ制御	67
	キャッシュ可能性	67
	キャッシュ・ライン・フィル	67
	キャッシュのフラッシュ	68
4.1.7	ハードウェア割り込み	68
4.1.8	バス・アービトレーション	68
4.1.9	A20M #	68

4.1.10	ページ・キャッシュアビリティ	68
4.1.11	数値演算エラー	68
4.2	バス動作	69
4.2.1	プロセッサ	69
4.2.2	ユーザ回路	69
	バースト転送	70
4.3	バス・サイクル・タイミング	70
4.3.1	基本的 2-2 バス・サイクル	70
4.3.2	基本的 3-3 バス・サイクル	70
4.3.3	非キャッシュアップル・非バースト、複数サイクル転送	70
4.3.4	非キャッシュアップル・バースト・サイクル	70
4.3.5	非バースト・キャッシュアップル・サイクル	70
4.3.6	キャッシュアップル・バースト・サイクル	70
4.3.7	KEN #の変化による効果	70
4.3.8	バースト中断サイクル	70
4.3.9	低速なバースト・サイクル	70
4.3.10	ロック・バス・サイクル	70
4.3.11	疑似ロック・バス・サイクル	71
4.3.12	内部キャッシュ無効化サイクル	71
4.4	特殊バス・サイクル	75
第 5 章	80486 のインターフェース設計	79
5.1	80486 のハードウェア	79
5.1.1	バス・サイクルの開始と終了	79
	開始	79
	終了	79
	サイクル中の動作	79
	ユーザ回路	79
	プロセッサ	79
5.1.2	データ転送の形態	79
5.1.3	RDY、BRDY の使い分け	80
5.1.4	ユーザ回路の機能	80
5.1.5	バス・サイクルの種類	80
5.1.6	キャッシュ	80
5.1.7	ダイナミック・バス・サイジング	80
5.1.8	データ転送と選択されるデータ・ポート	81
5.2	設計仕様	83
5.2.1	CPU	83
	キャッシュとキャッシュ・ラインフィル	83
	RDY	83
	BRDY	83
	今回の仕様	83

5.2.2	メモリ	83
	ROM	83
	RAM	83
5.2.3	I/O	83
5.3	ユーザ・インターフェース回路のブロック	83
5.3.1	チップの機能割付	83
5.3.2	順序回路	86
	PLD1	86
	シミュレーション	86
	実動観測	86
	PLD2	86
	シミュレーション	89
	実動観測	89
5.3.3	組み合わせ回路	89
	PLD3	89
	PLD4	89
	PLD5	91
	PLD6	91
	PLD7	92
	PLD8	93
	PLD9	94
5.4	実際の動作記録	94

第 III 部 ソフトウェア 103

第 6 章	命令	105
6.1	整数命令	105
6.1.1	データ転送命令	105
	汎用データ転送命令	105
	交換命令	105
	スタック操作命令	106
6.1.2	論理命令	108
	論理演算命令	108
	ビット移動命令	108
6.1.3	算術演算命令	109
	加減算命令	109
	インクリメント/デクリメント命令	110
	乗算/除算命令	110
6.1.4	10 進算術命令	110
6.1.5	比較命令	111
	CMP 命令	111

6.1.6	分岐命令	111
	無条件分岐命令	111
	条件分岐命令	112
	プロシジャ命令	112
6.1.7	ビット操作命令	112
6.1.8	ストリング命令	114
6.1.9	ループ命令	116
6.1.10	I/O 命令	117
6.1.11	フラグ制御命令	117
6.1.12	割り込み関連の命令	118
6.1.13	その他の命令	118
6.1.14	セグメント・レジスタ命令	119
6.1.15	システム命令	119
6.2	浮動小数点命令	122
6.2.1	データ転送命令	122
6.2.2	基本算術命令	123
6.2.3	比較命令	125
6.2.4	超越関数命令	126
6.2.5	定数ロード命令	127
6.2.6	FPU 制御命令	127
6.3	アドレッシング・モード	129
6.3.1	メモリ	129
6.3.2	I/O	129
6.4	命令コード・フォーマット	129
6.4.1	基本形式	129
6.4.2	プリフィックス	129
	命令プリフィックス	129
	アドレス・サイズ・プリフィックス 67h	129
	オペランド・サイズ・プリフィックス 66h	130
	セグメント・オーバーライド・プリフィックス	130
	使用例	130
6.4.3	オペコード	132
6.4.4	アドレッシング・モード・バイト	132
6.4.5	スケール・インデックス・ベース・バイト	132
6.4.6	ディスプレズメント・フィールド	133
6.4.7	イミディエイト・データ・フィールド	133
6.5	記号の説明	133
6.6	フラグ	135
6.7	プロセッサの変遷による命令の追加	135
6.7.1	8086	135
6.7.2	80186	135
6.7.3	80286	135

6.7.4	80386	147
6.7.5	80486	147
6.7.6	Pentium	147
6.7.7	PentiumPro	147
6.8	32 ビット命令の拡張	147

第 IV 部 文献	149
------------------	------------

参考文献	151
-------------	------------

1.1	汎用レジスタ	3
1.2	拡張されたアドレッシング・モード	3
1.3	8086 のアドレッシング・モード	3
1.4	セグメント・レジスタ	4
1.5	リアル・モードのアドレス変換	4
1.6	インストラクション・ポインタ	4
1.7	フラグ・レジスタ	4
1.8	浮動小数レジスタ	6
1.9	レジスタ・スタック	6
1.10	FPU タグ・ワード	7
1.11	FPU データのフォーマット	7
1.12	FPU ステータス・レジスタ	10
1.13	FPU コントロール・レジスタ	12
2.1	セグメント・セクタ	22
2.2	ディスクリプタ・テーブル・レジスタ	23
2.3	セグメント・ディスクリプタ	25
2.4	TSS ディスクリプタ	25
2.5	LDT ディスクリプタ	25
2.6	コール・ゲート	25
2.7	割り込みゲート/トラップ・ゲート	26
2.8	タスク・ゲート	26
2.9	スタック・セグメントの有効領域	26
2.10	ディスクリプタの分類	28
2.11	ディスクリプタとセグメント・セクタ	28
2.12	保護リング	30
2.14	32 ビット・セグメントの例外スタック・フレーム	32
2.15	16 ビット・セグメントの例外スタック・フレーム	32
2.16	ページ・フォルトのエラー・コード	32
2.17	エラー・コード	32
2.13	プロテクト・モードの割り込み構造	33
2.18	TSS タスク・ステート・セグメント	34
2.23	各特権レベルのスタック構造	35

2.19	16 ビット TSS	35
2.20	32 ビット TSS	36
2.21	I/O 許可ビットマップ	36
2.22	割り込みリダイレクション・ビットマップ	36
2.24	仮想 8086 モードの例外スタック・フレーム	37
3.1	リニア・アドレスと物理アドレスの変換	44
3.2	4M バイト・ページの変換	46
3.5	コントロール・レジスタ	46
3.6	テーブル・レジスタ	46
3.3	拡張物理アドレスの 4k バイト・ページ	47
3.4	拡張物理アドレスの 4M バイト・ページ	48
3.8	デバッグ・レジスタ構成 2	51
3.7	デバッグ・レジスタ構成 1	52
3.10	プロテクト・モードの設定	54
3.9	リニア・アドレスと物理アドレスの変換	55
3.11	リアル・モードへの移行	55
4.1	CPU の機能グループ	65
4.2	割り込みアクノレッジ・サイクル	69
4.3	基本的 2-2 バス・サイクル	71
4.4	基本的 3-3 バス・サイクル	71
4.5	非キャッシュャブル・非バースト、複数サイクル転送	72
4.6	非キャッシュャブル・バースト・サイクル	72
4.7	非バースト・キャッシュャブル・サイクル	73
4.8	キャッシュャブル・バースト・サイクル	73
4.9	KEN # の変化による効果	74
4.10	バースト中断サイクル	74
4.11	低速なバースト・サイクル	75
4.12	ロック・バス・サイクル	76
4.13	疑似ロック・バス・サイクル	76
4.14	高速な内部キャッシュ無効化サイクル	77
4.15	一般的な内部キャッシュ無効化サイクル	77
5.1	32 ビット・バス	80
5.2	16 ビット・バス	81
5.3	8 ビット・バス	81
5.4	80486CPU ボード・メモリマップ	84
5.5	IF486 ブロック	85
5.7	バス・サイクル検出	86
5.6	PLD1 シミュレーション	87
5.8	ウェイト・カウンタ	87
5.11	PLD2 シミュレーション (1)	88

5.12	PLD2 シミュレーション (2)	88
5.9	バス・サイクル進行	88
5.10	キャッシュ検出	88
5.13	バースト読み出し (ROM) キャッシュ・ラインフィル	96
5.14	非バースト読み出し (RAM)	96
5.15	非バースト書き込み (RAM)	97
5.16	割り込み応答	97
5.17	リセット・スタート	98
5.18	I/O 読み出し	98
5.19	I/O 書き込み	99
5.20	バス・サイクル検出	99
5.21	ウェイト・カウンタ	100
5.22	ウェイト・カウンタ (拡大)	100
5.23	バス・サイクル進行	101
5.24	キャッシュ・ラインフィル検出	101
6.1	スタック・フレーム	107
6.2	命令の基本的なフォーマット	130
6.3	アドレッシング・モード・バイト	132
6.4	スケール・インデックス・ベース・バイト	132

表目次

1.2	FPU のデータ型	7
1.1	タグの定義	7
2.1	システム・セグメントとゲートの定義	27
2.2	テーブルとディスクリプタの登録	27
2.3	セグメント・レジスタとディスクリプタ	29
2.4	IDT インタラプト・ディスクリプタ・テーブル	33
3.1	ページの保護属性	43
3.2	ページ・サイズと物理アドレス・サイズ	48
3.4	オンチップ浮動小数点ユニット制御	49
3.3	プロセッサの動作モード	49
3.5	オンチップ・キャッシュ制御モード	50
3.8	デバッグ・レジスタ構成 5	51
3.6	デバッグ・レジスタ構成 3	52
3.7	デバッグ・レジスタ構成 4	53
4.1	バイト・イネーブルとそれに対応するデータ端子	66
4.2	パリティ端子とそれに対応するデータ端子	66
4.3	ADS #で開始されるバスサイクルの定義	66
4.4	BE3 #～BE0 #から A1,A0 を生成	66
4.5	各種バスサイズで読み出されるデータ端子	67
4.6	16 ビット・バスをアドレスするための BHE #の生成	67
4.7	バス・サイズ	69
4.8	特殊バス・サイクルのエンコード	75
5.1	デバイス・コード	89
5.2	サイクル・コード	89
5.3	バースト・サイクルの組み合わせ	91
5.4	ウェイト数の設定	91
5.5	メモリ・デバイス選択	91
5.6	I/O デバイス選択	91
5.7	ステータス	92
5.8	8 ビット・バス・バイトイネーブル	93

5.9	16 ビット・バス・アドレス	93
5.11	16 ビット・バス・ライト・イネーブル	94
5.10	16 ビット・バス・バイト・イネーブル	95
5.12	32 ビット・バス・ライト・イネーブル	95
6.1	CMOVcc 命令	105
6.2	Jcc, SETcc 命令	113
6.3	FCMOVcc 命令	123
6.4	アドレッシング・モード・バイト 16 ビット	132
6.5	アドレッシング・モード・バイト 32 ビット	133
6.6	アドレッシング・モード・バイト 32 ビット	133
6.7	スケール・インデックス・ベース・バイト (ベース)	133
6.8	スケール・インデックス・ベース・バイト (インデックス)	134
6.9	オペコード (1/6)	136
6.10	オペコード (2/6)	137
6.11	オペコード (3/6)	138
6.12	オペコード (4/6)	139
6.13	オペコード (5/6)	140
6.14	オペコード (6/6)	141
6.15	FPU オペコード (1/2)	142
6.16	FPU オペコード (2/2)	143
6.17	EFLAGS クロス・リファレンス (1/3)	144
6.18	EFLAGS クロス・リファレンス (2/3)	145
6.19	EFLAGS クロス・リファレンス (3/3)	146

第Ⅰ部

アーキテクチャ

第 1 章

基本アーキテクチャ

図 1.1 汎用レジスタ

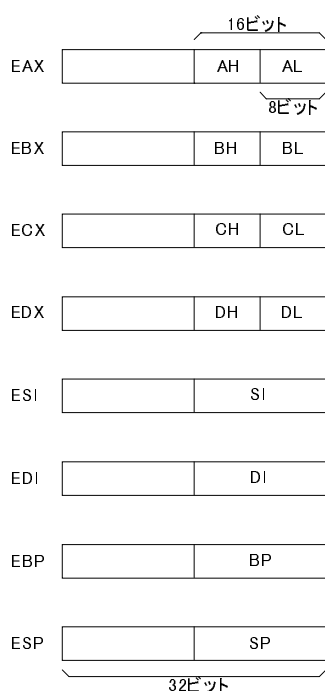


図 1.2 拡張されたアドレッシング・モード

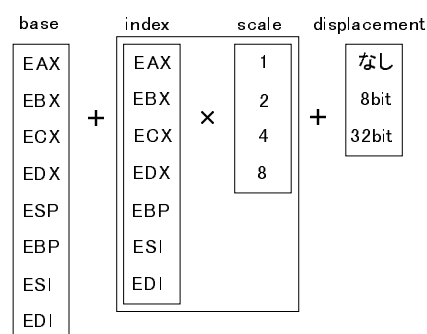
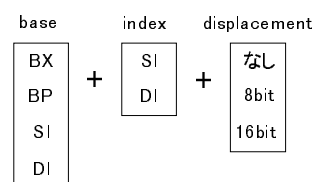


図 1.3 8086 のアドレッシング・モード



1.1 汎用レジスタ

1.1.1 構成

図 1.1 を参照して下さい、8086 との使い方の互換性を保って 32 ビット化されています。

ESP を含めて、8 本の汎用レジスタが使用出来るようになっています。

リアル・モードでも 32 ビット化されたレジスタは使用出来ます、このときは単純に 32 ビットになった 8086 として使えます。

1.1.2 リアル・モードの制限

32 ビット・レジスタでオペランドを指定した場合にアドレスが 64k を越える場合には、一般保護例外な

どが発生します。

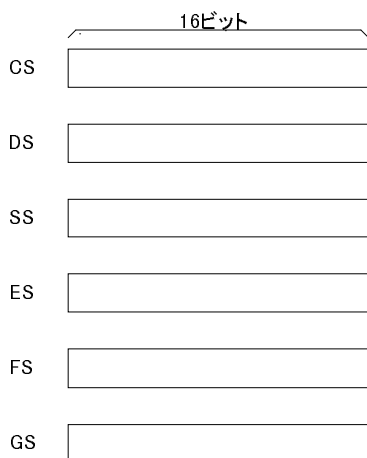
1.1.3 アドレッシング・モードの拡張

下記の式でオペランドを計算します。図 1.2 も参照して下さい。

$$offset = Base + (Index \times Scaling) + Displacement$$

- *Base* には、全ての汎用レジスタを指定出来ます。
- *Index* には、ESP を除く汎用レジスタを指定出来ます。
- *Scaling* は 1,2,4,8 倍が指定出来ます。
- *Displacement* は 16 ビット命令では 8,16 ビットが 32 ビット命令では 8,32 ビットの指定が出来ます。

図 1.4 セグメント・レジスタ



8086 の場合

- *Base* は BX, BP と SI, DI が指定できます。
 - *Index* には、SI, DI が指定出来ます。
 - *Scaling* はありません。
 - *Displacement* は 8, 16 ビットの指定が出来ます。
- 図 1.3 も参照して下さい。

1.2 セグメント・レジスタ

1.2.1 構成

図 1.4 を参照して下さい、80386 になってデータ・セグメント・レジスタに FS と GS が追加されました、PentiumPro も同じ構成です。

- CS はコード・セグメントで命令を実行する領域を指定します。
- DS はデータ・セグメントでデータ領域を指定します、他に ES, FS, GS があります。
- SS はスタック・セグメントでスタック領域を指定します。

1.2.2 セグメント・レジスタとリアル・モード

セグメント・レジスタの値を 4 ビット、左シフトしてベース・アドレスとして使います、図 1.5 を参照して下さい、このモードのリニア・アドレスはそのまま 20 ビットの物理アドレスになります。

1.2.3 セグメント・レジスタとプロテクト・モード

このモードではセグメント・レジスタにベース・アドレスを置かずにセグメント・セクタを置きます、このセクタの指すディスクリプタの中にベース・ア

図 1.5 リアル・モードのアドレス変換

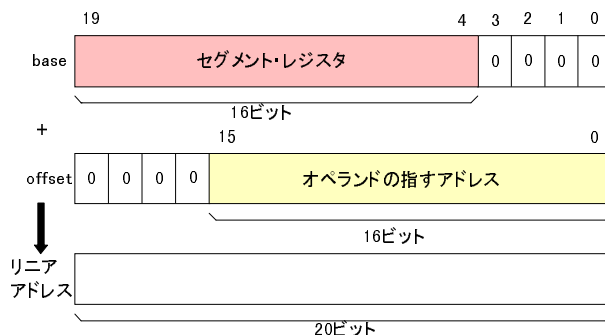


図 1.6 インストラクション・ポインタ

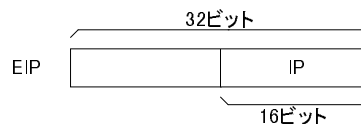


図 1.7 フラグ・レジスタ



ドレスがあります。

1.3 インストラクション・ポインタ

リアル・モードでは IP が使用されます、プロテクト・モードでは、セグメント・タイプを 16/32 ビットのどちらかに選択します、これで 16 ビットなら IP を 32 ビットなら EIP を使用します。図 1.6 を参照して下さい。

1.4 フラグ・レジスタ

図 1.7 を参照して下さい、PentiumPro の EFLAGS レジスタです。

8086 では下位の 11 ビット目までが定義されていました。

80286 ではプロテクト・モード用のビットの IOPL, NT が定義されました。

80386 ではレジスタが 16 ビットから 32 ビットに拡張されました、ビット 16,17 が定義されています。

80486 ではビット 18 が定義され、Pentium ではさらに上位に 3 ビットが定義されました。

1.4.1 CF キャリーフラグ

最上位ビットで桁上げまたは桁借りが出たときに 1 にされ、それ以外の場合に 0 にされます。

1.4.2 PF パリティフラグ

演算結果の下位 8 ビットで 1 になっているビットの数が偶数であれば 1 にされ、奇数であれば 0 にされます。

1.4.3 AF 補助キャリーフラグ

演算結果の下位 4 ビットから桁上げが出たり、上位 4 ビットから桁借りが出たときに 1 にされ、BCD の演算に対する 10 進補正に使用されます。

1.4.4 ZF ゼロフラグ

演算結果が 0 のときに 1 にされ、それ以外の場合に 0 にされます。

1.4.5 SF サインフラグ

演算結果の最上位ビットがセットされます。

1.4.6 TF トラップ・フラグ

通常、デバッグのときに使いますプロセッサは 1 つの命令を実行するたびに、シングル・ステップ例外 (割り込み番号 1) を発生します、例外の発生後の命令 (例外処理ルーチン) でまた例外が発生しそうですが、プロセッサは例外が発生すると TF を 0 にして無限ループを防ぎます。

1.4.7 IF 割り込み許可フラグ

マスク可能な割り込みを禁止します、例外や NMI 割り込み、ソフトウェア割り込みは禁止出来ません。

1.4.8 DF ダイレクション・フラグ

ストリング命令を実行する時にインデックス・レジスタをインクリメントするかデクリメントするかを決定します。

1.4.9 OF オーバーフローフラグ

符号付きの演算を行った時にオーバーフローやアンダーフローが起きたときに 1 にされます。

1.4.10 IOPL I/O 特権レベル

プロセッサは現在動作している特権レベルと I/O 特権レベルとを比較して、I/O 命令 (IN,OUT) を実行するかどうかを決めます、I/O 特権レベルは 2 ビットで構成され 0 のときに特権レベルは最も高く、3 のとき

に最も低くなります、動作レベルが I/O 特権レベルより大きかったら (低かったら) I/O 命令は実行されず一般保護例外 (割り込み番号 13) が発生します。

1.4.11 NT ネスト・タスク

プロセッサがタスク管理の為に使うフラグです、あるタスクの実行中に、これを中断して別のタスクを起動したときにプロセッサは NT を 1 にして、タスクがネスティングしている事を記憶します、そして現在のタスクを終了するときに NT が 1 なら元のタスクに戻す処理をします。

1.4.12 RF リジュームフラグ

一時的にデバッグ・フォールトを使用不能にします、これはプロセッサにより 1 になったり 0 になったりします。

例えばプロセッサのデバッグ機能によるハードウェア・ブレイク・ポイントを検出するとデバッグ・フォールト (割り込み番号 1) が発生します。

例外処理終了の RETI でデバッグ・フォールトを起こしたアドレスに戻るのですが、ちょっと考えるとプロセッサはここで又デバッグ・フォールトを起こして無限ループに入ってしまうそうですが、プロセッサは RF を 1 にして、この命令で再びデバッグ・フォールトになるのを禁止します、命令が終了すると RF はプロセッサにより 0 にされます。

1.4.13 VM 仮想 8086 モード

プロセッサを仮想 8086 モードにして、8086CPU のプログラム環境をエミュレートします。

1.4.14 AC アラインメント・チェック

アラインメントの合っていないオペランドを参照した時にセットされます、例えば奇数アドレスにあるワード・オペランドなどがそうです。

コントロール・レジスタ 0 (CR0) のアラインメント・マスク・ビット (AM) を 1 にすると、特権レベル 3 のプログラムの場合だけアラインメント・チェック例外 (割り込み番号 17) を発生させる事が出来ます。

すべてのデータがアラインメントを合わせなければならないプロセッサとデータを交換する時などに使います。

1.4.15 VIF 仮想割り込みフラグ

仮想 8086 モードや特権レベル 3 のプログラムが実行する STI, CLI 命令*1をシステム・レベルのプログラムがサポートするのを仮想的なフラグを設けて援助するものです。

1.4.16 VIP 仮想割り込み保留フラグ

VIF と同様です。

1.4.17 ID 識別フラグ

このフラグがセット・クリア出来る場合は CPUID 命令がサポートされていることを示します。

1.5 浮動小数点演算ユニット

1.5.1 歴史

8087 以前

インテルは浮動小数点演算の為の社内規格を定め、8080CPU 用の演算サブルーチン集の FPAL(Floating Point Arithmetic Library) を 1977 年に開発しました。

さらに、これをハード・ウェア化したワンボードの iSBC310 を作り、スピードは 10 倍以上の 100 μ s になりました。

次にこれをワンチップ IC 化した 8232 が 8080 ファミリの演算プロセッサとして登場しました。

インテルの社内規格は IEEE 754 に発展することになり、現在の IEEE 規格 854 に至っています。

8087 数値演算プロセッサ (NPX)

8086 ファミリのコ・プロセッサとして登場し、8232 と互換性があり、スピードは 10 倍になり、IEEE 規格 754 の初期提案に合致しています。

80287 数値演算プロセッサ

80286 はアドレス・バスとデータ・バスを分離したので 8087 は使えなくなり、80287 が登場しました、8087 の 2.5 倍の性能を持つとされています。

80387 数値演算プロセッサ

設計と製造プロセスが一新されました、機能的にも三角関数命令が充実して性能は 8087(8MHz) のおよそ 10 倍になり IEEE 規格 754 を完全に満足しています。

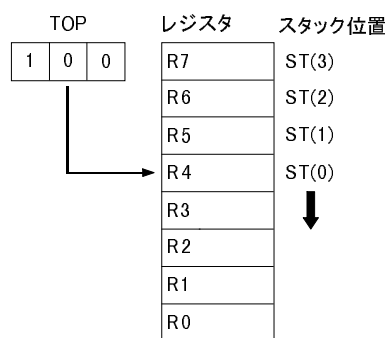
80486CPU 内蔵の FPU

80486 で数値演算プロセッサは 80387 と等価なものがオンチップになりました。

図 1.8 浮動小数レジスタ



図 1.9 レジスタ・スタック



IEEE 規格 754 と IEEE 規格 854 の両方に適合しています。

Pentium 内蔵の FPU

80486 の FPU を基に再設計され、80486 の FPU と比較して命令は 3 倍以上速くなっているそうです。

PentiumPro 内蔵の FPU

新しい命令が追加されています。

1.5.2 浮動小数点レジスタ

図 1.8 を参照して下さい、R0 から R7 の数値レジスタはスタック構成になっており、データのロード・セーブはプッシュ／ポップで行います。

数値レジスタを指定する時はスタック・トップからの相対位置で指定します、どのレジスタがスタック・トップになっているかは FPU のステータス・レジスタの中の 3 ビットに表されています。図 1.9 を参照して下さい。

また数値レジスタの状態については 16 ビットの FPU タグ・ワードに各 2 ビットで示されています、図 1.10 を参照して下さい、その意味を表 1.1 に示します。

*1 プロテクト・モードでは特権命令です。

表 1.2 FPU のデータ型

データ型	サイズ	精度	範囲	
ワード整数	16	15	$-2^{-15} \sim 2^{15} - 1$	$-32,768 \sim 32,767$
ショート整数	32	31	$-2^{-31} \sim 2^{31} - 1$	$-2.14 \times 10^9 \sim 2.14 \times 10^9$
ロング整数	64	63	$-2^{-63} \sim 2^{63} - 1$	$-9.22 \times 10^{18} \sim 9.22 \times 10^{18}$
パック BCD 整数	80	18 桁 (10 進)	—	$(-10^{18} + 1) \sim (10^{18} - 1)$
単精度実数	32	24	$2^{-126} \sim 2^{127}$	$1.18 \times 10^{-38} \sim 3.40 \times 10^{38}$
倍精度実数	64	53	$2^{-1022} \sim 2^{1023}$	$2.23 \times 10^{-308} \sim 1.79 \times 10^{308}$
拡張実数	80	64	$2^{-16382} \sim 2^{16383}$	$3.37 \times 10^{-4932} \sim 1.18 \times 10^{4932}$

図 1.10 FPU タグ・ワード

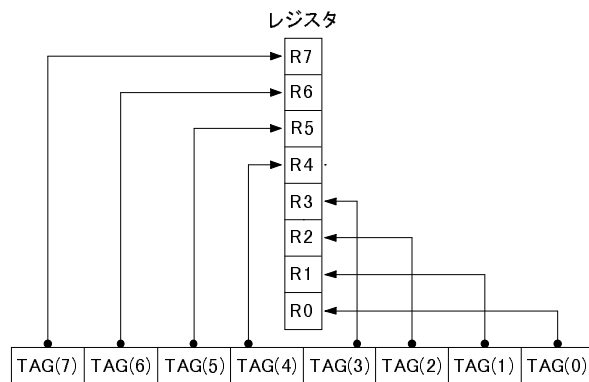


表 1.1 タグの定義

0	0	有効
0	1	ゼロ
1	0	*
1	1	空

*: QNaN, SNaN, 無限大、デノーマル、及びサポートされないフォーマットを示します。

数値データ型

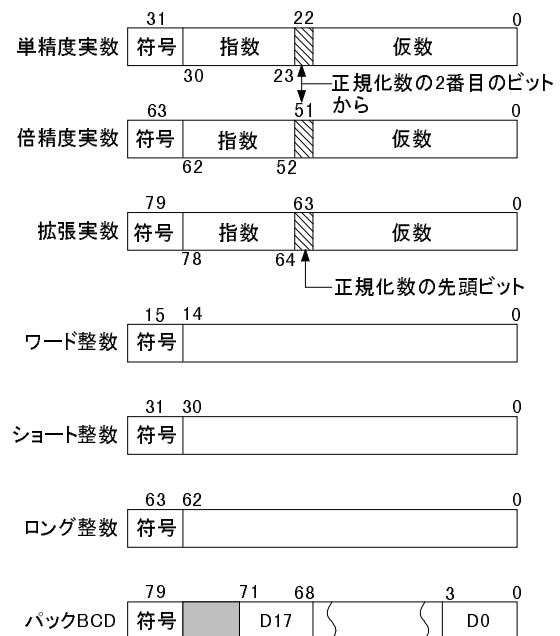
FPU では表 1.2 の数値データ型が使えます、フォーマットは図 1.11 を参照して下さい。

浮動小数点数がどのようなバイナリ表現になるかは、アセンブラがあればリスト 1 のような式で確認出来ます。

1.5.3 10 進 浮動小数点数の 2 進数変換

FPU を使う上で浮動小数点数についての基本的な知識として、10 進数から 2 進数への変換を手計算により行い FPU のフォーマットを理解したいと思います。

図 1.11 FPU データのフォーマット



1.5.4 12.34 を 2 進数に変換する

12.34 の内、少数点以上の数値を 2 進数にすると下記ようになります。

$$\begin{array}{r}
 2 \overline{) 12} \\
 \underline{2 \times 6 = 12} \\
 0 \\
 2 \overline{) 3} \\
 \underline{2 \times 1 = 2} \\
 1
 \end{array}$$

1100

少数点以下の数値を 2 進数にすると下記のようになります。

$$0.34 \rightarrow 0$$

$$0.34 \times 2 = 0.68 \rightarrow 0$$

$$0.68 \times 2 = 1.36 \rightarrow 1$$

$$(1.36 - 1) \times 2 = 0.72 \rightarrow 0$$

$$0.72 \times 2 = 1.44 \rightarrow 1$$

$$(1.44 - 1) \times 2 = 0.88 \rightarrow 0$$

$$0.88 \times 2 = 1.76 \rightarrow 1$$

$$(1.76 - 1) \times 2 = 1.52 \rightarrow 1$$

$$(1.52 - 1) \times 2 = 1.04 \rightarrow 1$$

$$(1.04 - 1) \times 2 = 0.08 \rightarrow 0$$

$$0.08 \times 2 = 0.16 \rightarrow 0$$

$$0.16 \times 2 = 0.32 \rightarrow 0$$

$$0.32 \times 2 = 0.64 \rightarrow 0$$

$$0.64 \times 2 = 1.28 \rightarrow 1$$

$$0.28 \times 2 = 0.56 \rightarrow 0$$

$$0.56 \times 2 = 1.12 \rightarrow 1$$

$$(1.12 - 1) \times 2 = 0.24 \rightarrow 0$$

$$0.24 \times 2 = 0.48 \rightarrow 0$$

$$0.48 \times 2 = 0.96 \rightarrow 0$$

$$0.96 \times 2 = 1.92 \rightarrow 1$$

$$(1.92 - 1) \times 2 = 1.84 \rightarrow 1$$

$$0.84 \times 2 = 1.68 \rightarrow 1$$

$$*$$

$$0.68 \times 2 = 1.36 \rightarrow 1$$

$$0.36 \times 2 = 0.72 \rightarrow 0$$

$$0.72 \times 2 = 1.44 \rightarrow 1$$

.

.

.

0.0101 0111 0000 1010 0011 1101

0 になるまでキリがなく続きますが、少数以下 24 ビットまで求めてみます。

両方、合わせて 1100.0101 0111 0000 1010 0011 1101 になります。

これは小数点を移動させて、次のようにも表せます、1.100 0101 0111 0000 1010 0011 1101 $\times 2^3$ で左辺が仮数部、右辺が指数部になります。

仮数部の先頭は常に 1 になるように、指数部で調整します、これを正規化と呼びます。

そして先頭は 1 であるとしてビットに含めないようにします、これをケチ表現と呼びます。

単精度実数、倍精度実数の仮数部はケチ表現を採用しています。

1.5.5 単精度実数

最上位に符号ビットを 1 ビット、次に指数部を 8 ビット、最後に 23 ビットの仮数部を付けると。

- 符号 0
- 指数部 7Fh+3=82h=1000 0010
- 仮数部 1000 1010 1110 0001 0100 0111

0100 0001 0100 0101 0111 0000 1010 0011
4 1 4 5 7 0 A 3

上記の 32 ビット・バイナリ値が、12.34 を単精度実数で表したものです。

この他に、倍精度実数、拡張精度実数がありますが、考え方は同じです。

1.5.6 倍精度実数

12.34 の少数点以下のビットをさらに求めます。

$$(1.44 - 1) \times 2 = 0.88 \rightarrow 0$$

$$0.88 \times 2 = 1.76 \rightarrow 1$$

$$(1.76 - 1) \times 2 = 1.52 \rightarrow 1$$

$$(1.52 - 1) \times 2 = 1.04 \rightarrow 1$$

$$(1.04 - 1) \times 2 = 0.08 \rightarrow 0$$

$$0.08 \times 2 = 0.16 \rightarrow 0$$

$$0.16 \times 2 = 0.32 \rightarrow 0$$

$$0.32 \times 2 = 0.64 \rightarrow 0$$

$$0.64 \times 2 = 1.28 \rightarrow 1$$

$$(1.28 - 1) \times 2 = 0.56 \rightarrow 0$$

$$0.56 \times 2 = 1.12 \rightarrow 1$$

$$(1.12 - 1) \times 2 = 0.24 \rightarrow 0$$

$$0.24 \times 2 = 0.48 \rightarrow 0$$

$$0.48 \times 2 = 0.96 \rightarrow 0$$

$$0.96 \times 2 = 1.92 \rightarrow 1$$

$$0.92 \times 2 = 1.84 \rightarrow 1$$

$$(1.84 - 1) \times 2 = 1.68 \rightarrow 1$$

$$(1.68 - 1) \times 2 = 1.36 \rightarrow 1$$

$$(1.36 - 1) \times 2 = 0.72 \rightarrow 0$$

$$0.72 \times 2 = 1.44 \rightarrow 1$$

$$(1.44 - 1) \times 2 = 0.88 \rightarrow 0$$

$$0.88 \times 2 = 1.76 \rightarrow 1$$

$$(1.76 - 1) \times 2 = 1.52 \rightarrow 1$$

$$(1.52 - 1) \times 2 = 1.04 \rightarrow 1$$

$$(1.04 - 1) \times 2 = 0.08 \rightarrow 0$$

$$0.08 \times 2 = 0.16 \rightarrow 0$$

$$0.16 \times 2 = 0.32 \rightarrow 0$$

$$0.32 \times 2 = 0.64 \rightarrow 0$$

$$0.64 \times 2 = 1.28 \rightarrow 1$$

$$(1.28 - 1) \times 2 = 0.56 \rightarrow 0$$

$$0.56 \times 2 = 1.12 \rightarrow 1$$

$$(1.12 - 1) \times 2 = 0.24 \rightarrow 0$$

$$0.24 \times 2 = 0.48 \rightarrow 0$$

$$0.48 \times 2 = 0.96 \rightarrow 0$$

$$0.96 \times 2 = 1.92 \rightarrow 1$$

$$(1.92 - 1) \times 2 = 1.84 \rightarrow 1$$

$$(1.84 - 1) \times 2 = 1.68 \rightarrow 1$$

$$(1.68 - 1) \times 2 = 1.36 \rightarrow 1$$

$$(1.36 - 1) \times 2 = 0.72 \rightarrow 0$$

$$0.72 \times 2 = 1.44 \rightarrow 1$$

$$(1.44 - 1) \times 2 = 0.88 \rightarrow 0$$

$$0.88 \times 2 = 1.76 \rightarrow 1$$

$$(1.76 - 1) \times 2 = 1.52 \rightarrow 1$$

$$(1.52 - 1) \times 2 = 1.04 \rightarrow 1$$

$$(1.04 - 1) \times 2 = 0.08 \rightarrow 0$$

*

$$0.08 \times 2 = 0.16 \rightarrow 0$$

$$0.16 \times 2 = 0.32 \rightarrow 0$$

$$0.32 \times 2 = 0.64 \rightarrow 0$$

最上位に符号ビットを 1 ビット、次に指数部を 11 ビット、最後に 52 ビットの仮数部を付けると。

- 符号 0

- 指数部 $3FFh+3=402h=100\ 0000\ 0010$

- 仮数部 $1000\ 1010\ 1110\ 0001\ 0100\ 0111\ 1010\ 1110\ 0001\ 0100\ 0111\ 1010\ 1110$

0100 0000 0010 1000 1010 1110 0001 0100 0111
4 0 2 8 A E 1 4 7

1010 1110 0001 0100 0111 1010 1110
A E 1 4 7 A E

上記の 64 ビット・バイナリ値が、12.34 を倍精度実数で表したものです。

1.5.7 拡張実数

12.34 の少数点以下のビットをさらに求めます。

$$0.64 \times 2 = 1.28 \rightarrow 1$$

$$(1.28 - 1) \times 2 = 0.56 \rightarrow 0$$

$$0.56 \times 2 = 1.12 \rightarrow 1$$

$$(1.12 - 1) \times 2 = 0.24 \rightarrow 0$$

$$0.24 \times 2 = 0.48 \rightarrow 0$$

$$0.48 \times 2 = 0.96 \rightarrow 0$$

$$0.96 \times 2 = 1.92 \rightarrow 1$$

$$0.92 \times 2 = 1.84 \rightarrow 1$$

最上位に符号ビットを 1 ビット、次に指数部を 15 ビット、最後に 64 ビットの仮数部を付けます。

拡張実数の仮数部は正規化した先頭の 1 は省略しません。

- 符号 0

- 指数部 $3FFFh+3=4002h=100\ 0000\ 0000\ 0010$

- 仮数部 $1100\ 0101\ 0111\ 0000\ 1010\ 0011\ 1101\ 0111\ 0000\ 1010\ 0011\ 1101\ 0111\ 0000\ 1010\ 0011$

0100 0000 0000 0010 1100 0101 0111 0000 1010 0011
4 0 0 2 C 5 7 0 A 3

1101 0111 0000 1010 0011 1101 0111 0000 1010 0011
D 7 0 A 3 D 7 0 A 3

上記の 80 ビット・バイナリ値が、12.34 を拡張実数で表したものです。

浮動小数点の用語について

- 正規化 (normalized)

仮数部の上位桁の無意味なゼロを存在させなくして、仮数部の桁を有効に活用する方式で、仮数部の有効数字の最上位桁に無意味なゼロが存在する時は、それを除く為、仮数部を上位方向に桁移動して、その位の上がった分だけ指数部で調整を行います、これを正規化するという、この場合、数値の表示桁数は、常に仮数部桁数の全部とし得るので精度が向上します。

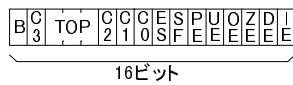
- 非数・NaN(not a number)

本来の数値としての意味はなく、演算に使用すると無効演算エラーになります、また無効演算の結果の不定値の表現に使用されます。IEEE 規格で扱うところの特殊数に定義され、特殊数には、ほかにゼロや無限大などがあります。

- 非正規化数 (unnormal)

正規化処理のされていない数値です。FPU でデノーマル化オペランドの例外が発生したとき、コントロール・レジスタのデノーマル化オ

図 1.12 FPU ステータス・レジスタ



ペランドの例外がマスクされている場合に、演算結果として格納される値です。

- 不正規格化数

IEEE 規格で扱うところの特殊数に定義されています。FPU でアンダーフローの例外が発生したとき、コントロール・レジスタのアンダーフロー例外がマスクされている場合に、演算結果として格納される値です。

1.6 ステータス・レジスタ

FPU のステータス・レジスタを読むには一旦メモリやレジスタに FSTSW 命令などでストアしてからそれを読みます。図 1.12 を参照して下さい。

1.6.1 IE 無効演算例外

$0 \div 0$ や負数の開平など結果が不定になる演算、非数 NAN に対する演算、空でない数値レジスタにデータをロードした場合、空の数値レジスタのストア、あるいは演算に使用した場合などで 1 になります。

1.6.2 DE デノーマル・オペランド例外

デノーマル数を数値レジスタにロードするか演算に使用した場合、非正規化実数を比較、テストした場合に 1 になります。

1.6.3 ZE ゼロ除算例外

被除数がゼロでない時にゼロで除算した場合に 1 になります。

1.6.4 OE オーバーフロー例外

結果が大きすぎて指定フォーマットに入らない場合に 1 にされます。

1.6.5 UE アンダーフロー例外

結果が小さすぎて指定フォーマットでは表せない、また、アンダーフロー例外がマスクされた場合は、デノーマル化によって精度が失われる場合に 1 になります。

1.6.6 PE 精度例外

値を数値レジスタやメモリに転送する時に丸めが行われた場合に 1 になります。

1.6.7 SF スタック・フォールト

FPU のスタックのオーバーフローあるいはアンダーフローにより 1 になります。

1.6.8 ES 例外要約ステータス・フラグ

マスクされていない例外が 1 つ以上発生している場合に 1 になります。

1.6.9 C3-0 コード・フラグ

数値演算の結果がこのビットに反映されます。

1.6.10 TOP スタック・トップ

数値レジスタ R0 から R7 のどれがスタック・トップにあるかを 3 ビットで示します。

1.6.11 B ビジー

8087 からの互換性の為にあり、ES ビットと同じ値になっています。

リスト1 LST2A.ASM

Turbo Assembler Version 3.5 98/07/07 17:06:11 Page 1
rei2a.ASM

```

1 ; =====;
2 ;
3 ;
4 ; assembler : Turbo assembler   ver 1.01                kenji.F
5 ;
6 ; =====;
7
8                                .486P
9                                .8087
10 00000000                      .MODEL    SMALL
11 00000000                      .CODE
12
13 ; =====;
14 ;
15 ;      単精度実数
16 ;
17 ; =====;
18
19 00000000 00000000                DD      0
20 00000004 3F800000                DD      1.0
21 00000008 BE000000                DD     -0.125
22 0000000C 3E000000                DD      0.125
23 00000010 3E800000                DD      0.25
24
25 ; =====;
26 ;
27 ;      倍精度実数
28 ;
29 ; =====;
30
31 00000014 0000000000000000        DQ      0
32 0000001C 3FF0000000000000        DQ      1.0
33 00000024 BFC0000000000000        DQ     -0.125
34 0000002C 3FC0000000000000        DQ      0.125
35 00000034 3FD0000000000000        DQ      0.25
36
37 ; =====;
38 ;
39 ;      8 0 桁浮動小数点数
40 ;
41 ; =====;
42
43 0000003C 00000000000000000000    DT      0
44 00000046 3FFF8000000000000000    DT      1.0
45 00000050 BFFC8000000000000000    DT     -0.125
46 0000005A 3FFC8000000000000000    DT      0.125
47 00000064 3FFD8000000000000000    DT      0.25
48 0000006E 4003F000000000000000    DT      30.
49 00000078 4006B400000000000000    DT     180.
50
51 ; =====;
52 ;
53 ;      浮動小数点数の各フォーマットの比較
54 ;
55 ; =====;
56
57 00000082 414570A4                DD      12.34
58 00000086 4028AE147AE147AE        DQ      12.34
59 0000008E 4002C570A3D70A3D70A4    DT      12.34
60
61                                END

```

図 1.13 FPU コントロール・レジスタ



1.7 コントロール・レジスタ

コントロール・レジスタの読み書きは、一旦レジスタの内容を FSTCW でメモリにストアして、この内容を変更して FLDCW でメモリの内容をコントロール・レジスタにロードする事により行います。図 1.13 を参照して下さい。

1.7.1 IM 無効演算マスク

1 にすると無効演算例外の発生を禁止します、演算結果には非数 NAN か不定値になります。

1.7.2 DM デノーマル・オペランド・マスク

1 にするとデノーマル・オペランド・マスク例外の発生を禁止します、例外の原因が発生した時は通常の処理が継続されます。

1.7.3 ZM ゼロ除算マスク

1 にするとゼロ除算例外の発生を禁止します、演算結果は ∞ になります。

1.7.4 OM オーバーフローマスク

1 にするとオーバーフロー例外の発生を禁止します、結果は最大の有限の値、又は ∞ になります。

1.7.5 UM アンダーフローマスク

1 にするとアンダーフロー例外の発生を禁止します、結果はデノーマル化されるか、またはゼロになります。

1.7.6 PM 精度マスク

1 にすると精度例外の発生を禁止します、例外の原因が発生した時は通常の処理が継続されます。

1.7.7 PC 精度制御

2 ビットで構成され FPU の内部演算精度を設定します、ADD, SUB, DIV, MUL, AQRT の命令に影響があります。

- | | | |
|----|--------|------|
| 00 | 24 ビット | 単精度 |
| 01 | — | |
| 10 | 53 ビット | 倍精度 |
| 11 | 64 ビット | 拡張精度 |

1.7.8 RC 丸め制御

2 ビットで構成され次の命令に影響があります、FST, FSTP, FIST、全ての算術演算命令 (FPREM, FPREM1, FXTRACT, FABS, FCHS を除く)、及び全ての超越関数命令。

- | | |
|----|-----------------------|
| 00 | 最近値か偶数への丸め |
| 01 | 切り下げ ($-\infty$ 方向へ) |
| 10 | 切り上げ ($+\infty$ 方向へ) |
| 11 | 切り捨て (0 方向へ) |

1.8 命令ポインタとデータポインタ

CPU は FPU の実行する命令のアドレスとオペランドのアドレスを保存するレジスタを持っています、これらのレジスタは FSTENV 命令によりメモリにストアしてから内容を確認します。

FSTENV 命令は他にオペコード、タグワード、ステータス・レジスタ、コントロール・レジスタの内容も一緒にストアします。

1.9 FPU の命令

1.9.1 ロード命令

- | | |
|----------------|------|
| ● 実数ロード | FLD |
| ● 整数ロード | FILD |
| ● パック化 10 進ロード | FBLD |

1.9.2 ストア命令

- | | |
|--------------------|-------|
| ● 実数ストア | FST |
| ● 整数ストア | FIST |
| ● 実数ストアとポップ | FSTP |
| ● 整数ストアとポップ | FISTP |
| ● パック化 10 進ストアとポップ | FBSTP |

1.9.3 定数命令

- | | |
|----------------------|--------|
| ● 0 のロード | FLDZ |
| ● 1 のロード | FLD1 |
| ● π のロード | FLDPI |
| ● $\log_2 10$ のロード | FLDL2T |
| ● $\log_2 e$ のロード | FLDL2E |
| ● $\log_{10} 2$ のロード | FLDLG2 |
| ● $\log_e 2$ のロード | FLDLN2 |

1.9.4 交換命令

- | | |
|--------|------|
| レジスタ交換 | FXCH |
|--------|------|

1.9.5 比較命令

- 実数比較 FCOM,FUCOM
- 整数比較 FICOM
- 実数比較とポップ FCOMP,FUCOMP
- 実数比較と 2 回ポップ FCOMP,FUCOMP
- 整数比較とポップ FICOMP

◎ 上記命令で変化するコンディション・コード

C3	C2	C0	内容
0	0	0	スタック・トップ $ST(0) > x$
0	0	1	スタック・トップ $ST(0) < x$
1	0	0	スタック・トップ $ST(0) = x$
1	1	1	比較不能

- テスト FTST

◎ 上記命令で変化するコンディション・コード

C3	C2	C0	内容
0	0	0	スタック・トップ $ST(0) > 0$
0	0	1	スタック・トップ $ST(0) < 0$
1	0	0	スタック・トップ $ST(0) = 0$
1	1	1	比較不能

- 検査 FXAM

◎ 上記命令で変化するコンディション・コード

C3	C2	C1	C0	内容
0	0	0	0	+ アンノーマル
0	0	0	1	+ NAN
0	0	1	0	- アンノーマル
0	0	1	1	- NAN
0	1	0	0	+ ノーマル
0	1	0	1	$+\infty$
0	1	1	0	- ノーマル
0	1	1	1	$-\infty$
1	0	0	0	+ 0
1	0	0	1	空
1	0	1	0	- 0
1	0	1	1	空
1	1	0	0	+ デノーマル
1	1	0	1	空
1	1	1	0	- デノーマル
1	1	1	1	空

1.9.6 算術演算命令

減算命令

- 実数減算 FSUB
- 整数減算 FISUB
- 実数減算とポップ FSUB,FSUBP
- 実数逆減算 FSUBR
- 整数逆減算 FISUBR

- 実数逆減算とポップ FSUBR,FSUBRP

加算命令

- 実数加算 FADD
- 整数加算 FIADD
- 実数加算とポップ FADD,FADDP

乗算命令

- 実数乗算 FMUL
- 整数乗算 FIMUL
- 実数乗算とポップ FMUL,FMULP

除算命令

- 実数除算 FDIV
- 整数除算 FIDIV
- 実数除算とポップ FDIV,FDIVP
- 実数逆除算 FDIVR
- 整数逆除算 FIDIVR
- 実数逆除算とポップ FDIVR,FDIVRP

その他

- 平方根 FSQRT
- 整数への丸め FRNDINT
- 絶対値 FABS
- 符号反転 FCHS
- 部分剰余 FPREM

◎ 上記命令で変化するコンディション・コード

C3	C2	C1	C0	内容
.	1	.	.	剰余はまだ求まっていない
Q1	0	Q0	Q2	剰余は求まった

Q2-0 に商の下位 3 ビットがセットされる。

- スケール FSCALE
- 指数と有効数字の分離 FEXTRACT

1.9.7 超越関数

- 部分正接 FPTAN
- 部分逆正接 FPATAN
- $x^2 - 1$ F2XM1
- $y \log_2 x$ FYL2X
- $y \log_2(x + 1)$ FYL2XP1
- $\sin$ FSIN
- $\cos$ FCOS
- $\sin \times \cos$ FSINCOS

1.9.8 プロセッサ制御命令

- プロセッサの初期化 FINIT, FNINIT
- コントロール・レジスタのロード FLDCW
- コントロール・レジスタのストア FSTCW
- FNSTCW
- ステータス・レジスタのストア FSTSW
- FNSTSW
- 例外クリア FCLEX
- FNCLEX
- 環境のストア FSTENV
- FNSTENV
- 環境のロード FLDENV
- ステートのセーブ FSAVE
- FNSAVE
- ステートのリストア FRSTOR
- スタック・ポインタのインクリメント
- FINCSTP
- スタック・ポインタのデクリメント
- FDECSTP
- 数値レジスタの解放 FFREE
- ノー・オペレーション FNOP
- ウェイト FWAIT

1.9.9 分岐命令

FPU は比較命令を持っていますが、分岐命令はありません、FPU の比較命令の後にステータス・レジスタの C3-C0 のビットを判断して各処理に分岐させます、例を [リスト 2](#) に示しますので参考にして下さい、ビットの対応は次のようになります。

FPU フラグ	CPU フラグ
C0	CF
C1	—
C2	PF
C3	ZF

CPU の分岐命令との対応は次のようになります。

JE/JZ	ST(0) = x
JNE/JNZ	ST(0) ≠ x
JB/JNAE	ST(0) < x
JNB/JAE	ST(0) ≥ x
JBE/JNA	ST(0) ≤ x
JNBE/JA	ST(0) > x

x はスタック・トップと比較される値で FCOM 命

令では数値レジスタやメモリにある値で、FTST 命令では 0.0 になります。

1.10 FPU の使い方

$\sin(60\pi/180) \times 1000$ の計算をさせてみます、[リスト 3](#) の★ 1 から実際に FPU を操作する処理が始まります、FINIT は FPU を初期化する命令です。

1.10.1 FPU の初期化

現在実行中の FPU の命令は中断され、FPU は下記の状態になります。

- 例外は全部マスク。
- 丸め制御は最近値
- 精度は 64 ビット
- ステータス・ワードはクリア
- 数値レジスタ R0-R7 はすべて空
- 数値レジスタ R0 がスタック・トップになります

次にパラメータの 1 をスタック・トップの数値レジスタに★ 2 の FLD 命令でセットします、FPU はスタックをプッシュ・ダウンします、次にパラメータの 2 をスタック・トップの数値レジスタに★ 3 の FLD 命令でセットします、この時点でスタックの状態は下記の様になっています。

スタック・トップ	数値レジスタ R6	パラメータ 2
スタック・2 番目	数値レジスタ R7	パラメータ 1
—	数値レジスタ R0	空
—	数値レジスタ R1	空
—	数値レジスタ R2	空
—	数値レジスタ R3	空
—	数値レジスタ R4	空
—	数値レジスタ R5	空

数値レジスタにロードされた数値は FPU の内部表現である拡張実数に変換してセットされます。

★ 11 はパラメータセットの前、★ 12 はパラメータのセットの後の FPU の状態です。

次に $60 \div 180$ の計算をします、スタック・トップとスタック・2 番目の値を★ 4 の FDIV 命令で割り算します、結果はスタック・トップにあります、FPU の状態を★ 13 に示します。

次に π を掛けますが、FPU には専用の★ 5 の FLDPI 命令がありますので、これを使ってスタック・トップ

に円周率を積みます、FPU は他に定数をロードする命令を数種類持っています、そしてスタック・トップの π とスタック・2 番目を★6 の FMUL 命令で掛け算します、FPU の状態を★14 に示します。

次に $\sin$ を計算させます、スタック・トップの値を★7 の FSIN 命令で計算して結果はスタック・トップに置かれます、FPU の状態を★15 に示します。

次にこれを 1000 倍してから、★10 の FIST 命令で整数化してメモリにセーブします、最終結果を★16 に示します。

リスト 2 は製作ボード上のリモート・デバッガでステップ実行させたものです。

リスト3 LST2C.ASM

```

Turbo Assembler Version 3.5      96/09/20 16:27:45      Page 1
1;=====;
2;
3;      FPU(80387) を使ったプログラムのデバッグ (PROTECT MODE)
4;
5;      sin (60 / 180 * pi) * 1000 = 866 (362h)
6;=====;
7
8                      .486P
9
10                     DGROUP    GROUP    _DATA,_BSS
11                     ASSUME    CS:RAM,DS:DGROUP
12
13
14                     0000      RAM      SEGMENT DWORD PUBLIC USE16 'CODE'
15
16                     0000      task0    PROC    FAR
17                     0000      BC F000      mov    sp,0F000h
18                     0003      B8 00C8      mov    ax,0C8h
19                     0006      8E D0      mov    ss,ax
20
21                     0008      B8 00C0      mov    ax,0C0h
22                     000B      8E C0      mov    es,ax
23                     000D      B8 00B8      mov    ax,0B8h
24                     0010      8E D8      mov    ds,ax
25
26                     0012      33 DB      xor    bx,bx
27                     0014      B9 000A      mov    cx,10
28
29                     0017      26: 8B 07      L1:    mov    ax,es:[bx]
30                     001A      89 07      mov    [bx],ax
31                     001C      83 C3 02      add    bx,2
32                     001F      E2 F6      loop   L1
33
34                     0021      9B DB E3      L2:    finit
35                     0024      D9 06 0000r  ★ 1      fld    dword ptr [p1]          ; 60.0
36                     0028      D9 06 0004r  ★ 2      fld    dword ptr [p2]          ; 180.0
37                     002C      D8 F1      ★ 3      fdiv  st,st[1]              ; / = 0.33333333...
38                     002E      D9 EB      ★ 4      fldpi
39                     0030      D8 C9      ★ 5      fmul  st,st[1]              ; * = 1.047197551
40                     0032      D9 FE      ★ 6      fsin
41                     0034      D9 06 0008r  ★ 7      fld    dword ptr [p3]          ; sin = 0.866025403
42                     0038      D8 C9      ★ 8      fmul  st,st[1]              ; 1000.
43                     003A      DB 16 0000r  ★ 9      fist  [a1]                  ; * = 866
44                     003E      EB E1      ★ 10     jmp    L2
45
46                     0040      task0    ENDP
47
48                     0040      RAM      ENDS
49
50                     0000      _DATA    SEGMENT PARA PUBLIC USE16 'DATA'
51                     0000      43340000      p1      DD      180.0
52                     0004      42700000      p2      DD      60.0
53                     0008      447A0000      p3      DD      1000.0
54                     000C      _DATA    ENDS
55
56                     0000      _BSS      SEGMENT PARA PUBLIC USE16 'BSS'
57                     0000      ?????????      a1      DD      ?
58                     0004      _BSS      ENDS
59
60                     END

```

★ 11

>fp

FPU コントロール レジスタ - 37F

IM invalid operation	RRPP.....	24bit タンセイド
DM denormalized	CCCC..PUOZDI	マルメセイギョグウスウ
ZM zero divide	1010..MMMMMM	
OM overflow	0000001101111111	
UM underflow		
PM precision		
PC precision control		
RC rounding control		

FPU ステータス レジスタ - 0

IE invalid operation	..TTT.....	ZCAS
DE denormalized operand	..000.....	0000
ZE zero divide	..CPPPCCCESPUOZDI	top of stack 0
OE overflow	B3210210SFEEEEEE	
UE underflow	0000000000000000	
PE precision		
SF stack fault		
ES error summary status		
C condition code		
TOP top of stack pointer		
B fpu busy		

FPU タグ ワード レジスタ - FFFF

```

tag0 - empty
tag1 - empty
tag2 - empty
tag3 - empty
tag4 - empty
tag5 - empty
tag6 - empty
tag7 - empty
ip offset ..... - 0
op code / cs selector - 0
data operand offset.. - 0
operand selector..... - 0
st0 - 0000 0000000000000000
st1 - 0000 0000000000000000
st2 - 0000 0000000000000000
st3 - 0000 0000000000000000
st4 - 0000 0000000000000000
st5 - 0000 0000000000000000
st6 - 0000 0000000000000000
st7 - 0000 0000000000000000

```

★ 12

>t

mode 2

```

50028: D9-06 04 00          FLD    dword ptr [0004]
EAX 00000000 EBX 00000014 ECX 00000000 EDX 00000000 (ESP )00000000
ESI 00000000 EDI 00000000 EBP 00000000 ESP 0000F000 (ESP+4 )00000000
CS    00D0 DS    00B8 ES    00C0 SS    00C8 (ESP+8 )00000000
FS    0010 GS    0010          (ESP+12)00000000
M0 E000D7A7 M1 E000D7A7 M2 E000D7A7 M3 E000D7A7 (ESP+16)00000000
EFLAGS 00000306 EIP 0000002C count 1[0]

```

AVR NIODITSZ A P C

CME TPF F F F F F F

010.00001100.0.1.0

5002C: D8-F1

FDIV ST,ST[1]

>fp

FPU コントロール レジスタ - 37F

IM invalid operation	RRPP.....	24bit タンセイド
DM denormalized	CCCC..PUOZDI	マルメセイギョグウスウ
ZM zero divide	1010..MMMMMM	
OM overflow	0000001101111111	
UM underflow		
PM precision		
PC precision control		
RC rounding control		

FPU ステータス レジスタ - 3000

IE invalid operation	..TTT.....	ZCAS
DE denormalized operand	..000.....	0000
ZE zero divide	.CPPPCCCESPUOZDI	top of stack 6
OE overflow	B3210210SFEEEEEE	
UE underflow	0011000000000000	
PE precision		
SF stack fault		
ES error summary status		
C condition code		
TOP top of stack pointer		
B fpu busy		

FPU タグ ワード レジスタ - FFF

```

tag0 - empty
tag1 - empty
tag2 - empty
tag3 - empty
tag4 - empty
tag5 - empty
tag6 - valid
tag7 - valid
ip offset ..... - 28
op code / cs selector - D0
data operand offset.. - 4
operand selector..... - B8
st0 - 4004 F000000000000000 ; 180.
st1 - 4006 B400000000000000 ; 60.
st2 - 0000 0000000000000000
st3 - 0000 0000000000000000
st4 - 0000 0000000000000000
st5 - 0000 0000000000000000
st6 - 0000 0000000000000000
st7 - 0000 0000000000000000

```

★ 13

>t

mode 2

```

5002C: D8-F1          FDIV  ST,ST[1]
EAX 00000000 EBX 00000014 ECX 00000000 EDX 00000000 (ESP )00000000
ESI 00000000 EDI 00000000 EBP 00000000 ESP 0000F000 (ESP+4 )00000000
CS    00D0 DS    00B8 ES    00C0 SS    00C8 (ESP+8 )00000000

```

```

FS      0010 GS      0010      (ESP+12)00000000
M0 E000D7A7 M1 E000D7A7 M2 E000D7A7 M3 E000D7A7 (ESP+16)00000000
EFLAGS 00000306 EIP 0000002E count 1[0]

```

```

AVR NIODITSZ A P C
CME TFFFFFF F F F
.....010.00001100.0.1.0

```

```

5002E: D9-EB          FLDPI

```

```

>fp
FPU コントロール レジスタ - 37F

```

```

-----
省略

```

```

st0 - 3FFD AAAAAAAAAAAAAAAB ; 60./180.=0.33333...
st1 - 4006 B400000000000000
st2 - 0000 0000000000000000
st3 - 0000 0000000000000000
st4 - 0000 0000000000000000
st5 - 0000 0000000000000000
st6 - 0000 0000000000000000
st7 - 0000 0000000000000000

```

```

★ 14

```

```

>t

```

```

mode 2
50030: D8-C9          FMUL ST,ST[1]

```

```

-----
省略

```

```

>fp
FPU コントロール レジスタ - 37F

```

```

-----
省略

```

```

st0 - 3FFF 860A91C16B9B2C24 ; 60./180. × π=1.047..
st1 - 3FFD AAAAAAAAAAAAAAAB
st2 - 4006 B400000000000000
st3 - 0000 0000000000000000
st4 - 0000 0000000000000000
st5 - 0000 0000000000000000
st6 - 0000 0000000000000000
st7 - 0000 0000000000000000

```

```

★ 15

```

```

>t

```

```

mode 2
50032: D9-FE          FSIN

```

```

-----
省略

```

```

>fp
FPU コントロール レジスタ - 37F

```

```

-----
省略

```

```

st0 - 3FFE DDB3D742C265539E ; sin = 0.866025403..
st1 - 3FFD AAAAAAAAAAAAAAAB
st2 - 4006 B400000000000000
st3 - 0000 0000000000000000
st4 - 0000 0000000000000000
st5 - 0000 0000000000000000
st6 - 0000 0000000000000000
st7 - 0000 0000000000000000

```

```

★ 16

```

```

>t

```

```

mode 2
5003A: DB-16 10 00      FIST dword ptr [0010]

```

```

-----
省略

```

```

>fp
FPU コントロール レジスタ - 37F

```

```

-----
省略

```

```

FPU タグ ワード レジスタ - FF

```

```

tag0 - empty
tag1 - empty
tag2 - empty
tag3 - empty
tag4 - valid
tag5 - valid
tag6 - valid
tag7 - valid

```

```

ip offset ..... - 3A

```

```

op code / cs selector - D0

```

```

data operand offset.. - 10

```

```

operand selector..... - B8

```

```

st0 - 4008 D881A03731D6F3A8 ; sin(30./180.*pai)*1000=866.0254038...
st1 - 3FFE DDB3D742C265539E
st2 - 3FFD AAAAAAAAAAAAAAAB
st3 - 4006 B400000000000000
st4 - 0000 0000000000000000

```

```
st5 - 0000 0000000000000000
st6 - 0000 0000000000000000
st7 - 0000 0000000000000000
>d 51000
51000:00 00 34 43 00 00 70 42 00 00 7A 44 00 00 00 00 ..4C..pB...zD....
51010:62 03 00 00 00 00 00 00 00 00 00 00 00 00 00 00 b.....
```

↑
整数化した計算結果

整数化した計算結果

リスト2 LST2B.ASM

```

=====;
;
;      FPU(80387) を使ったプログラム      (PROTECT MODE)
;      比較命令と分岐
;
=====;
      .486P

DGROUP  GROUP    _DATA,_BSS

      ASSUME CS:RAM,DS:DGROUP

RAM      SEGMENT DWORD PUBLIC USE16 'CODE'

task0    PROC     FAR
      ARPL      BX,DX
      mov       sp,0F000h
      mov       ax,0C8h
      mov       ss,ax

      mov       ax,0C0h
      mov       es,ax
      mov       ax,0B8h
      mov       ds,ax

      xor       bx,bx
      mov       cx,10

L1:      mov     ax,es:[bx]
      mov     [bx],ax
      add     bx,2
      loop    L1

L2:      finit
      fld     dword ptr [p1]      ; ST=1.2345
      fld     dword ptr [p2]      ; ST=2.3456
      fcom    ; ST(0) と ST(1) を比較
      fstsw   word ptr [a1]      ; ステータス・レジスタをメモリにストア
      mov     ax,word ptr [a1]    ; AX にロード
      sahf    ; AH を CPU のフラグ・レジスタにロード
      jp      L10                 ; ST は比較出来ない
      ja      L11                 ; ST > 0.0
      jae     L12                 ; ST >= 0.0
      je      L13                 ; ST = 0.0
      jbe     L14                 ; ST <= 0.0
      jb      L15                 ; ST < 0.0

L10:     mov     [a2],1
      jmp      L3
L11:     mov     [a2],2
      jmp      L3
L12:     mov     [a2],3
      jmp      L3
L13:     mov     [a2],4
      jmp      L3
L14:     mov     [a2],5
      jmp      L3
L15:     mov     [a2],6
      jmp      L3

L3:      jmp     L2

task0    ENDP
RAM      ENDS

_DATA    SEGMENT PARA PUBLIC USE16 'DATA'
p1       DD      1.23456
p2       DD      2.34567
_DATA    ENDS

_BSS     SEGMENT PARA PUBLIC USE16 'BSS'
a1       DW      ?
a2       DW      ?
_BSS     ENDS

      END

```

第 2 章

リアル・モードとプロテクト・モード

2.1 リアル・モード

8086 のアーキテクチャと互換性を保っている動作モードで 80286 以降のプロセッサは、リセット後このモードになります。

80386 からは 32 ビット化された汎用レジスタを、また追加された FS,GS の 2 本のセグメント・レジスタを使う事が出来ます、80486 以降はキャッシュも使えるようになっています。

リアル・モードではメモリ空間は 1M バイト、セグメントの大きさは 64k バイトです、汎用レジスタが 32 ビットになったので 1M バイト以上のメモリがアクセス出来そうですが、MOV AX,[EBX] の EBX は 0-FFFF でないと例外 (割込み番号 13) が発生してしまいます。

リセット後は割込みベクタ・テーブルは 0~3FFh 番地にありますが、割込みディスクリプタ・テーブル・レジスタ (IDTR) の値で変更できます、リセットの時にはベース・アドレスは 0 に上限は 3FFh になり 8086 との互換性のある位置にあります、このようにリアル・モードでも IDTR は有効ですから割込みを有効にしている時にプロテクト・モードへの移行準備で IDTR を変更する時は、この点を留意して行って下さい。

2.2 プロテクト・モード

80286 から導入された動作モードです、80286 のメモリ空間は 16M バイト、セグメントの大きさは 64k バイトです。

このモードでは次の機能を提供しています。

- メモリ管理機能
プログラムが直接的にアドレスを指定してコー

ドを実行したり、データをアクセスさせないようになっています、プログラムはディスクリプタと呼ばれるものを使ってコードやデータを間接的に指定します、しかもコードやデータには 0~3 の 4 段階のレベルのいずれかに属するように設定され、異なるレベル間のコードの実行やデータのアクセスはプロセッサからチェックされて規定に違反したアクセスであれば、例外が発生してシステムを保護するようになっています。

- マルチ・タスク機能

プロセッサの機能としてタスク・スイッチの命令やコンテキストの取り扱いなどをサポートしています。

80386 ではメモリ空間が 4G バイトに拡大しました、セグメントも最大 4G バイトまで扱える様になり、これによりディスクリプタのアドレス記述枠などの拡張がなされました、また新機能としてメモリ管理機能にページングが追加され仮想記憶を実現出来るようになりました、マルチ・タスク機能には仮想 8086 モードが追加されました。

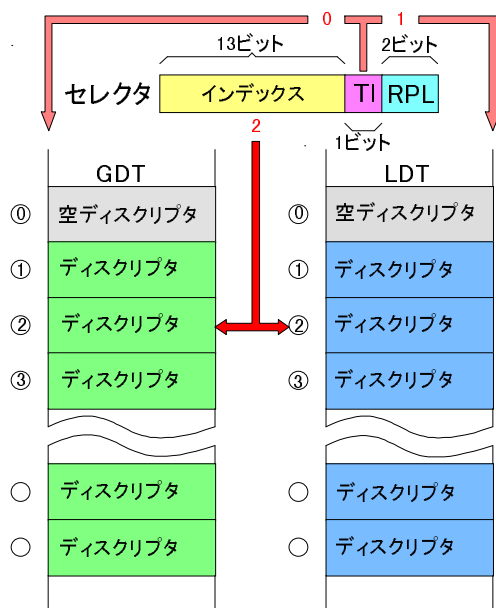
Pentium のプロテクト・モードでは仮想 8086 モードが強化されました。

8086 ではシステム・プログラムとアプリケーション・プログラムをプロセッサが区別して取り扱う機構はありませんでした、プロテクト・モードの導入で両プログラムはプロセッサからレベルの異なるものと認識され違う取り扱いが出来るようになりました。

2.2.1 リニア・アドレス

セグメント方式により生成されるアドレスでセグメント・ベース・アドレスとオフセット・アドレスにより得られます。

図 2.1 セグメント・セクタ



2.2.2 物理アドレス

最終的にメモリをアクセスする時に使うアドレスで、CPU のアドレス端子に出力されているものです。

ページングがイネーブルされていないときはリニア・アドレスが物理アドレスになり、イネーブルされているときはリニア・アドレスをページ・テーブルにより物理アドレスに変換し仮想記憶もサポートされます。

2.3 セグメント・セクタ

リアル・モードでは最終的な物理アドレスを意識してセグメント・レジスタに値をセットしていましたが、プロテクト・モードでは直接その値から最終的な物理アドレスを連想出来ないセグメント・セクタをセグメント・レジスタにセットします。

図 2.1 にセクタの書式を示します、インデックスとテーブル選択ビットと保護属性から構成されています。

2.3.1 インデックス

インデックスでディスクリプタ・テーブルの何番目のディスクリプタであるかを指します、また 0 番目にはディスクリプタは登録されないことになっており、インデックスを 0 に設定したときは空設定を示します。

2.3.2 テーブル・インジケータ

CPU は GDT と LDT の 2 つのディスクリプタ・テーブルを持ちますから TI (TABLE INDICATOR) ビットでインデックスがどちらのテーブルを指しているかを示します、0 なら GDT、1 なら LDT の指定になります、これでどのテーブルにある何番目のディスクリプタであるかを示します。

2.3.3 RPL

セクタは他に特権順位を示すビットを持っています、これは RPL (REQUESTOR PRIVILEGE LEVEL) 要求者特権レベルでこのセクタをセグメント・レジスタにロードしたプログラムの特権レベルを設定します、このビットの目的は特権レベルの低いプログラムが特権レベルの高いプログラムを呼び出して、呼び出し元の特権レベルの低いプログラムが間接的に特権レベルの高いデータ領域をアクセスするのを禁止させる事です、CPU は RPL を使用して特権レベルをチェックしますが、RPL の設定は呼び出されたプログラム (システム・プログラム) がします。

保護の例

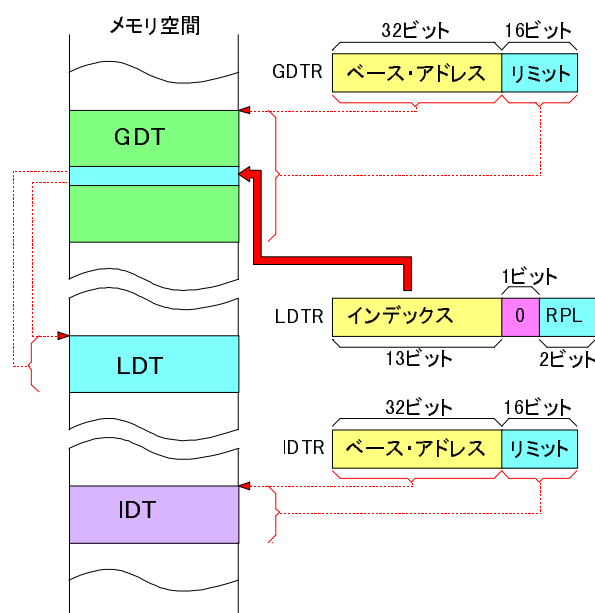
特権レベル 3 専用の特権レベル 0 のサービス・プログラムがあったとします、呼び出し元プログラムは特権レベル 3 であるとして RPL はいつも 3 にします、すると呼び出しプログラムが特権レベル 0 のセクタを使ってアクセスを要求してきても、サービス・プログラムは特権レベル 0 ですから特権レベル 0 のセクタを使う事は許されますが RPL が 3 なのでセクタ・レジスタにロードした時に例外が発生します、これで特権レベルの低いプログラムの不正なアクセスから、特権レベル 0 にあるシステムのデータ領域を保護します。

2.4 ディスクリプタ・テーブル

ディスクリプタを並べた領域をディスクリプタ・テーブルと呼びます、これはメモリ上に置きプログラムで使用する為にはプロセッサの専用レジスタにベース・アドレスと大きさ (最大 64k バイト) を登録します、

ディスクリプタ・テーブルはグローバル・ディスクリプタ・テーブル (GDT) とローカル・ディスクリプ

図 2.2 ディスクリプタ・テーブル・レジスタ



タ・テーブル (LDT) とインタラプト・ディスクリプタ・テーブル (IDT) の 3 種類があります。

2.5 ディスクリプタ・テーブル・レジスタ

ディスクリプタ・テーブルを登録する為の専用のレジスタです、グローバル・ディスクリプタ・テーブル・レジスタ (GDTR) とローカル・ディスクリプタ・テーブル・レジスタ (LDTR) とインタラプト・ディスクリプタ・テーブル・レジスタ (IDTR) の 3 種類があります。

図 2.2 にディスクリプタ・テーブル・レジスタの書式を示します。

2.5.1 GDTR と IDTR

ベース・アドレスはテーブルの先頭アドレスでメモリ空間のどこにでも指定出来ます、リミット値でテーブルの大きさを指定します、最大で 64k バイトの大きさを指定できます、そのときはディスクリプタを 8192 個まで登録出来ます。

また、これでテーブルの大きさを越えた番号のディスクリプタをセグメント・レジスタにロードした時に例外を発生してテーブルの大きさを越えてディスクリプタをロードする事を防止しています、ベース・アドレスでディスクリプタ・テーブルの先頭をリニア・アドレスで指定します。

2.5.2 LDTR

LDTR は直接、ベース・アドレスやサイズを指定せずに、それを指定した LDT ディスクリプタを指す GDT 中のインデックスを指定するセグメント・セクタを設定します。

LDTR はタスク・スイッチで切り替えられるので、各タスクが専用のディスクリプタ・テーブルを持つことが出来るようになっています。

2.6 ディスクリプタ

プロセッサがプロテクト・モードでメモリを管理してプログラムを動作させるために必要な情報を書き込んであるカードのようなものでセグメント・セクタで指定します。

ディスクリプタはセグメント・ディスクリプタとシステム・セグメント・ディスクリプタの 2 つに大別されさらに下記の様に細分類されます。

2.6.1 セグメント・ディスクリプタ

- コード・セグメント・ディスクリプタ 図 2.3
プログラム領域のベース・アドレスとサイズや特権レベルを指定します。
- データ・セグメント・ディスクリプタ 図 2.3
データ領域のベース・アドレスとサイズや特権レベルを指定します。

2.6.2 システム・セグメント・ディスクリプタ

- 16 ビット TSS ディスクリプタ 図 2.4
タスクのコンテキストを保存する領域のベース・アドレスとサイズや特権レベルを指定します。
- 32 ビット TSS ディスクリプタ 図 2.4
16 ビット TSS ディスクリプタと同様です。
- LDT ディスクリプタ 図 2.5
LDT ディスクリプタ・テーブルのベース・アドレスとサイズや特権レベルを指定します。
- ゲート・ディスクリプタ
ゲートにはベース・アドレスやサイズは指定せずにセクタを指定します。
 - 16 ビット・コール・ゲート 図 2.6
コード・セグメント・セクタと、そのセクタの指すディスクリプタのベース・アドレスからのオフセットや特権レベルを指

定します。

また、コール・ゲートにはパラメータ・カウンタを設定します、詳細は後述します。

- 16 ビット・タスク・ゲート 図 2.8
16 ビット・コール・ゲートと同様です。
- 16 ビット割込み・ゲート 図 2.7
コード・セグメント・セクタと、そのセクタの指すディスクリプタのベース・アドレスからのオフセットや特権レベルを指定します。
- 16 ビットトラップ・ゲート 図 2.6
16 ビット割り込みゲートと同様です。
- 32 ビット・コール・ゲート 図 2.6
16 ビット割り込みゲートと同様です。
- 32 ビット・タスク・ゲート 図 2.8
TSS ディスクリプタのセクタを指定します。
- 32 ビット割込み・ゲート 図 2.7
16 ビット割り込みゲートと同様です。
- 32 ビットトラップ・ゲート 図 2.6
16 ビット割り込みゲートと同様です。

2.6.3 G (GRANULARITY)

このビットはリミットで指定した値の倍率を指定します、0 なら 1 倍、1 なら 4096 倍です。

これでセグメントの最大サイズは 0 の時はバイト単位で 1M バイト、1 の時は 4k バイト単位で 4G バイトになります。

2.6.4 D (DEFAULT OPERATION SIZE)

セグメント・ディスクリプタのみが持つビットで次の意味があります。

コード・セグメント

0: オペランドや実行アドレスを 16 ビットで操作する。

1: オペランドや実行アドレスを 32 ビットで操作する。

スタック・セグメント

0: 16 ビットの SP レジスタを使う。

1: 32 ビットの ESP レジスタを使う。

2.6.5 AVL (AVAILABLE FOR USE BY SYSTEM SOFTWARE)

インテルがユーザに解放しているビットで将来のプロセッサで使われる心配のない、システム・プログラマが自由に使えるビットです。

2.6.6 リミット

セグメントの大きさを定義します、G ビットとも関連しています。

2.6.7 P (SEGMENT PRESENT)

仮想メモリ・システムをサポートする為のビットでシステム・プログラマが管理します、このビットが 0 のディスクリプタをセグメント・レジスタにロードすると CPU はセグメント不在例外 (割り込み番号 11) を発生します。

2.6.8 DPL (DESCRIPTOR PRIVILEGE LEVEL)

ディスクリプタの特権レベルを定義します。

2.6.9 S (SEGMENT DESCRIPTOR)

このビットが 0 ならディスクリプタの定義するセグメントはシステム・セグメントです。

2.6.10 ベース・アドレス

ディスクリプタの定義するセグメントの先頭のリニア・アドレスを指定します。

2.6.11 タイプ

ディスクリプタの種類を定義しています表 2.1 を参照して下さい。

セグメント・ディスクリプタではタイプ・ビットは次のような意味を持ちます、図 2.3 も参照して下さい。

データ・セグメント

• E (EXPANSION DIRECTION)

上方伸張セグメントと下方伸張セグメントを識別します、実際の効果としてはリミット値に対する保護の方向が変わります。

– 0: 上方伸張セグメントとして扱う

アドレスがリミット値より上位にある場合は例外が発生します。

– 1: 下方伸張セグメントとして扱う

スタックにプッシュされる時はアドレスが減少する方向に向かうのでリミット値に対するアドレスの有効範囲がデータ・セグメントのアドレスの有効範囲とは反対になります、アドレスがリミット値より下位にある場合に例外が発生します。

図 2.3 セグメント・ディスクリプタ

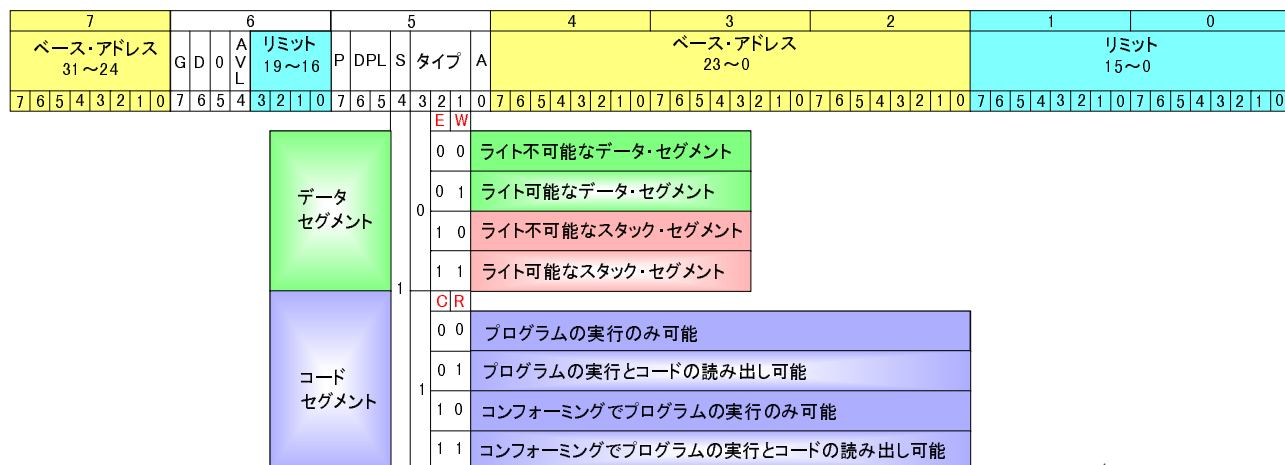


図 2.4 TSS ディスクリプタ

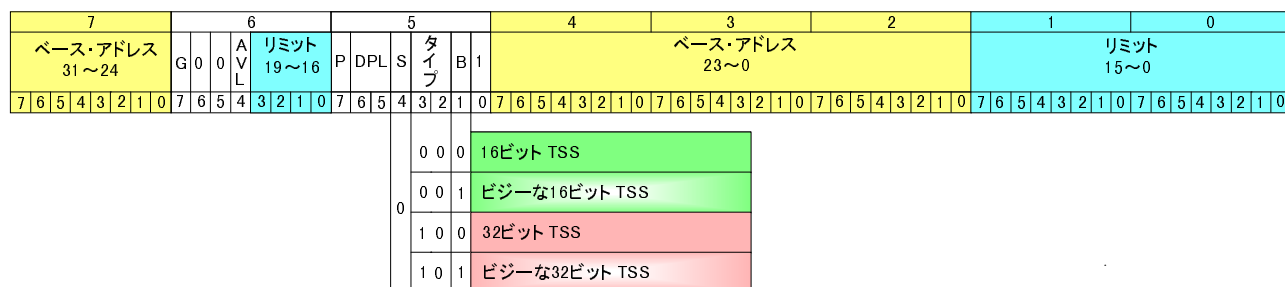


図 2.5 LDT ディスクリプタ

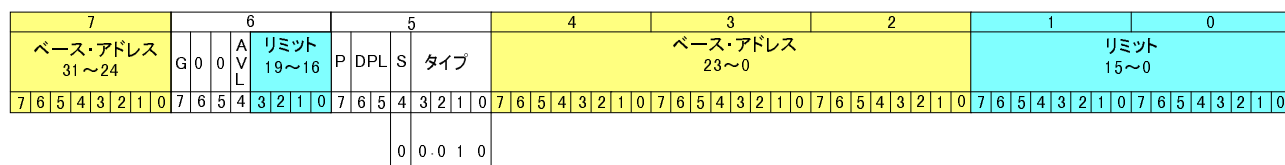


図 2.6 コール・ゲート

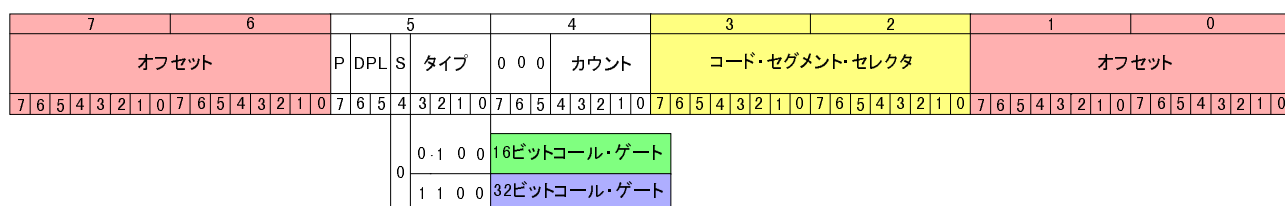


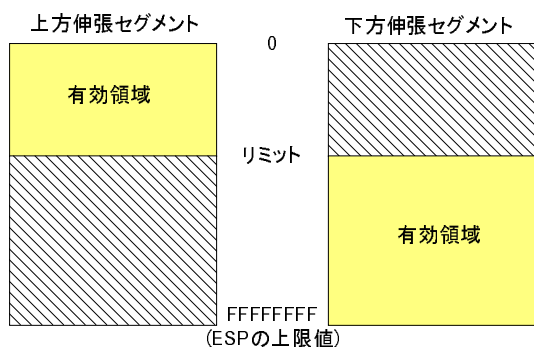
図 2.7 割り込みゲート/トラップ・ゲート

7							6							5				4				3				2				1				0																					
オフセット														P	DPL	S	タイプ	0	0	0	コード・セグメント・セクタ												オフセット																						
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0								
															0	4	3	2	1	0																																			

図 2.8 タスク・ゲート

7								6								5				4								3								2								1								0															
																P	DPL	S	タイプ																	TSS ディスクリプタのセクタ																															
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0												
																		0	0	1	0	1																																													

図 2.9 スタック・セグメントの有効領域



リミット値でスタック・ポインタに下限を設定出来ます。

下方伸張セグメントはスタック・セグメントに適合するものですが、上方伸張セグメントでも構いません。

図 2.9 を参照して下さい。

- W (WRITE ABLE)

ディスクリプタの定義するセグメントがライト可能なときに 1 を指定します、0 を指定するとリード・オンリーになります。

コード・セグメント

- C (CONFORMING)

ディスクリプタの定義するセグメントがコンフォーミングなときに 1 を指定します。

■コンフォーミング・セグメント

通常は特権レベルの低いプログラムが特権レベルの高いプログラムを呼び出すときはコール・ゲートを使っています。

この時は特権レベルの移行が伴います、特権レベルを移行して保護を万全にするためですが、しかしその処理に高い特権レベルのデータ領域を使わないのなら、その必要は無いわけです。このようなプログラムをコンフォーミング・セグメントに指定してやると、特権レベルの低いプログラムからコール・ゲートを使わずに特権レベルも移行させないので直接 JMP, CALL で呼び出す事が出来ます。

- R (READ ABLE)

ディスクリプタの定義するセグメントがリード可能なときに 1 を指定します、0 を指定すると実行は出来ませんがコードは読めなくなります。

2.6.12 A (ACCESSED)

ディスクリプタがセグメント・レジスタにロードされていることを示すためにプロセッサによって 1 にされます、プロセッサが 0 にすることはありません。

表 2.1 システム・セグメントとゲートの定義

タイプ	ディスクリプタ
0000	予約
0001	16 ビット TSS
0010	LDT
0011	ビジーな 16 ビット TSS
0100	16 ビット・コール・ゲート
0101	タスク・ゲート
0110	16 ビット割り込みゲート
0111	16 ビット・トラップ・ゲート
1000	予約
1001	32 ビット TSS
1010	予約
1011	ビジーな 32 ビット TSS
1100	32 ビット・コール・ゲート
1101	予約
1110	32 ビット割り込みゲート
1111	32 ビット・トラップ・ゲート

カウント

コール・ゲートにのみあるビットです、CALL 命令で特権レベルの異なるプログラムを呼び出したときにはスタックも特権レベルに合わせて切り替わります、この為パラメータをスタックにプッシュした場合は、呼び出されたプログラムがスタックをポップしても、既に切り替わったスタックですからパラメータが受け取れません、プロセッサはカウント値が 0 でなければ切替前のスタックから切替後のスタックにカウント値が 1 なら 16 ビット・コール・ゲートで 2 バイト、32 ビットコール・ゲートで 4 バイトを自動的にコピーします、カウントは 5 ビットで設定しますから 16 ビット・コール・ゲートで 64 バイト、32 ビット・コール・ゲートで 128 バイトまでコピー出来ます。

2.6.13 ディスクリプタ・テーブルとディスクリプタ

ディスクリプタは図 2.10 のように分類されますが、ゲート・ディスクリプタは 16/32 ビットですから、全部で 12 種類のディスクリプタがあります。

それぞれのテーブルには表 2.2 のディスクリプタが登録出来ます。

GDT グローバル・ディスクリプタ・テーブル

割り込みゲートとトラップ・ゲートを除く全てのディスクリプタを登録できます。

LDT ローカル・ディスクリプタ・テーブル

各タスクが専用のディスクリプタ・テーブルを持てるようにしたものです使わない場合もあります、GDT とほぼ同じですが TSS は登録出来ません、LDT は登録する意味がありません。

IDT 割り込みディスクリプタ・テーブル

割り込みゲート、トラップ・ゲートなどの例外処理用のディスクリプタとタスク・スイッチ用のタスク・ゲートが登録出来ます。

表 2.2 テーブルとディスクリプタの登録

ディスクリプタ	GDT	LDT	IDT
コード	○	○	×
データ	○	○	×
TSS	○	×	×
LDT	○	×	×
コール・ゲート	○	○	×
タスク・ゲート	○	○	○
割り込みゲート	×	×	○
トラップ・ゲート	×	×	○

2.6.14 セレクタ・レジスタとディスクリプタ

それぞれのセグメント・レジスタには下記のディスクリプタを指すセレクタを登録します、図 2.11 を参照して下さい。

各セグメント・レジスタで使えるディスクリプタは表 2.3 を参照して下さい。

ディスクリプタの属性に対するプロセッサの保護

- コード・セグメントに書き込むことは出来ない。
読み出し可能なコード・セグメントの設定は出来ませんが書き込み可能にすることは出来ません。
- データ・セグメントのデータを命令として実行することは出来ない。
CS にデータ・セグメント・ディスクリプタを指すセレクタをロードすることは出来ません。
- コード・セグメントをスタック領域として指定することは出来ない。

図 2.10 ディスクリプタの分類

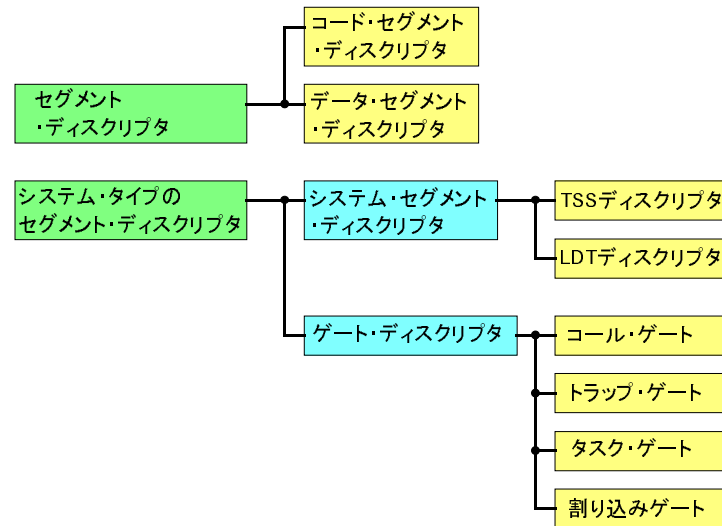


図 2.11 ディスクリプタとセグメント・セクタ

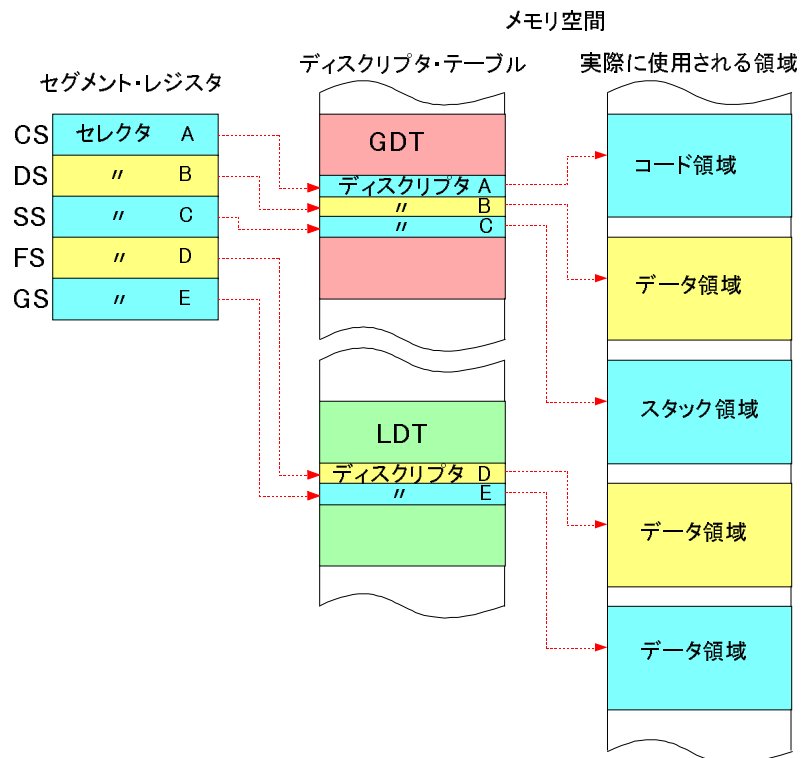


表 2.3 セグメント・レジスタとディスクリプタ

ディスクリプタ	CS	DS	SS	ES,FS,GS
コード	○	○	×	○
データ	×	○	○	○
TSS	●	×	×	×
LDT	×	×	×	×
コール・ゲート	●	×	×	×
タスク・ゲート	●	×	×	×
割り込みゲート	●	×	×	×
トラップ・ゲート	●	×	×	×

SS にデータ・セグメント・ディスクリプタ以外を指すセレクトをロードすることは出来ません。

上記の操作では一般保護例外 (割り込み番号 13) が発生します。

コード・セグメント・レジスタに対するディスクリプタのロード

一般的に次の場合があります。

- セグメント間の JMP, CALL 命令の実行
- 割り込み/例外処理の発生
- タスク切り替え

レジスタに直接セレクトをロードすることも出来ませんが、通常することはありません。

CS コード・セグメント・レジスタ

- コード・セグメント・ディスクリプタを指すセレクトでプログラムの実行領域を指定します。TSS やゲート・ディスクリプタを使った場合でも最終的には、それらで間接的に指定されたコード・セグメント・ディスクリプタを指すセレクトがセットされます。
- TSS はタスク・スイッチをするときに JMP や CALL 命令で使います、CS には TSS ディスクリプタの指定する、TSS の CS の値がセットされます。
- コール・ゲートは上位の特権レベルにあるプログラムを呼び出したいときに JMP や CALL 命令で使います、CS にはゲートの指定するセレクトのコード・セグメント・ディスクリプタ

のセレクトがセットされます。

- タスク・ゲートはタスク・スイッチをするときに JMP や CALL 命令、または例外処理で使います、CS にはゲートの指定するセレクトの TSS ディスクリプタの指定する TSS の CS の値がセットされます。
- 割り込みゲートとトラップ・ゲートは例外処理で使います、CS にはゲートの指定するセレクトのコード・セグメント・ディスクリプタのセレクトがセットされます。

DS データ・セグメント・レジスタ

- 通常はプログラムの使用するデータ領域を指定するデータ・セグメント・ディスクリプタを指すセレクトを指定します。
- 読みとり可能に設定されたコード・セグメント・ディスクリプタを指すセレクトは指定出来ません。

SS スタック・セグメント・レジスタ

書き込み可能なデータ・セグメント・ディスクリプタを指すセレクトを指定します。

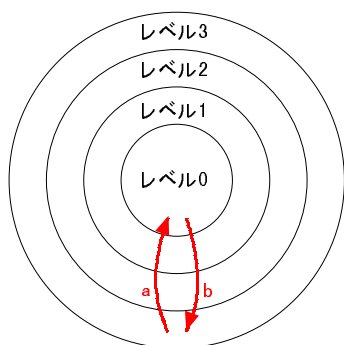
ES,FS,GS その他のセグメント・レジスタ

DS と同様です。

2.6.15 特権レベル

- プロテクト・モードではプログラムが直接に物理アドレスを指定出来ない構造になっています、プログラムはセグメント・レジスタにセットしたセレクトでプロテクト・モードの仕様に基いてシステム・レベルで用意されたディスクリプタを指定して、その情報によりプロセッサが物理アドレスを生成してメモリにアクセスします。
- しかし実際にメモリにアクセス出来るのはディスクリプタに記述された条件がプロセッサにチェックされて満足された時だけです、あるプログラムの要求は許可され、あるプログラムの要求は許可されません、これらのディスクリプタなどの情報を準備する、あらゆるプログラムの最上位にあるプログラムというものを設定し、これをプロセッサが認識する為に特権レベルという考え方を導入しています。
- 特権レベルは 4 段階に分けられレベル 0 が最高

図 2.12 保護リング



でレベル3が最低です、通常は最上位の特権レベルはオペレーティング・システムが使い、それ以下のレベルはアプリケーション・プログラムなどに割り当てます。

図 2.12 を参照して下さい。

特権レベル間には通常、次の様な規則があります。

1. 特権レベルの高いコード・セグメントから、特権レベルの低いコード・セグメントに移行出来ない。
2. 特権レベルの低いコード・セグメントが特権レベルの高いデータ・セグメントにアクセス出来ない。
3. 特権レベルの低いコード・セグメントから特権レベルの高いコード・セグメントに移行することは条件付きで出来ます。

図 2.12 の矢印 a、b のように下位のプログラムが上位のサービプログラムを一時的 (OS のシステム・コールなど) に利用する場合です。

■その条件は コール・ゲートを使うことで、31 頁 2.6.17 のコール・ゲートを参照して下さい。

■この時に スタック・セグメントも移行したコード・セグメントと同じレベルのものに切り替わります、プロセッサはこれらの事柄を制御する為に 2 ビットのフィールドをセグメント・セクタやディスクリプタに設けています。

- CPL (カレント・プライビレッジ・レベル)
現行特権レベル
実行中のプログラムの特権レベルを示し CS レ

ジスタの下位 2 ビットにあり SS レジスタの下位 2 ビットにもコピーされています。

- DPL (ディスクリプタ・プライビレッジ・レベル)

ディスクリプタ特権レベル

ディスクリプタ特権レベルと呼びディスクリプタに記述されたセグメントの特権レベルを示します。

- RPL (リクエスト・プライビレッジ・レベル)

リクエスト特権レベル

要求者特権レベルと呼び、セグメント・セクタの下位 2 ビットにそのセクタでメモリ・アクセスを要求しているプログラムの特権レベルを設定します。

特権レベルが DPL とチェックされる時は CPL と RPL の内で低い方が使われます。

2.6.16 コード・セグメント

- JMP 命令のオペランドに指定した場合は CPL と DPL が等しいときにのみ使えます。
すなわち同一レベル間のプログラムに限ります。
- CALL 命令のオペランドに指定した場合は CPL が DPL と等しいときにのみ使えます、しかしオペランドにコール・ゲートを使用して間接的にコード・セグメントを指定したときには上位のプログラムを呼び出すことが出来ます。

■その条件としては コール・ゲートが使用可能であることですが、CPL と DPL が等しいか CPL が高いことが必要です。

■そのコール・ゲート のセクタで指された上位のコード・セグメント・ディスクリプタで指定された領域のプログラムを呼び出せるようになります。

2.6.17 プログラムの実行レベル

- プロテクト・モードのプログラムはシステム保護の目的のために、各種制限が設けられていますが、特権レベルの移行に関する制限について考えてみました。
- リセット後にリアル・モードからプロテクト・モードに移行した時点では最高位のレベル 0 にいます、規則によれば、タスクスイッチ以外で

は、そこから他レベルのプログラムに移行出来ません。

- レベルの移行はコール・ゲートを使ったときに下位のプログラムが上位のプログラムを一時的に呼び出す場合と RET で下位レベルのプログラムに戻るときだけのようです。
- CALL 命令と割り込み処理によるタスク・スイッチが下位レベルに移行する方法ですが、どちらも復帰命令で戻るのが前提にしています。
- 結論としては、プロテクト・モードのシステムは複数の特権レベルを使ったマルチタスク・システムとして構築して、最上位の特権をシステムで、最下位の特権をアプリケーションに割り当てて、アプリケーション・プログラムに実行を移すときは、タスク・スイッチで行う。

これでシステム・プログラムとアプリケーション・プログラムが独立した環境で実行されます。

● そして、アプリケーション・レベルのプログラムが、システム・レベルのプログラムのサービスを受けるときは、コール・ゲートを使って行う。

TSS タスク・ステート・セグメント

タスク・スイッチを直接行うディスクリプタです、GDT だけに登録出来ます。

- JMP 命令のオペランドに指定した場合には CPL と DPL が等しいときにのみ使えます。
- CALL 命令のオペランドに指定した場合は CPL が DPL と等しいかより高い特権レベルにあるときにのみ使えます。

スイッチされるタスクの特権レベルは TSS の CS セレクタの RPL にありますが、TSS の特権レベルが満足されていれば、どの特権レベルのタスクにもスイッチ出来ます。

データ・セグメント

- 通常のデータ・セグメント
DS,ES,FS,GS にセットされます。

- スタックに使われるデータ・セグメント
SS にセットされるデータ・セグメントはスタックとして扱われます、書き込み可能に設定されていなければいけません。
プロテクト・モードでは TSS に特権レベルごとの SS と ESP を設定する領域が用意されており、レベルの移行があったときは、そのレベルの SS と ESP がプロセッサによりロードされます。
- プログラムのレベルとの関係
同じレベルか下位レベルのデータ・セグメントはアクセス出来ます、上位レベルのデータ・セグメントはアクセス出来ません。

コール・ゲート

コード・セグメント・ディスクリプタを間接的に指定する方法です、コール・ゲートを使った CALL はレベルの低いプログラムがレベルの高いプログラムを呼び出せる唯一の方法です*1。

- JMP では CPL とゲートの DPL が同じか、低い場合でゲートの指すセレクタの指定するコード・セグメント・ディスクリプタの DPL が CPL と同じ場合に使えます。
- CALL では JMP と同じ条件を含んで、最終的に指定するコード・セグメント・ディスクリプタの DPL が CPL より高い場合にも使えます。
- システム・レベルのプログラムは特権レベルの低いプログラムにサービスするためのコード・セグメント・ディスクリプタを GDT、あるいは LDT に登録します。

タスク・ゲート

- タスク・スイッチを間接的に行う方法です、CPL とのチェックはコール・ゲートと同様です、タスクの特権レベルはスイッチ前の CPL とは無関係でどの特権レベル*2にもタスク・スイッチ出来ます。
- 低い特権レベルのプログラムに移行するのはタスク・スイッチが唯一の方法です。

*1 タスク・スイッチは除きます。

*2 TSS の CS の項目が指定するセレクタの指すコード・セグメント・ディスクリプタの特権レベル

- GDT,LDT,IDT に登録できます、割り込みや例外処理でタスク・ゲートが使われるときは、ソフトウェア割り込みを除いて CPL と DPL(ゲート、TSS、スイッチ後タスク) がチェックされることはありません。

割り込みゲート／トラップ・ゲート

IDT にのみ登録できます。

- 割り込み/例外処理
ゲートの DPL は CPL とチェックされません。
- ソフトウェア割り込み
ゲートの DPL は CPL と同じか低いレベルである必要があります。

ゲートの特権レベル

システム・レベルのプログラムはゲートの特権レベルの設定によって各特権レベルに対するサービスを制御することが出来ます。

■**インテルのマニュアル** では特権レベル 0 はオペレーティング・システムのカーネル・プログラム、特権レベル 1,2 はデバイス・ドライバ、特権レベル 3 はアプリケーションで使うことを勧めています。

ワンボード・マイコンでもプロセッサに備わった機能を使って簡単に高機能なシステムを実現出来ます、高機能を求めなくても、プロテクト・モードでシステム・レベルやアプリケーション・レベルのプログラムを作成することでマルチ・タスクや仮想記憶なども自然に理解されると思います。

2.7 割り込みと例外

- 割り込み
外部的原因により命令の実行とは非同期に発生するもので NMI, INTR 端子により引き起こされるものです。
- 例外
命令の実行により発生して 3 タイプのアボート、フォールト、トラップに分類されます。

図 2.14 32 ビット・セグメントの例外スタック・フレーム

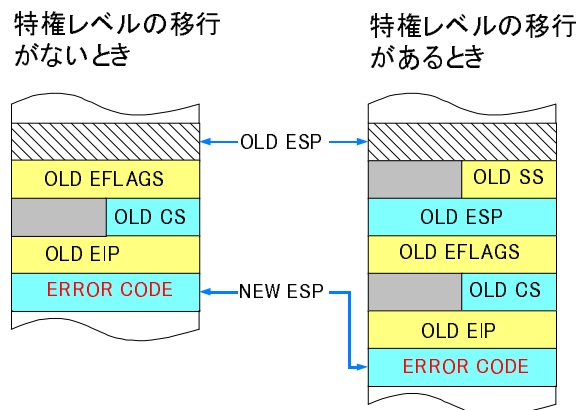


図 2.15 16 ビット・セグメントの例外スタック・フレーム

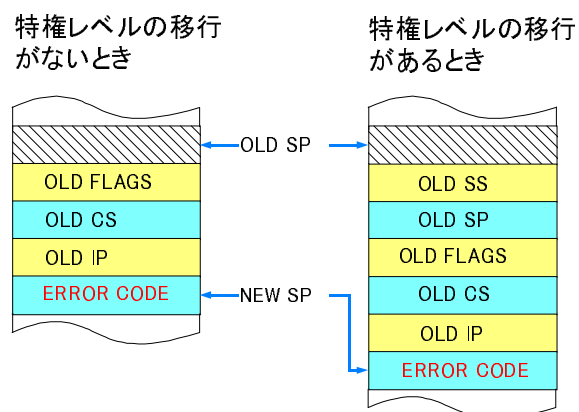


図 2.16 ページ・フォルトのエラー・コード

31~3	2	1	0
RESERVED	U/S	W/R	P

U/S 1: ユーザ・モードでフォルト発生
0: スーパ・バイザ・モードでフォルト発生
W/R 1: 書き込みでフォルト発生
0: 読み出しでフォルト発生
P 1: ページ不在でフォルト発生
0: ページ・レベルの保護違反

図 2.17 エラー・コード

31~16	15~3	2	1	0
RESERVED	セクタ	TI	IDT	EXT

セクタ : 例外が発生する原因になったセグメント・セクタの上位 13 ビット

TI 1: LDT のセクタ
0: GDT のセクタ
IDT 1: IDT のセクタ
EXT 1: 外部的要因でエラー発生

図 2.13 プロテクト・モードの割り込み構造

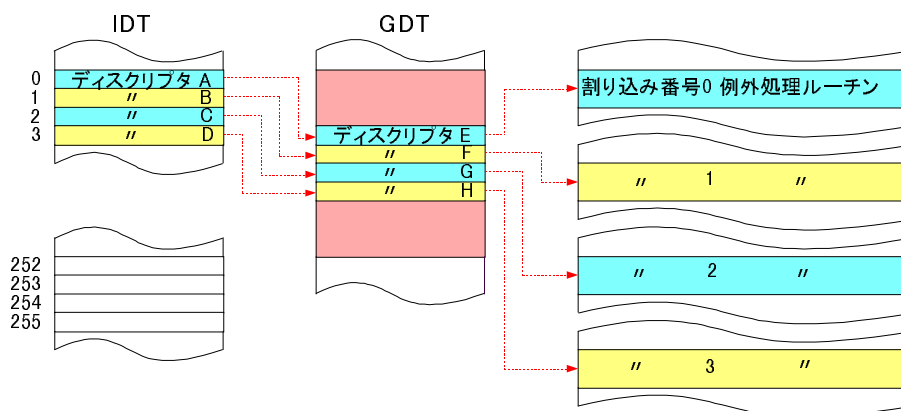


表 2.4 IDT インタラプト・ディスクリプタ・テーブル

番号	内容	タイプ	例外を発生した CS:IP の保持	エラー・コード
0	除算エラー	フォルト	○	×
1	デバッグ例外	トラップ/フォルト	○	×
2	NMI 割り込み	トラップ	—	×
3	ブレーク・ポイント	ソフトウェア割り込み/トラップ	—	×
4	INTO オーバ・フロー	ソフトウェア割り込み/トラップ	—	×
5	BOUND 範囲超過	ソフトウェア割り込み/フォルト	—	×
6	無効オペコード	フォルト	○	×
7	コプロセッサ不在	フォルト	○	×
8	ダブル・フォールト	アボート	×	○
9	コプロセッサ・ セグメント・オーバラン	フォルト	○	×
10	無効タスク・ ステート・セグメント	フォルト	×	○
11	セグメント不在	フォルト	○	○
12	スタック・フォールト	フォルト	○	○
13	一般保護	フォルト/トラップ	○	○
14	ページ・フォルト	フォルト	○	○
15	インテル予約	—	—	—
16	コプロセッサ・エラー	フォルト	○	×
17	アラインメント・チェック	フォルト	○	○
18	マシン・チェック例外	アボート	—	×
19～31	インテル予約	—	—	—
32～255	ユーザ割り込み	—	—	—

アボート:修復不能な重大なエラーがあった時、発生します。

フォールト:原因となる命令を実行する前に例外が発生させる CS:EIP がスタックに保存されているので、原因を取り除いた後、命令の再実行が可能で、この例外から復帰するとき、原因となった命令に戻ります。

トラップ:原因となる命令を実行している間に発生する例外で CS:EIP は原因となった次の命令を指しています。

● リアル・モード

割り込みベクタ・テーブルは 8086 の 1M バイトのメモリ空間を前提にしており、ひとつのベクタのフォーマットは IP にロードされる 16 ビット値と CS にロードされる 16 ビット値の 4 バイトです。

● プロテクト・モード

IDT は割り込みディスクリプタ・テーブルとよばれ、ひとつのベクタにはひとつのディスクリプタが置かれます、ベクトル番号に対する例外発生要因は両モードで同じです。

図 2.13 を参照して下さい。

● IDT には割り込みゲート、トラップ・ゲート、タスク・ゲートを登録出来ます。

● 割り込みゲート

割り込み・例外が発生した時に割り込みフラグを 0 にして、割り込み禁止にする。

● トラップ・ゲート

割り込み・例外が発生した時に割り込みフラグを変化させない。

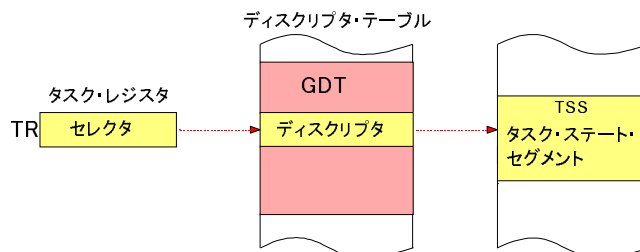
— 16/32 ビットの注意 —

32 ビット・セグメントのプログラムの実行中に割り込みや例外が発生して 16 ビットのゲートで処理に移行すると EIP で動作していたプログラムを 16 ビットの IP で保存することになるので、戻り番地が IP のサイズを越えていた場合、正しく元に戻れなくなります。

2.8 タスク

- 80286 からサポートされている機能でプロセッサのアーキテクチャでマルチ・タスク機能を実

図 2.18 TSS タスク・ステート・セグメント



現します、ハードウェアによりタスクをスイッチするので高速にタスク・スイッチします。

- 各タスクは現在のレジスタの値を保持する領域を、TSS(タスク・ステート・セグメント)に与えられています、また各タスクは LDT(ローカル・ディスクリプタ・テーブル)により、ローカルな資源^{*3}をシステム・レベルのプログラムから割り当てられます。
- タスク・スイッチが起きると汎用レジスタ、セグメント・レジスタ、LDTR、TR、インストラクション・ポインタ、フラグ・レジスタを現タスクに保存して、新タスクからロードします。
- またタスク・スイッチによって現タスクに戻された時には TSS からレジスタに値をロードして前回から継続した処理を再開出来ます。

2.8.1 TSS (タスク・ステート・セグメント)

ひとつのタスクにひとつの TSS が割り当てられ、コンテキストを保存しています、図 2.18 を参照して下さい。

TSS のリミットは 67h 以上の設定でないと例外が発生します。

● LDT セレクタ

LDTR にロードされる値です、LDT を使わない場合でも空セレクタを設定しておいて下さい。

● CR3 (PDBR:ページ・ディレクトリ・ベース・レジスタ)

ページングをイネーブルにしているなら設定が必要です。

● 特権レベル 0,1,2 の SS と ESP

^{*3} LDT に登録されたディスクリプタが指定するセグメント

図 2.23 各特権レベルのスタック構造

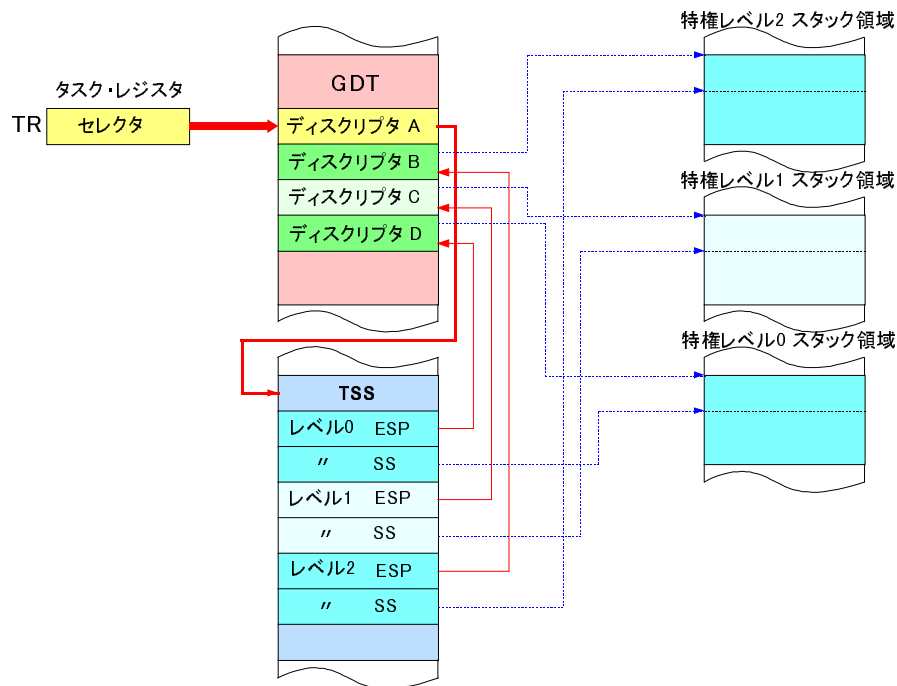


図 2.19 16 ビット TSS

バックリンク・セレクタ	0h
特権0のSP	2h
特権0のSSセレクタ	4h
特権1のSP	6h
特権1のSSセレクタ	8h
特権2のSP	Ah
特権2のSSセレクタ	Ch
FLAGS	Eh
IP	10h
AX	12h
CX	14h
DX	16h
BX	18h
SP	1Ah
BP	1Ch
SI	1Eh
DI	20h
ESセレクタ	22h
CSセレクタ	24h
SSセレクタ	26h
DSセレクタ	28h
LDTセレクタ	2Ah

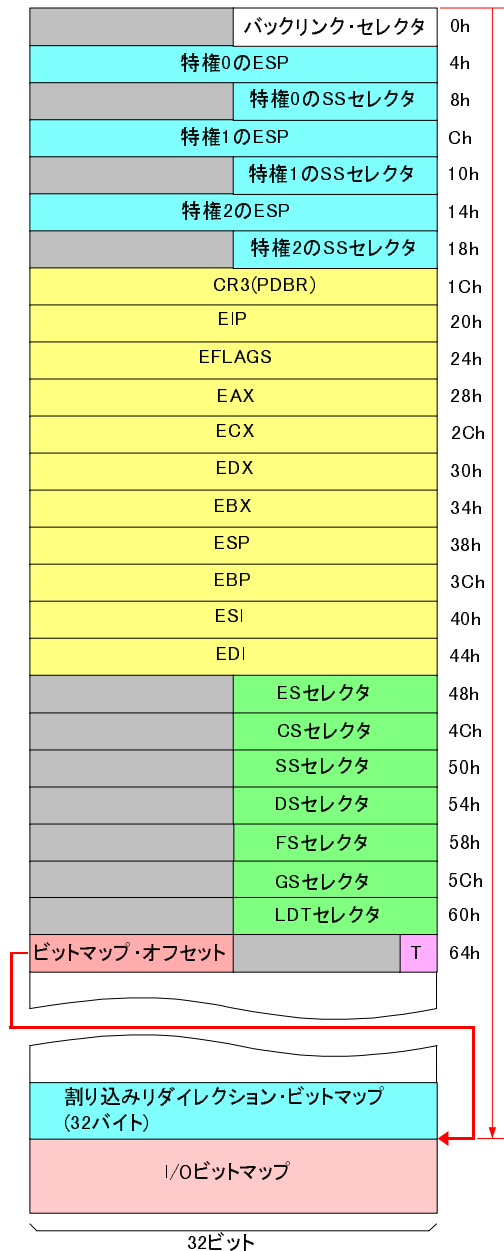
16ビット

各特権レベルはそれぞれのスタックを持ちますから設定しておきます、図 2.23 を参照して下さい。

2.8.2 I/O ビット・マップ・オフセット

- I/O ビット・マップ・オフセットは TSS の先頭からの I/O ビット・マップのオフセットを設定します。
- 1 ポート・アドレスのアクセスの許可を 1 ビットで指定します、1 なら I/O 特権レベルに従い CPL が IOPL と同じか高い時にそのポートのアクセスを許可します、0 ならこのタスクに限りアクセスを許可します。
- プロセッサの I/O 領域は 64k バイトですが I/O ビット・マップの大きさは 8k バイトになりますが TSS のリミットを越えた分は全部 1 と解釈されます。
- また I/O ビット・マップを使わないときはオフセットを 0 にしますこのときは I/O ビット・マップは無いものとしビットが全部 1 と解釈されます。

図 2.20 32 ビット TSS



T 1: タスク・スイッチのときデバッグ例外 (割り込み番号 1) を発生

図 2.21 I/O 許可ビットマップ

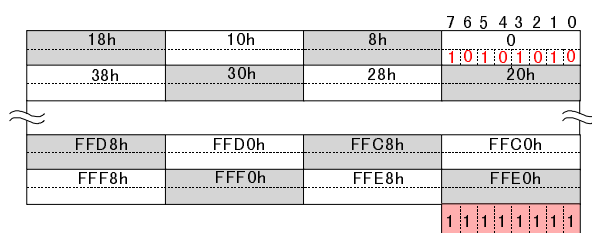
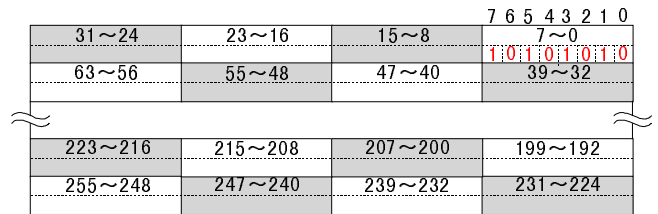


図 2.22 割り込みリダイレクション・ビットマップ



2.8.3 セグメント・レジスタ

CS,DS,SS の設定は最低限必要です、使わないセグメント・レジスタには空セクタを設定して下さい。

2.8.4 EIP

タスクがスイッチされたときに実行するプログラムのアドレスを設定します。

2.8.5 EFLAGS

タスクがスイッチされたときの状態を定義します、例えば VM ビットを 1 にしておけば仮想 8086 モードで特権レベル 3 のプログラムをタスク・スイッチで起動したり出来ます。

2.8.6 T ビット

このビットを 1 にしておくとタスク・スイッチが起きた時にデバッグ例外を発生させます。

2.9 タスク・スイッチの準備

特権レベル 0 のプログラムが次の手順でタスク・スイッチの準備をします。

1. TSS をメモリ中に作成して GDT に TSS ディスクリプタを登録します。
2. ローカルなメモリ空間を設定して LDT ディスクリプタを GDT に登録します。
3. TR に現実行中のプログラムの TSS ディスクリプタをロードして現在タスクとして登録します。

2.9.1 タスク・スイッチ

セグメント間 JMP/CALL 命令を使って GDT にある TSS ディスクリプタやタスク・ゲートのセクタを指定する場合と、IDT にタスク・ゲートを登録して割り込みでタスクをスイッチさせる場合があります。

JMP

JMP 命令でタスク・スイッチをした場合はスイッチしたタスクは終了して TSS ディスクリプタの B(ビュージ) ビットは 0 にされ、スイッチされたタスクの TSS ディスクリプタを指す GDT のセクタが TR にロードされそのディスクリプタの B ビットは 1 にされます。

CALL

CALL 命令でタスク・スイッチをした場合にはプロセッサはスイッチされたタスクの TSS のバック・リンク・セクタに元のタスクの TSS ディスクリプタを指すセクタをセットして元のタスクに戻るようになります。

2.9.2 タスクのネスティング

- CALL によるタスク・スイッチではフラグ・レジスタの NT ビットを 1 にしてタスクがネストしている証拠を残します、元タスクの TSS ディスクリプタの B ビットはタスクは中断していますが 1 のままにしています。
- タスクは再入が不可能なのでタスクをネストする際に TSS ディスクリプタの B ビットをプロセッサが調べてネストしているタスクに再びスイッチしようとしたら例外を起こしてこれを防止しています。

2.9.3 タスクのネスティングからの復帰

通常の復帰

IRET を実行するとプロセッサはフラグ・レジスタの NT ビットを見て 1 なら TSS のバック・リンク・セクタを TR(タスク・レジスタ) にロードしてネストしたタスクをさかのぼって元のタスクにスイッチします。

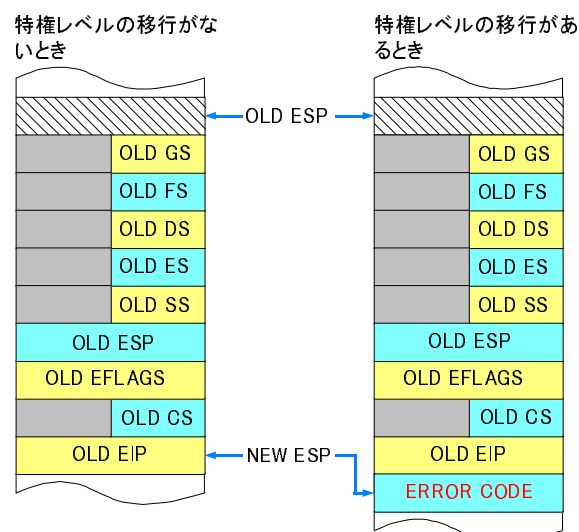
ネスト・タスクに割り込みがあった場合

- IRET は割り込みの復帰にも使われる命令なので、すでにタスクがネストしており、そこに割り込みが入って IRET で処理を終了した場合は割り込まれたタスクのその前のタスクに戻ってしまいうるに思えます。
- タスク・ゲートを使わない割り込みは NT を 0 にしますから通常の割り込みの復帰処理の IRET で現タスクを終わらせる心配はありません、また復帰処理でフラグ・レジスタは割り込

み前のものに戻されますから現タスクの NT も復元されます。

2.10 仮想 8086 モード

図 2.24 仮想 8086 モードの例外スタック・フレーム



- プロテクトモードの管理の中で 8086CPU のプログラムをひとつのタスクとして動作させるモードです、このモードに入るにはフラグ・レジスタの VM ビットを 1 にする事により行われます。
- 特権レベルは常に 3 で最低のレベルに設定されています、割り込みが発生した場合は自動的にプロテクトモードに移行して処理終了後に仮想 8086 モードに戻って来ます。
- このモードのプログラムではセグメント・レジスタの使い方はリアル・モードの場合と同じです。
仮想 8086 モードではリニア・アドレスを指します。
- 8086CPU は 1M バイトのメモリ空間でしたが、このモードでも 1M バイトのメモリ空間をサポートします、この空間はページング機能により実メモリのどこにでも置けますので仮想 8086 モードの複数のタスクが複数の 1M バイトのメモリ空間を持てばマルチ・タスクにより複数の 8086 環境が存在して動作している様に

見えます。

2.10.1 仮想 8086 モードにあってリアル・モードに無いもの

1. ページング機能を使える。
2. マルチ・タスクに出来る。
3. I/O 許可マップによる I/O の保護機能が働く。
4. ページングによるメモリ保護機能が働く。

2.10.2 リアル・モードにあって仮想 8086 モードに無いもの

1. 特権命令を実行できる。
2. 8086CPU と同じ割り込みベクタテーブルを使う。

2.10.3 注意点

1. 仮想 8086 モードは 80386CPU から取り入れられたので 80286CPU と互換性のある 16 ビット TSS は使えませんフラグ・レジスタの枠が 16 ビットなので 18 ビット目にある VM ビットが表現出来ないためです。
2. IOPL (I/O 特権レベル) が 3 でないと使えない命令があります。
CLI, STI, PUSHF, POPF, INT, IRET
がこれに当たります。

■IOPL が 3 ではない時にこれらの命令を使うと一般保護例外が発生します。

■INT n はソフトウェアで割り込みを起動する命令ですが仮想 8086 モードでは割り込みはプロテクト・モードで処理されますからプロテクト・モードのプログラムのサービスを簡単に受ける事が出来ます。

2.10.4 割り込み

仮想 8086 モードで割り込みが起きるとセグメント・レジスタがすべてスタックに保存されます、

図 2.24 を参照して下さい、そして CS と SS を除いて 0 にされますプロテクト・モードでリアル・モードのフォーマットで設定された値は意味を持たないからです、また VM ビットも 0 にされます。

2.10.5 仮想 8086 モードへの移行

- フラグ・レジスタの VM ビットを 1 にすれば移行出来ますが、そのときセグメント・レジスタにはディスクリプタを指すセクタが入っています CS レジスタの解釈がいきなりリアル・

モードの解釈に変わるのでこれは実際には使えません。

- モードの移行は VM ビットだけの問題ですし、そのとき各レジスタが仮想 8086 モードに合った値になっていればいいわけです、これを 1 度で行うにはフラグ・レジスタの VM ビットを 1 にした仮想 8086 モード用の TSS を準備して、これにタスク・スイッチすればうまくモード移行が行えます。
- ページングをイネーブルにしているなら TSS 中のページ・ディレクトリ・ベース・レジスタの枠がありますから仮想 8086 モードのメモリ空間を物理メモリのどこにマップするかを決定出来ます。

2.10.6 仮想 8086 モードの拡張機能

Pentium になってから、このモードに割り込みに関する拡張機能が、選択出来るフラグと共に追加されています。

拡張機能の許可・禁止

CR4^{*4} の VME(仮想 8086 モード拡張) フラグを 1 にすると許可され、0 にすると禁止されます。

ソフトウェア割り込み

INT n や INTO 命令を使った割り込みを 8086CPU を使ったときのように、リニア・アドレス 0 から配置される割り込みベクタ・テーブルで処理出来るようになりました。

- 準備
8086 割り込みベクタ・テーブルと IDT のどちらを使うかを割り込み番号ごとに定義出来ますので、それを定義したビット・マップを作ります。
- 割り込みリダイレクション・ビット・マップ
TSS 中にあり、I/O ビット・マップ・オフセットで指定された位置を割り込みリダイレクション・ビット・マップの後尾として配置します。
図 2.20 を参照して下さい。
図 2.22 を参照して下さい、1 ビットで 1 ベクタについて割り込み番号 32~255 まで設定します。

^{*4} コントロール・レジスタ 4

1 で IDT を 0 で 8086 割り込みベクタテーブルを使う指定になります。

マスク可能なハードウェア割り込み

仮想 8086 モードで動作するプログラムが実行する STI, CLI 命令に対応するプロセッサの機能とフラグが追加されました。

- VIF 仮想割り込みフラグ
CLI が実行されると VIF が 1 にされる。
STI が実行されると VIP の値により次の処理をプロセッサが行う。
- VIP 仮想割り込み保留フラグ
0: プロセッサは VIF を 0 にする。
1: プロセッサは一般保護例外を生成する。
- 上記がプロセッサの提供する機能です、プロセッサは VIP をセットしたり、クリアしたりすることはありません、VIF が 1 でも割り込みは発生しますから、その処理の中で該当する割り込みプログラムを実行せず VIP を 1 にセットし保留の扱いとして、割り込まれたプログラムに戻すのはシステム・レベルのプログラムの判断で処理します。
- STI を実行したときも、VIP が 1 なら一般保護例外が発生しますから、保留中の割り込みプログラムを実行するのも、しないのも、VIF を 0 にするのもシステム・レベルのプログラムの仕事になります。
- マニュアル中には特権レベル 0 の例外ハンドラと仮想 8086 モニタの仕事として記述されています。

2.10.7 仮想 8086 モードとページング

仮想 8086 モードのメモリの使い方におけるページングの効果を説明します、8086CPU は 1M バイトのメモリ空間ですから 1 タスクの仮想 8086 モードのプログラムは 1M バイトの範囲内でアクセスされています、仮想 8086 モードのプログラムが複数のタスクで動作している時は同じメモリアドレスを使っていますが、並行動作が可能です、これはそれぞれのタスクの論理アドレス空間は同じでも、それぞれのタスクの物理アドレス空間を違う場所に指定出来るからです。

2.10.8 保護モード仮想割り込み

特権レベル 3 のプログラムに対して、VIF,VIP フラグをサポートします、但しマスク可能ハードウェア割り込みにしか影響をあたえません。

機能は仮想 8086 モードにおける場合と同様です。

拡張機能の許可・禁止

CR4 の PVI(保護モード仮想割り込み) フラグを 1 にすると許可され、0 にすると禁止されます。

2.11 特権レベルと命令

特権レベルによる保護機能として、最上位の特権レベルのみが実行出来る命令があります。

- 保護モードの各種設定をする命令
 - ディスクリプタ・テーブルを定義する命令
 - * ディスクリプタ・テーブル・レジスタにロードする命令
 - ・ LGDT : GDTR にメモリからロード
 - ・ LIDT : IDTR にメモリからロード
 - ・ LLDT : LDTR にセクタをロード
 - タスク・レジスタに関する命令
 - * LTR : TR にセクタをロード
 - * STR : TR からオペランドにストア
 - コントロール・レジスタに関する命令
 - * MOV CRn,r32
 - * MOV r32,CRn
 - * LMSW
- タスクに関する命令
 - CLTS : CR0 の TS(タスク切り替え) ビットをクリアする。
- I/O 操作に関する命令
 - フラグ・レジスタの IOPL を操作する命令
POPF,POPF,IRET,IRETD で IOPL を更新出来るのは特権レベル 0 だけです。
- その他の命令
 - TLB エントリを無効化する命令
INVLPG
 - キャッシュに関する命令
INVD,WBINVD

– デバッグ・レジスタに関する命令

* **MOV DRn,r32**

* **MOV r32,DRn**

– **HLT**

プロセッサによる特権レベルのチェックによって実行出来ない命令があります。

● 割り込みに関する命令

– フラグ・レジスタの IF の変更に関する命令

CPL が IOPL と同じか、上位の場合に IF を更新出来る命令です。

* **STI**

* **CLI**

* **POPF,POPF,IRET,IRETD**

● I/O 操作に関する命令

IN,OUT : CPL が IOPL と同じか、上位の場合には命令を実行します、下位の場合は TSS の I/O ビット・マップが参照され該当ビットが 1 の場合は一般保護例外を発生します、0 の場合は命令を実行します。

第 3 章

ページングとその他

3.1 ページング

80386 からメモリ管理の方法がもうひとつ導入されました、メモリ空間を本のページの様にある決まった大きさで順番を持つものに分割しています。

ページングの選択はセグメント方式で生成したリニア・アドレスにページング方式のメモリ管理を使うか使わないかという事になります。

3.1.1 メモリ空間

プロセッサの扱うメモリ空間はセグメント方式で生成されるリニア・アドレスの空間と、そのアドレスを使ってページング方式が物理メモリに割り付ける空間の二つがあります。

前者を論理アドレス空間と呼び後者を物理アドレス空間と呼びます。

3.1.2 仮想記憶

この両空間はページ分割され、論理アドレスで指定したページに対する物理アドレス・ページの割付をプロセッサ・レベルでシステム・レベルのプログラムに押しサポートします。

論理アドレスと物理アドレスの変換にシステム・プログラムを介在させて仮想記憶システムを作ることが出来ます。

3.1.3 ページング方式

論理アドレス空間の 32 ビット・アドレスの下位 12 ビットをページ内のオフセット値として使い、上位 20 ビットの値でテーブルを引いて物理アドレス空間のページを選択します。

テーブル

論理アドレス空間の上位 10 ビットで 1 番目のテーブルのエントリを選択します、次の 10 ビットで 2 番目のテーブルのエントリを選択します。

2 段階にテーブルを使用して物理アドレス空間のページを選択します。

- 1 番目のテーブルの選択

プロセッサの PDBR(ページ・ディレクトリ・ベース・レジスタ) でテーブルの先頭アドレスをリニア・アドレスで指定します。

このテーブルを PDT(ページ・ディレクトリ・テーブル) と呼びます。

- 2 番目のテーブルの選択

1 番目のアドレスで選択されたエントリにより 2 番目のテーブルの先頭アドレスを指定します。

このテーブルを PT(ページ・テーブル) と呼びます。

エントリ

テーブルには 4 バイトの大きさのエントリが置かれます、[図 3.1](#) を参照して下さい。

- PD には PDE(ページ・ディレクトリ・エントリ) と呼ばれるものが置かれ、PT の先頭アドレスを指定します。
- PT には PTE(ページ・テーブル・エントリ) と呼ばれるものが置かれ、物理アドレス空間にあるページの先頭アドレスを指定します。

テーブルの構成

テーブルは 2 段階の階層構造になっています、上位に PD があり、下位に PT があります。

- ひとつの PT は 1024 個のページを持ち、ページひとつの大きさは 4k バイトです、PT ひとつで 4M バイトのメモリ空間に対応しています。
- プロセッサは 4G バイトのメモリ空間を持ちま

すが、4M バイト以内の物理メモリ空間で仮想記憶のような使い方をしないのなら PD にひとつだけエントリを登録して PT をひとつだけでも十分です。

テーブルの大きさ

PD,PT とも 4 バイトのエントリを 1024 個登録できます。

その大きさは 4k バイトになります、PD の中のひとつのエントリは、ひとつの PT を指しますから、PDE は 1024 個の PTE を従えています、PD 全体としては $1024 \times 1024 = 1018576$ 個の PTE を持てます、その大きさは 4G バイトになります。

ページの保護

ページごとに特権レベルによる保護属性を設定出来ます、どのような属性になるかは PDE,PTE のエントリにより決定されます。

特権レベル 0,1,2 はスーパー・バイザ、特権レベル 3 はユーザのページに設定して、2 レベルの特権保護を受けられます。

レベル 3 のプログラムはスーパー・バイザ・ページにアクセス出来ません、また、アクセスが許可されるページでも書き込みが出来ないような設定にも出来ます。

- R/W,U/S

CR0(コントロール・レジスタ 0) の WP ビットと合わせてページの保護属性を設定します、**表 3.1** を参照して下さい。

WP ビットが 1 の時は特権 0 のプログラムでもリード・オンリに設定されたページに書き込むと例外を発生します。

ページのキャッシング

ページごとにキャッシングの設定をすることが出来ます、キャッシュの有効・無効とライト・バック、ライト・スルーを設定します。

PDE,PTE にこれらの設定を行いますが、PDE のキャッシング設定は PT に適用され、PTE のキャッシング設定はページに適用されます。

- PWT (ページ・ライト・スルー)

- 0:ライト・バック・キャッシュを選択
- 1:ライト・スルー・キャッシュを選択

PWT ビットはページのキャッシュ方式を PWT 端子で外部キャッシュに通知します、CPU の内部キャッシュは影響を受けません。

- PCD (ページ・キャッシュ・ディセーブル)

- 0:内蔵キャッシュをイネーブルにします。
- 1:内蔵キャッシュをディセーブルにします。

仮想記憶のサポート

プロセッサはシステム・レベルのプログラムに対して仮想記憶をサポートするビットを PDE,PTE に持ちます。

そのページが物理アドレス空間に、あるかどうか(割り付けているかどうか)を示します。

物理アドレス空間に、無いとされているページにアクセスした場合はページ・フォルト例外を発生します。

- P (プレゼント)

エントリの指すリニア・アドレスに物理メモリがマッピングされているかどうかを示します。

- 0:物理メモリが割り当てられていない。
- 1:物理メモリが割り当てられている。

仮想記憶をサポートするシステムは、どこにも割り当てていない物理メモリがあれば、それを割り当ててプログラムを再実行します。

物理メモリが無いときは物理メモリを割り当てたページのなかで、あまり使っていないページを捜してディスク装置などにセーブします。

その後、割り当てを解除して、そこに参照されたページのデータをディスク装置からロードして物理メモリを割り付けます。

ページのチェック

プロセッサはシステム・レベルのプログラムに対してページの使用状況の調査をサポートするビットを持ちます。

PDE,PTE には、そのページがアクセスされたときにプロセッサが 1 にするビットがあります、プロセッサはこのビットを 0 にすることはありません。

PTE には、そのページに書き込みされたときにプロセッサが 1 にするビットがあります、プロセッサはこのビットを 0 にすることはありません。

表 3.1 ページの保護属性

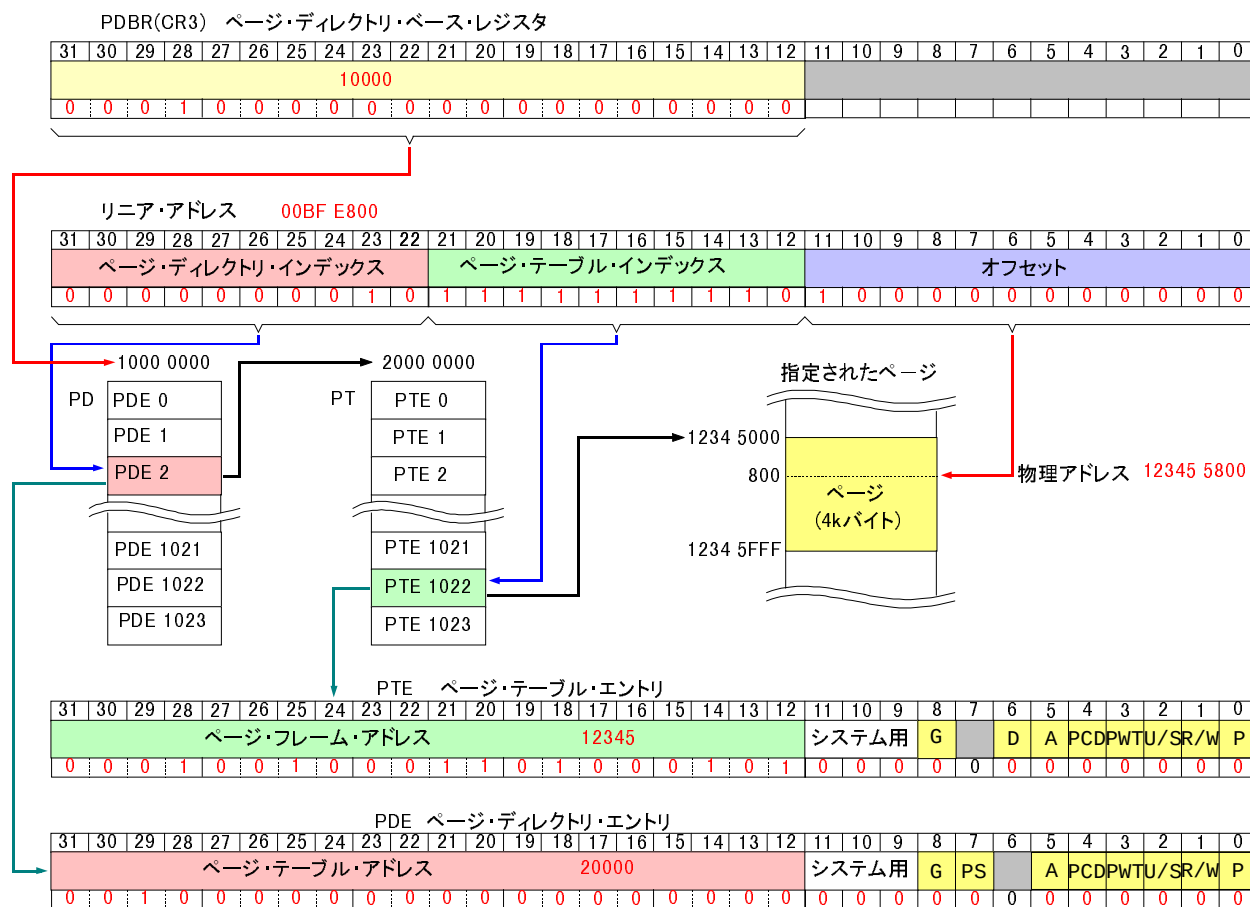
CR0	PDE		PTE		適用されるページ属性		ユーザ属性	スーパ・バイザ ・アクセス
	WP	U/S	R/W	U/S	R/W	U/S	R/W	
0		U	R	U	R	U	R	読み出し/実行
0		U	R	U	W	U	R	読み出し/実行
0		U	W	U	R	U	R	読み出し/実行
0		U	W	U	W	U	W	読み出し/書き込み/実行
0		U	R	S	R	S	W	不可
0		U	R	S	W	S	W	不可
0		U	W	S	R	S	W	不可
0		U	W	S	W	S	W	不可
0		S	R	U	R	S	W	不可
0		S	R	U	W	S	W	不可
0		S	W	U	R	S	W	不可
0		S	W	U	W	S	W	不可
0		S	R	S	R	S	W	不可
0		S	R	S	W	S	W	不可
0		S	W	S	R	S	W	不可
0		S	W	S	W	S	W	不可
1		U	R	U	R	U	R	読み出し/実行
1		U	R	U	W	U	R	読み出し/実行
1		U	W	U	W	U	R	読み出し/実行
1		U	W	U	W	U	W	読み出し/書き込み/実行
1		U	R	S	R	S	R	不可
1		U	R	S	W	S	R	不可
1		U	W	S	R	S	R	不可
1		U	W	S	W	S	W	読み出し/書き込み/実行
1		S	R	U	R	S	R	不可
1		S	R	U	W	S	R	不可
1		S	W	U	R	S	R	不可
1		S	W	U	W	S	W	読み出し/書き込み/実行
1		S	R	S	R	S	R	不可
1		S	R	S	W	S	R	不可
1		S	W	S	R	S	R	不可
1		S	W	S	W	S	W	読み出し/書き込み/実行

U:ユーザ (特権レベル 3) S:スーパ・バイザ (特権レベル 0,1,2)

R:リード・オンリ W:リード/ライト

U/S 0:スーパ・バイザ 1:ユーザ

R/W 0:リード・オンリ 1:リード/ライト



- A (アクセス)

PDE, PTE でカバーされるページがアクセスされたかどうかを示す。

- 0: アクセスされていない。
- 1: アクセスされた。

- D (ダーティ)

PTE でカバーされるページに書き込みが行われたかどうかを示す。

- 0: 書き込まれていない。
- 1: 書き込まれた。

プロセッサは、このビットを 1 にしても 0 にしません、仮想記憶システムではこのページに物理メモリを割り付けたときに、このビットを 0 にしていれば、このページの割り当てを解除するときにこのビットが 0 ならば内容の変更は無くディスクなどに書き戻さなくてよいことが知られます。

エントリによるシステム提供ビット

システム・プログラマが自由に定義して使えるビットを PDE, PTE のビット 9, 10, 11 に用意しています。

将来のプロセッサでもインテルが使用しないことを保証しています。

ページング使用中のページ・テーブルの変更

プロセッサは最近使った PTE をプロセッサ内部に記憶する TLB(Translation Lookaside Buffer) と呼ばれる場所を持っています。

80486 では 32 個の PTE を格納しています、プロセッサは TLB の PTE から使用しますから、テーブルの変更をした場合は専用の命令で TLB の PTE を無効にして更新した PTE が使われるようにします。

3.2 Pentium 以降のページングの拡張

ページ・サイズを 2,4M バイトで扱えるようになり、そのイネーブル・ビットが追加されています。

PentiumPro ではアドレス・ピンが追加されて物理アドレス空間を 36 ビットで指定出来るようになった

ので、これを使えるようにするためのビットが追加されています。

3.2.1 コントロール・レジスタ

CR4 に次のビットが追加されました。

PAE (物理アドレス拡張)

- 0 : 物理アドレスは 32 ビットまで。
- 1 : // を 36 ビットに拡張。

PSE (ページ・サイズ拡張)

- 0 : 4k バイト・ページのみ有効。
- 1 : 4M バイト・ページをイネーブル。

これらのビットの組み合わせにより、決定されるページ・サイズや物理アドレス・サイズについては表 3.2 を参照して下さい。

3.2.2 ページ・サイズの拡張

32 ビットの物理アドレス空間で 4M バイト・ページ、36 ビットの物理アドレス空間で 2M バイト・ページを使うことが出来ます。

- 4M バイト・ページ
PT を使わずに PDE が直接、物理アドレス空間のページを指します、PT の PTE を指すのに使われていたリニア・アドレスの 10 ビットはオフセットに加えられました。
32 ビットのリニア・アドレスの内、上位 10 ビットで PDE を指定して、後の 22 ビットをページ中のオフセットに使います、図 3.2 を参照して下さい。
- 2M バイト・ページ
物理アドレス空間を拡張した際には 4M バイト・ページは使えず 2M バイト・ページを使います、次項を参照して下さい。

3.2.3 物理アドレス空間の拡張

PentiumPro のハードウェアのアドレスが 36 ビットになったことにより、サポートされました。

4G バイトの論理アドレス空間を 4k バイト・ページまたは 2M バイト・ページで 16G バイトの物理アドレス空間にマップします。

拡張された物理アドレス空間はページングでのみアクセス出来ます。

リニア・アドレスを 32 ビットのままで、4k ページとのテーブルの互換性を保ったので 4M バイトのページは使えず 2M バイト・ページになっています。

テーブル

32 ビットでは PD,PT の 2 階層のテーブルを使っていましたが、36 ビットでは PDPT(ページ・ディレクトリ・ポインタ・テーブル),PD,PT の 3 階層のテーブルを使います、図 3.3 を参照して下さい。

CR3

32 ビットでは CR3 の上位 20 ビットを使って PD の先頭リニア・アドレスを指定していましたが、36 ビットでは上位 27 ビットを使って PDPT の先頭リニア・アドレスを指定します。

PDPT

36 ビットでのページングで使うテーブル階層の最上位に位置するもので、4 個の DPE(ディレクトリ・ポインタ・エントリ)を持ちます。

PDE,PTE

PD,PT に登録されるエントリのサイズが 36 ビットの物理アドレス空間のページを指定するために 8 バイトになりました。

これでひとつのテーブルのエントリの個数は半分の 512 個になりました。

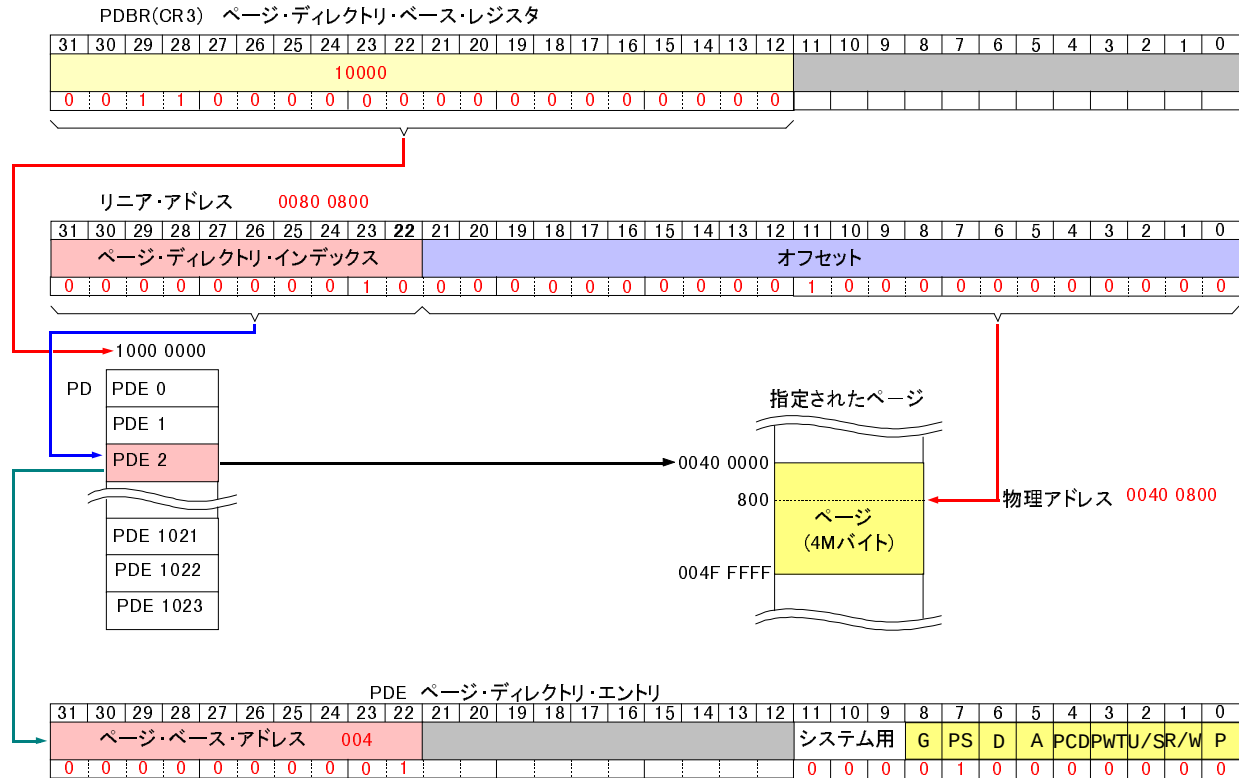
PTE(PDE) の個数

- 4k バイト・ページ
PD,PT に登録出来るエントリの個数が両テーブルで $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$ になりましたが、PDPT が 4 個の PD を持つのでページングで使われる PTE の個数は 4k ページの場合は 32 ビットと変わりません。
- 2M バイト・ページ
PD テーブルの 512 個の PDE で物理アドレス空間のページを指定します、PDPT が 4 個の PD を指定出来るので $512 \times 4 = 2048$ 個の PDE で 4G バイトにマップしています、図 3.4 を参照して下さい。

論理アドレスから物理アドレスへの変換

PD,PT に登録出来るエントリが半分になったので、それぞれのテーブルのエントリを指定するのに必要なビット数は 9 ビットです、32 ビットの場合はリニア・アドレスの上位 20 ビットを使っていましたが、この内の 18 ビットを PDE,PTE の指定に使って、残りの 2 ビットを最上位に置いて PDPT の DPE の指定に使います。

図 3.2 4M バイト・ページの変換



- 4k バイト・ページ

PDPT と PD,PT のために上位 20 ビットを使います、残りの 12 ビットはページ内のオフセットになります。

- 2M バイト・ページ

PDPT と PD のために上位 12 ビットを使います、残りの 20 ビットはページ内のオフセットになります。

3.3 コントロール・レジスタ

80286 で MSW(マシン・ステータス・ワード)と呼ばれていた 16 ビットのレジスタが拡張されたものです。

プロセッサの動作モードを決定したり、状態を知るための 32 ビットのレジスタで 5 本あります、図 3.5 を参照して下さい。

3.3.1 CR0

PE (プロテクション・イネーブル)

- 0:リアル・モード

図 3.5 コントロール・レジスタ

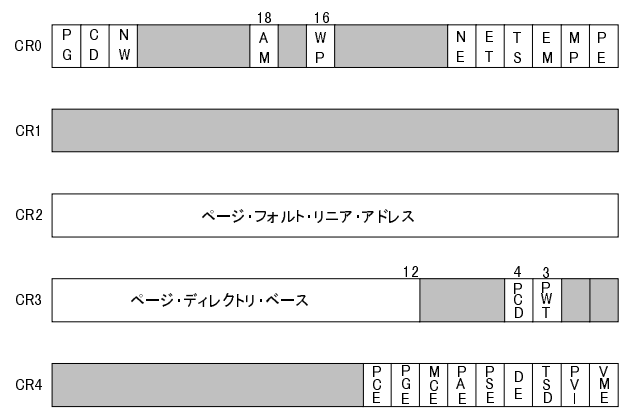


図 3.6 テーブル・レジスタ

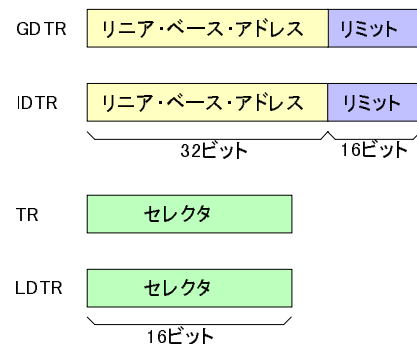


図 3.3 拡張物理アドレスの 4k バイト・ページ

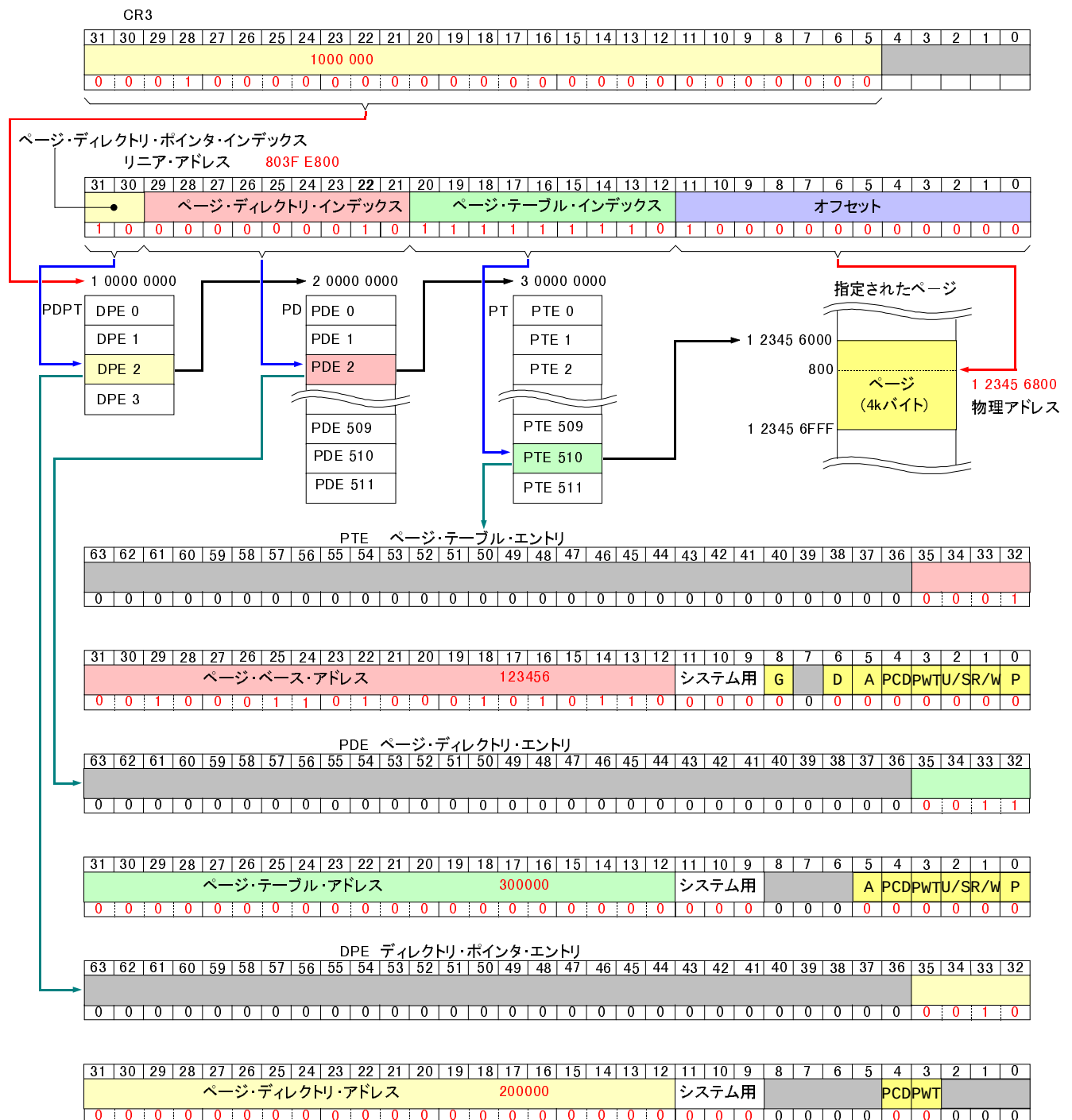


図 3.4 拡張物理アドレスの 4M バイト・ページ

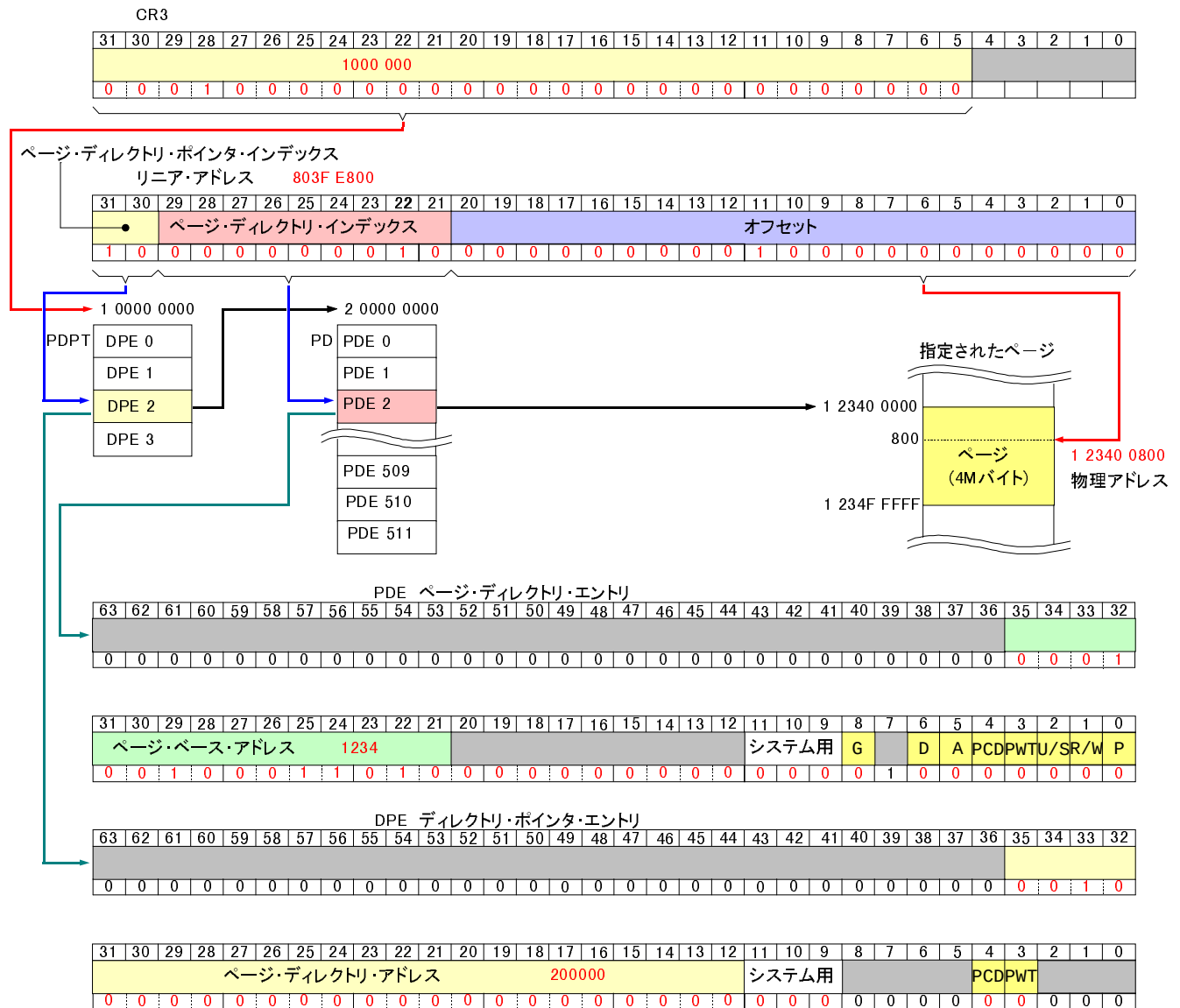


表 3.2 ページ・サイズと物理アドレス・サイズ

CR0	CR4		PDE		アドレス サイズ
PG	PAE	PSE	PS	サイズ	
0	—	—	—	—	ページング不可
1	0	0	—	4k	32 ビット
1	0	1	0	4k	32 ビット
1	0	1	1	4M	32 ビット
1	1	—	0	4k	36 ビット
1	1	—	1	2M	36 ビット

● 1:プロテクト・モード

セグメント・レベルの保護機能をイネーブルにします、表 3.3 を参照して下さい。

MP (モニタ・コプロセッサ)

● 0:コプロセッサが無い時

● 1:コプロセッサが在る時 (FPU は該当する)

MP ビットは TS ビットと組合わせて WAIT 命令でトラップするかどうかを決めます、WAIT 命令はコプロセッサとの同期をとる命令です、一般的には浮動小数点命令の間に挿入されます、タスク・スイッチが起

表 3.4 オンチップ浮動小数点ユニット制御

CR0			命令のタイプ		TS のチェック	ソフトウェア エミュレーション $EM = 1$
EM	TS	MP	浮動小数点	WAIT		
0	0	0	実行	実行	×	×
0	0	1	実行	実行	○	×
0	1	0	トラップ	実行	×	×
0	1	1	トラップ	トラップ	○	×
1	0	0	トラップ	実行	×	○
1	0	1	トラップ	実行	○	×
1	1	0	トラップ	実行	×	○
1	1	1	トラップ	トラップ	○	×

表 3.3 プロセッサの動作モード

PG	PE	モード
0	0	リアル・モード
0	1	プロテクト・モード
1	0	未定義
1	1	ページングのプロテクト・モード

きたときには FPU のレジスタは自動的に保存されませんから、この例外で前のタスクの FPU のレジスタを保存して現在のタスクの FPU レジスタを復帰させて処理を継続すればタスク間の FPU のプログラムの独自性を確保出来ます、表 3.4 を参照して下さい。

EM (エミュレート・コプロセッサ)

- 0:浮動小数点命令でトラップしない。
- 1:浮動小数点命令でトラップする。

FPU がなくて浮動小数点命令をプログラムでエミュレートする場合などに使いますから EM が 1 なら MP は 0 にします、表 3.4 を参照して下さい。

TS (タスク・スイッチ)

プロセッサはタスク・スイッチが起きた時にこのビットを 1 にします、自動的に 0 にされませんから適切な時期に命令により 0 にします、FPU との関連において利用する場合は表 3.4 を参照して下さい。

ET (拡張タイプ)

80386 や 80286 では FPU は外付けの 80387 や 80287 が在る時は 1 に無い時は 0 になります、80486

や Pentium では FPU 内蔵ですから 1 に固定されています。

NE (数値演算エラー)

浮動小数点エラー例外を下記のように処理します。

- 0:外部割り込みを待って処理します。
- 1:割り込み番号 16 を発生させます。

ビットが 0 のときは PC スタイル (DOS) のエラー・レポートを実現します、詳細は割愛。

WP (書き込み保護)

読みだし専用のページにスーパー・バイザ・レベルのプログラムから書き込みがあったときに例外を発生するかどうかを指定します。

- 0:例外は発生しない。
- 1:例外を発生する。

80486 で追加したフラグなので 80386 の動作は 0 のときと同じです。

AM (整列マスク)

フラグ・レジスタの AC ビットによる整列チェック例外をマスクするかしないかを指定します。

- 0:例外をマスクする。
- 1:例外をマスクしない。

NW (ライト・スルー・ビット)

キャッシュのライト・スルーと無効化サイクルについて設定します、表 3.5 を参照して下さい。

- 0:イネーブルにする。
- 1:ディスエーブルにする。

表 3.5 オンチップ・キャッシュ制御モード

CD	NW	動作モード
0	0	ライト・スルー・キャッシュをイネーブル
0	1	不正組み合わせ
1	0	キャッシュ不使用
1	1	キャッシュ不使用

CD (キャッシュ・ディスエーブル)

オンチップ・キャッシュの使用について設定します、表 3.5 を参照して下さい。

キャッシュへのアクセスを完全になくすには、さらにキャッシュを無効化します。

PG (ページング・イネーブル)

ページングの使用について設定します、表 3.3 を参照して下さい。

ページングが使用されない場合はリニア・アドレスがそのまま物理アドレスに使用されます。

3.3.2 CR1

インテル予約（機能の定義はありません）。

3.3.3 CR2

ページ・フォルト・リニア・アドレス・レジスタと呼び、最後にページ・フォルトを起こしたリニア・アドレスを保存しています。

3.3.4 CR3

ページ・ディレクトリ・テーブルの先頭アドレスを上位 20 ビットに指定します。

拡張物理アドレスをイネーブルしている場合は上位 27 ビットでページ・ディレクトリ・ポインタ・テーブルの先頭アドレスを指定します。

PWT (ページ・ライト・スルー)

PDT(ページ・ディレクトリ・テーブル)をアクセスしたときのキャッシュ方式を指定します。

- 0: ライト・バック方式。
- 1: ライト・スルー方式。

PCD (ページ・キャッシュ・ディスエーブル)

PDT(ページ・ディレクトリ・テーブル)をアクセスしたときのキャッシュの使用を指定します。

- 0: 使う。
- 1: 使わない。

3.3.5 CR4

Pentium で追加されたレジスタです。

VME 仮想 8086 モード拡張

仮想 8086 モードでの割り込みの拡張をイネーブルにして VIF(仮想割り込みフラグ)をサポートします。

- 0: ディセーブル
- 1: イネーブル

PVI 保護モード仮想割り込み

保護モードでの割り込みの拡張をイネーブルにして VIF をサポートします。

- 0: ディセーブル
- 1: イネーブル

TSD タイム・スタンプ・ディスエーブル

RDTSC 命令を特権レベル 0 に制限します。

- 0: 全特権レベルで実行可能。
- 1: 特権レベル 0 だけ実行可能。

DE デバッグ拡張

デバッグ・レジスタの DR4, DR5 を参照する命令に対して、未定義オペコードの例外を発生させます。

- 0: 例外を発生さない。
- 1: 例外を発生させる。

PSE ページ・サイズ拡張

4M バイト・ページをイネーブルにします、物理アドレスが拡張されているときは 2M バイト・ページになります。

- 0: 4M(2M) バイト・ページをディセーブル
- 1: 4M(2M) バイト・ページをイネーブル

PAE 物理アドレス拡張

PentiumPro で設けられたビットで、物理アドレスを 36 ビットに拡張します。

- 0: 拡張しない。
- 1: 拡張する。

MCE マシン・チェック・イネーブル

マシン・チェック例外の設定をします。

- 0: 例外を発生さない。
- 1: 例外を発生させる。

PGE ページ・グローバル・イネーブル

PentiumPro で設けられたビットで、グローバル・ページ機能の設定をします。

- 0: ディセーブル
- 1: イネーブル

PCE パフォーマンス・モニタリング・カウンタ・インネブル

PentiumPro で設けられたビットで、RDPMC 命令を特権レベル 0 に制限します。

- 0 : 特権レベル 0 だけ実行可能。
- 1 : 全特権レベルで実行可能。

3.4 デバッグ

3.4.1 8086 のデバッグ機能

8086 でサポートされたデバッグを支援する機能には 2 種類があります、シングル・ステップ割り込みとブレイク・ポイント割り込みです、フラグ・レジスタの TF ビットを 1 にするとその次の命令からシングル・ステップ例外が発生します、ブレイク・ポイント割り込みは 1 バイトのソフトウェア割り込み命令をプログラム中の 1 バイトを置き換えておいて命令の実行によりブレイク・ポイント例外が発生します。

3.4.2 80286 のデバッグ機能

80286 でプロテクト・モードなどの大幅な機能拡張がなされましたがデバッグ機能については 8086 と変わりません。

3.4.3 80386/80486 のデバッグ機能

80386 では専用レジスタ群の追加によりデバッグ機能が大幅に強化されました、4 本の 32 ビットのブレイク・ポイント設定レジスタがありメモリ空間のどこにでもリニア・アドレスで設定できます、またデータ・アクセスによるブレイクも可能になりました、これらの機能はリアル・モードでも使えます。

ある特定のタスクにスイッチした時にデバッグ例外を起こす事も可能です。

3.4.4 Pentium のデバッグ機能

Pentium では 80486 のデバッグ機能に加えて I/O アクセスに対するブレイクが可能になりました。

3.4.5 デバッグ・レジスタ

デバッグをサポートするレジスタが 8 本あります。

DR0 から DR3 までの 4 本がブレイク・ポイントをリニア・アドレスで指定するレジスタです。

DR4 と DR5 はインテル予約で PentiumPro でも機能は定義されていません。

DR6 はステータス・レジスタの様に使います、デバッグ例外が発生したときにそれがどのブレイク・ポイント・レジスタで発生したのか、あるいはシングル・

図 3.8 デバッグ・レジスタ構成 2

DR5	インテル予約
DR4	インテル予約
DR3	ブレイク・ポイント 3 リニア・アドレス
DR2	ブレイク・ポイント 2 リニア・アドレス
DR1	ブレイク・ポイント 1 リニア・アドレス
DR0	ブレイク・ポイント 0 リニア・アドレス

表 3.8 デバッグ・レジスタ構成 5

⑤	B0	DR0 のブレイク検出	0	未検出
			1	検出
	B1	DR1 のブレイク検出	0	未検出
			1	検出
	B2	DR2 のブレイク検出	0	未検出
			1	検出
⑥	BS	シングル・ステップ・トラップ	0	発生していない
			1	発生している
	BD	デバッグ・レジスタのアクセス	0	発生していない
			1	発生している
	BT	タスク・スイッチ	0	発生していない
			1	発生している

ステップ・トラップなのか、またはタスク・スイッチによるのかなどを調べます、このレジスタのビットはデバッグ例外が発生した時に更新されています。

DR7 はコントロール・レジスタの様に使います、4 本のブレイク・ポイント・レジスタのブレイクする条件を設定します、それぞれのブレイク・ポイントが命令実行でなのか、データの書き込みか読み出し、I/O のアクセスなのかを指定するビットを持ちます。

図 3.7、図 3.8 を参照して下さい。

3.4.6 DR7

LEN0~3

ブレイク・アドレスのサイズを設定します。

命令の実行でブレイクさせる場合は 1 バイト長の指

図 3.7 デバッグ・レジスタ構成 1

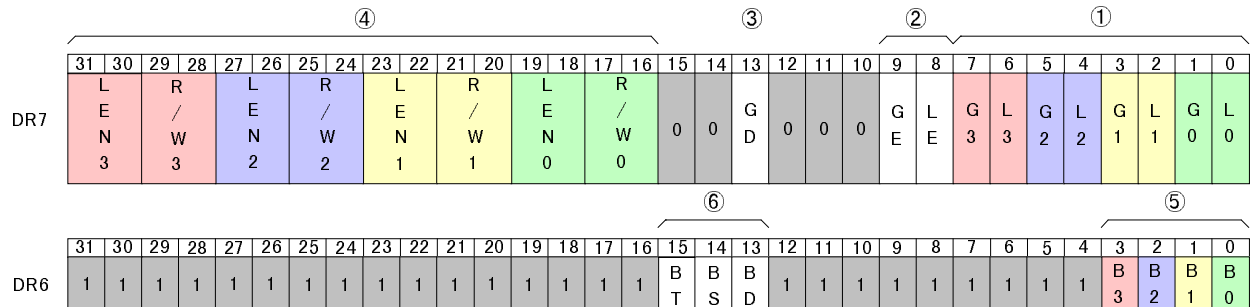


表 3.6 デバッグ・レジスタ構成 3

①	L0~3	ローカル・ブレイク・イネーブル	0	ディセーブル
			1	イネーブル
	G0~3	グローバル・ブレイク・イネーブル	0	ディセーブル
			1	イネーブル
②	LE	ローカル・ブレイク	0	ディセーブル
			1	イネーブル
	GE	グローバル・ブレイク	0	ディセーブル
			1	イネーブル
③	GD	グローバル・デバッグレジスタ・アクセス検出設定	0	例外を発生しない
			1	例外を発生する

定にします、データ・アクセスでブレイクさせるときに LEN を 2 バイト長にしたときにはブレイク・ポイントをワード境界に設定します、LEN を 4 バイト長にしたときにはブレイク・ポイントをダブル・ワード境界に設定します。

R/W0~3

ブレイク・ポイントの条件を設定します。

Pentium で I/O アクセスが追加されました、CR4 の DE ビットでデバッグ拡張がイネーブルしているときに使用できます。

GD 一般検出イネーブル

デバッグ・レジスタがアクセスされたときに例外を発生させます、これはターゲット・プログラムがデバッグ・レジスタをアクセスしてデバッグが意図しない状態に陥る事を防ぐことが出来ます。

LE,GE

このビットは 80486 から PentiumPro では使いません、80386 ではデータ・アクセスのブレイク・ポイントを引き起こした命令を正確に捉える為に CPU の処理を遅くしてもよいかどうかを指定します。

G0~3

ブレイク・ポイントをグローバルに有効にします、次項も参照して下さい。

L0~3

ブレイク・ポイントを現タスク内でのみ有効にします、タスク・スイッチが起こるとこのビットは 0 にされますから、別のタスクでのブレイク・ポイントによりデバッグ例外が発生する事はありません。

3.4.7 DR6

BT

TSS の T ビットが 1 になっているタスクにタスク・スイッチした事を示します。

表 3.7 デバッグ・レジスタ構成 4

④	R/W 0～3	ブレーク条件設定	0	0	命令実行時
			0	1	データ書き込み
			1	0	I/O 読みとり・書き込み
			1	1	データ読みとり・書き込み
	LEN 0～3	ブレーク・ポイント ロケーション・ サイズ設定	0	0	1 バイト長
			0	1	2 バイト長
			1	0	未定義
			1	1	4 バイト長

BS

シングル・ステップ・トラップが発生したことを示します。

BD

DR7 の GD ビットを 1 に設定しているときにデバッグ・レジスタがアクセスされたことを示します。

B0～3

DR0～3 のブレーク・ポイントの条件が成立しているかどうかを示します。

このビットは G または L ビットが 1 になっていなくても条件が成立していれば 1 にされます。

※ DR6 の各ビットはプロセッサによってクリアされませんからデバッグ例外処理プログラムでクリアします。

3.4.8 DR4,5

デバッグ拡張がイネーブルされていないときには、DR4 は DR6 を DR5 は DR7 を参照します。

3.5 プロテクト・モードのプログラム**3.5.1 ディスクリプタ・テーブル・レジスタ**

GDTR と IDTR はベース・アドレスの 32 ビットとリミットの 16 ビットで 48 ビットをレジスタにロードするわけですが汎用レジスタで一度にロード出来ませんから、一旦メモリ上に設定値を置いてからレジスタにロードします。

LDTR と TR はセクタの 16 ビットですから汎用レジスタで指定出来ます。 [リスト 4](#) を参照して下さい。

3.5.2 コントロール・レジスタ/デバッグ・レジスタ

これらのレジスタは 32 ビットの汎用レジスタでロード出来ます、 [リスト 5](#) を参照して下さい。

3.5.3 プロテクト・モードへの移行

モードの移行は CR0 の PE ビットを 1 にするだけで移行しますが、その前にこのモードで使われるテーブルをメモリ上に作成したりプロセッサのレジスタにテーブルの先頭アドレスを登録したりとの準備が必要です。

GDT と IDT は必ず TSS はほとんどの場合は必要です、LDT は必要に応じて、PDT と PT はページングを使用するのなら必要です。

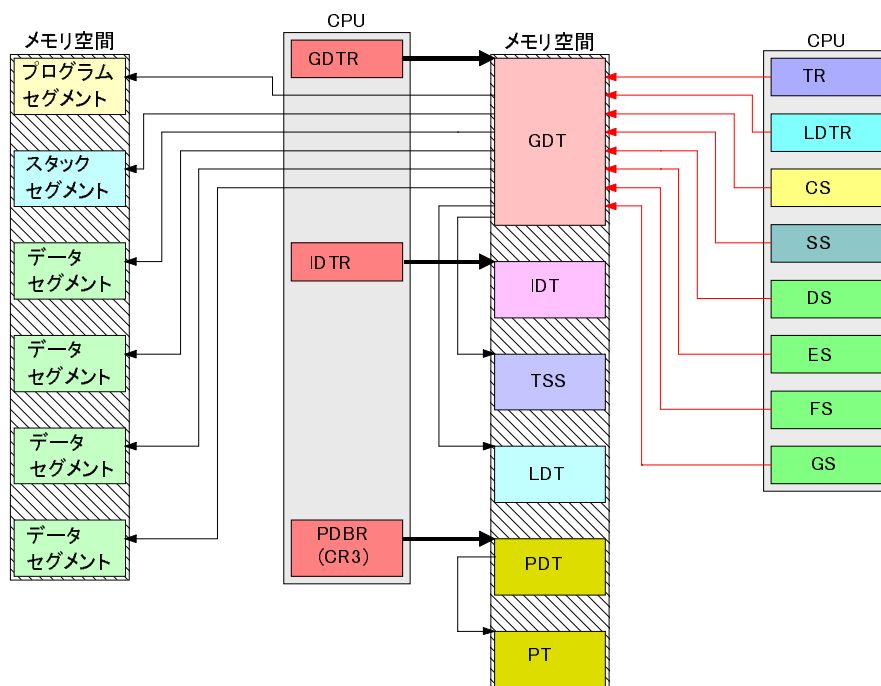
モードの移行直後はセグメント・レジスタが更新されるまでリアル・モードのアドレス解釈で動作しています、CS にセグメント・セクタをロードするセグメント間 JMP 命令を実行すれば CS のディスクリプタ・レジスタにセクタで指定されたディスクリプタをロードしてその情報からアドレスを生成するプロテクト・モードの動作になります。 [リスト 6](#) と [図 3.9](#) を参照して下さい。

3.5.4 プロテクト・モードからリアル・モードに移行

モードの移行は CR0 の PE ビットを 0 にするだけで移行しますが、セグメント・レジスタの値によっては移行したとたん暴走する可能性があります。

モードの移行直後はプログラムの実行はどこにしているのかと言いますと、CS のディスクリプタ・レジスタの値が CS の更新まで使われます、リミット値なども有効なので移行する前にセグメント・レジスタにリアル・モードでも適合性のあるディスクリプタをロードしてから移行します。 [リスト 7](#) と [図 3.11](#) を

図 3.10 プロテクト・モードの設定



参照して下さい。

3.5.5 ページングのイネーブル

プロセッサがプロテクト・モードになっていて、PDTやPT、PDPT^{*1}が準備されている必要があります。

PGビットを操作するプログラムはリニア・アドレスと物理アドレスが一致するようなページで実行したほうがよいでしょう。

3.5.6 ページングのディセーブル

ページングがイネーブルの間にリニア・アドレスと物理アドレスが一致するようなマップで、CR0のPGビットを0にしてページングをディセーブルにします、CR3のページ・ディレクトリ・ベース・レジスタに0をロードして無効にしておきます。

3.5.7 キャッシュのイネーブル

ページングを使わない場合

CR0のCDビットとNWビットを0にすればプロセッサの条件は揃います、外部ハードウェアはメモリ領域によってキャッシュを有効にしたり無効にしたりします、両方の条件が有効になった時、キャッシュはイネーブルにされます。

ページングを使う場合

ページングでは専用のテーブルを使ってメモリを4kバイト単位^{*2}で管理し保護属性の設定などを行っています。

この中にキャッシュのイネーブル・ビットも含まれます、

CR0のCD,NWビットによりプロセッサ・レベルのキャッシュ設定が有効になっていて、PTE^{*3}のPCD,PWTビットでページ・レベルのキャッシュを有効にした場合に該当ページにキャッシュが適用されます。

CR3やPDEのキャッシュ設定ビットはそれぞれが指定するテーブルに対するキャッシュに適用されます。

3.5.8 キャッシュのディセーブル

CR0のCDビットを1にすれば、プロセッサのキャッシュ使用を禁止します。

この禁止とはキャッシュ・ライン・フィルなどでキャッシュ・メモリのデータを更新しないということで、ヒットした場合はキャッシュ・メモリのデータが

^{*1} 物理アドレス拡張がイネーブルされている場合。

^{*2} 拡張機能で2M,4Mバイト・テーブルも使えます。

^{*3} 2M,4Mページの場合はPDEになります。

図 3.9 リニア・アドレスと物理アドレスの変換

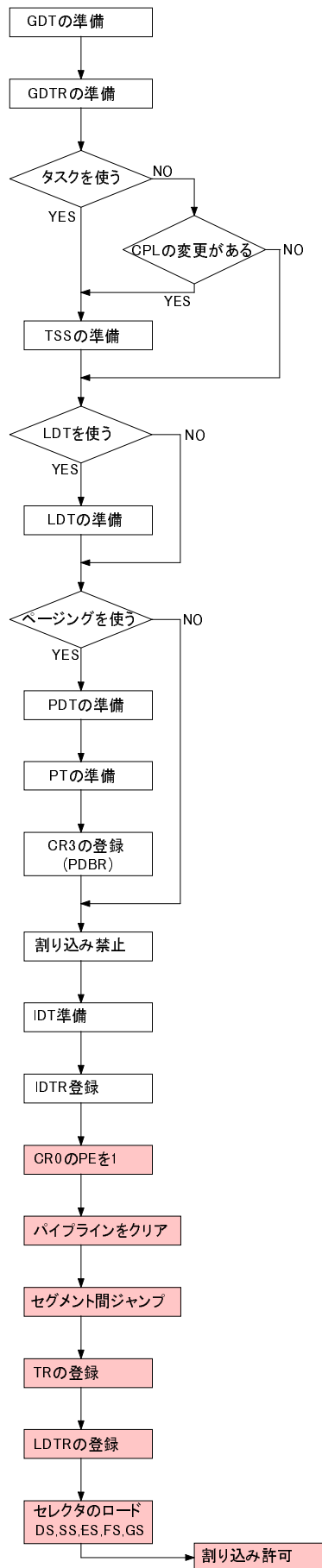
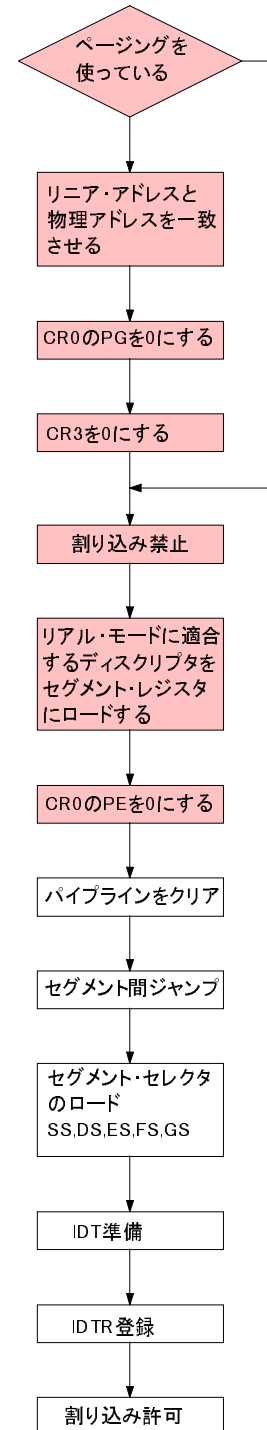


図 3.11 リアル・モードへの移行



使われます。

キャッシュ・メモリのデータを使わないようにするには専用の命令によりキャッシュを無効化してヒットが起きないようにします。

リスト4 LST4A.ASM

```

;=====;
;
;      プロテクトモード用 レジスタの使い方
;
;=====;
.486P

GDT_ENT EQU 10000h ; GDT のベース・アドレス
GDT_LIM EQU 100*8 ; ディスクリプタを 100 個登録出来るリミット

DGROUP GROUP _BSS

ASSUME CS:RAM,DS:DGROUP

RAM SEGMENT DWORD PUBLIC USE16 'CODE'

task0 PROC FAR

;-----;
; グローバル・ディスクリプタ・テーブル・レジスタのロードとセーブ
;-----;
MOV AX,GDT_LIM
MOV word ptr [TABLEG],AX ; リミットのセット
MOV EAX,GDT_ENT
MOV dword ptr 2[TABLEG],EAX ; テーブル・ベース・アドレスのセット

LGDT [TABLEG] ; メモリ -> グローバル・ディスクリプタ・テーブル・レジスタ ロード
SGDT [TABLEG] ; グローバル・ディスクリプタ・テーブル・レジスタ -> メモリ セーブ

;-----;
; ローカル・ディスクリプタ・テーブル・レジスタのロードとセーブ
;-----;
MOV ax,50h ; GDT の 10 番目の (TI=0,RPL=00) LDT ディスクリプタを設定
MOV [TABLEL],ax ; セレクタのセット
LLDT [TABLEL] ; ローカル・ディスクリプタ・テーブル・レジスタのロード
SLDT [TABLEL] ; ローカル・ディスクリプタ・テーブル・レジスタのセーブ

;-----;
; 割り込みディスクリプタ・テーブル・レジスタの ロードとセーブ
;-----;
MOV AX,1024
MOV word ptr [TABLEI],AX ; リミットのセット
MOV EAX,00000h
MOV dword ptr 2[TABLEI],EAX ; テーブル・ベース・アドレスのセット

LIDT [TABLEI] ; メモリ -> 割り込みディスクリプタ・テーブル・レジスタ ロード
SIDT [TABLEI] ; 割り込みディスクリプタ・テーブル・レジスタ -> メモリ セーブ

;-----;
; タスク・レジスタのロードとセーブ
;-----;
MOV [TABLET],58h ; GDT の 11 番目の (TI=0,RPL=00) TSS ディスクリプタを設定
LTR [TABLET] ; タスク・レジスタのロード
STR [TABLET] ; タスク・レジスタのセーブ

task0 ENDP

RAM ENDS

_BSS SEGMENT PARA PUBLIC USE16 'BSS'

TABLEG DF ?
TABLEI DF ?
TABLEL DW ?
TABLET DW ?

_BSS ENDS

END

```

リスト5 LST4B.ASM

```

;=====;
;
; コントロール・レジスタ/デバッグ・レジスタ 操作命令
;
;=====;
        .486P

        ASSUME CS:RAM

RAM      SEGMENT DWORD PUBLIC USE16 'CODE'

task0    PROC    FAR

;-----;
;      コントロール・レジスタ
;-----;

        MOV     CR0,EAX
        MOV     CR2,EAX
        MOV     CR3,EAX

;-----;
;      デバッグ・レジスタ
;-----;

        MOV     DR0,EAX
        MOV     DR1,EAX
        MOV     DR2,EAX
        MOV     DR3,EAX
        MOV     DR6,EAX
        MOV     DR7,EAX

task0    ENDP

RAM      ENDS

        END

```

リスト6 LST4C.ASM

```

;=====;
;      real mode -> protect mode
;=====;
@prot$qv PROC NEAR

    cli                                ; 割り込み禁止

    mov     ax, IDT_LIM
    mov     word ptr [_IDTR], ax
    mov     eax, IDT_ENT
    mov     dword ptr 2[_IDTR], eax

    lidt     [_IDTR]                  ; プロテクト・モード用 IDTR の登録

    mov     eax, cr0
    or      eax, 1
    mov     cr0, eax                ; プロテクト・モードに移行

    jmp     prot_L1                  ; リアル・モードでパイプラインに先読み
    ; した命令を捨てる.

prot_L1:
    mov     ax, 10h
    mov     ds, ax                  ; DS にセクタ ロード
    mov     es, ax                  ; ES にセクタ ロード
    mov     fs, ax                  ; FS にセクタ ロード
    mov     gs, ax                  ; GS にセクタ ロード

    mov     ax, 18h
    mov     ss, ax                  ; SS にセクタ ロード

    sti                                ; 割り込み許可

    db      0EAh                    ; JMP CS セクタ:prot_L2
    dw      OFFSET prot_L2          ;
    dw      00D8h                    ; CS にセクタを指定してセグメント間 JMP

prot_L2:
    mov     ax, 50h
    mov     word ptr [_LDTR], ax
    lldt     word ptr [_LDTR]        ; LDTR の登録

    str     ax
    cmp     ax, 58h
    jz      prot_L3                  ; TR に既にロードされているのと同じ値
    ; はロード出来ないので検査

    mov     word ptr [_TR], 58h
    ltr     word ptr [_TR]          ; TR のセクタが 58h でなければ 58h
    ; をロード

prot_L3:
    ret

@prot$qv ENDP

;=====;
;      ディスクリプタ
;=====;

;-----;
;      DS      no.2   10h   データ・セグメント
;-----;
gdt2      DW      0FBFFh      ; segment limit  0 ~ 15
          DW      0400h      ; base address   0 ~ 15
          DB      00h        ; base address  16 ~ 23
          DB      10010010b   ; type

```

```

        DB      00000000b      ; type - segment limit 16 ~ 19
        DB      00h           ; base address 24 ~ 31

;-----;
;          SS      no.3   18h   スタック・セグメント
;-----;

gdt3      DW      0C000h      ; segment limit 0 ~ 15
          DW      00400h      ; base address 0 ~ 15
          DB      00h         ; base address 16 ~ 23
          DB      10010110b    ; type
          DB      00000000b    ; type - segment limit 16 ~ 19
          DB      00h         ; base address 24 ~ 31

;-----;
;  TSS 0      no.11   58h      タスク・ステート・セグメント
;-----;

gdt11     DW      00080h      ; segment limit 0 ~ 15
          DW      2A00h       ; base address 0 ~ 15
          DB      02h         ; base address 16 ~ 23
          DB      10001001b    ; type TSS
          DB      00000000b    ; type - segment limit 16 ~ 19
          DB      00h         ; base address 24 ~ 31

;-----;
;          CS      no.27   D8h   コード・セグメント
;-----;

gdt27     DW      0FFFFh      ; segment limit 0 ~ 15
          DW      0000h       ; base address 0 ~ 15
          DB      0Eh         ; base address 16 ~ 23
          DB      10011010b    ; type
          DB      00000000b    ; type - segment limit 16 ~ 19
          DB      00h         ; base address 24 ~ 31

;=====;
;          タスク・ステート・セグメント
;=====;

;-----;
;  一番最初のタスク・スイッチがあった時、原タスクのコンテキスト
;  をここに保存する
;-----;

tss0      DW      0000h       ; back link
          DW      0000h       ;
          DD      00000000h    ; ESP 0
          DW      0000h       ; SS 0
          DW      0000h       ;
          DD      00000000h    ; ESP 1
          DW      0000h       ; SS 1
          DW      0000h       ;
          DD      00000000h    ; ESP 2
          DW      0000h       ; SS 2
          DW      0000h       ;
          DD      00000000h    ; CR3
          DD      00000000h    ; EIP
          DD      00000000h    ; EFLAGS
          DD      00000000h    ; EAX
          DD      00000000h    ; ECX
          DD      00000000h    ; EDX
          DD      00000000h    ; EBX
          DD      00000000h    ; ESP
          DD      00000000h    ; EBP
          DD      00000000h    ; ESI

```

```

DD      00000000h      ; EDI
DW      0000h          ; ES
DW      0000h          ;
DW      0000h          ; CS
DW      0000h          ;
DW      0000h          ; SS
DW      0000h          ;
DW      0000h          ; DS
DW      0000h          ;
DW      0000h          ; FS
DW      0000h          ;
DW      0000h          ; GS
DW      0000h          ;
DW      0000h          ; LDT
DW      0000h          ; T
DW      0000h          ; I/O

DB      24 DUP (?)      ; user area

;-----;
; タスク・スイッチがあった時このコンテキストを CPU にロードする
;-----;

tss1     DW      0000h      ; back link  0
         DW      0000h      ;           2
         DD      00000EFFFh  ; ESP  0      4   特権レベル 0 スタック領域
         DW      00A0h      ; SS   0      8
         DW      0000h      ;           A
         DD      00000AFF0h  ; ESP  1      C   特権レベル 1 スタック領域
         DW      00A8h      ; SS   1     10
         DW      0000h      ;           12
         DD      000009FF0h  ; ESP  2     14   特権レベル 2 スタック領域
         DW      00B0h      ; SS   2     18
         DW      0000h      ;           1A
         DD      00024000h    ; CR3      1C
         DD      00000000h    ; EIP      20
         DD      00003206h    ; EFLAGS   24
         DD      00000000h    ; EAX      28
         DD      00000000h    ; ECX      2C
         DD      00000000h    ; EDX      30
         DD      00000000h    ; EBX      34
         DD      00000FFFh    ; ESP      38   タスク・スイッチ後の ESP
         DD      00000000h    ; EBP      3C
         DD      00000000h    ; ESI      40
         DD      00000000h    ; EDI      44
         DW      007Bh      ; ES       48
         DW      0000h      ;         4A
         DW      0043h      ; CS       4C
         DW      0000h      ;         4E
         DW      0073h      ; SS       50   タスク・スイッチ後の SS
         DW      0000h      ;         52
         DW      007Bh      ; DS       54
         DW      0000h      ;         56
         DW      0000h      ; FS       58
         DW      0000h      ;         5A
         DW      0000h      ; GS       5C
         DW      0000h      ;         5E
         DW      0053h      ; LDT      60
         DW      0000h      ;         62
         DW      0000h      ; T        64
         DW      0000h      ; I/O      66

DB      24 DUP (?)      ; user area

```

リスト6 LST4C.ASM

```

;=====;
;      protect mode -> real mode      ;
;=====;
@real$qv PROC NEAR

    cli                                ; 割り込み禁止
    mov     ax,1024
    mov     word ptr [_IDTR],ax
    mov     eax,0000h
    mov     dword ptr 2[_IDTR],eax

    lidt     [_IDTR]                  ; リアル・モード用 IDTR の登録

    mov     ax,30h                    ; リアル・モード復帰用 セクタ
    mov     ds,ax                    ; DS にセクタ ロード
    mov     es,ax                    ; ES にセクタ ロード
    mov     fs,ax                    ; FS にセクタ ロード
    mov     gs,ax                    ; GS にセクタ ロード
    mov     ss,ax                    ; SS にセクタ ロード

    db      0EAh                      ; JMP CS セクタ:real_L3
    dw      OFFSET real_L3            ;
    dw      00E0h                    ; CS にセクタを指定してセグメント間 JMP

real_L3:

    mov     eax,cr0
    and     eax,07FFFFFFEh
    mov     cr0,eax                  ; リアル・モードに移行

    jmp     real_L1                  ; リアル・モードでパイプラインに先読み
real_L1:                                     ; した命令を捨てる。

    jmp     far ptr real_L2          ; セグメント間 JMP 命令で CS を更新
real_L2:

    mov     ax,DGROUP
    mov     ds,ax                    ; DS にベース・アドレスをロード
    mov     es,ax                    ; ES にベース・アドレスをロード
    mov     fs,ax                    ; FS にベース・アドレスをロード
    mov     gs,ax                    ; GS にベース・アドレスをロード
    mov     ss,ax                    ; SS にベース・アドレスをロード

    sti                                ; 割り込み許可
    ret

@real$qv ENDP

;=====;
;      ディスクリプタ      ;
;=====;

;-----;
;      DS      no.6      30h      リアルモード復帰準備      ;
;-----;

gdt6      DW      0FBFFh      ; segment limit  0 ~ 15
          DW      0400h      ; base address   0 ~ 15
          DB      00h        ; base address  16 ~ 23
          DB      10010010b   ; type
          DB      00000000b   ; type - segment limit 16 ~ 19
          DB      00h        ; base address  24 ~ 31

```

第Ⅱ部

ハードウェア

第 4 章

80486 のハードウェア

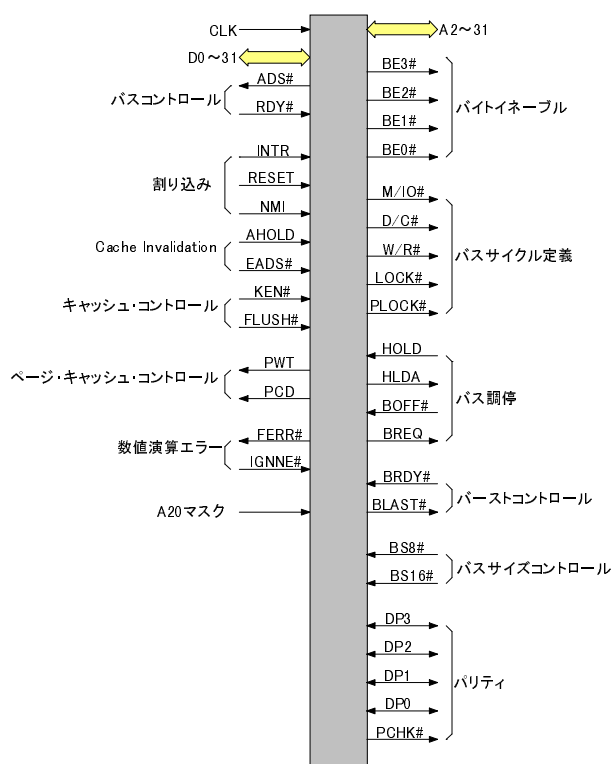
4.1 80486 の概要

80486 は、80386 の後継のプロセッサです、80386 と比べて改善された点は次のようなものです。

- 倍クロックが不用
80386 では 33MHz のプロセッサでは 66MHz のクロックを供給していましたが、80486DX ではプロセッサの表示と同クロックでよくなり、80486DX2 では半分、80486DX4 では $\frac{1}{3}$ になりました。
- ダイナミック・バス・サイジングのサポート
80386 でも 16 ビット・バスをサポートしていましたが、80486 から 8 ビット・バスもサポートしています。
- 浮動小数点コプロセッサの内蔵
80386 には 80387 を外付けしましたが、80486 では内蔵になり基板上のチップ占有面積と配線を大幅に減少出来るようになりました。
- キャッシュの内蔵とバースト・サイクル
キャッシュがプロセッサに内蔵されるようになりました、これによりソフトウェアが高速に実行されますが、外部メモリからキャッシュに効率よくデータを転送するためかデータ転送の形態にバースト・サイクルが導入されました。

個人的に設計・製作するコンピュータ・システムでは 80486 はその後の Pentium よりも使いやすい面も持っています、Pentium にはダイナミック・バスサイジングはありませんからユーザ回路は大きくなります、また DX や DX2 では電源が 5V ですし、パッケージが 2.54mm の PGA ですから扱いやすい 32 ビット・プロセッサです。

図 4.1 CPU の機能グループ



4.1.1 クロック

プロセッサの入力・出力信号はこのクロックの立ち上がりエッジを基準にして指定されます。

クロックの仕様は TTL レベルとして構いません。

4.1.2 リセット

電源投入時は Vcc と CLK の安定後、少なくとも 1ms、ウォームスタート時は 15CLK の時間の間リセットをアサートします。

表 4.1 バイト・イネーブルとそれに対応するデータ端子

バイト・イネーブル	対応するデータ・バス
BE0#	D7~0
BE1#	D15~8
BE2#	D23~16
BE3#	D31~24

表 4.2 パリティ端子とそれに対応するデータ端子

パリティ	対応するデータ・バス
DP0	D7~0
DP1	D15~8
DP2	D23~16
DP3	D31~24

4.1.3 アドレス・バス

プロセッサは A31~2 と BE3~0# によりアドレスを指定します、メモリ空間で 4G バイト、I/O 空間で 64k バイトを持ちます。

BE3~0# は 32 ビット・バス中のデータ・バイトを指定します。

アドレス・バスは双方向になっていて、外部からプロセッサにアドレスを渡す場合もあります。

プロセッサ内部のキャッシュ・ラインを外部からアドレスを指定して無効化させる場合などです。

4.1.4 データ・バス

D31~0 の 32 ビット・データ・バスとダイナミック・バス・サイジング機能を使って、8/16/32 ビット・バスをサポートします。

バイトイネーブル

- 32 ビットバス

表 4.1 を参照して下さい。

- 16 ビットバス

A1 と BHE# と BLE# を表 4.4、表 4.6 から作ります、BLE# は A0 と同じです。

データ端子との関係は表 4.5 の「BS16 # ありのとき」を見て下さい。

- 8 ビットバス

A1, A0 を表 4.4 から作ります。

データ端子との関係は表 4.5 の「BS8 # ありのとき」を見て下さい。

パリティ・チェック

データをバイト単位でパリティ・チェックする機能を持っています、パリティ・データを DP3~0 から入出力しますので、これをパリティ用メモリに接続します。

パリティ端子とデータとの関係は表 4.2 のようになっています。

表 4.3 ADS # で開始されるバスサイクルの定義

M/IO#	D/C#	W/R#	開始されるバス・サイクル
0	0	0	割り込みアクノレッジ
0	0	1	ホールド/特殊サイクル
0	1	0	I/O 読み出し
0	1	1	I/O 書き込み
1	0	0	コード読み出し
1	0	1	—
1	1	0	メモリ読み出し
1	1	1	メモリ書き込み

表 4.4 BE3 # ~ BE0 # から A1, A0 を生成

BE3#	BE2#	BE1#	BE0#	A1	A0
—	—	—	0	0	0
—	—	0	1	0	1
—	0	1	1	1	0
0	1	1	1	1	1

プロセッサはライト時にパリティ・ジェネレータでパリティ・データ*1を出力して、リード時に、そのパリティ・データとデータ・バス上のデータをチェックして、その結果を PCHK# に出力しています。

パリティを使わない場合は DP3~0 をプルアップします、またパリティ・チェックに対応するプロセッサの機能は PCHK# 以外にはありません。

4.1.5 バスサイクルの定義

プロセッサにより ADS# がアサートされて開始されるサイクルは次のようにプロセッサとユーザ回路により定義されます。

表 4.5 各種バスサイズで読み出されるデータ端子

BE3#	BE2#	BE1#	BE0#	BS8#/BS16#なしのとき	BS8#ありのとき	BS16#ありのとき
1	1	1	0	D7~0	D7~0	D7~0
1	1	0	0	D15~0	D7~0	D15~0
1	0	0	0	D23~0	D7~0	D15~0
0	0	0	0	D31~0	D7~0	D15~0
1	1	0	1	D15~8	D15~8	D15~8
1	0	0	1	D23~8	D15~8	D15~8
0	0	0	1	D31~8	D15~8	D15~8
1	0	1	1	D23~16	D23~16	D23~16
0	0	1	1	D31~24	D23~16	D31~16
0	1	1	1	D31~24	D31~24	D31~24

表 4.6 16 ビット・バスをアドレスするための BHE #の生成

BE3#	BE2#	BE1#	BE0#	BHE#
1	1	1	1	—
1	1	1	0	1
1	1	0	1	0
1	1	0	0	0
1	0	1	1	1
1	0	1	0	—
1	0	0	1	0
1	0	0	0	0
0	1	1	1	0
0	1	1	0	—
0	1	0	1	—
0	1	0	0	—
0	0	1	1	0
0	0	1	0	—
0	0	0	1	0
0	0	0	0	0

プロセッサ側

M/IO#, D/C#, W/R#で表 4.3 の様に定義します。

ユーザ側

BS16#, BS8#でバス・サイズを KEN#でキャッシュ可能かどうかを定義します、RDY#, BRDY#で単一の転送かバースト転送かを決定します。

4.1.6 キャッシュ

80486 では命令/データ用に 8k バイトのキャッシュが内蔵されました。

キャッシュ制御

開始されたバスサイクルのアドレス領域を、ユーザがキャッシュすることを望むのであれば KEN#をアクティブにします。

キャッシュ可能性

メモリ・リードと命令プリフェッチだけがキャッシュ出来ます、ソフトウェア上ではコントロール・レジスタ 0 の CD, NW ビットにより制御します、ハードウェア上では KEN#がアサートされるメモリ領域に限定されます、実際には上記のバス・サイクルでソフトウェアとハードウェアの条件が揃ったときに、キャッシュ動作が可能になります。

キャッシュ・ライン・フィル

プロセッサがキャッシュ可能であれば、キャッシュ・ライン・フィル・サイクルが実行されます。

キャッシュ・ラインは 16 バイト長あります、キャッシュ・ライン・フィル・サイクル中の最初と最後のデータ転送で KEN#がアサートであったならプロセッサはライン・フィル・サイクルは成功したとみなしますし、どちらかの KEN#がネゲートなら失敗とみなし

*1 偶数パリティ

データは使われません。

キャッシュ・ライン・フィル・サイクルは8ビットバスや16ビットバスでも可能です。

キャッシュのフラッシュ

ユーザがハードウェアでキャッシュをフラッシュさせるときは、FLUSH#を1クロックだけアサートします。

キャッシュ・ライン・フィル：

キャッシュ可能なメモリにプロセッサがアクセスしたときに、そのアドレスがキャッシュ中に無かった場合には、キャッシュ・ライン・フィルのバスサイクルを開始してキャッシュの1ライン分のデータを読み出してプロセッサのキャッシュ・ラインに格納します。

フラッシュ：

ここではキャッシュ・ラインのデータを無効にすることを言います。

4.1.7 ハードウェア割り込み

下記の2本の割り込み入力を持ちます。

1. INTR マスカブル割り込み要求入力

プロセッサはこの入力に応答して割り込みアクノレッジ・バス・サイクル(図4.2)を生成します、このサイクルは割り込みコントローラ8259AのINTAに使えます。

INTRは割り込みアクノレッジ・バス・サイクルが開始されるまでアサートに保持しなければなりません。

2. NMI ノンマスカブル割り込み要求入力

この割り込みのベクタ番号は2番と決まっているので割り込みアクノレッジ・バス・サイクルは生成されません、NMIはエッジトリガで前後の4クロックは安定させます。

4.1.8 バス・アービトレーション

ユーザがプロセッサにバスマスタから降りてほしいときにHOLDをアサートします、プロセッサは大部分の入出力端子をフロートにしてHLDAをアサートします。

フロートする端子

BE3#~BE0# ,PWT,PCD ,W/R# ,D/C# ,M/IO#
,LOCK# ,PLOCK# ,ADS# ,BLAST# ,PCHK#
,D31~D0 ,DP3~DP0 ,A31~A4

この状態はユーザがHOLDをネゲートにするまで続きます。

もうひとつの方法にBOFF#を使う場合があります、HOLDの場合のときプロセッサは現在のバスサイクルを完了させてからバスを解放しますが、BOFF#をアサートしたときは現在のバスサイクルを中断して、HOLDの場合と同じ入出力端子を次のクロックで解放します、HLDAは出しません、これはRDY#よりもBRDY#よりも最優先で受け付けられます、BOFF#の割り込んだバスサイクルは、BOFF#がネゲートになった後にADS#を出してから再実行されます、BOFF#の割り込んだサイクルのデータは使われません。

4.1.9 A20M#

8086との互換性をとるためにあります、8086ではアドレスはA19~0で20ビットありますので、それ以上にアドレスがすすむとFFFFFhから00000hになります(アドレス・ラップ・アラウンドと呼ばれます)、80486はアドレスは32ビットあるので、FFFFFhから100000hになります、これを8086と同じ動作にするためにA20M#をアサートします。

4.1.10 ページ・キャッシュビリティ

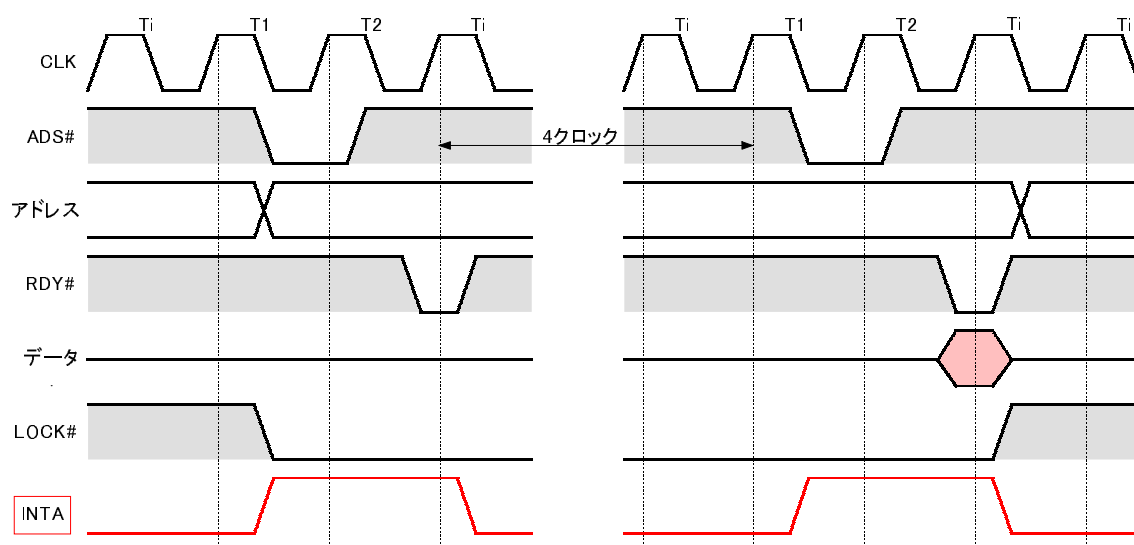
PWT,PCDはコントロールレジスタ3かPDE(ページ・ディレクトリ・エントリ)かPTE(ページ・テーブル・エントリ)の同名のビットを端子に出力したもので、ページ・レベルのキャッシング設定を外部キャッシュに通知します。

4.1.11 数値演算エラー

FERR#は浮動小数点エラーが起きたときにアサートされます。

IGNNE#は浮動小数点エラーが発生した時にプロセッサが動作を継続するか、停止するかを決めます、アサートにした時は浮動小数点エラーを無視して動作を継続します。

図 4.2 割り込みアクノレッジ・サイクル



4.2 バス動作

4.2.1 プロセッサ

プロセッサは命令コードの読み出しやデータのリード/ライト、やキャッシュ・ラインフィルなどの内部的な要求か、又は割り込みなどの外部的要求によりバス・サイクルを開始します。

1回で最大32ビットのデータ転送が可能ですが、16ビット・バスで2回、8ビット・バスで4回のデータ転送にすることが出来ます。

それぞれのデータ転送に対してはRDY#、BRDY#がアサートされることで終了したと見なします。

一連のデータ転送はプロセッサがBLAST#をアサートしたときの、ユーザ回路がRDY#、BRDY#をアサートするデータ転送を最後に終了します。

プロセッサはRDY#がアサートで返された場合は、次の残されたデータ転送を複数サイクル転送で開始します、BRDY#がアサートで返された場合はバースト転送を開始します。

4.2.2 ユーザ回路

ユーザ回路はプロセッサが開始したサイクルに対応して次の情報を返します。

- バス・サイズ
バス・サイズはBS8#、BS16#によって定義します。

表 4.7 バス・サイズ

BS8#	BS16#	サイズ
0	0	16
0	1	8
1	0	16
1	1	32

これらの信号はクロックごとにサンプルされていますがRDY#、BRDY#がアサートされたひとつ前のクロックの値が採用されます。

BLAST#はバス・サイズの決定に左右されますから、BS8#、BS16#を確定した次のクロックからの値が有効です。

- キャッシュ可能・不可能

キャッシュはKEN#によって定義します。

KEN#のサンプルはバス・サイズ定義信号と同じですが、少なくとも最初と最後のデータ転送でアサートされないとキャッシュ・ラインフィルで読み取られた16バイトのデータは有効とされません。

また、I/Oサイクルや書き込みサイクルではKEN#は無視されます。

- データ転送の終了

プロセッサのデータ転送要求はバス・サイズの

都合や 32 ビット以上のデータ転送^{*2}が必要なことから、1 個または複数のデータ転送から構成されています、個々のデータ転送を終了させるのは RDY#または BRDY#です、これらをアサートする時期を制御することでデバイスのアクセス・タイムを確保します。

バースト転送

このサイクルはデータ転送を BRDY#をアサートして終了させた場合に、次のデータ転送をバースト転送で開始させます、次のデータ転送がない場合にはサイクルは終了します。

BRDY#で終了した、次のデータ転送ではプロセッサは ADS#をアサートしませんので、ユーザ回路は BRDY#で、この期間中のデータ転送の開始、終了を制御します。

4.3 バス・サイクル・タイミング

4.3.1 基本的 2-2 バス・サイクル

80486 の最短のバス・サイクルを図 4.3 に示します、最大 32 ビットのデータを 2 クロックで転送しています。

プロセッサがサイクルのステータスを確定してから、ADS#をアサートしてサイクルの開始を外部に通知します、ユーザ回路は RDY#をアサートしてサイクルを終了します。

4.3.2 基本的 3-3 バス・サイクル

RDY#のアサートを 1 クロック遅らせて、サイクルを 3 クロックで終了させています、図 4.4 を参照して下さい。

RDY#はデバイスのアクセス・タイムを確保するなどのために必要なだけ遅らせてアサートすることが出来ます。

4.3.3 非キャッシュャブル・非バースト、複数サイクル転送

図 4.5 を参照して下さい、最初のサイクルの RDY#のアサートのタイミングで BLAST#がネゲートされているので、次のデータ転送が続くことが分かります。

4.3.4 非キャッシュャブル・バースト・サイクル

図 4.6 を参照して下さい、最初のサイクルの BRDY#のアサートのタイミングで BLAST#がネゲートされているので、次のデータ転送が続くことが分かります。

バースト転送の場合は ADS#を使わないので、1 クロックだけ短くて済みます、キャッシュ・ラインフィルは 16 バイトを転送しますから、32 ビットのバス幅なら 4 クロック短くなります。

4.3.5 非バースト・キャッシュャブル・サイクル

複数サイクル転送による、キャッシュ・ラインフィルです、図 4.7 を参照して下さい。

最初と最後のサイクルで KEN#をアサートしていますが、これは最低限度の条件なので、ラインフィル中はずっとアサートしていても差し支えありません。

4.3.6 キャッシュャブル・バースト・サイクル

バースト転送による、キャッシュ・ラインフィルです、図 4.8 を参照して下さい。

4.3.7 KEN #の変化による効果

図 4.9 の KEN#、BLAST#に注目して下さい。

最初のサイクルの KEN#でキャッシュ・ラインフィルの決定をしますが、最初の KEN#のアサートで次クロックの BLAST#がネゲートされています、しかし KEN#がネゲートされたので次クロックの BLAST#がアサートされています。

このような 2 転、3 転があっても

次クロックで RDY#がアサートされたので 1 クロック前の KEN#の状態が採用されて、キャッシュ・ラインフィルが確定します。

KEN#や BS8#、BS16#の変化によって BLAST#の値は変化します。

プロセッサは RDY#か BRDY#をアサートした 1 クロック前の KEN#や BS8#、BS16#の値を採用します。

4.3.8 バースト中断サイクル

図 4.10 を参照して下さい、RDY#と BRDY#によってバースト転送と複数サイクル転送は混用することが可能です。

4.3.9 低速なバースト・サイクル

バースト転送でも、BRDY#のアサート時期を制御することで任意のウェイトを確保出来ます。

4.3.10 ロック・バス・サイクル

プロセッサは複数のサイクルの間でユーザにバスを解放しない特別のバス・サイクルを持ちます。

^{*2} キャッシュ・ラインフィルなど

図 4.3 基本的 2-2 バス・サイクル

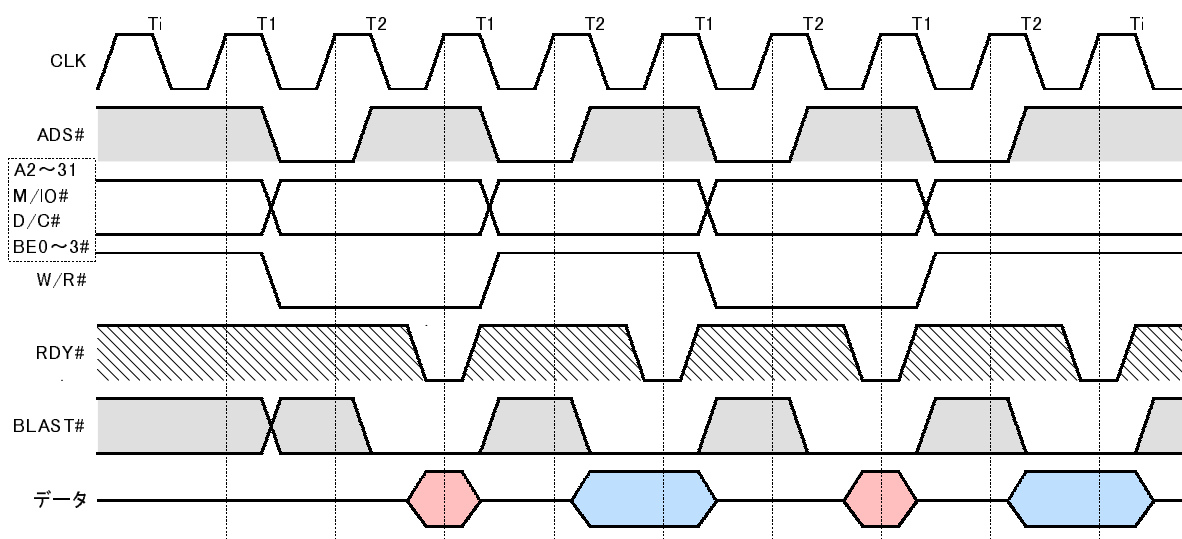
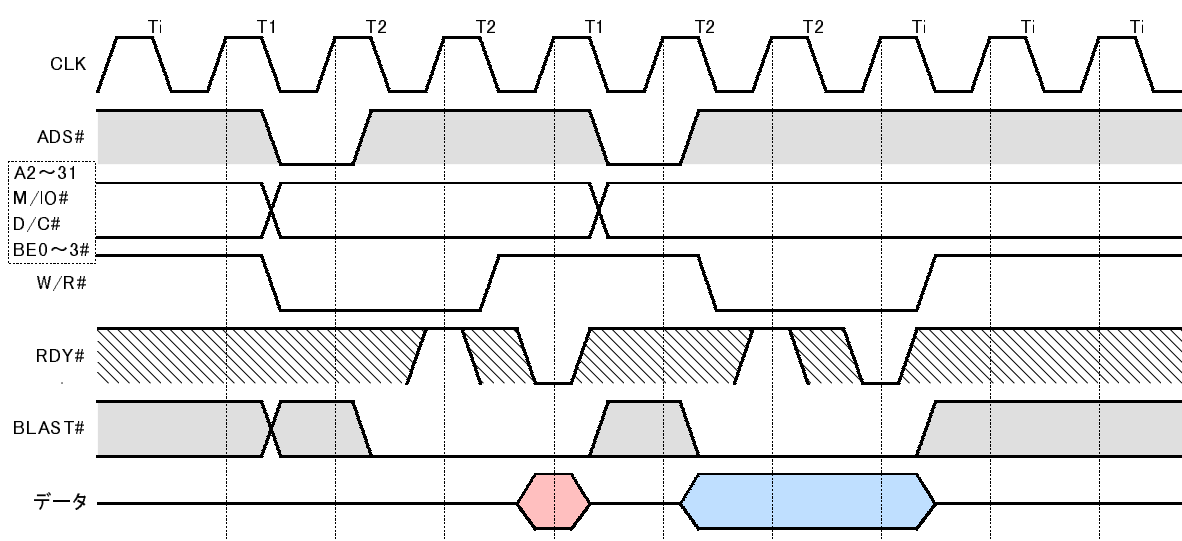


図 4.4 基本的 3-3 バス・サイクル



リード・モディファイ・ライトなどのソフトウェア上の必要と割り込みアクノレッジ・サイクルにより生成されます、図 4.12 参照して下さい。

ユーザは LOCK#がアサートになる事により、このサイクルに入った事がわかります。ロック・サイクル中は AHOLD と BOFF#は認識しますが、HOLD は認識しません。

4.3.11 疑似ロック・バス・サイクル

1 つ以上のバス・サイクルを必要とするオペランド転送などに対して生成されます。

ユーザは PLOCK#がアサートになることにより、こ

のサイクルに入った事がわかります、図 4.13 参照して下さい。

疑似ロック・サイクル中は AHOLD#と BOFF#は認識しますが、HOLD は認識しません。

4.3.12 内部キャッシュ無効化サイクル

ユーザはアドレスを指定してプロセッサの持つキャッシュラインを無効化することができます、図 4.14・図 4.15 を参照して下さい。

80486 のアドレスバスは双方向になっています。

1. AHOLD をアサートすると次のクロックでプロ

図 4.5 非キャッシュブル・非バースト、複数サイクル転送

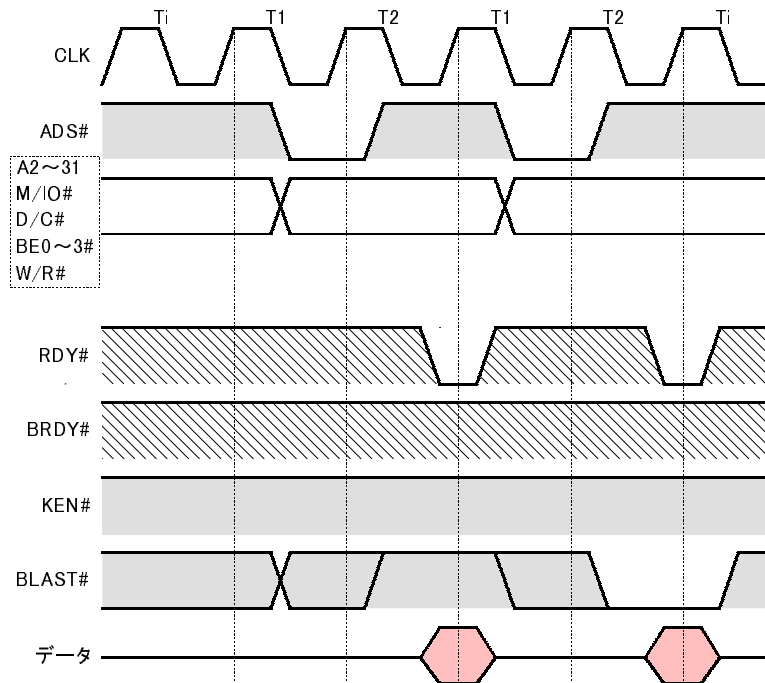


図 4.6 非キャッシュブル・バースト・サイクル

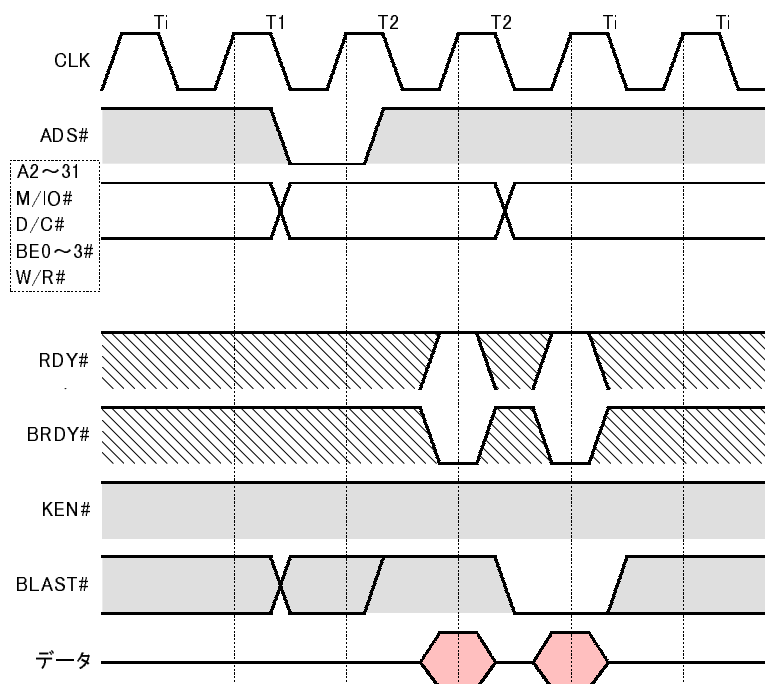


図 4.7 非バースト・キャッシュャブル・サイクル

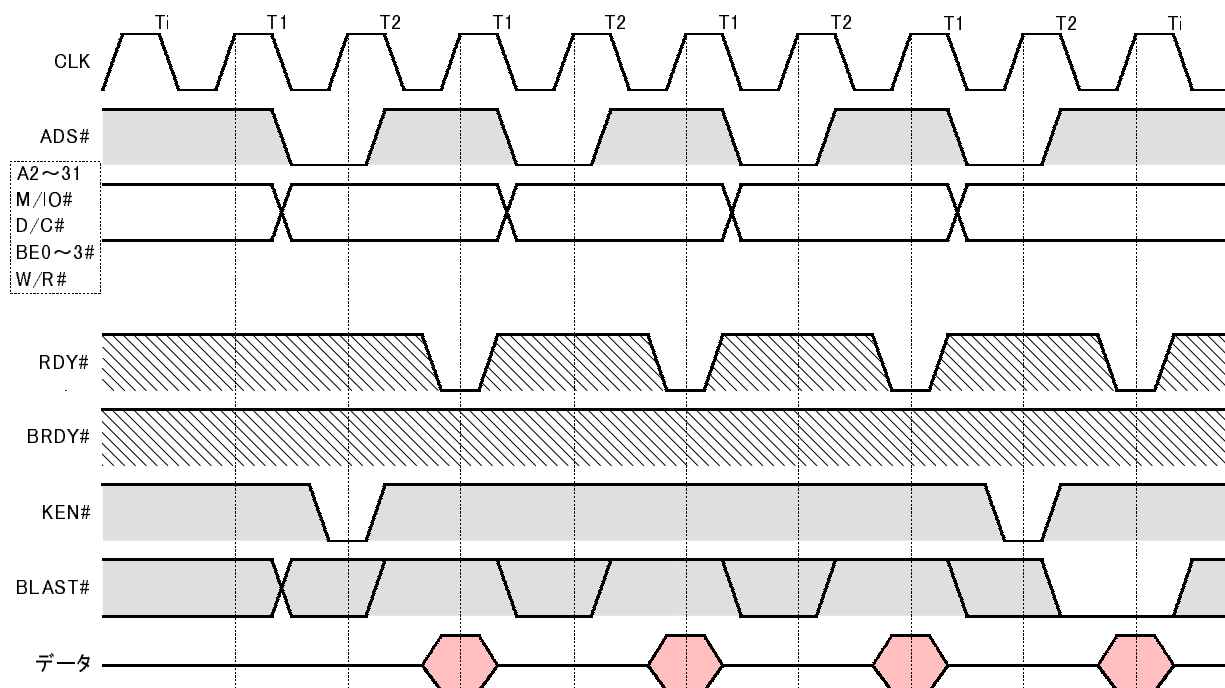


図 4.8 キャッシュャブル・バースト・サイクル

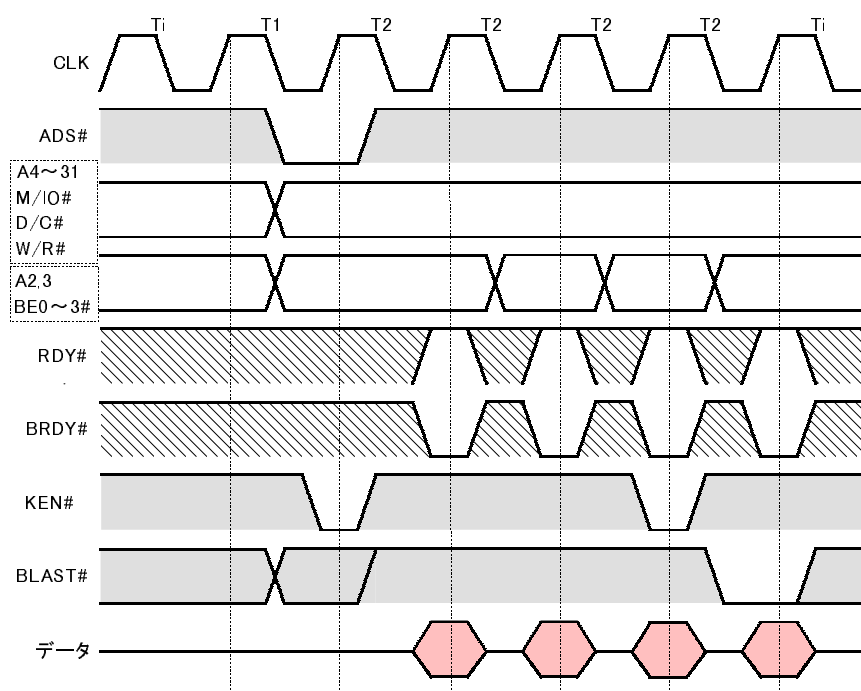


図 4.9 KEN # の変化による効果

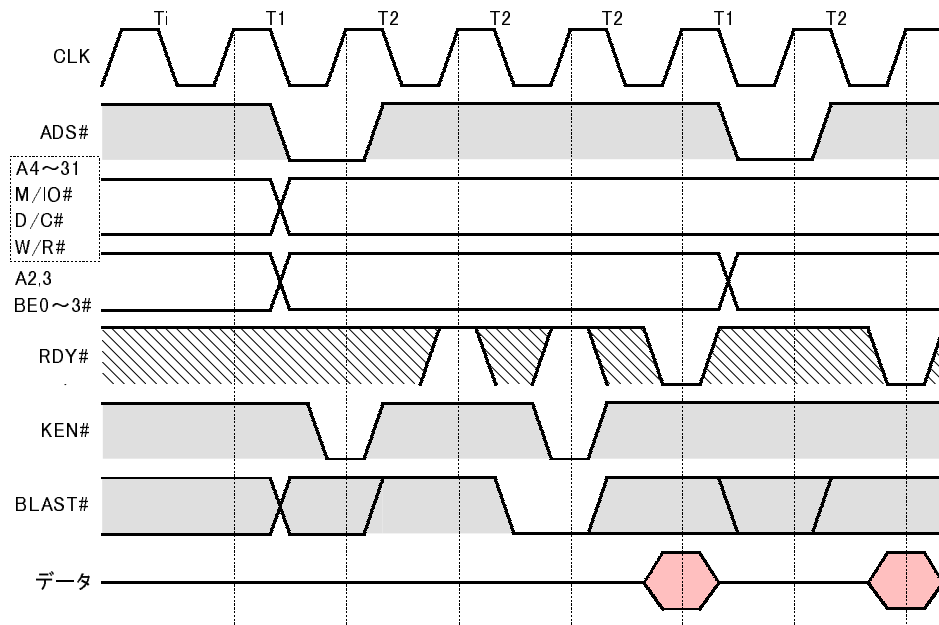


図 4.10 バースト中断サイクル

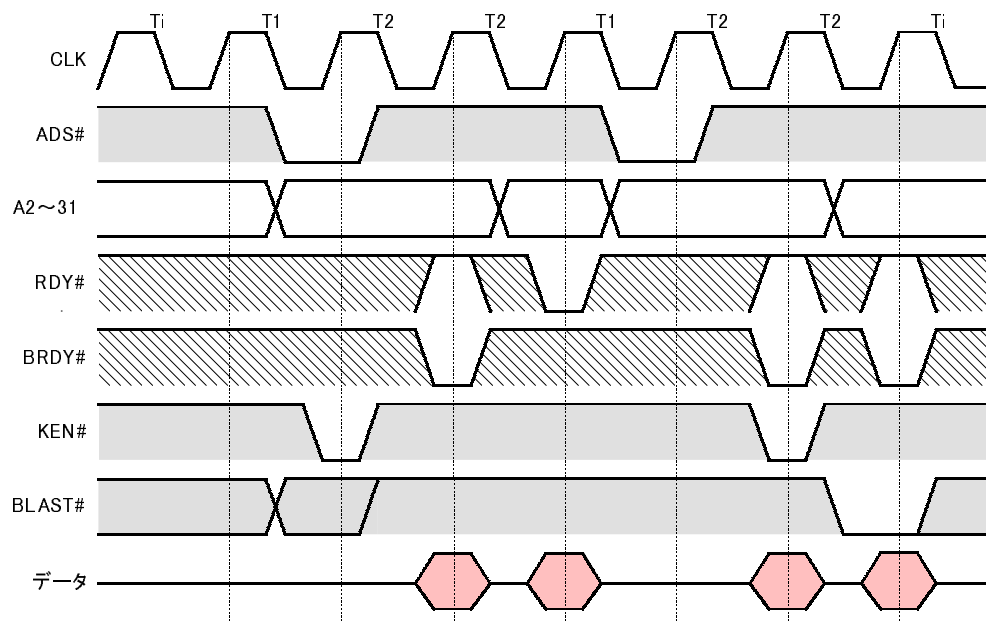
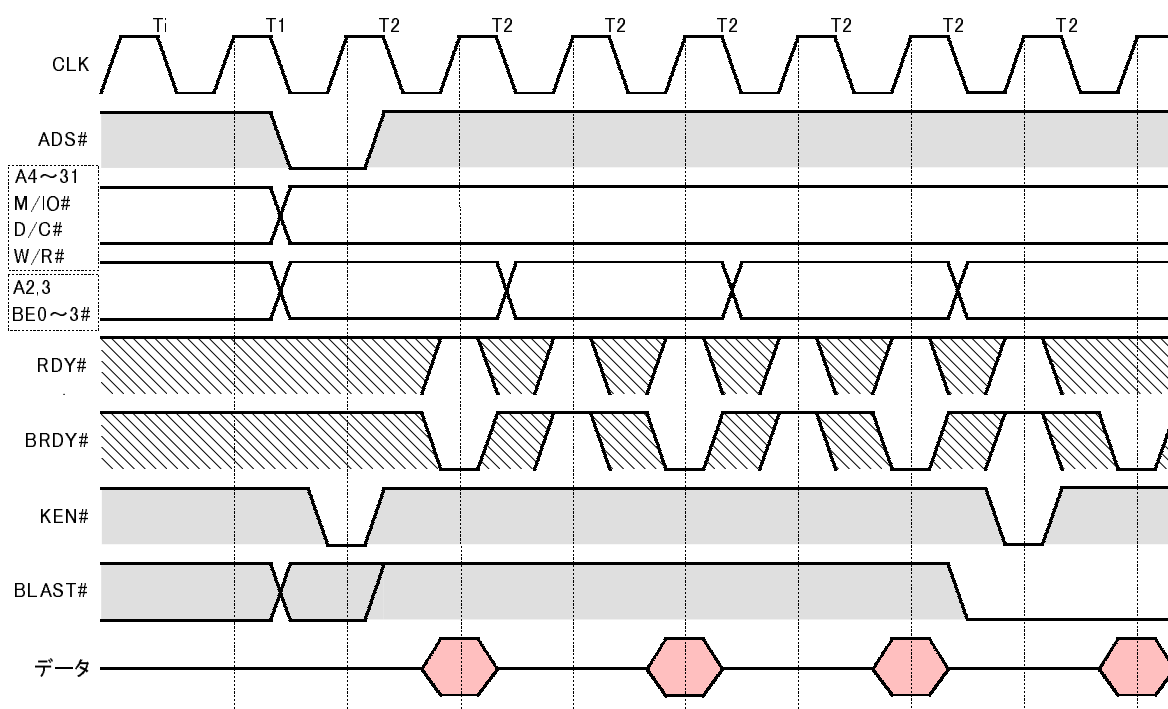


図 4.11 低速なバースト・サイクル



セッサは A4~31 だけをフロートします。

- ユーザはここにプロセッサのキャッシュ中の無効化させたいアドレスをドライブして、EADS# を 1 クロック、アサートします。
- プロセッサはここで無効化アドレスを受け取ります、プロセッサは 1 クロックごとに EADS# をサンプルし無効化することが出来ます、他に無効化したいアドレスがなければ AHOLD# をネゲートします、これでプロセッサは通常動作に戻ります。

プロセッサはいつでも AHOLD# を受け付けます。
バスサイクルの進行中でも受け付けますが、そのときにはアドレスがフロートしますのでメモリアクセスなどでは困った事になります。
このためにアドレスラッチを付けます、これで通常のバスサイクルも無効化サイクルも並行して行うことが出来ます。

表 4.8 特殊バス・サイクルのエンコード

BE3	BE2	BE1	BE0	内容
1	1	1	0	シャット・ダウン
1	1	0	1	フラッシュ
1	0	1	1	ホールド
0	1	1	1	ライト・バック

4.4 特殊バス・サイクル

80486 は 4 つの特殊サイクルを持ちます。表 4.3 のホールド／特殊サイクルのバス・サイクルの時に、BE3#~BE0# に表 4.8 のように定義されます。

図 4.12 ロック・バス・サイクル

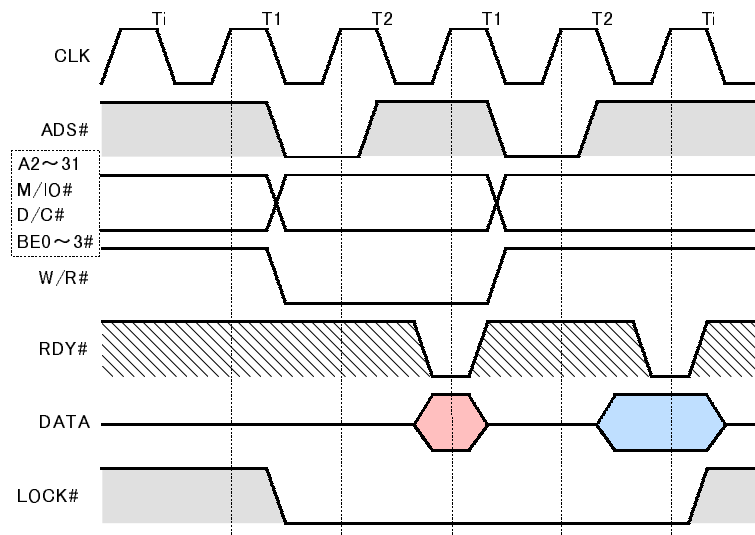


図 4.13 疑似ロック・バス・サイクル

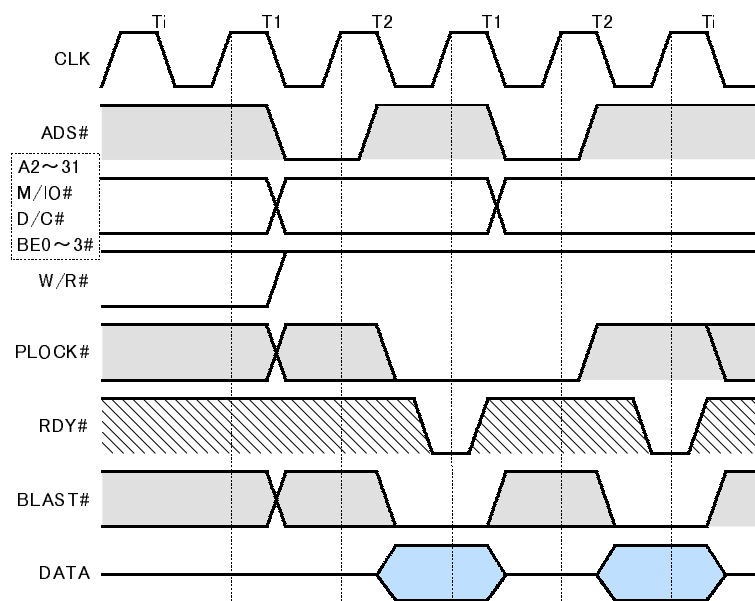


図 4.14 高速な内部キャッシュ無効化サイクル

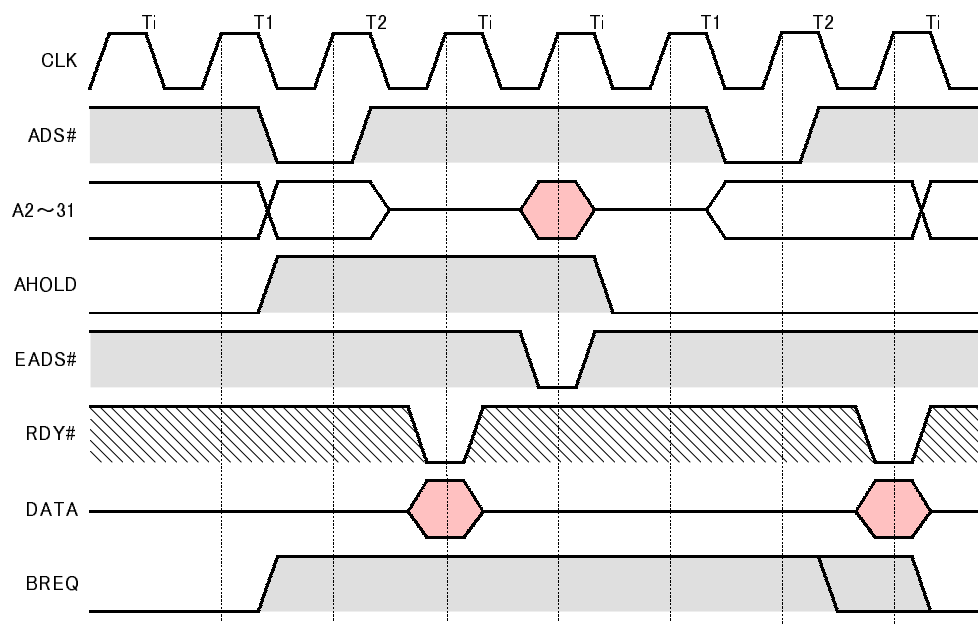
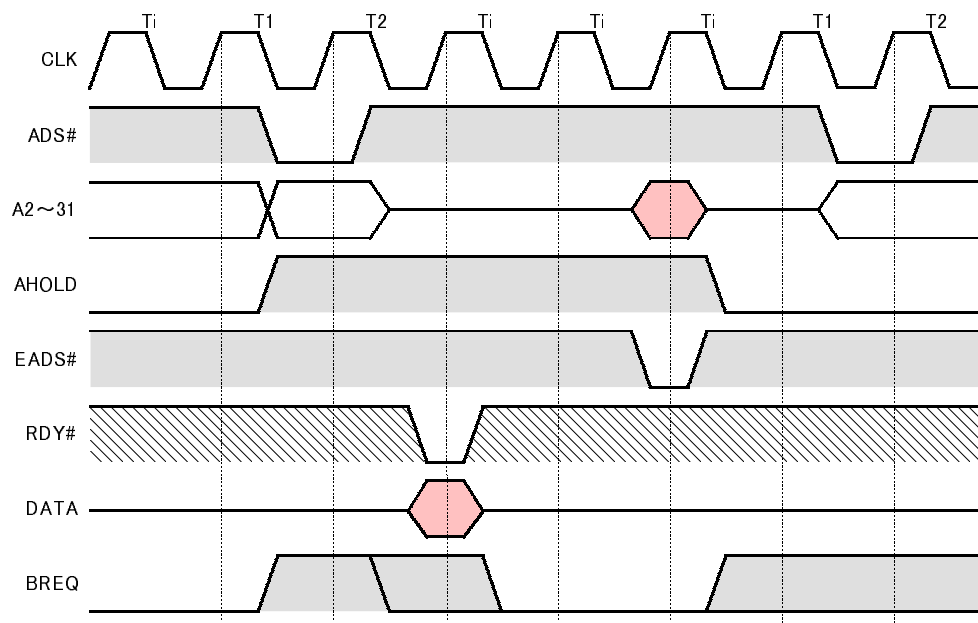


図 4.15 一般的な内部キャッシュ無効化サイクル



第 5 章

80486 のインターフェース設計

80486 プロセッサとメモリや I/O とのインターフェースについて考えます。

5.1 80486 のハードウェア

80486 プロセッサはダイナミック・バス・サイジングで 32 ビット以外に 8 ビット、16 ビットのバスをサポートしていますので、プロセッサの内部のデータ転送要求に対する外部のデータ転送サイクルの個数がバス・サイズにより動的に可変しますから、バス仕様は Pentium などと比較すると複雑です、しかしバースト転送時のアドレスはプロセッサから供給されるのでインターフェース回路の規模は 80486 の方が小さくて済みそうです。

インターフェースに関連した部分について概説します。

5.1.1 バス・サイクルの開始と終了

開始

プロセッサは ADS# を 1 クロック期間アサートすることでバス・サイクルの開始を知らせます。

バースト・サイクルの 2 番目以降のサイクルはユーザ回路の BRDY# のアサートにより開始されます。

終了

ユーザ回路が RDY# または BRDY# をアサートすることで一つのデータ転送を終了させます。

プロセッサの内部のデータ転送要求を満たす外部のデータ転送の最後のサイクルは BLAST# がアサートされます、これによりユーザ回路はプロセッサの一連のデータ転送の最後のサイクルを検知出来ます。

サイクル中の動作

■ユーザ回路 は、サイクルの終了時にはプロセッサのデータ読み出しに対してデータを確定させておかなければならないし、書き込みに対してデータを確実に

受け取れる様になっていなければなりません。

■プロセッサ はデータの読み出しにおいては RDY# または BRDY# のアサートをサンプリングしたクロックでデータをプロセッサ内部に取り込みます。

また BS8# と BS16#、KEN# をサンプリングして、バス・サイズとキャッシュの可・不可を検知後にプロセッサの外部に対するデータ転送サイクル数を決定しています。

5.1.2 データ転送の形態

1. 単一転送

ADS# のアサートで開始されます、それから 1 クロック後に BLAST# がアサートされていれば単一転送サイクルです。

しかし、プロセッサが 32 ビット・データを転送するときにユーザ回路がバス・サイズを 8 ビットや 16 ビットで受けたときには、プロセッサは追加のバス・サイクルを起動します。

2. 複数サイクル転送

ADS# のアサートで開始されます、それから 1 クロック後に BLAST# がネゲートされていれば (転送すべきデータがまだ残っている) バースト転送要求があることを示します。

このときユーザ回路が RDY# をアサートしてバースト転送を拒否すれば、次のプロセッサの転送は ADS# をアサートする単一転送と同様のものになりますが、プロセッサはデータ転送を完了する為に必要な個数のサイクルを発行することになります。

3. バースト転送

ADS# のアサートで開始されます、それから 1 クロック後に BLAST# がネゲートされていればバースト転送要求があることを示します。

このときユーザ回路が BRDY# をアサートしてバースト転送を許可すれば、次のプロセッサの転送は ADS# をアサートしないバースト転送になります。

5.1.3 RDY、BRDY の使い分け

● RDY

単一転送や複数サイクル転送の一つのデータ転送を終了させる為にアサートします。

またバースト転送を中断させる場合にもアサート出来ます、その場合は次のデータ転送は ADS# をアサートして複数サイクル転送になります。

● BRDY

バースト転送の一つのデータ転送を終了させる為と次のデータ転送を要求する為にアサートします。

プロセッサは次のデータ転送の為に ADS# をアサートしませんがアドレスは更新されます。

同じクロックで RDY# と BRDY# がアサート返された場合は BRDY# は無視されますし、またバースト出来ないサイクルで BRDY# がアサートされた場合は RDY# と同じ効果を持ちます。

5.1.4 ユーザ回路の機能

プロセッサのバス・サイクルの開始に対して、ユーザ回路が対応すべき要件は次の様なものです。

1. バス・サイズ

BS8# で 8 ビット・バス、BS16# で 16 ビット・バスを指定します、両方がネゲートされていれば 32 ビット・バスを指定したことになります。

2. キャッシュ

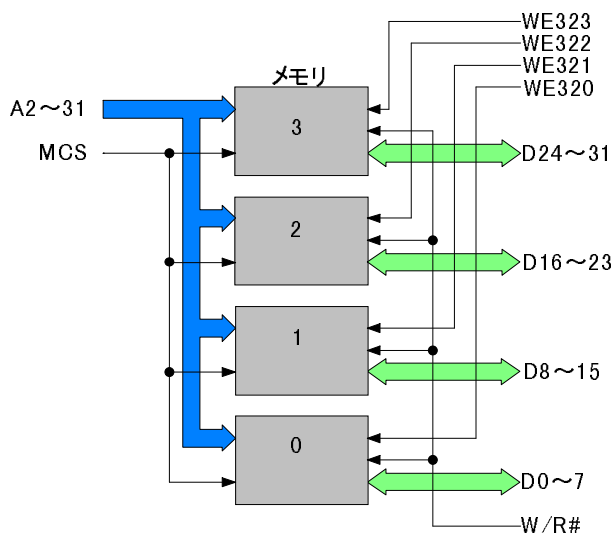
KEN# をアサートしてキャッシュが可能であることを示します。

3. バースト

データ転送を BRDY# をアサートして終了することで、次の転送をバーストで転送出来ることを示します。

これらの要件は遅くとも RDY# または BRDY# をアサートする 1 クロック前には確定しておきます。

図 5.1 32 ビット・バス



5.1.5 バス・サイクルの種類

66 頁の表 4.3 を参照して下さい、これでメモリや I/O のリード・ライト信号や 8259A の INTA 信号を作ります。

5.1.6 キャッシュ

ユーザのメモリ・マップでキャッシュ可能な領域で KEN# をアサートすればプロセッサにキャッシュ可能なアドレスと認識されキャッシュ・ラインフィルなどが実行されます。

但し I/O アクセスはキャッシュされることはありません。

5.1.7 ダイナミック・バス・サイジング

80486 プロセッサはダイナミック・バス・サイジングで 32, 16, 8 ビットのバス・サイズをサポートします。

しかし 16, 8 ビット・バスのデータ・ポートの位置は固定ではないので、プロセッサの指定するバイト位置にデータを置きます。

● 32 ビット・バス

A31~2 と BE3#~0# でデバイスをアクセスします。66 頁の表 4.1 と図 5.1 を参照して下さい。

● 16 ビット・バス

A31~2 と BE3#~0# から作成した A1 でデバイスをアクセスします。66 頁の表 4.4 を参照して下さい。

図 5.2 16 ビット・バス

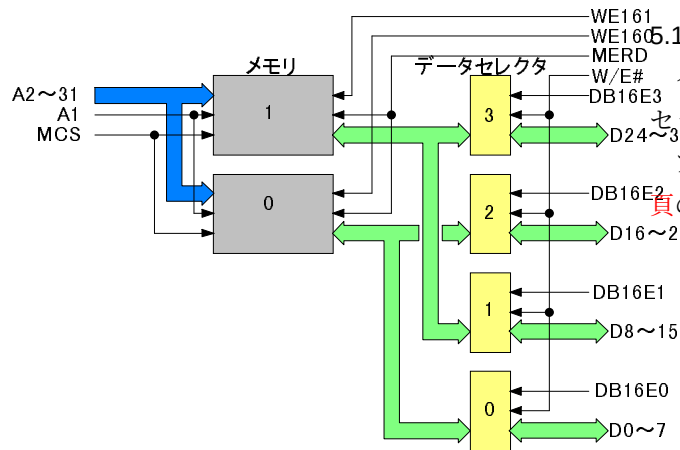
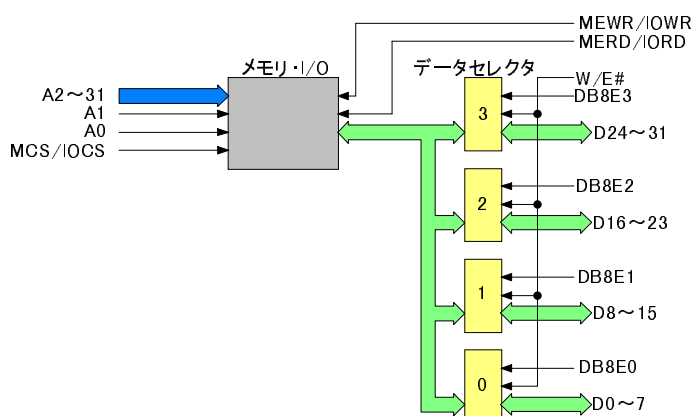


図 5.3 8 ビット・バス



プロセッサのデータ・ポートと 16 ビット・バスのメモリ (I/O) との間にある、バイト・スワップ回路は 67 頁の表 4.5 の BS16#の項を見ればデータ・ポートの D31~16、D15~0 にデータを配置出来ればよいことがわかります。図 5.2 を参照して下さい。

● 8 ビット・バス

A31~2 と BE3#~0#から作成した A0 と A1 でデバイスをアクセスします。66 頁の表 4.4 を参照して下さい。

プロセッサのデータ・ポートと 8 ビット・バスのメモリ (I/O) との間にある、バイト・スワップ回路は 67 頁の表 4.5 の BS8#の項を見ればデータ・ポートの D31~24、D23~16、D15~

8、D7~0 にデータをマップ出来ればよいことがわかります。図 5.3 を参照して下さい。

5.1.8 データ転送と選択されるデータ・ポート

インターフェース回路を設計する前に 80486 プロセッサのバス操作をイメージしてみたいと思います。プロセッサの動作は推定によるものですが、67 頁の表 4.5 を参考にしました。

- 32 ビットデータを 32 ビット・バスのメモリの 0 番地にリード・アクセス

1. プロセッサ

A31~2 に 0、BE3#~0#を LLLL にセットして、ADS#を 1 クロックアサート。

2. ユーザ

BS16#、BS8#、KEN#をネゲートしてデータを D31~0 にセット、ADS#から 5 クロック^a後に RDY#をアサート。

^a メモリのアクセス・タイム

3. プロセッサ

バス・サイクル中の領域が 32 ビット・バスでキャッシュ出来ないことを検知して、D31~0 からデータをプロセッサ内部に取り込んでサイクルを終了。

- 32 ビットデータを 16 ビット・バスのメモリの 0 番地にリード・アクセス

1. プロセッサ

A31~2 に 0、BE3#~0#に LLLL をセットして、ADS#を 1 クロックアサートしてサイクル開始。

2. ユーザ

BS16#をアサート、BS8#、KEN#をネゲートしてデータを D15~0 にセット、ADS#から 5 クロック後に RDY#をアサート。

3. プロセッサ

バス・サイクル中の領域が 16 ビット・バスでキャッシュ出来ないことを検知して、**D15～0** からデータをプロセッサ内部に取り込んで最初のサイクルを終了。

4. プロセッサ

A31～2 に 0、BE3#～0#を LLHH にセットして、ADS #を 1 クロックアサートして 2 番目のサイクルを開始。

5. ユーザ

BS16#をアサート、BS8#、KEN#をネゲートしてデータを **D31～16** にセット、ADS#から 5 クロック後に RDY#をアサート。

6. プロセッサ

バス・サイクル中の領域が 16 ビット・バスでキャッシュ出来ないことを検知して、**D31～16** からデータをプロセッサ内部に取り込んでサイクルを終了して 32 ビット・データの転送を完了する。

- 32 ビットデータを 8 ビット・バスのメモリの 0 番地にリード・アクセス

1. プロセッサ

A31～2 に 0、BE3#～0#に LLLL をセットして、ADS#を 1 クロックアサートしてサイクル開始。

2. ユーザ

BS8#をアサート、BS16#、KEN#をネゲートしてデータを **D7～0** にセット、ADS#から 5 クロック後に RDY#をアサート。

3. プロセッサ

バス・サイクル中の領域が 8 ビット・バスでキャッシュ出来ないことを検知して、**D7～0** からデータをプロセッサ内部に取り込んで最初のサイクルを終了する。

4. プロセッサ

A31～2 に 0、BE3#～0#を LLHH にセットして、ADS #を 1 クロックアサートして 2 番目のサイクル開始。

5. ユーザ

BS8#をアサート、BS16#、KEN#をネゲートしてデータを **D15～8** にセット、ADS#から 5 クロック後に RDY#をアサート。

6. プロセッサ

バス・サイクル中の領域が 8 ビット・バスでキャッシュ出来ないことを検知して、**D15～8** からデータをプロセッサ内部に取り込んで 2 番目のサイクルを終了する。

7. プロセッサ

A31～2 に 0、BE3#～0#を LLHH にセットして、ADS #を 1 クロックアサートして 3 番目のサイクル開始。

8. ユーザ

BS8#をアサート、BS16#、KEN#をネゲートしてデータを **D23～16** にセット、ADS#から 5 クロック後に RDY#をアサート。

9. プロセッサ

バス・サイクル中の領域が 8 ビット・バスでキャッシュ出来ないことを検知して、**D23～16** からデータをプロセッサ内部に取り込んで 3 番目のサイクルを終了する。

10. プロセッサ

A31~2 に 0、BE3#~0#を LHHH に 0 をセットして、ADS#を 1 クロック アサートして 4 番目のサイクル開始。

11. ユーザ

BS8#をアサート、BS16#、KEN# をネゲートしてデータを **D31~24** にセット、ADS#から 5 クロック後に RDY#をアサート。

12. プロセッサ

バス・サイクル中の領域が 8 ビット・バスでキャッシュ出来ないことを検知して、**D31~24** からデータをプロセッサ内部に取り込んでサイクルを終了して 32 ビット・データの転送を完了する。

5.2 設計仕様

インターフェースの設計に入る前に、プロセッサとインターフェースする、メモリや I/O などのハードウェアを規定します。

メモリ・マップと I/O マップは図 5.4 を参照して下さい。

5.2.1 CPU

80486DX の 33MHz を使います、DX2 の 66MHz でも差し替えだけで使えそうです。

DX4 は DX や DX2 のように電源電圧が 5V ではありませんが、その他は互換性があると思われます。

キャッシュとキャッシュ・ラインフィル

キャッシュ可能なアドレスを指定するときには KEN#をアサートしますが、メモリ・マップを参照しながら、メモリのチップ・セレクトを作る要領で作ります。

RDY

通常はこれをバス・サイクルを終了させる為に使うことにします、具体的には BLAST#がアサートされるバス・サイクルでは RDY#をアサートとします。

BRDY

BLAST#がネゲートされるバス・サイクルでは BRDY#をアサートとします、これがバースト転送の開始になります。

このように開始されるバス・サイクルはプロセッサが 32 ビット・データを 2 個以上の転送要求を持つ場合ですが、実際にはキャッシュ・ライン・フィルのデータ転送がそれに当たります。

■今回の仕様 では、8,16 ビット・バスのアクセスではバースト転送を考慮していませんが、キャッシュ・ラインフィルではない通常の 32 ビット・データ転送も 8,16 ビット・バスならバースト転送出来るので、そのように設計するのもよいでしょう。

5.2.2 メモリ

ROM 領域は 128k バイト、RAM 領域は 512k バイトで 1M バイト以内にマップしていますのでアドレス・ラインは A19~0 をだけを使います。

ROM

16 ビット・バスに配置します、キャッシュ可能でバースト・アクセスが出来るようにします。

64k バイトの容量で 8 ビット・データの EP-ROM の 27512 を 2 個使います、アクセス・タイムは 200ns を想定します。

RAM

32 ビット・バスに配置します、キャッシュ可能でバースト・アクセスが出来るようにします。

128k バイトの容量で 8 ビット・データの SRAM の 628128 を 4 個使います、アクセス・タイムは 120ns を想定します。

5.2.3 I/O

8 ビット・バスに配置します。

割り込みコントローラの 8259A など 8 ビットの 80 系周辺 LSI を 4 個使います。

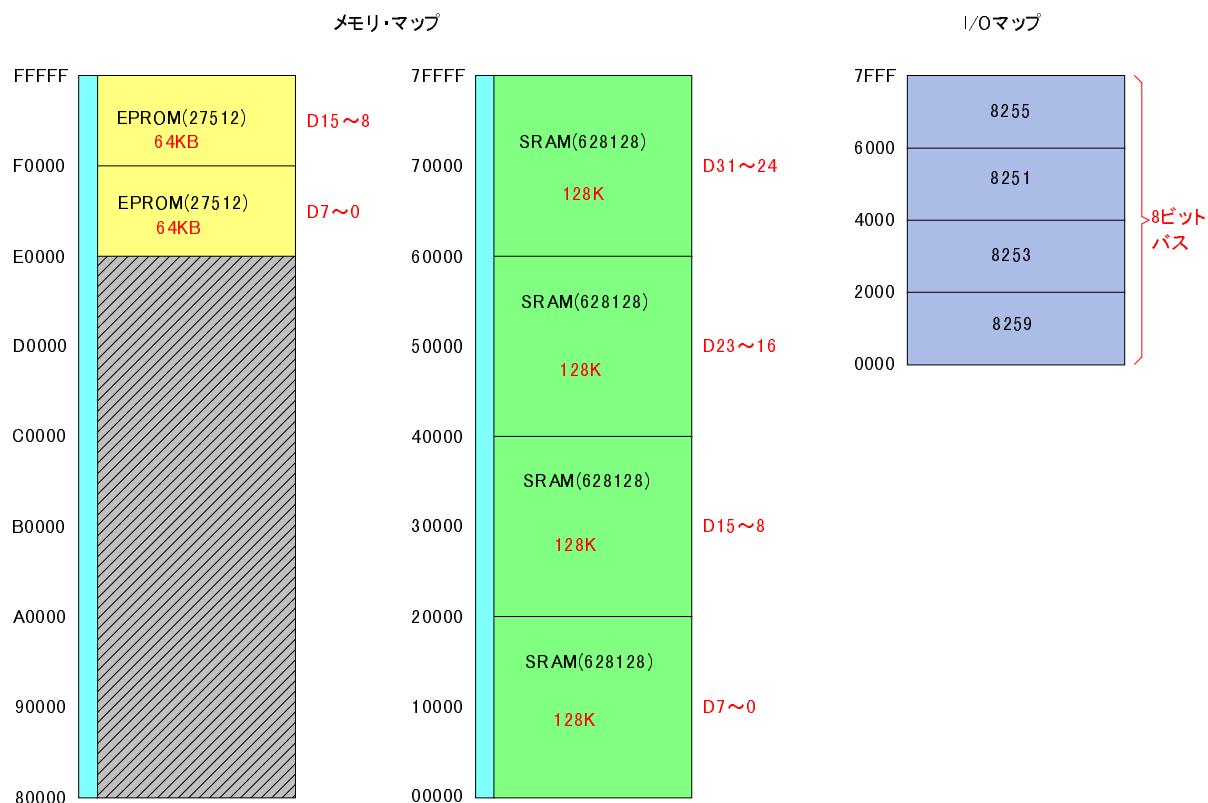
5.3 ユーザ・インターフェース回路のブロック

ユーザ回路のブロックを図 5.5 で参照して下さい、IF486 と呼びます。

5.3.1 チップの機能割付

PLD の GAL16V8 や GAL20V8 を 9 個を使います、それらに下記の機能を割り付けていきます。

図 5.4 80486CPU ボード・メモリマップ



- タイミング・コントロール

CPU の ADS#, BLAST#によりバス・サイクルの開始・終了の根拠としてタイミング信号を作ります。

1. PLD1 GAL16V8

バス・サイクルの検出とウェイト・カウンタ

2. PLD2 GAL16V8

バス・サイクルとキャッシュ・サイクルの進行

- 状態デコード

CPU のアドレス、M/IO#, D/C#, W/R #によりバス・サイクルの種類を内部コードに変換したものを、他の機能グループに渡す信号とします。

PLD3 GAL16V8

- コマンド・コントロール

メモリや I/O のリード/ライト信号、割り込みアクノレッジ信号、RDY#, BRDY#を作ります。

PLD4 GAL20V8

- チップ・コントロール

メモリや I/O のチップ・セレクトと KEN#や BS8#, BS16#を作ります。

1. PLD5 GAL16V8

メモリ・チップ・セレクト、I/O チップ・セレクト

2. PLD6 GAL16V8

KEN#, BS8#, BS16#

- データ・コントロール

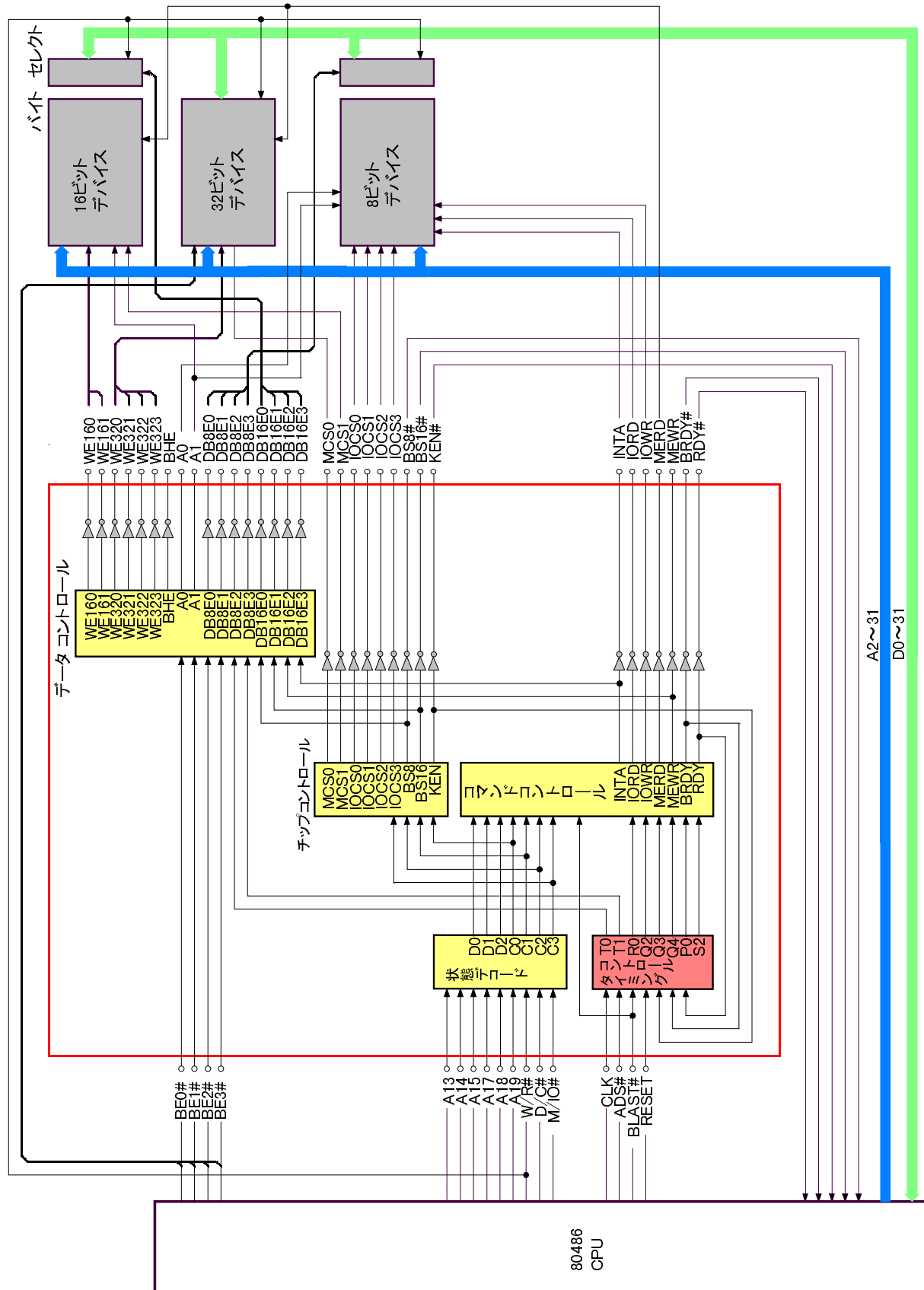
8ビット・バスや 16ビット・バスのバイト・イネーブル、A1,0を作ります。

1. PLD7 8ビット・バス制御信号 GAL16V8

2. PLD8 16ビット・バス制御信号 GAL16V8

3. PLD9 32ビット・バス制御信号 GAL16V8

図 5.5 IF486 ブロック



5.3.2 順序回路

それぞれの PLD1、2 について、LHDL 式を見ていきます。

PLD1

リスト8 PLD1.LH

```
module pld1;

defst sta[2]
  $0:[00];
  $1:[01];
  $2:[11];
endefst;

defst stb[5]
  $0:[00000];
  $1:[00001];
  $2:[00011];
  $3:[00111];
  $4:[01111];
  $5:[11111];
  $6:[11110];
  $7:[11100];
  $8:[11000];
  $9:[10000];
endefst;

input reset,ads,blast,rdy,brdy;
output Ps[2].reg;
output Q[5].reg;
output R0;

begin

{バス・サイクル検出}

state Ps(reset,ads,blast,rdy,brdy):sta;
  case $0: {idle}
    if (0, 1,x, x,x) then $0;      {idle}
    if (0, 0,x, x,x) then $1; endcase; {start cycle}
  case $1: {non burst}
    if (0, 1,x, 1,0) then $0;      {one cycle end}
    if (0, 1,0, 0,1) then $0;      {one cycle end}

    if (0, 1,x, 0,0) then $1;      {wait}
    if (0, 1,1, 0,1) then $2; endcase; {start burst cycle}
  case $2: {burst}
    if (0, 1,0, 0,1) then $0;      {burst cycle end}
    if (0, 1,x, 1,0) then $0;      {cycle end}

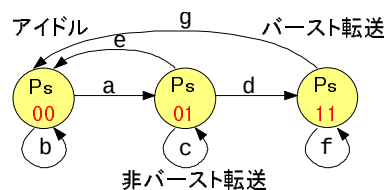
    if (0, 1,0, 0,0) then $2;      {wait}
    if (0, 1,1, 0,x) then $2; endcase; {burst cycle}
  endstate;

{ウェイト・カウンタ}

state Q(reset,ads,rdy,brdy,Ps.0):stb;
  case $0: if (0, 1,0,0,1) then $1; endcase;
  case $1: if (0, 1,0,0,1) then $2; endcase;
  case $2: if (0, 1,0,0,1) then $3; endcase;
  case $3: if (0, 1,0,0,1) then $4; endcase;
  case $4: if (0, 1,0,0,1) then $5; endcase;
  case $5: if (0, 1,0,0,1) then $6; endcase;
  case $6: if (0, 1,0,0,1) then $7; endcase;
  case $7: if (0, 1,0,0,1) then $8; endcase;
  case $8: if (0, 1,0,0,1) then $9; endcase;
  case $9: if (0, 1,0,0,1) then $0; endcase;
endstate;

{ライト・パルス}
```

図 5.7 バス・サイクル検出



```
table ([ads,rdy,brdy,Ps.0] -> [R0] )
  [x, x,x, 0]->[1]; {idle }
  [1, 1,0, 1]->[1]; {rdy end}
  [1, 0,1, 1]->[1]; {brdy end}
endtable;

end;

endm;
```

1. バス・サイクル検出

リスト 8 を参照して下さい、2 ビットのステート・マシン Ps はバス・サイクルの状態を図 5.7 のように示します、Ps が [00] の時はどのバス・サイクルも開始されておらず、Ps が [x1] の時はバス・サイクル中にあることを示します。

2. ウェイト・カウンタ

5 ビットのジョンソン・カウンタの Q はアイドル時は [00000] でバス・サイクル中は無制御でカウント・アップします、図 5.8 を参照して下さい、メモリや I/O の制御信号のタイミングを決定するために使っています。

3. ライト信号補助

サイクル期間中は 0 にしている信号でライト信号のアサート・タイミングに使っています。

シミュレーション

図 5.6 を参照して下さい、リセットの後の最初のサイクルに非バースト、次にバーストの設定をしています。

実動観測

• Ps

99 頁の図 5.20 を参照して下さい。

• Q

100 頁の図 5.21、100 頁の図 5.22 を参照して下さい。

PLD2

リスト9 PLD2.LH

図 5.6 PLD1 シミュレーション

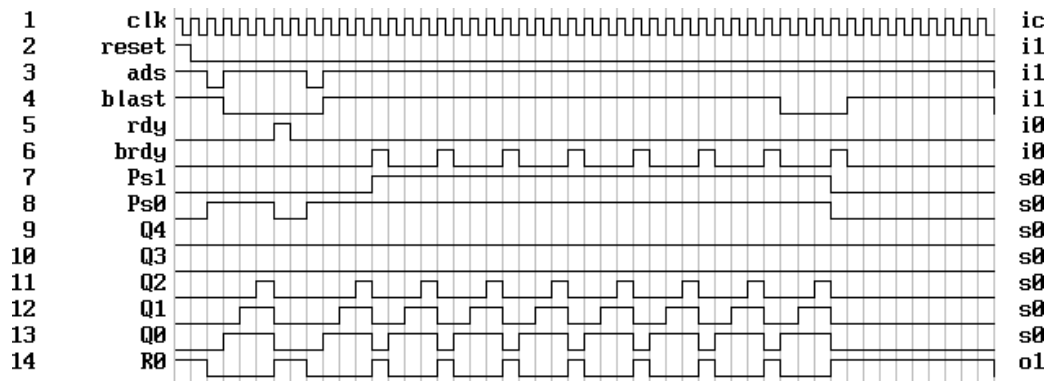
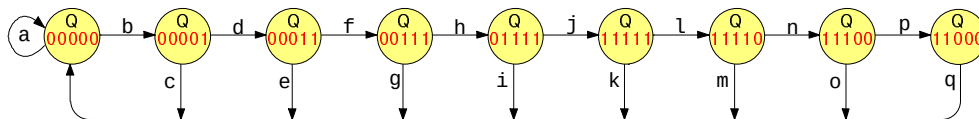


図 5.8 ウェイト・カウンタ



```

module pld2;

defst sta[3]
  $0:[000];
  $1:[001];
  $2:[010];
  $3:[100];
  $4:[011];
endefst;

defst stb[2]
  $0:[00];
  $1:[01];
  $2:[11];
  $3:[10];
endefst;

input reset;
input ads,ken,blast,rdy,brdy;
output S[3].reg;
output t[2].reg;

begin

{バス・サイクル進行}

state S(reset,ads,blast,rdy,brdy):sta;
  case $0: {idle}
    if (0, 1,x, x,x) then $0;          {idle}
    if (0, 0,x, x,x) then $1; endcase; {start cycle}
  case $1: {start cycle}
    if (0, 1,0, 1,0) then $0;          {one cycle}
    if (0, 1,x, 0,0) then $1;          {wait}
    if (0, 1,1, 1,0) then $2;          {multiple cycle}
    if (0, 1,1, 0,1) then $3; endcase; {burst cycle}
  case $2: {ads wait for multiple cycle}
    if (0, 0,x, x,x) then $4; endcase; {multiple cycle}
  case $3: {burst cycle}
    if (0, 1,0, 1,0) then $0;          {end burst cycle}
    if (0, 1,0, 0,1) then $0;          {end burst cycle}
    if (0, 1,1, 1,0) then $2;          {go multiple cycle}
    if (0, 1,0, 0,0) then $3;          {last burst cycle}
    if (0, 1,1, 0,x) then $3; endcase; {wait}

    case $4: {multiple cycle}
      if (0, 1,0, 1,0) then $0;          {go idle}
      if (0, 1,1, 1,0) then $2;          {ads wait}
      if (0, 1,1, 0,1) then $3;          {go burst cycle}
      if (0, 1,x, 0,0) then $4; endcase; {wait}
    endstate;

{キャッシュ検出}

state t(reset,ads,ken,blast,rdy,brdy):stb;
  case $0: {non cache}
    if (0, 0,1,x, x,x) then $1; endcase; {cache line fill start}
  case $1: {cache data 1}
    if (0, x,x,1, 1,0) then $2;          {multiple continue}
    if (0, x,x,1, 0,1) then $2;          {burst continue}
    if (0, x,x,1, 0,0) then $1;          {wait}

    if (0, x,x,0, 0,0) then $1;          {wait}
    if (0, x,x,0, 0,1) then $0;          {burst last}
    if (0, x,x,0, 1,0) then $0; endcase; {multiple last}
  case $2: {cache data 2}
    if (0, x,x,1, 0,0) then $2;          {wait}
    if (0, x,x,1, 0,1) then $2;          {burst continue}
    if (0, x,x,1, 1,0) then $2;          {multiple continue}
    if (0, x,x,0, 0,0) then $2;          {wait}
    if (0, x,x,0, 0,1) then $0;          {burst last}
    if (0, x,x,0, 1,0) then $0; endcase; {multiple last}
  endstate;

end;

endm;

```

1. バス・サイクル進行

バス・サイクルの状態を S の 3 ビット・ラッチに保持しています、[リスト 9](#) と [図 5.9](#) を参照して下さい、主に S の第 3 ビットをバースト状態の検出に使っています。

図 5.11 PLD2 シミュレーション (1)

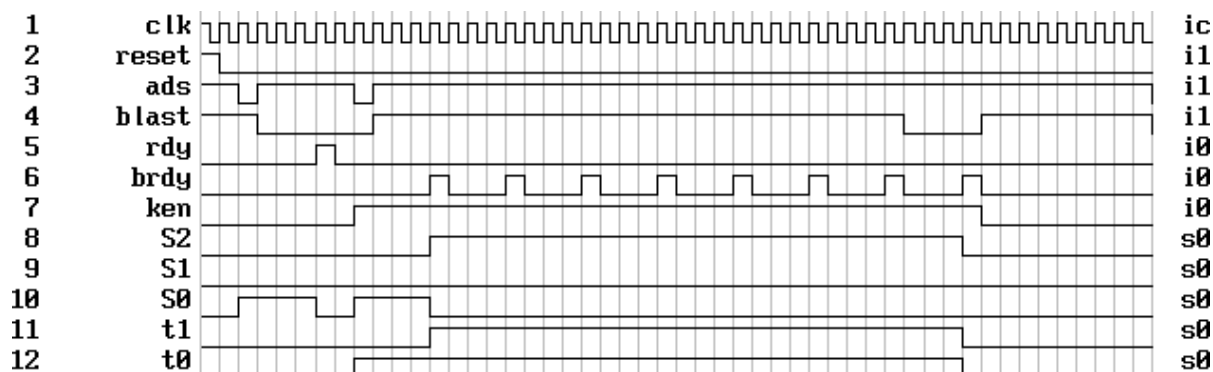


図 5.12 PLD2 シミュレーション (2)

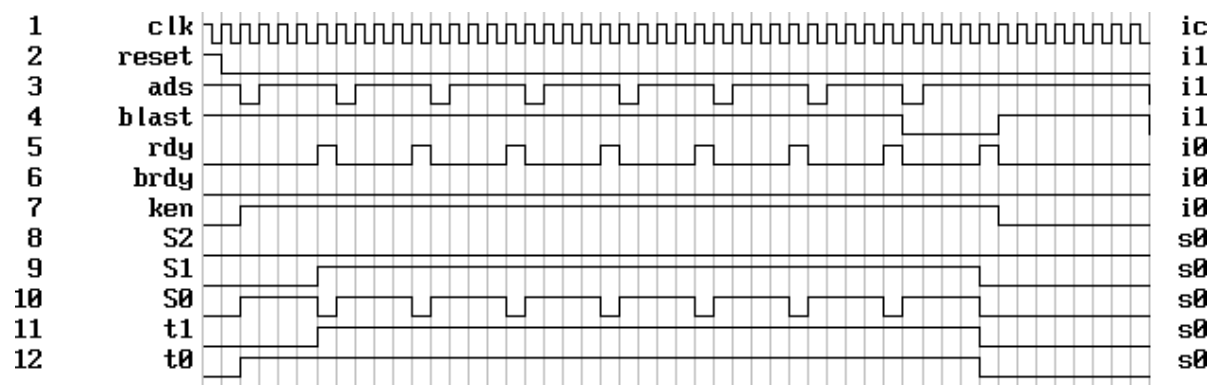


図 5.9 バス・サイクル進行

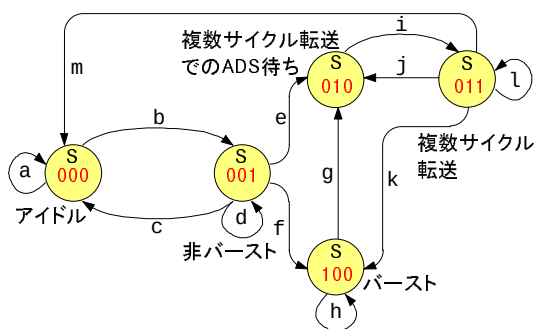
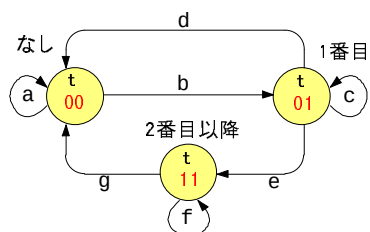


図 5.10 キャッシュ検出



2. キャッシュ検出

インテルのマニュアル*1に次の記述がありました。

外部システムは、キャッシュ・ラインフィルの最初のサイクルでは、すべてのバイト・イネーブルがアクティブであるとみなす必要がある。

インテル 486 マイクロプロセッサは、ラインフィルのそれ以降のサイクルに対して正しいバイト・イネーブルを生成する。

この為にキャッシュ・ラインフィルの最初とそれ以降のサイクルを区別してバイト・イネーブル信号を制御する必要があります。

図 5.10 を参照して下さい、t の 2 ビットにキャッシュ・ラインフィルの状態を保持しています。

*1 インテル 486TMDX マイクロプロセッサ・データブック
資料番号:240440J-004 102 頁

シミュレーション

図 5.11 と図 5.12 を参照して下さい、図 5.11 はリセットの後の最初のサイクルに単一転送、2 番目にキャッシュ・ラインフィルのバーストの設定をしています。

図 5.12 はリセットの後にキャッシュ・ラインフィルの複数サイクル転送の設定をしています。

実動観測

- S 101 頁の図 5.23 を参照して下さい。
- t 101 頁の図 5.24 を参照して下さい。

5.3.3 組み合わせ回路

図 5.5 の中のタイミング・コントロール以外のものがこれに当たります。

PLD3

リスト10 PLD3.LH

```
module pld3;

input  mio,dc,wr;
input  a19,a18,a17;
input  a15,a14,a13;
input  spbc;

output c[4],d[3];

begin

{デバイス・デコード}

table([mio,dc,wr,a19,a18,a17,a15,a14,a13,spbc]->[c:4])
  [1,x,x, 1,0,0, x,x,x, x] -> [8:4] ; {memory:no device}
  [1,x,x, 1,0,1, x,x,x, x] -> [9:4] ; {memory:no device}
  [1,x,x, 1,1,0, x,x,x, x] -> [A:4] ; {memory:no device}
  [0,0,1, x,x,x, x,x,x, 1] -> [B:4] ; {special bus cycle}
  [0,1,x, x,x,x, 0,0,1, x] -> [4:4] ; {I/O}
  [0,1,x, x,x,x, 0,1,0, x] -> [5:4] ; {I/O}
  [0,1,x, x,x,x, 0,1,1, x] -> [6:4] ; {I/O}
  [0,0,0, x,x,x, x,x,x, x] -> [7:4] ; {inta}
  [1,x,x, 1,1,1, x,x,x, x] -> [2:4] ; {ROM(16)}
  [0,1,x, x,x,x, 0,0,0, x] -> [3:4] ; {I/O}
  [1,x,x, 0,0,0, x,x,x, x] -> [1:4] ; {RAM(32)}
  [1,x,x, 0,0,1, x,x,x, x] -> [1:4] ; {RAM(32)}
  [1,x,x, 0,1,0, x,x,x, x] -> [1:4] ; {RAM(32)}
  [1,x,x, 0,1,1, x,x,x, x] -> [1:4] ; {RAM(32)}
endtable;

{サイクル・デコード}

table([mio,dc,wr]->[d:3])
  [0,0,0] -> [5:3] ; {割り込みアクノレッジ}
  [0,0,1] -> [6:3] ; {ホールド／特殊サイクル}
  [0,1,1] -> [4:3] ; {I/O 書き込み}
  [1,0,0] -> [7:3] ; {コード読み出し}
  [0,1,0] -> [3:3] ; {I/O 読み出し}
  [1,1,1] -> [2:3] ; {メモリ書き込み}
  [1,1,0] -> [1:3] ; {メモリ読み出し}
endtable;

end;

endm;
```

c の 4 ビットにデバイス・コードを割り付けています、リスト 10 と表 5.1 を参照して下さい。

表 5.1 デバイス・コード

コード	デバイス	バスサイズ
1	RAM	32 ビット
2	ROM	16 ビット
3	I/O	8 ビット
4	I/O	8 ビット
5	I/O	8 ビット
6	I/O	8 ビット
7	INTA*2	8 ビット
8	—	—
9	—	—
A	—	—
B	**3	—

d の 3 ビットにサイクル・コードを割り付けています、リスト 10 と表 5.2 を参照して下さい。

表 5.2 サイクル・コード

コード	サイクル
1	メモリ読み出し
2	メモリ書き込み
3	I/O 読み出し
4	I/O 書き込み
5	割り込みアクノレッジ
6	ホールド／特殊サイクル
7	コード読み出し

PLD4

リスト11 PLD4.LH

```
module pld4;

input  blast,S2,Q4,Q3,Q2,c[4],d[3],R0,Ps0;
output rdy,brdy,mewr,iowr,inta,merd,iord;

{非バースト・デバイス rdy}

begin
table([blast,S2,Q4,Q3,Q2,c:4]->[rdy])
  [0, 0, x,0,1, &1:4]->[1]; {RAM(32) 2wait}
end;
```

*2 割り込みコントローラ

*3 スペシャル・バス・サイクル

```

[0, 0, 0, 1, x, &2:4] -> [1]; {ROM(16) 3wait}

[0, 0, 1, 0, x, &3:4] -> [1]; {I/O 8wait}
[0, 1, 1, 0, x, &3:4] -> [1]; { ^ }
[1, 0, 1, 0, x, &3:4] -> [1]; { ^ }
[1, 1, 1, 0, x, &3:4] -> [1]; { ^ }

[0, 0, 1, 0, x, &4:4] -> [1]; {I/O 8wait}
[0, 1, 1, 0, x, &4:4] -> [1]; { ^ }
[1, 0, 1, 0, x, &4:4] -> [1]; { ^ }
[1, 1, 1, 0, x, &4:4] -> [1]; { ^ }

[0, 0, 1, 0, x, &5:4] -> [1]; {I/O 8wait}
[0, 1, 1, 0, x, &5:4] -> [1]; { ^ }
[1, 0, 1, 0, x, &5:4] -> [1]; { ^ }
[1, 1, 1, 0, x, &5:4] -> [1]; { ^ }

[0, 0, 1, 0, x, &6:4] -> [1]; {I/O 8wait}
[0, 1, 1, 0, x, &6:4] -> [1]; { ^ }
[1, 0, 1, 0, x, &6:4] -> [1]; { ^ }
[1, 1, 1, 0, x, &6:4] -> [1]; { ^ }

[x, x, 1, 0, x, &7:4] -> [1]; {inta 8wait}

{memory 2wait}
[0, 0, x, 0, 1, &8:4] -> [1]; { ^ single/multiple last}
[0, 1, x, 0, 1, &8:4] -> [1]; { ^ burst last}
[1, 0, x, 0, 1, &8:4] -> [1]; { ^ single/multiple}
[1, 1, x, 0, 1, &8:4] -> [1]; { ^ burst}

{memory 3wait}
[0, 0, x, 0, 1, &9:4] -> [1]; { ^ single/multiple last}
[0, 1, x, 0, 1, &9:4] -> [1]; { ^ burst last}
[1, 0, x, 0, 1, &9:4] -> [1]; { ^ single/multiple}
[1, 1, x, 0, 1, &9:4] -> [1]; { ^ burst}

[0, 0, x, 0, 1, &B:4] -> [1]; {special bus cycle}
[0, 1, x, 0, 1, &B:4] -> [1]; { ^ }
[1, 0, x, 0, 1, &B:4] -> [1]; { ^ }
[1, 1, x, 0, 1, &B:4] -> [1]; { ^ }
endtable;

{バースト・デバイス brdy}

table ([blast, S2, Q4, Q3, Q2, c:4] -> [brdy])

{RAM(32) 2wait}
[0, 1, x, 0, 1, &1:4] -> [1]; { ^ burst last}
[1, 0, x, 0, 1, &1:4] -> [1]; { ^ single/multiple}
[1, 1, x, 0, 1, &1:4] -> [1]; { ^ burst}

{ROM(16) 3wait}
[0, 1, 0, 1, x, &2:4] -> [1]; { ^ burst last}
[1, 0, 0, 1, x, &2:4] -> [1]; { ^ single/multiple}
[1, 1, 0, 1, x, &2:4] -> [1]; { ^ burst}
endtable;

{メモリ・ライト}

table ([c:4, d:3, Q4, Q3, R0] -> [mewr])

[&1:4, &2:3, x, 0, 0] -> [1]; {RAM(32) 4wait}
[&2:4, &2:3, 0, x, 0] -> [1]; {ROM(16) 8wait}
[&8:4, &2:3, x, 0, 0] -> [1]; {memory(32) 4wait}
[&9:4, &2:3, x, 0, 0] -> [1]; {memory(32) 4wait}
endtable;

{I/O ライト}

table ([c:4, d:3, R0] -> [iowr])
[&3:4, &4:3, 0] -> [1];
[&4:4, &4:3, 0] -> [1];
[&5:4, &4:3, 0] -> [1];

```

```

[&6:4, &4:3, 0] -> [1];
endtable;

{割り込みアクノレッジ}

table ([d:3, Ps0] -> [inta])
[&5:3, 1] -> [1];
endtable;

{メモリ・リード}

table ([c:4, d:3] -> [merd])

{data read}
[&1:4, &1:3] -> [1]; { ^ RAM(32)}
[&2:4, &1:3] -> [1]; { ^ ROM(16)}
[&8:4, &1:3] -> [1]; { ^ memory(32)}
[&9:4, &1:3] -> [1]; { ^ memory(32)}
[&A:4, &1:3] -> [1]; { ^ special bus cycle}

{code read}
[&1:4, &7:3] -> [1]; { ^ RAM(32)}
[&2:4, &7:3] -> [1]; { ^ ROM(16)}
[&8:4, &7:3] -> [1]; { ^ memory(32)}
[&9:4, &7:3] -> [1]; { ^ memory(32)}
[&A:4, &7:3] -> [1]; { ^ special bus cycle}
endtable;

{I/O リード}

table ([c:4, d:3] -> [iord])
[&3:4, &3:3] -> [1];
[&4:4, &3:3] -> [1];
[&5:4, &3:3] -> [1];
[&6:4, &3:3] -> [1];
endtable;

end;

endm;

```

プロセッサに返す信号の rdy、brdy とメモリに対するコマンド信号の mewr、merd、iowr、iord を作っています、[リスト 11](#) を参照して下さい。

- rdy

I/O サイクルはすべて RDY#を返します。

メモリ・サイクル^{*4}のキャッシュ・ラインフィル以外のサイクル^{*5}では RDY#を返します。

- brdy

メモリ・サイクルでキャッシュ・ラインフィルのサイクルでは BRDY#を返します。

rdy と brdy の入力項目は同じです、意味は [表 5.5](#) と [表 5.6](#) を参照して下さい。

- merd、iord

デバイス・コードとサイクル・コードをデコードするだけでリード信号を作っています。

^{*4} デバイス・コードが 1 と 2 のサイクル

^{*5} blast が 0 で開始されるサイクル

表 5.3 バースト・サイクルの組み合わせ

状態	blast	S2
①シングル転送	0	0
②バースト・ラスト	0	1
③バースト開始	1	0
④バースト継続	1	1

表 5.4 ウェイト数の設定

ウェイト数	Q4	Q3	Q2
2	×	0	1
3	0	1	×
8	1	0	×

サイクル・コードは表 5.2 の①メモリ読み出し、⑦コード読み出し、③I/O 読み出しを必要条件にしています。

- mewr

デバイス・コードとサイクル・コードにウェイト・カウンタと R0 を加えています。ウェイト・カウンタと R0 を条件に入れたのはライト信号のアサート・タイミングを得るためです。

- iowr

mewr とほぼ同じですが、I/O サイクルはバーストされないのでウェイトカウンタの条件は外しています。

- inta

割り込みアクノレッジ信号をデバイス・コードと Ps0 の組み合わせで作っています、Ps0 が 1 のときはサイクル中であることを示しますので、両条件が揃ったときは、割り込みアクノレッジ・サイクルであり、この信号は 8259A の INTA に接続します。

PLD5

リスト12 PLD5.LH

```
module pld5;
input c[4];
output mcs[4], iocs[4];
begin
```

{チップ・セレクト}

```
table ([c:4]->[mcs:4,iocs:4])
  [&1:4]->[0,0,0,1, 0,0,0,0]; {RAM(32)}
  [&2:4]->[0,0,1,0, 0,0,0,0]; {ROM(16)}
  [&8:4]->[0,1,0,0, 0,0,0,0]; {memory(32)}
  [&9:4]->[1,0,0,0, 0,0,0,0]; {memory(32)}
  [&3:4]->[0,0,0,0, 0,0,0,1]; {I/O}
  [&4:4]->[0,0,0,0, 0,0,1,0]; {I/O}
  [&5:4]->[0,0,0,0, 0,1,0,0]; {I/O}
  [&6:4]->[0,0,0,0, 1,0,0,0]; {I/O}
endtable;

end;

endm;
```

デバイス・コードを使ってチップ・セレクトを作っていますリスト 12 と、メモリ・チップセレクトの場合は表 5.5、I/O チップセレクトの場合は表 5.6 を参照して下さい。

表 5.5 メモリ・デバイス選択

コード	MCS3	MCS2	MCS1	MCS0	デバイス
1	×	×	×	○	RAM(32)
2	×	×	○	×	ROM(16)
8	×	○	×	×	—
9	○	×	×	×	—

表 5.6 I/O デバイス選択

コード	IOCS3	IOCS2	IOCS1	IOCS0	デバイス
3	×	×	×	○	8259A
4	×	×	○	×	8253
5	×	○	×	×	8251
6	○	×	×	×	8255

PLD6

リスト13 PLD6.LH

```
module pld6;
input c[4];
output ken,bs8,bs16;
begin
```

{バス・サイズ／キャッシュ制御}

```
table ([c:4]->[ken,bs8,bs16])
  [&1:4]->[1,0,0]; {RAM(32)}
  [&2:4]->[1,0,1]; {ROM(16)}
  [&8:4]->[1,0,0]; {memory(32)}
  [&3:4]->[0,1,0]; {I/O}
  [&4:4]->[0,1,0]; {I/O}
```

```
[&5:4]->[0,1,0]; {I/O}
[&6:4]->[0,1,0]; {I/O}
endtable;

end;

endm;
```

デバイス・コードによってプロセッサに返す信号の真偽を決めています、**リスト 13**を参照しています。

キャッシュの可否を KEN#で、バス・サイズを BS8#、BS16#で指定しています**リスト 13**と**表 5.7**を参照して下さい。

表 5.7 ステータス

コード	ken	bs8	bs16	デバイス
1	○	×	×	RAM(32)
2	○	×	○	ROM(16)
8	○	×	×	—
9	×	×	×	—
3	×	○	×	I/O
4	×	○	×	I/O
5	×	○	×	I/O
6	×	○	×	I/O

PLD7

リスト14 PLD7.LH

```
module pld7;

input be[4],t[2],bs8,inta;
output db8e[4];
output a1,a0,bhe;
output spbc;

begin

{8ビット・バス バイト・イネーブル}

table ([be:4,bs8,inta,t:2]->[db8e:4])
[0,1,1,1, 1,0, 0,0] -> [1,0,0,0] ; {byte non cache}
[0,1,1,1, 1,0, 1,1] -> [1,0,0,0] ; { ^ cache2}

[1,0,1,1, 1,0, 0,0] -> [0,1,0,0] ; {byte non cache}
[0,0,1,1, 1,0, 0,0] -> [0,1,0,0] ; {word non cache}
[1,0,1,1, 1,0, 1,1] -> [0,1,0,0] ; {byte cache2}
[0,0,1,1, 1,0, 1,1] -> [0,1,0,0] ; {word cache2}

[1,1,0,1, 1,0, 0,0] -> [0,0,1,0] ; {byte non cache}
[1,0,0,1, 1,0, 0,0] -> [0,0,1,0] ; {word non cache}
[0,0,0,1, 1,0, 0,0] -> [0,0,1,0] ; {trip non cache}
[1,1,0,1, 1,0, 1,1] -> [0,0,1,0] ; {byte cache2}
[1,0,0,1, 1,0, 1,1] -> [0,0,1,0] ; {word cache2}
[0,0,0,1, 1,0, 1,1] -> [0,0,1,0] ; {trip cache2}

[1,1,1,0, 1,0, 0,0] -> [0,0,0,1] ; {byte non cache}
[1,1,0,0, 1,0, 0,0] -> [0,0,0,1] ; {word non cache}
[1,0,0,0, 1,0, 0,0] -> [0,0,0,1] ; {trip non cache}
```

```
[0,0,0,0, 1,0, 0,0] -> [0,0,0,1] ; {dword non cache}
[1,1,1,0, 1,0, 1,1] -> [0,0,0,1] ; {byte cache2}
[1,1,0,0, 1,0, 1,1] -> [0,0,0,1] ; {word cache2}
[1,0,0,0, 1,0, 1,1] -> [0,0,0,1] ; {trip cache2}
[0,0,0,0, 1,0, 1,1] -> [0,0,0,1] ; {dword cache2}
```

```
[x,x,x,x, 1,0, 0,1] -> [0,0,0,1] ; {cache1}
```

```
[x,x,x,x, x,1, x,x] -> [0,0,0,1] ; {inta}
endtable;
```

{16ビット・バス制御}

```
table ([be:4,t:2] -> [a1,a0,bhe])
[1,0,1,1, 0,0] -> [1,0,0] ; {byte non cache}
[0,0,1,1, 0,0] -> [1,0,1] ; {word non cache}
[0,1,1,1, 0,0] -> [1,1,1] ; {byte non cache}
[1,0,1,1, 1,1] -> [1,0,0] ; {byte cache2}
[0,0,1,1, 1,1] -> [1,0,1] ; {word cache2}
[0,1,1,1, 1,1] -> [1,1,1] ; {byte cache2}
```

```
[1,1,0,1, 0,0] -> [0,1,1] ; {byte non cache}
[1,0,0,1, 0,0] -> [0,1,1] ; {word non cache}
[0,0,0,1, 0,0] -> [0,1,1] ; {trip non cache}
[1,1,0,1, 1,1] -> [0,1,1] ; {byte cache2}
[1,0,0,1, 1,1] -> [0,1,1] ; {word cache2}
[0,0,0,1, 1,1] -> [0,1,1] ; {trip cache2}
```

```
[1,1,0,0, 0,0] -> [0,0,1] ; {word non cache}
[1,0,0,0, 0,0] -> [0,0,1] ; {trip non cache}
[0,0,0,0, 0,0] -> [0,0,1] ; {dword non cache}
[1,1,0,0, 1,1] -> [0,0,1] ; {word cache2}
[1,0,0,0, 1,1] -> [0,0,1] ; {trip cache2}
[0,0,0,0, 1,1] -> [0,0,1] ; {dword cache2}
```

```
[x,x,x,x, 0,1] -> [0,0,1] ; {cache1}
endtable;
```

{特殊バス・サイクル}

```
table ([be:4] -> [spbc])
[1,1,0,1] -> [1] ;
[0,1,1,1] -> [1] ;
endtable;

end;
```

8ビット・バスのバイト・イネーブル信号、16ビット・バスのアドレス信号を作っています、**リスト 14**を参照して下さい。

● 8ビット・バス・イネーブル

$bs8 = 1, inta = 0$ で8ビットバスが選択されていて、かつ割り込みアクノレッジ・サイクルでないこと。

$t = [00]$ か $[11]$ でキャッシュ・ラインフィルでないか、その最初のサイクルで無いことを条件に**表 5.8**が成立します。

● 16ビット・バス・アドレス

16ビット・バス用の下位アドレスです**表 5.9**を参照して下さい。

表 5.8 8ビット・バス・バイトイネーブル

BE3	BE2	BE1	BE0	DB8E3	DB8E2	DB8E1	DB8E0
0	1	1	1	○	×	×	×
1	0	1	1	×	○	×	×
0	0	1	1	×	○	×	×
1	1	0	1	×	×	○	×
1	0	0	1	×	×	○	×
0	0	0	1	×	×	○	×
1	1	1	0	×	×	×	○
1	1	0	0	×	×	×	○
1	0	0	0	×	×	×	○
0	0	0	0	×	×	×	○

表 5.9 16ビット・バス・アドレス

BE3	BE2	BE1	BE0	A1	A0(BLE)	BHE
1	0	1	1	1	0 (○)	×
0	0	1	1	1	0 (○)	○
0	1	1	1	1	1 (×)	○
1	1	0	1	0	1 (×)	○
1	0	0	1	0	1 (×)	○
0	0	0	1	0	1 (×)	○
1	1	0	0	0	0 (○)	○
1	0	0	0	0	0 (○)	○
0	0	0	0	0	0 (○)	○

PLD8

リスト15 PLD8.LH

```

module pld8;

input  be[4],t[2],bs16,mewr;
output db16e[4];
output we16[2];

begin

{16ビット・バス・バイト・イネーブル}

table ([be:4,bs16,t:2] -> [db16e:4])
[0,0,1,1, 1, 0,0] -> [1,1,0,0] ; {word non cache}
[0,1,1,1, 1, 0,0] -> [1,0,0,0] ; {byte non cache}
[0,0,1,1, 1, 1,1] -> [1,1,0,0] ; {word cache2}
[0,1,1,1, 1, 1,1] -> [1,0,0,0] ; {byte cache2}

[1,0,1,1, 1, 0,0] -> [0,1,0,0] ; {byte non cache}
[1,0,1,1, 1, 1,1] -> [0,1,0,0] ; {byte cache2}

```

```

[1,1,0,0, 1, 0,0] -> [0,0,1,1] ; {word non cache}
[1,0,0,0, 1, 0,0] -> [0,0,1,1] ; {trip non cache}
[0,0,0,0, 1, 0,0] -> [0,0,1,1] ; {dword non cache}
[1,1,0,1, 1, 0,0] -> [0,0,1,0] ; {byte non cache}
[1,0,0,1, 1, 0,0] -> [0,0,1,0] ; {word non cache}
[0,0,0,1, 1, 0,0] -> [0,0,1,0] ; {trip non cache}
[1,1,0,0, 1, 1,1] -> [0,0,1,1] ; {word cache2}
[1,0,0,0, 1, 1,1] -> [0,0,1,1] ; {trip cache2}
[0,0,0,0, 1, 1,1] -> [0,0,1,1] ; {dword cache2}
[1,1,0,1, 1, 1,1] -> [0,0,1,0] ; {byte cache2}
[1,0,0,1, 1, 1,1] -> [0,0,1,0] ; {word cache2}
[0,0,0,1, 1, 1,1] -> [0,0,1,0] ; {trip cache2}

```

```

[x,x,x,x, 1, 0,1] -> [0,0,1,1] ; {cache1}
[1,1,1,0, 1, 0,0] -> [0,0,0,1] ; {byte non cache}
[1,1,1,0, 1, 1,1] -> [0,0,0,1] ; {byte cache2}

```

endtable;

{16ビット・バス・ライト・イネーブル}

```

table ([be:4,bs16,mewr] -> [we16:2])
[1,1,0,0, 1, 1] -> [1,1] ; {word}
[1,0,0,0, 1, 1] -> [1,1] ; {trip}
[0,0,0,0, 1, 1] -> [1,1] ; {dword}}
[1,1,0,1, 1, 1] -> [1,0] ; {byte}
[1,0,0,1, 1, 1] -> [1,0] ; {word}
[0,0,0,1, 1, 1] -> [1,0] ; {trip}
[0,0,0,1, 1, 1] -> [1,1] ; {word}
[0,1,1,1, 1, 1] -> [1,0] ; {byte}
[1,1,1,0, 1, 1] -> [0,1] ; {byte}
[1,0,1,1, 1, 1] -> [0,1] ; {byte}

```

endtable;

end;

endm;

16ビット・バスのバイト・イネーブルとライト・イネーブル信号を作っています、[リスト 15](#)を参照して下さい。

● バイト・イネーブル

bs16 = 1 で 16ビットバスが選択されて、t =

[00] か [11] でキャッシュ・ラインフィルでないか、その最初のサイクルで無いことを条件に表 5.10 が成立します。

- ライト・イネーブル
bs16 = 1 で 16 ビットバスが選択されて、mewr = 1 であることを条件に表 5.11 が成立します。

表 5.11 16 ビット・バス・ライト・イネーブル

BE3	BE2	BE1	BE0	WE16E1	WE16E0
1	1	0	0	○	○
1	0	0	0	○	○
0	0	0	0	○	○
1	1	0	1	○	×
1	0	0	1	○	×
0	0	0	1	○	×
0	0	1	1	○	○
0	1	1	1	○	×
1	1	1	0	×	○
1	0	1	1	×	○

PLD9

リスト 16 PLD9.LH

```
module pld9;
input be[4],bs8,bs16,mewr;
output we32[4];

begin
{32 ビット・バス・ライト・イネーブル}

table ([be:4,bs8,bs16,mewr] -> [we32:4])
[0,1,1,1, 0,0, 1] -> [1,0,0,0];
[1,0,1,1, 0,0, 1] -> [0,1,0,0];
[1,1,0,1, 0,0, 1] -> [0,0,1,0];
[1,1,1,0, 0,0, 1] -> [0,0,0,1];
endtable;

end;

endm;
```

32 ビット・バスのライト・イネーブル信号を作っています、リスト 16 を参照して下さい。

bs16 = 0, bs8 = 0 で 16 ビットバスも 8 ビットバスも選択されていない場合で*6、mewr = 1 であること

を条件に表 5.12 が成立します。

5.4 実際の動作記録

ロジック・アナライザ*7 で観測したデータをプリントアウトしたものです。

- バースト読み出し (ROM) キャッシュ・ラインフィル

96 頁 図 5.13 を参照して下さい。

ROM 領域はキャッシュとバーストを可能な領域にしています、キャッシュ・ラインフィルは 16 ビット・バスですから、バーストで 16 ビットの 8 個のデータ転送をしています、個々の転送においてプロセッサはアドレスを変化させています。

- 非バースト読み出し (RAM)

96 頁 図 5.14 を参照して下さい。

サイクルの最初から BLAST#がアサートされていますから、非バーストの読み出しです。

- 非バースト書き込み (RAM)

97 頁 図 5.15 を参照して下さい。

- 割り込み応答

97 頁 図 5.16 を参照して下さい。

割り込みがプロセッサに受付られると、割り込みコントローラの 8259A のために 2 回割り込み応答サイクルを実行されます、このサイクルは RDY#をアクティブにして終了させますが、デバイスに対応したウェイトを確保します。

- リセット・スタート

98 頁 図 5.17 を参照して下さい。

リセット・パルスが終了してから 7μs 程で ROM をバースト読み出ししています。

リセット・ベクタは ROM 領域でキャッシュ可能ですが、このときはリセット直後なのでソフト的にはイネーブルされていません、しかし 80486 はキャッシュ・ラインフィルを実行しているようです。

- I/O 読み出し

98 頁 図 5.18 を参照して下さい。

メモリの読み出しと基本的に変わりませんが、

*6 32 ビット・バスの選択。

*7 ロジアナ 98 50MHz

表 5.10 16 ビット・バス・バイト・イネーブル

BE3	BE2	BE1	BE0	DB16E3	DB16E2	DB16E1	DB16E0
0	0	1	1	○	○	×	×
0	1	1	1	○	×	×	×
1	0	1	1	×	○	×	×
1	1	0	0	×	×	○	○
1	0	0	0	×	×	○	○
0	0	0	0	×	×	○	○
1	1	0	1	×	×	○	×
1	0	0	1	×	×	○	×
0	0	0	1	×	×	○	×
1	1	1	0	×	×	×	○

表 5.12 32 ビット・バス・ライト・イネーブル

BE3	BE2	BE1	BE0	WE323	WE322	WE321	WE320
0	1	1	1	○	×	×	×
1	0	1	1	×	○	×	×
1	1	0	1	×	×	○	×
1	1	1	0	×	×	×	○

デバイスに合わせてウェイトを多くしてます。

- I/O 書き込み

99 頁 図 5.19 を参照して下さい。

- バス・サイクル検出^{*8}

99 頁 図 5.20 を参照して下さい。

アイドル状態と非バースト転送、バースト転送を検出します。

- ウェイト・カウンタ^{*6}

100 頁 図 5.21 を参照して下さい。

RDY#や BRDY#をアサートするタイミングの作成に使う信号です。

- ウェイト・カウンタ (拡大) ^{*6}

100 頁 図 5.22 を参照して下さい。

- バス・サイクル進行^{*6}

101 頁 図 5.23 を参照して下さい。

サイクルの進行状況を示す信号です、サイクルの開始から終了までのサイクルの遷移を把握するのに使っています。

- キャッシュ・ラインフィル検出^{*6}

101 頁 図 5.24 を参照して下さい。

キャッシュ・ラインフィルのサイクルの遷移を把握するのに使っています。

^{*8} ※プロセッサの動作クロックを 8MHz にして測定しました。

図 5.13 バースト読み出し (ROM) キャッシュ・ラインフィル

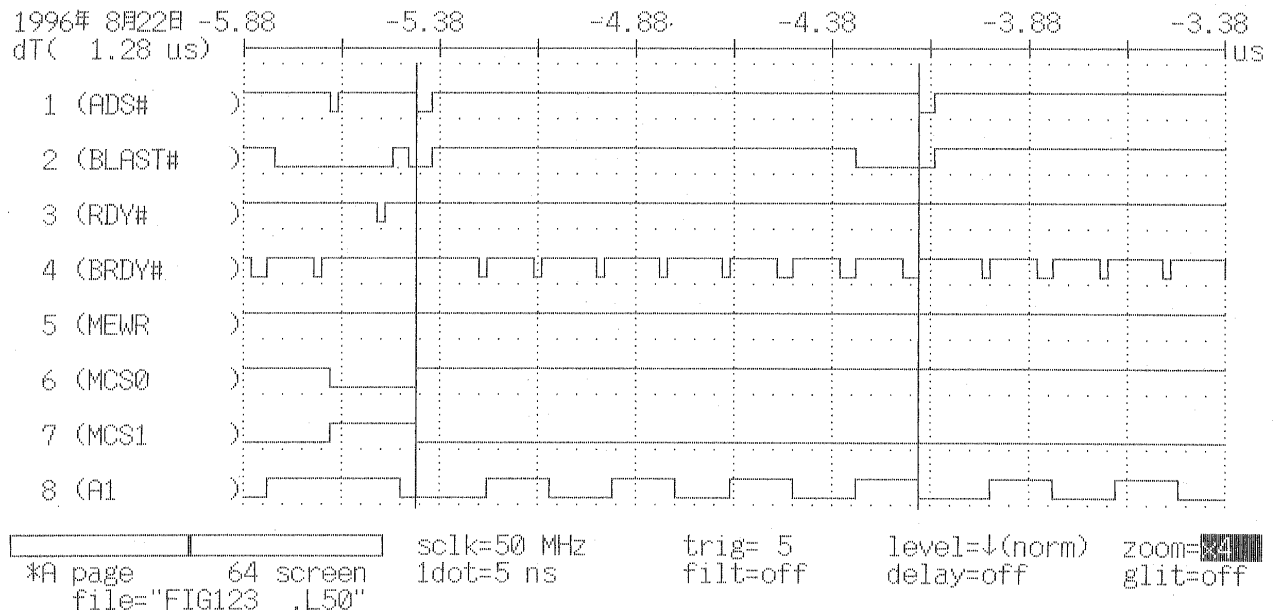


図 5.14 非バースト読み出し (RAM)

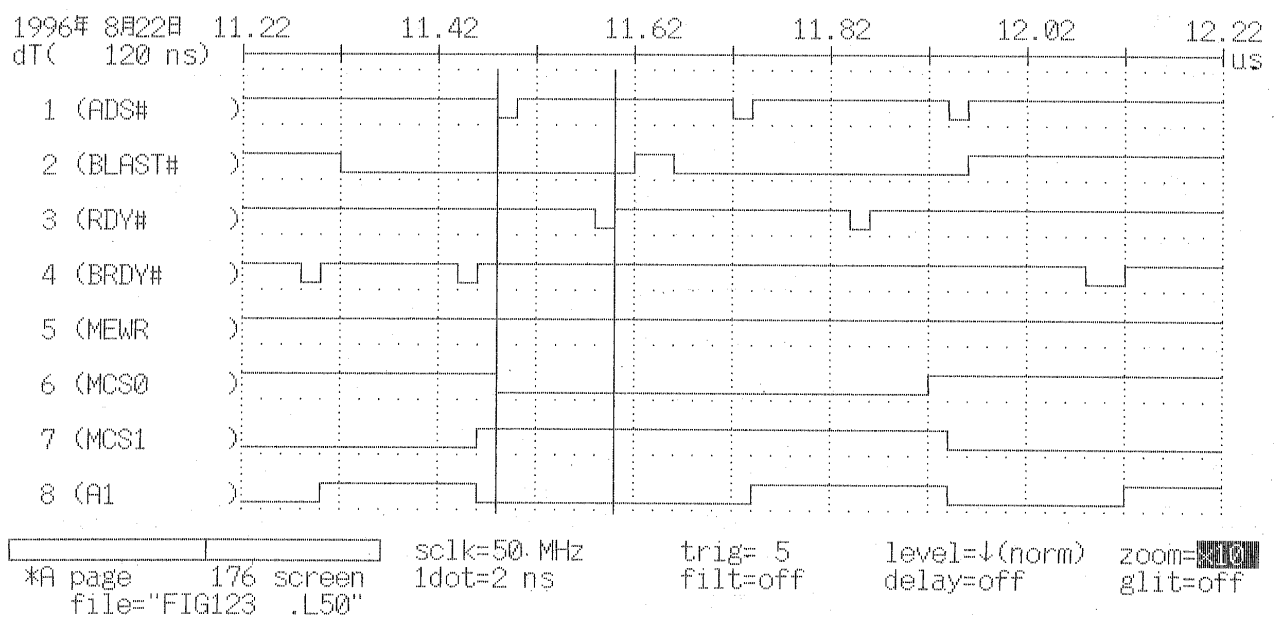


図 5.15 非バースト書き込み (RAM)

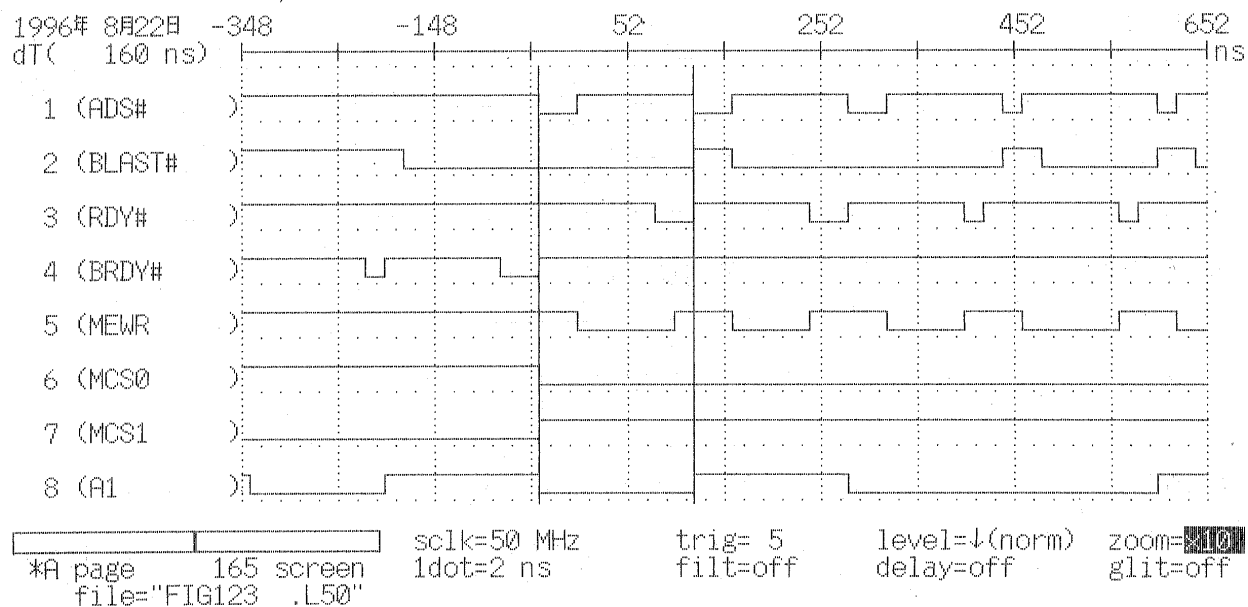


図 5.16 割り込み応答

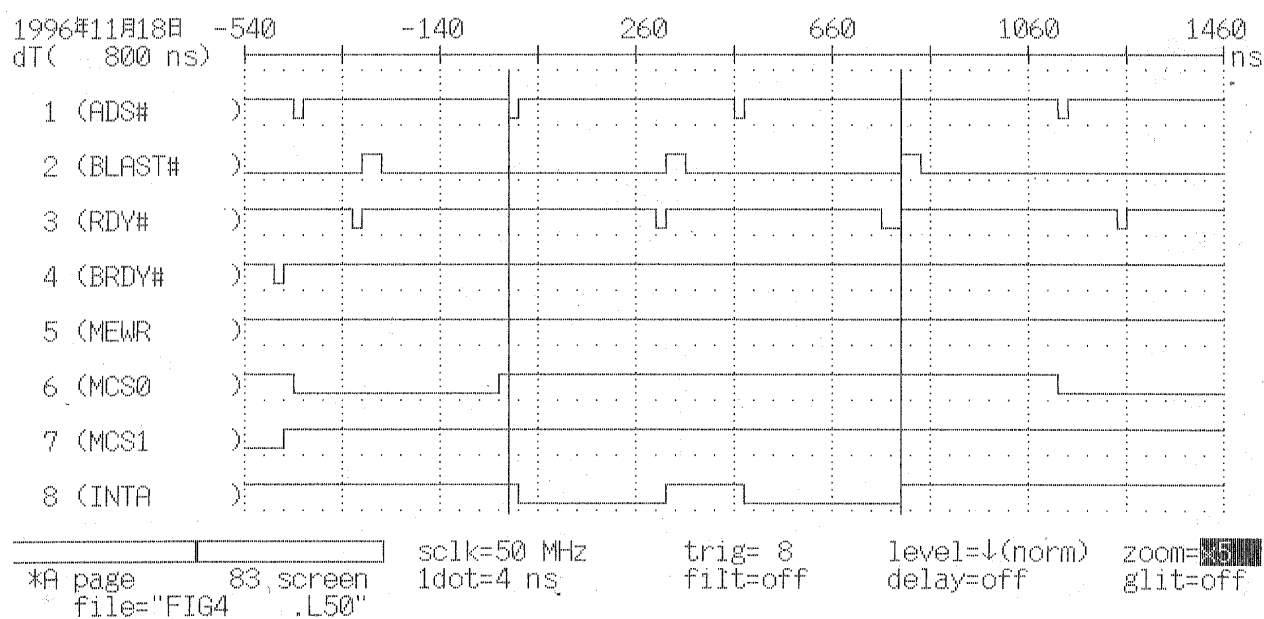


図 5.17 リセット・スタート

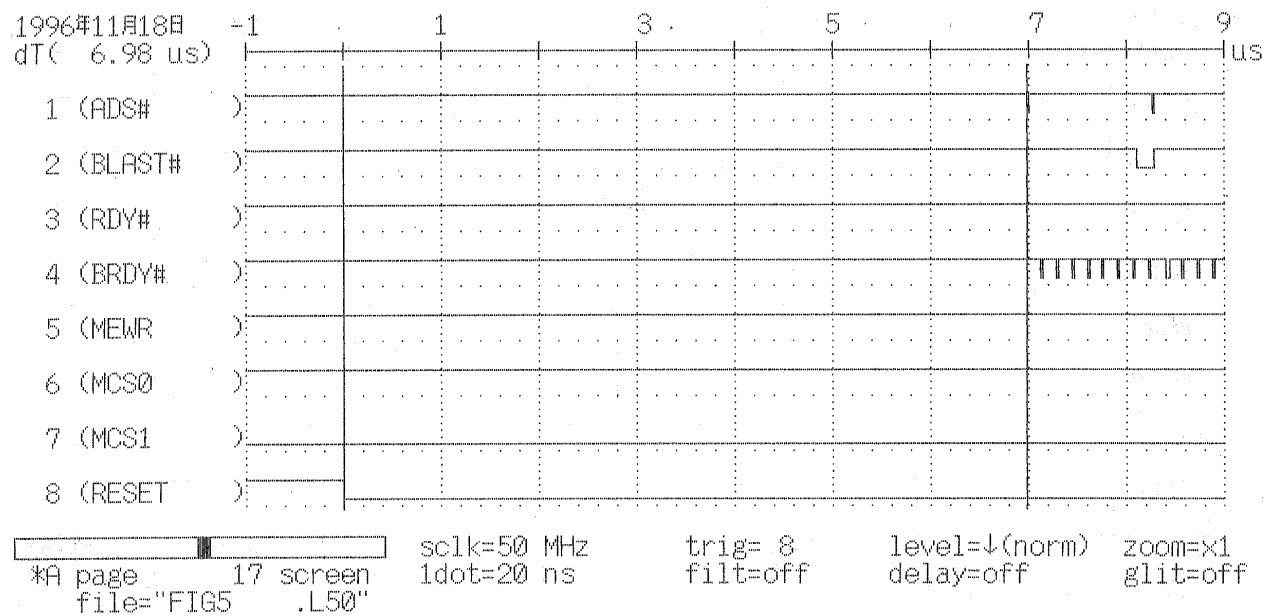


図 5.18 I/O 読み出し

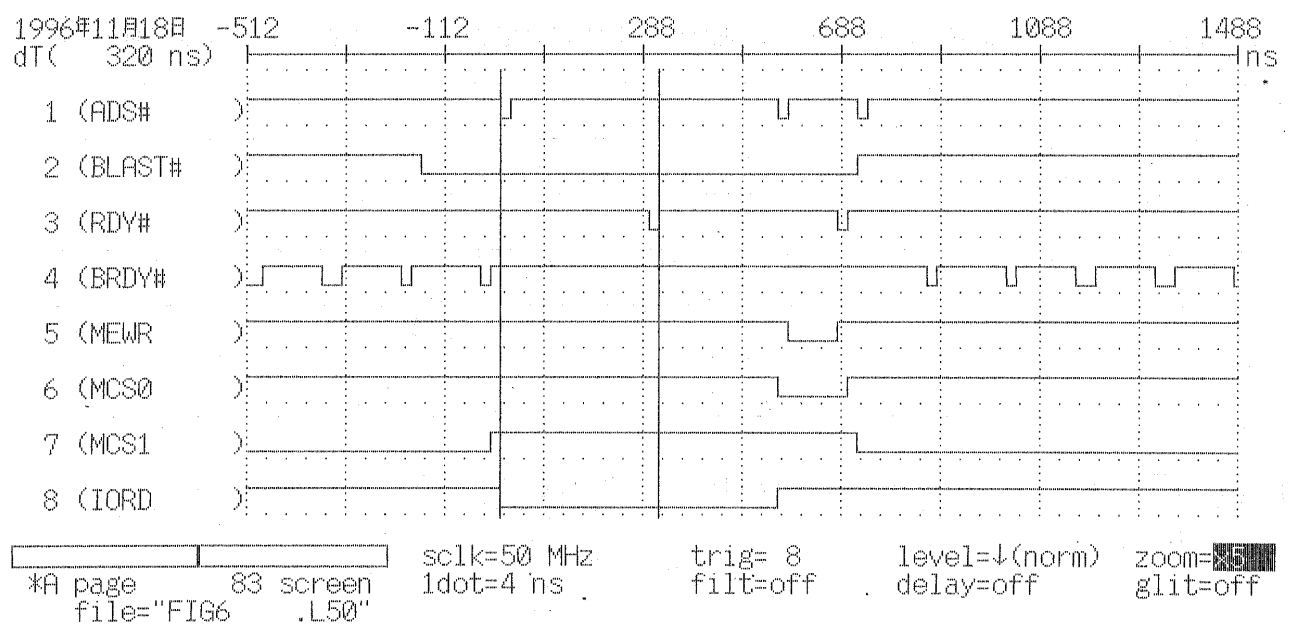


図 5.19 I/O 書き込み

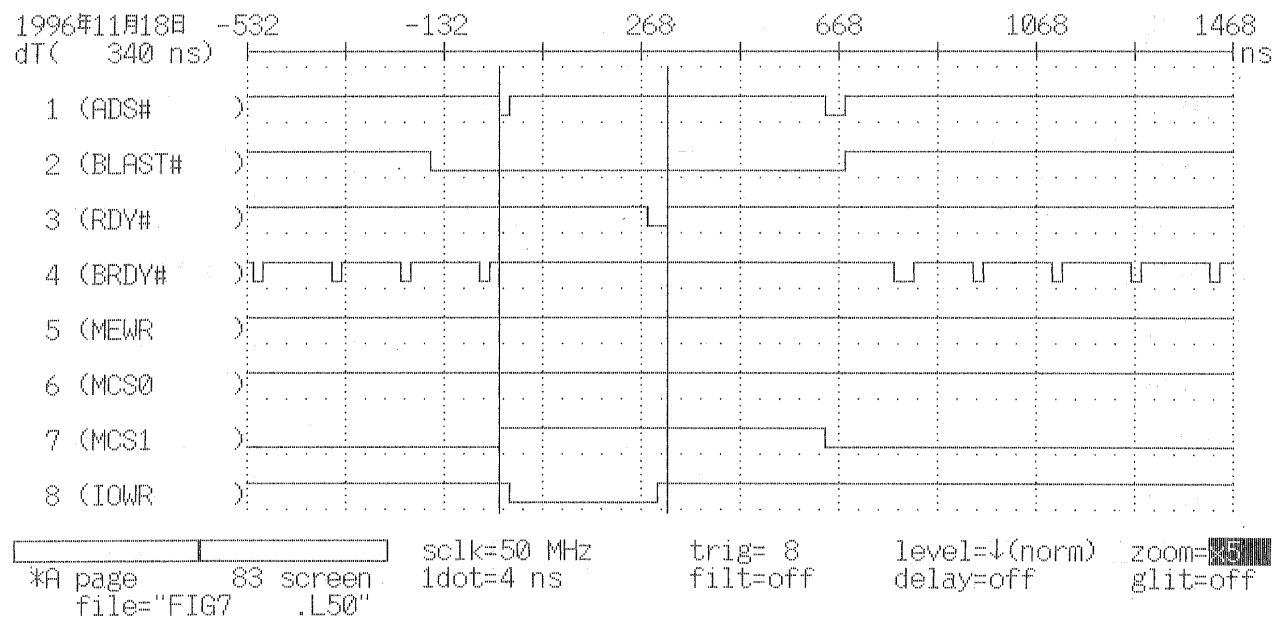


図 5.20 バス・サイクル検出

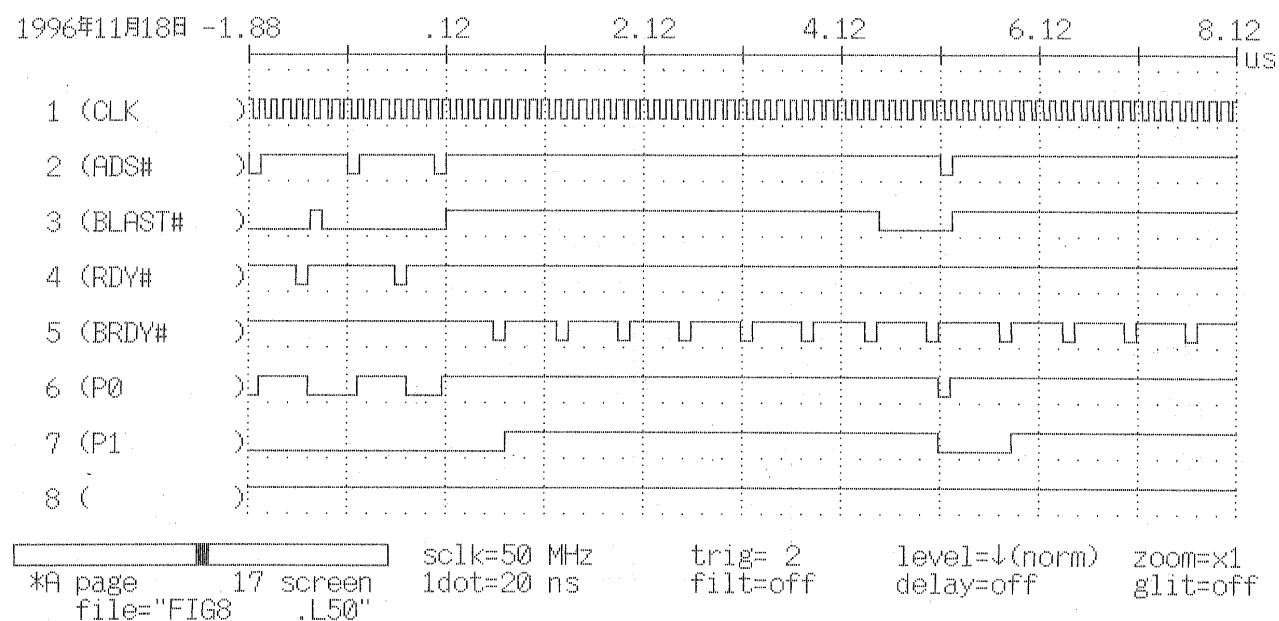


図 5.21 ウェイト・カウンタ

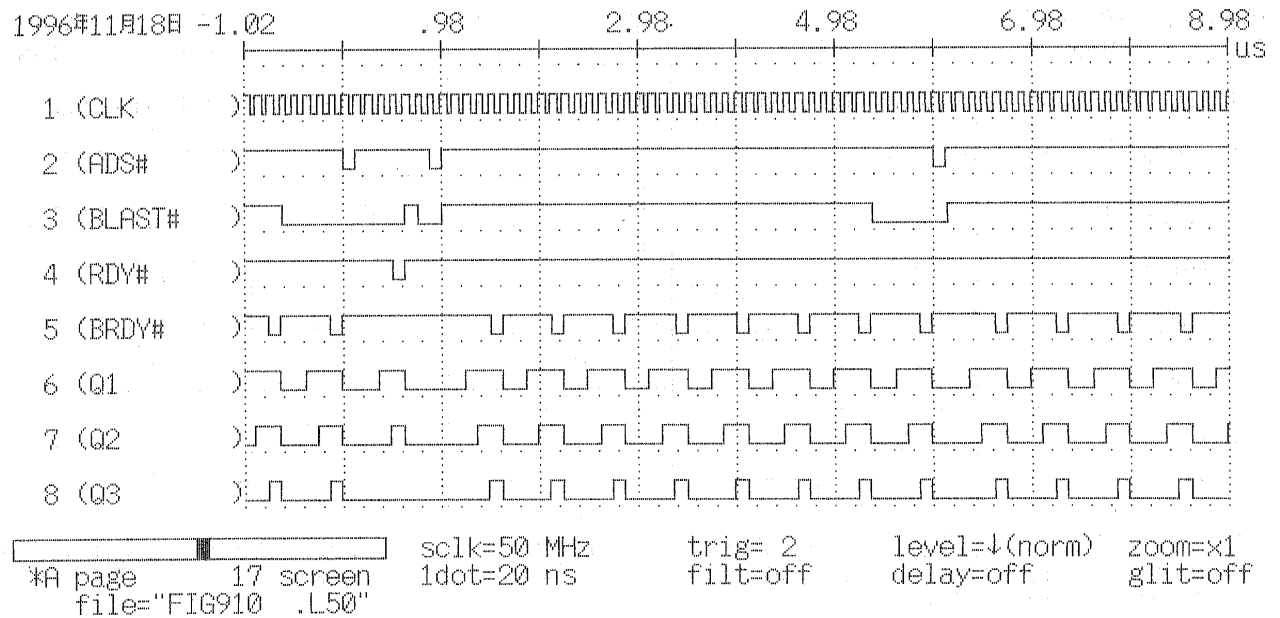


図 5.22 ウェイト・カウンタ (拡大)

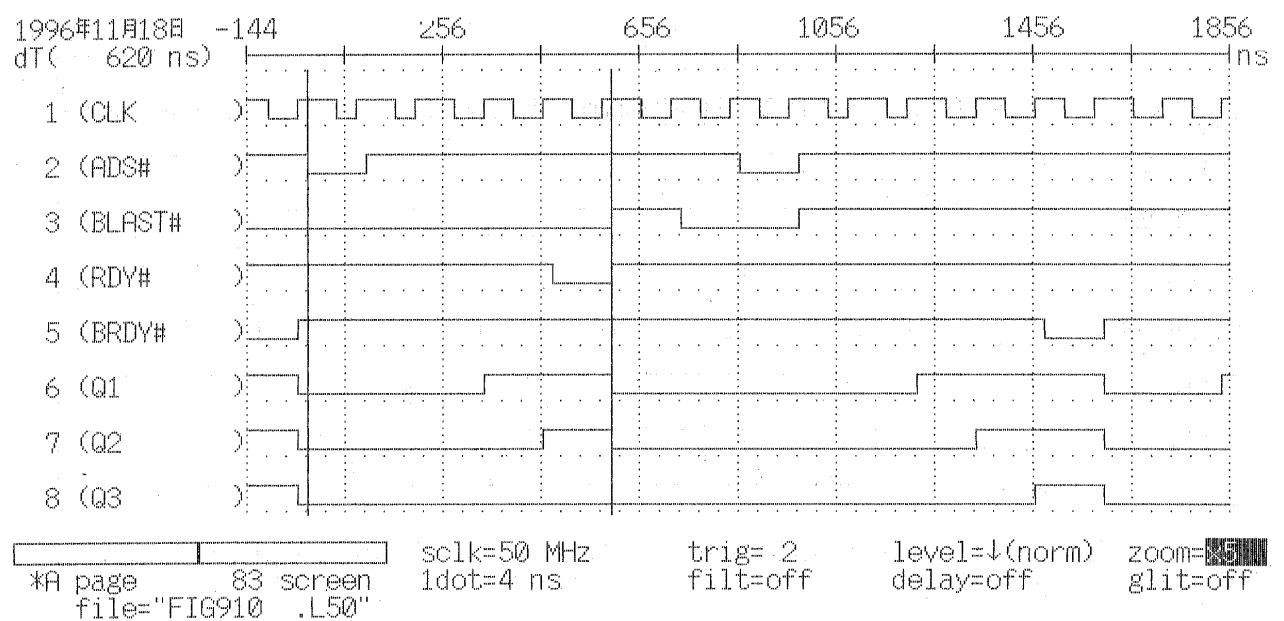


図 5.23 バス・サイクル進行

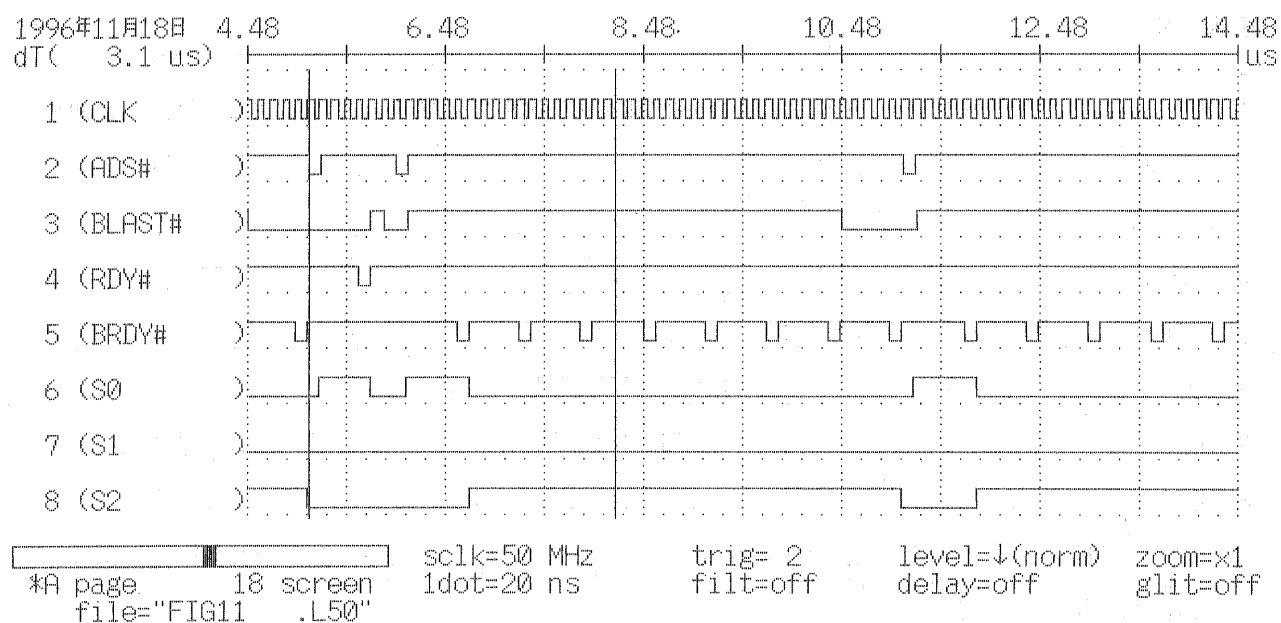
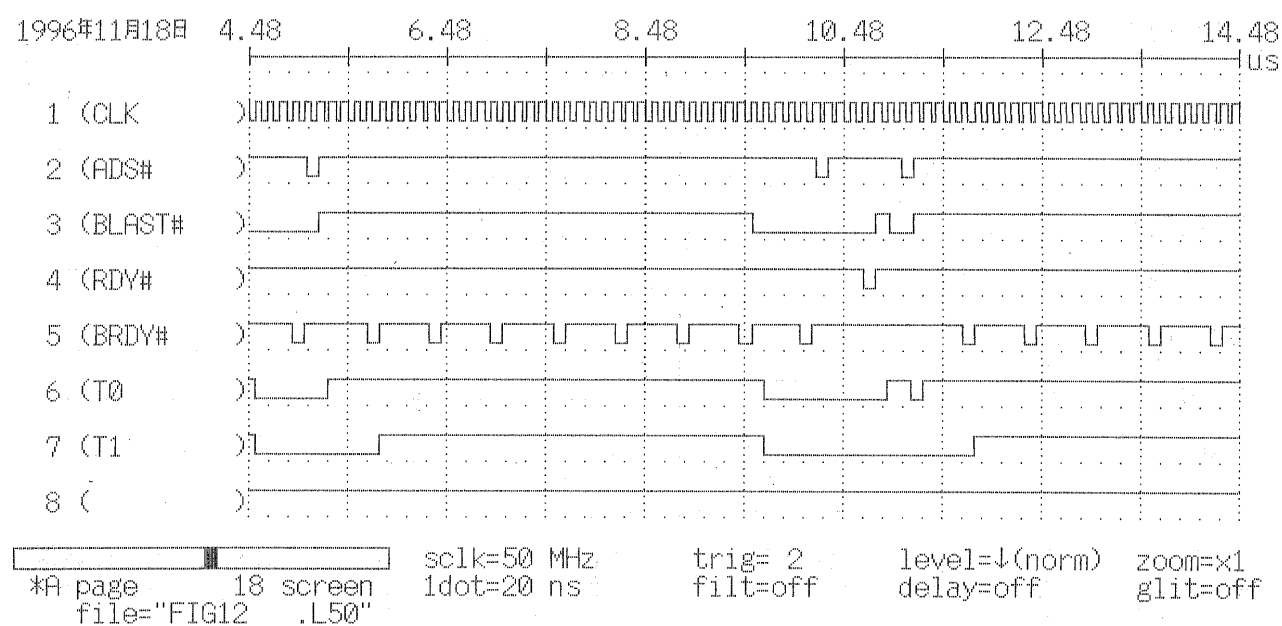


図 5.24 キャッシュ・ラインフィル検出



第 III 部

ソフトウェア

第 6 章

命令

6.1 整数命令

6.1.1 データ転送命令

汎用データ転送命令

- MOV 命令

記述形式 MOV opd,ops

最も基本的なデータ転送命令です、アドレッシング・モードで指定する、組み合わせ以外にセグメント・レジスタやシステム専用レジスタにデータを転送する機能を持っています。

- CMOVcc 命令

記述形式 CMOVcc opd,ops

条件付きのデータ転送命令です、EFLAGS レジスタのフラグをチェックして、条件が満たされれば命令は実行され、満たされなければ次の命令に移ります。

この命令は PentiumPro からサポートされた命令ですが、使用する場合は CPUID 命令でサポートを確認して下さい。

条件により表 6.1 の種類があります。

表 6.1 CMOVcc 命令

命令 (cc)	条件フラグ
A/NBE	(CF and ZF)=0
AE/NB	CF=0
NC	CF=0
B/NAE	CF=1
C	CF=1
BE/NA	(CF or ZF)=1
E/Z	ZF=1
NE/NZ	ZF=0
P/PE	PF=1
NP/PO	PF=0
GE/NL	SF = OF
L/NGE	SF <> OF
LE/NG	ZF=1 or SF <> OF
O	OF=1
NO	OF=0
S	SF=1
NS	SF=0

交換命令

- XCHG 命令

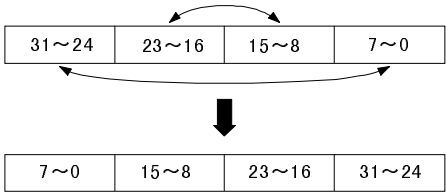
記述形式 XCHG op1,op2

2 つのオペランドの内容を交換します、オペランドがメモリを指す場合は LOCK 信号がアサートされます。

- BSWAP 命令

記述形式 BSWAP op1,op2

32 ビット・レジスタの内容をバイト単位で交換してリトル・エンディアンからビッグ・エンディアンのデータ形式に交互に変換する機能を持っています。



この命令は 80486 からサポートされています。

- XADD 命令

記述形式 XADD opd,ops

2 つのオペランドの内容を交換し、その後に加算した結果をデスティネーション・オペランド

に格納します。

LOCK プリフィックスが適用出来ます。

ソース・オペランドは必ずレジスタです。

この命令は 80486 からサポートされています。

● CMPXCHG 命令

記述形式 **CMPXCHG** *opd,ops*

マルチ・プロセッサ・システムのセマフォのテストや変更に用いられる命令です。

詳細はインテル・マニュアルの命令セット・リファレンスを参照して下さい。

● CMPXCHG8B 命令

記述形式 **CMPXCHG8** *op*

CMPXCHG 命令と同じです。

スタック操作命令

● PUSH 命令

記述形式 **PUSH** *op*

スタック・ポインタをデクリメントしてから、オペランドをスタック・トップをコピーします。

● POP 命令

記述形式 **POP** *op*

スタック・トップのデータをオペランドの指定する場所にコピーして、スタック・ポインタをインクリメントします。

● PUSHA,PUSHAD 命令

記述形式 **PUSHA**

PUSHAD

8 個の汎用レジスタをスタックに格納する命令です、格納される順番は

EAX,ECX,EDX,EBX,ESP,EBP,ESI,EDI

又は

AX,CX,DX,BX,SP,BP,SI,DI

になります、ESP 又は SP は、この命令によるスタック操作が開始される前の値が格納されます。

PUSHA,PUSHAD は前が 16 ビット・レジスタを、後が 32 ビット・レジスタを格納する場合に使うニモニックですが、オペコードは同じです、格納されるレジスタのサイズはオペランド・サイズ属性*1によります。

● POPA,POPAD 命令

記述形式 **POPA**

POPAD

PUSHA 命令の逆の操作を行います。

スタック中の ESP 又は SP 位置の値は ESP 又は SP レジスタにはロードされません。

● ENTER 命令

Make Stack Frame for Procedure Parameters

記述形式 **ENTER** *op1,op2*

ブロック構造言語に対して、スタック・フレームの作成をサポートします。

op1 はスタック・フレームのサイズを指定します、*op2* はレキシカル・ネスティング・レベルを指定します。

命令の説明の前に、この命令の前提としている考え方を考察してみます。

PL/M 言語による例としてリスト 17 を参照して下さい。

リスト17 TEST.SRC

```
TEST: DO;
DECLARE ANS BYTE;

CONOUT:
PROCEDURE(PRM)EXTERNAL;
    DECLARE PRM BYTE;
END CONOUT;

/* ===== MAIN ===== */
ANS = 30H;

MAIN:DO; /* LEVEL.1 */
    DECLARE X1 BYTE;
    X1 = 1;

    A:DO; /* LEVEL.2 */
        DECLARE X2 BYTE;
        X2 = 2;

        B:DO; /* LEVEL.3 */
            DECLARE X3 BYTE;
            X3 = 3;

            C:DO; /* LEVEL.4 */
                DECLARE X4 BYTE;
                X4 = 4;

                ANS = ANS+X1+X2+X3+X4;
            END;
        END;
    END;
END;

/* (NG) X4 = 1; */
CALL CONOUT(ANS);
END TEST;
```

これは MAIN,A,B,C と 4 レベルまで、ブロッ

*1 セグメント・ディスクリプタの D ビット

ク構造が多重化されています、この深さがレキシカル・ネスティング・レベルです。

ENTER 命令は、レキシカル・ネスティング・レベルの設定によって上位のブロックで定義している、スタック・フレームのポインタを下位のブロックのスタック・フレームにコピーしています。

これで下位のブロックは、上位のブロックで定義された変数を参照することが出来ます。

又、上位のブロックは、下位のブロックの変数を参照することは出来ません、例を示します。リスト中の (NG) の行は、X4 が参照出来ない、コメントを外すと、コンパイラがエラーを報告します。

これらの仕様を **op2** の設定を操作することで実現します。

リスト 18 では次のように ENTER 命令が使われます。

ブロック C まで、プログラムが実行されたときは、**図 6.1** のようなスタック・フレームになっています。

リスト18 ENTER 使用例

```

MAIN:      LEVEL.1
          ENTER  frame_size_A,1
          CALL   A
          LEAVE
          RET

A:         LEVEL.2
          ENTER  frame_size_B,2
          CALL   B
          LEAVE
          RET

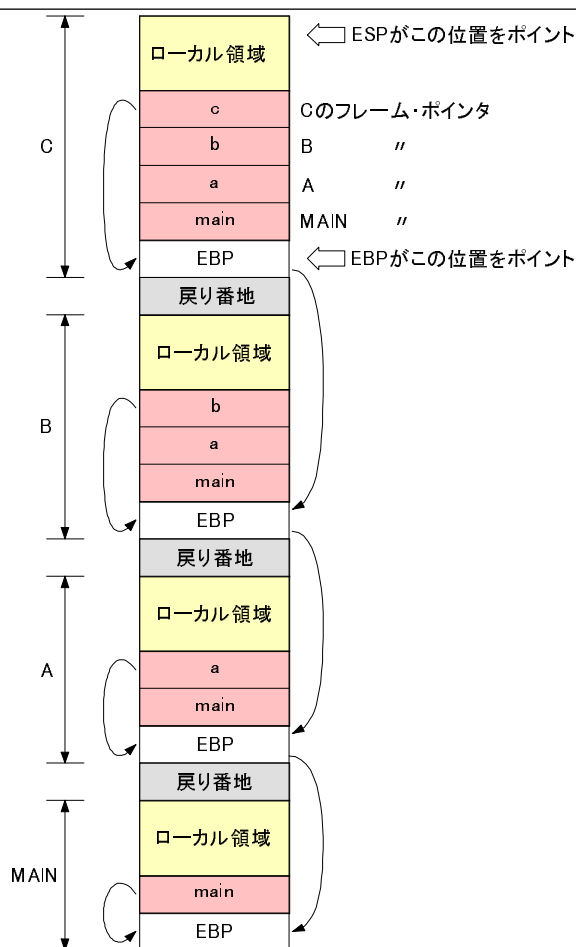
B:         LEVEL.3
          ENTER  frame_size_C,3
          CALL   C
          LEAVE
          RET

C:         LEVEL.4
          ENTER  frame_size_C,4
          .
          .
          LEAVE
          RET

```

ENTER 命令の動作の順序を次に示します。

1. EBP をスタック・ポインタにプッシュします。



2. レキシカル・ネスティング・レベルが 0 より大きいなら下記の順序で操作を実施します。

(このとき EBP はまだ、前ブロックのスタック・フレームを指しています)

(a) EBP の指すスタック・フレームの 2 番目の値から (**op2** -1) の個数を現スタックにコピーします。

(b) EBP をスタックにプッシュした後の、ESP の値をスタックにプッシュします。

3. EBP をスタックにプッシュした後の ESP の値を EBP にロードします。

(現ブロックのスタック・フレーム・ポインタ)

4. ESP から **op1** を減算してローカル領域を確保します。

ESP と EBP やスタック操作サイズはオペランド・サイズ属性によります。

- **LEAVE 命令**

High Level Procedure Exit

記述形式 **LEAVE**

ENTER 命令によって作成された、スタック・フレームを開放します。

MOV ESP, EBP

POP EBP

の命令と同じ操作を実行します。

ESP と EBP やスタック操作サイズはオペランド・サイズ属性によります。

6.1.2 論理命令

論理演算命令

- **AND 命令**

記述形式 **AND opd,ops**

デスティネーション・オペランドとソース・オペランドの AND 演算を行い、結果をデスティネーション・オペランドに格納します。

- **OR 命令**

記述形式 **OR opd,ops**

デスティネーション・オペランドとソース・オペランドの OR 演算を行い、結果をデスティネーション・オペランドに格納します。

- **XOR 命令**

Exclusive or

記述形式 **AND opd,ops**

デスティネーション・オペランドとソース・オペランドの XOR 演算を行い、結果をデスティネーション・オペランドに格納します。

- **NOT 命令**

記述形式 **NOT op**

オペランドのビットを反転します。

ビット移動命令

- **SAR 命令**

Shift arithmetic right

記述形式 **SAR opd,opc**

デスティネーション・オペランドをカウント・オペランドで指定したビット数だけ右にシフトします。

この命令は最上位ビット (符号) を維持します。



カウント・オペランドは即値かレジスタ CL ですが、カウント範囲は 0~31 までです。

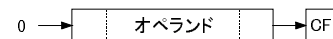
- **SHR 命令**

Shift logical right

記述形式 **SHR opd,opc**

デスティネーション・オペランドをカウント・オペランドで指定したビット数だけ右にシフトします。

最上位ビットには 0 が入ります。



カウント・オペランドは即値かレジスタ CL ですが、カウント範囲は 0~31 までです。

- **SAL/SHL 命令**

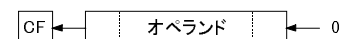
Shift arithmetic left/Shift logical left

記述形式 **SAL opd,opc**

SHL opd,opc

デスティネーション・オペランドをカウント・オペランドで指定したビット数だけ左にシフトします。

最下位ビットには 0 が入ります。



カウント・オペランドは即値かレジスタ CL ですが、カウント範囲は 0~31 までです。

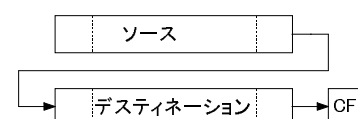
- **SHRD 命令**

Shift right double

記述形式 **SHRD opd,ops,opc**

ソース・オペランドを上位に、デスティネーション・オペランドを下位に連結して、カウント・オペランドで指定したビット数だけ右にシフトします。

ソース・オペランドの最上位ビットに入る値は不定です。



カウント・オペランドは即値かレジスタ CL ですが、カウント範囲は 0~31 までです。

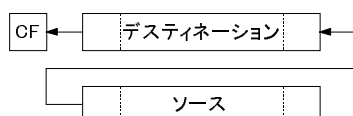
• SHLD 命令

Shift left double

記述形式 **SHLD** *opd,ops,opc*

デスティネーション・オペランドを上位に、ソース・オペランドを下位に連結して、カウント・オペランドで指定したビット数だけ左にシフトします。

ソース・オペランドの最下位ビットに入る値は不定です。



カウント・オペランドは即値かレジスタ CL ですが、カウント範囲は 0~31 までです。

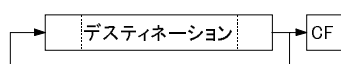
• ROR 命令

Rotate right

記述形式 **ROR** *opd,opc*

デスティネーション・オペランドをカウント・オペランドで指定したビット数だけ右に循環します。

デスティネーション・オペランドの最下位をでたビットは CF にも入ります。



カウント・オペランドは即値かレジスタ CL ですが、カウント範囲は 0~31 までです。

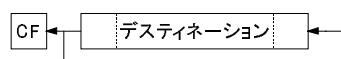
• ROL 命令

Rotate left

記述形式 **ROL** *opd,opc*

デスティネーション・オペランドをカウント・オペランドで指定したビット数だけ左に循環します。

デスティネーション・オペランドの最上位をでたビットは CF にも入ります。



カウント・オペランドは即値かレジスタ CL ですが、カウント範囲は 0~31 までです。

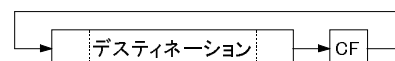
• RCR 命令

Rotate through carry right

記述形式 **RCR** *opd,opc*

デスティネーション・オペランドをカウント・オペランドで指定したビット数だけ右に循環します。

デスティネーション・オペランドの最下位ビットは CF に入り、CF のビットはデスティネーション・オペランドの最上位に入ります。



カウント・オペランドは即値かレジスタ CL ですが、カウント範囲は 0~31 までです。

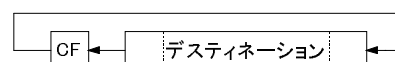
• RCL 命令

Rotate through carry left

記述形式 **RCL** *opd,opc*

デスティネーション・オペランドをカウント・オペランドで指定したビット数だけ左に循環します。

デスティネーション・オペランドの最上位ビットは CF に入り、CF のビットはデスティネーション・オペランドの最下位に入ります。



カウント・オペランドは即値かレジスタ CL ですが、カウント範囲は 0~31 までです。

6.1.3 算術演算命令

加減算命令

• ADD 命令

記述形式 **ADD** *opd,opc*

2 つのオペランドを加算して結果をデスティネーション・オペランドに格納します。

• ADC 命令

記述形式 **ADC** *opd,opc*

2 つのオペランドと CF を加算して結果をデスティネーション・オペランドに格納します。

この命令はキャリを考慮して計算する場合があります。

• SUB 命令

記述形式 **SUB** *opd,opc*

デスティネーション・オペランドからソース・オペランドを減算して結果をデスティネーション・オペランドに格納します。

ン・オペランドに格納します。

- **SBB 命令**

記述形式 **SBB** *opd,opc*

デスティネーション・オペランドからソース・オペランドと CF の和を減算して結果をデスティネーション・オペランドに格納します。

この命令はボローを考慮して計算する場合に用います。

インクリメント/デクリメント命令

- **INC 命令**

記述形式 **INC** *op*

オペランドに 1 を加算します、この命令では CF は影響を受けません。

- **DEC 命令**

記述形式 **DEC** *op*

オペランドから 1 を減算します、この命令では CF は影響を受けません。

乗算/除算命令

- **MUL 命令**

記述形式 **MUL** *ops*

2 つのオペランドを乗算して結果をデスティネーション・オペランドに格納します。

デスティネーション・オペランドはサイズにより **下表** のように暗黙的に決められています。

サイズ	オペランド <i>opd</i>	積
バイト	AL	AX
ワード	AX	DX:AX
ダブルワード	EAX	EDX:EAX

- **IMUL 命令**

記述形式 ① **IMUL** *ops*

② **IMUL** *opd,ops*

③ **IMUL** *opd,ops1,ops2*

2 つの符号付きオペランドを乗算して結果をデスティネーション・オペランドに格納します。

1. 記述形式 ① のオペランドは MUL 命令と同じです。

$opd \leftarrow opd \times ops$

2. 記述形式 ② はデスティネーション・オペランドを汎用レジスタで指定出来ます。

$opd \leftarrow opd \times ops$

3. 記述形式 ③ は 2 個のソース・オペランドを乗算します、2 番目のソース・オペランド *ops2* には即値を指定します。

$opd \leftarrow ops1 \times ops2$

- **DIV 命令**

記述形式 **DIV** *ops*

暗黙的に決められたレジスタの被除数の値を、ソース・オペランドの除数で除算して、結果を暗黙的に決められたレジスタに格納します。

サイズによって **下表** のようになっています。

サイズ	被除数	商	余り
バイト	AX	AL	AH
ワード	DX:AX	AX	DX
ダブルワード	EDX:EAX	EAX	EDX

- **IDIV 命令**

記述形式 **IDIV** *ops*

演算に使う値を符号付きとして、暗黙的に決められたレジスタの被除数の値を、ソース・オペランドの除数で除算して、結果を暗黙的に決められたレジスタに格納します。

DIV 命令の **表** を参照して下さい。

6.1.4 10 進算術命令

- **DAA 命令**

Decimal Adjust AL after addition

記述形式 **DAA**

ADD 命令による、2 つのパック形式 BCD の値の加算結果を調整して、適正な 2 桁のパック形式 BCD の結果にします。

この命令は次の条件の演算後に有効です。

ADD AL,ops

デスティネーション・レジスタには AL を指定して、演算結果が AL に格納されるようにします。

演算データは 2 桁のパック形式 BCD 値です。

- **DAS 命令**

Decimal Adjust AL after Subtraction

記述形式 **DAS**

SUB 命令による、2 つのパック形式 BCD の値の減算結果を調整して、適正な 2 桁のパック形

式 BCD の結果にします。

この命令は次の条件の演算後に有効です。

SUB AL,ops

デスティネーション・レジスタには AL を指定して、演算結果が AL に格納されるようにします。

演算データは 2 桁のパック形式 BCD 値です。

• AAA 命令

ASCII Adjust AL After Addition

記述形式 AAA

ADD 命令による、2 つのアンパック形式 BCD の値の加算結果を調整して、適正なアンパック形式 BCD の結果にします。

10 進キャリーが発生した場合は、CF と AF がセットされて AH がインクリメントされます、発生しない場合は CF と AF がクリアされます。この命令は次の条件の演算後に有効です。

ADD AL,ops

デスティネーション・レジスタには AL を指定して、演算結果が AL に格納されるようにします。

演算データはアンパック形式 BCD 値です。

• AAS 命令

ASCII Adjust AL After Subtraction

記述形式 AAS

SUB 命令による、2 つのアンパック形式 BCD の値の減算結果を調整して、適正なアンパック形式 BCD の結果にします。

10 進ボローが発生した場合は、CF と AF がセットされて AH がデクリメントされます、発生しない場合は CF と AF がクリアされます。この命令は次の条件の演算後に有効です。

SUB AL,ops

デスティネーション・レジスタには AL を指定して、演算結果が AL に格納されるようにします。

演算データはアンパック形式 BCD 値です。

• AAM 命令

ASCII Adjust AX After Multiply

記述形式 AAM

MUL 命令による、2 つのアンパック形式 BCD の値の乗算結果を調整して、適正な 2 桁のアン

パック形式 BCD の結果にします。

この命令は次の条件の演算後に有効です。

MUL ops

バイト・サイズの乗算を指定して、演算結果が AX に格納されるようにします。

演算データはアンパック形式 BCD 値です。

• AAD 命令

ASCII Adjust AX Before Division

記述形式 AAD

AX レジスタの 2 桁のアンパック形式 BCD の値を、バイナリ値に変換して AL レジスタに格納します、AH レジスタはクリアされます。

この命令は次の条件の演算前に有効です。

DIV ops

バイト・サイズの除算を指定して、演算結果が AL に格納されるようにします。

演算結果はアンパック形式 BCD 値が得られます。

6.1.5 比較命令

CMP 命令

Compare Two Operands

記述形式 CMP op1,op2

ソース・オペランド 1 からソース・オペランド 2 を減算して、フラグを設定します。

6.1.6 分岐命令

JMP や CALL 命令に関してはプロテクトモードの項も参照して下さい。

無条件分岐命令

• JMP 命令

記述形式 JMP op

オペランドの指定する位置に、命令の実行位置を移します。

オペランドには次の 2 種類があります。

1. near ジャンプ

EIP のみを対象とした、コード・セグメント内を範囲とする分岐に使います。

オペランドの指定方法は次の 2 種類があります。

(a) 絶対オフセット*2

*2 コード・セグメント・ベースからのオフセット

絶対オフセットの置かれたレジスタ又はメモリを間接的に指定したり、オフセットを直接指定したりします。

(b) 相対オフセット

EIP を基準にした、相対的な位置を符号付きの値でオペランドに指定します。

2. far ジャンプ

EIP と CS を対象とした、コード・セグメントを変更して異なるセグメントへの分岐に使います。

通常の方岐以外にリアル/プロテクトのモード変更や、タスク・スイッチなどを行うときにも使います。

オペランドには CS:EIP を直接指定する場合と、CS:EIP の値を置いたメモリで間接的に指定する場合があります。

最終的な EIP の値はオペランド・サイズ属性の制限を受けます。

条件分岐命令

• Jcc 命令

記述形式 Jcc op

フラグの条件を満足した場合にジャンプします。

条件により表 6.2 の種類があります。

条件の意味の less 及び greater は符号付き整数の比較に、above 及び below は、符号なし整数に使用します。

オペランドは near ジャンプの相対オフセットを指定します。

プロシジャ命令

• CALL 命令

記述形式 CALL op

次の命令の実行位置をスタックにプッシュした後、オペランドの指定する位置に、命令の実行位置を移します。

オペランドの形式は JMP 命令と同じですが、8 ビットの相対オフセットは指定出来ません。スタックの内容はコールの方法により次のようになります。

1. near コールの場合はスタックに EIP をプッシュします。
2. far コールの場合はスタックに CS と EIP

をプッシュします。

スタックに保存される EIP のサイズはオペランド・サイズ属性の影響を受けます。

• RET 命令

記述形式 RET

RET op

CALL 命令がスタックにプッシュした情報をポップして、実行位置を移します。

オペランド付きの RET 命令は、CALL 命令の前にスタックに積まれたパラメータ領域を解放することが出来ます。

near コールには near リターン、far コールには far リターンを使います。

6.1.7 ビット操作命令

• BT

Bit test

記述形式 BT op1,op2

ビット列から、ビット・オフセットで指定したビットの値を CF に格納します。

ビット列は op1 でレジスタかメモリを指定します。

ビット・オフセットは op2 で 0~31 の即値か、符号付きの 32 ビット値で指定出来ます。

• BTS

Bit test and set

記述形式 BTS op1,op2

ビット列から、ビット・オフセットで指定したビットの値を CF に格納して、その後にビットを 1 にします。

オペランドの形式は BT 命令を参照して下さい。

• BTR

Bit test and reset

記述形式 BTR op1,op2

ビット列から、ビット・オフセットで指定したビットの値を CF に格納して、その後にビットを 0 にします。

オペランドの形式は BT 命令を参照して下さい。

• BTC

Bit test and complement

表 6.2 Jcc, SETcc 命令

命令 (cc)	条件フラグ	意味	説明
A	CF=0 and ZF=0	above	符号なし演算の結果が正のとき
AE	CF=0	above or equal	符号なし演算の結果が 0 以上のとき
B	CF=1	below	符号なし演算の結果が負のとき
BE	CF=1 and ZF=1	below or equal	符号なし演算の結果が 0 以下のとき
C	CF=1	carry	符号なし演算でオーバー・フローがあったとき
CXZ ^a	–	CX レジスタ=0	
ECXZ	–	ECX レジスタ=0	
E	ZF=1	equal	演算結果が 0 のとき
G	ZF=0 and SF=OF	greater	符号付き演算の結果が正のとき
GE	SF=OF	greater or equal	符号付き演算の結果が 0 以上のとき
L	SF<>OF	less	符号付き演算の結果が負のとき
LE	ZF=1 or SF<>OF	less or equal	符号付き演算の結果が 0 以下のとき
NA	CF=1 or ZF=1	not above	符号なし演算の結果が 0 以下のとき
NAE	CF=1	above or equal	符号なし演算の結果が負のとき
NB	CF=0	not below	符号なし演算の結果が 0 以上のとき
NBE	CF=0 and ZF=0	below or equal	符号なし演算の結果が正のとき
NC	CF=0	not carry	符号なし演算でオーバー・フローがないとき
NE	ZF=0	not equal	演算結果が 0 でないとき
NG	ZF=1 or SF<>OF	not greater	符号付き演算の結果が 0 以下のとき
NGE	SF<>OF	not greater or equal	符号付き演算の結果が負のとき
NL	SF=OF	not less	符号付き演算の結果が 0 以上のとき
NLE	ZF=0 and SF=OF	not less or equal	符号付き演算の結果が正のとき
NO	OF=0	not overflow	符号付き演算でオーバー・フローがなかったとき
NP	PF=0	not parity	下位バイトに 1 が奇数個あるとき
NS	SF=0	not sign	符号付き演算の結果が 0 から 127 のとき
NZ	ZF=0	not zero	演算結果が 0 でないとき
O	OF=1	overflow	符号付き演算でオーバー・フローがあったとき
P	PF=1	parity	下位バイトに 1 が偶数個あるとき
PE	PF=1	parity even	下位バイトに 1 が偶数個あるとき
PO	PF=0	parity odd	下位バイトに 1 が奇数個あるとき
S	SF=1	sign	符号付き演算の結果が -1 から -128 のとき
Z	ZF=1	zero	演算結果が 0 のとき

^a CXZ と ECXZ は Jcc 命令のみに適用されます。

記述形式 **BTC op1,op2**

ビット列から、ビット・オフセットで指定したビットの値を CF に格納して、その後にビットの補数をとります。

オペランドの形式は BT 命令を参照して下さい。

- **BSF**

Bit scan forward

記述形式 **BSF opd,ops**

ソース・オペランドのレジスタ又はメモリの、下位ビットから上位方向に捜して最初の 1 になるビット位置を、デスティネーション・オペランドに格納します。

- **BSR**

Bit scan reverse

記述形式 **BSR opd,ops**

ソース・オペランドのレジスタ又はメモリの、上位ビットから下位方向に捜して最初の 1 になるビット位置を、デスティネーション・オペランドに格納します。

- **SETcc**

Set Byte on Condition

記述形式 **SETcc op**

フラグの条件を満足した場合にオペランドを 1 にします、満足しない場合は 0 にします。

オペランドは 8 ビット・レジスタかバイトのメモリです。

条件により表 6.2 の種類があります。

- **TEST**

Logical Compare

記述形式 **TEST op1,op2**

2 つのビットの論理積をとり、フラグを設定します。

SF,ZF,PF が演算結果により設定されます。

6.1.8 スtring命令

- **MOVS**

Move Data from String to String

記述形式 バイト **MOVS ES:(E)DI, DS:(E)SI**

ワード **MOVS ES:DI, DS:SI**

ダブルワード **MOVS ES:EDI, DS:ESI**

ソース・オペランドからデスティネーション・

オペランドにデータを転送します。

デスティネーション・オペランドのセグメント・レジスタは ES が固定です。

ソース・オペランドのセグメント・レジスタは DS 以外も使えます。

インデックス・レジスタが EDI になるか DI になるか、または ESI になるか SI になるかは、オペランド・サイズ属性によります。

ワード転送とダブル・ワード転送は同じオペコードです、どちらになるかはオペランド・サイズ属性によります。

命令の実行後に EDI,ESI または DI,SI は、インクリメントかデクリメントされますが、EFLAGS レジスタの DF フラグによります。

DF=0 インクリメント

DF=1 デクリメント

サイズによってバイト が 1、ワードが 2、ダブル・ワードが 4 バイト単位で増減します。

この命令は REP プリフィックスを用いてブロック転送にすることが出来ます。

- **MOVSB/MOVSW/MOVS**

Move Data from String to String

記述形式 バイト **MOVSB**

ワード **MOVSW**

ダブルワード **MOVS**

MOVS 命令の短縮形でオペランドはありませんが MOVS と同じです。

- **CMPS**

Compare String Operand

記述形式 バイト **CMPS DS:(E)SI,ES:(E)DI**

ワード **CMPS DS:SI,ES:DI**

ダブルワード **CMPS DS:ESI,ES:EDI**

第 1 オペランドから第 2 オペランドを減算して、その結果によりフラグを設定します。

インデックス・レジスタが EDI になるか DI になるか、または ESI になるか SI になるかは、オペランド・サイズ属性によります。

ワード比較とダブル・ワード比較は同じオペコードです、どちらになるかはオペランド・サイズ属性によります。

命令の実行後に EDI,ESI または DI,SI は、インクリメントかデクリメントされますが、

EFLAGS レジスタの DF フラグによります。

DF=0 インクリメント

DF=1 デクリメント

サイズによってバイトが1、ワードが2、ダブル・ワードが4バイト単位で増減します。

この命令は REP プリフィックスを用いてブロック転送にすることが出来ます。

- **CMPSB/CMPSW/CMPSD**

記述形式 バイト **CMPSB**

ワード **CMPSW**

ダブルワード **CMPSD**

CMPS 命令の短縮形でオペランドはありませんが CMPS と同じです。

- **SCAS**

Scan String Data

記述形式 バイト **SCAS ES:(E)DI**

ワード **SCAS ES:DI**

ダブルワード **SCAS ES:EDI**

レジスタからオペランドを減算して、その結果によりフラグを設定します。

レジスタはオペランドのサイズにより、

AL,AX,EAX が暗黙的に選択されます。

ワード比較とダブル・ワード比較は同じオペコードです、どちらになるかはオペランド・サイズ属性によります。

命令の実行後に EDI または DI は、インクリメントかデクリメントされますが、EFLAGS レジスタの DF フラグによります。

DF=0 インクリメント

DF=1 デクリメント

サイズによってバイトが1、ワードが2、ダブル・ワードが4バイト単位で増減します。

この命令は REP プリフィックスを用いてブロック転送にすることが出来ます。

- **SCASB/SCASW/SCASD**

記述形式 バイト **SCASB**

ワード **SCASW**

ダブルワード **SCASD**

SCAS の短縮形でオペランドはありませんが SCAS と同じです。

- **LODS**

Load String Operand

記述形式 バイト **LODS DS:(E)SI**

ワード **LODS DS:SI**

ダブルワード **LODS DS:ESI**

オペランドからレジスタにデータを転送します。

その他のオペランドについての詳細は SCAS 命令と同様です。

- **LODSB/LODSW/LODSD**

記述形式 バイト **LODSB**

ワード **LODSW**

ダブルワード **LODSD**

LODS の短縮形でオペランドはありませんが LODS と同じです。

- **STOS**

Store String Data

記述形式 バイト **STOS ES:(E)DI**

ワード **STOS ES:DI**

ダブルワード **STOS ES:EDI**

レジスタからオペランドにデータを転送します。

オペランドの詳細は SCAS 命令と同じです。

- **STOSB/STOSW/STOSD**

記述形式 バイト **STOSB**

ワード **STOSW**

ダブルワード **STOSD**

STOS の短縮形でオペランドはありませんが STOS と同じです。

- **INS**

Input from Port to String

記述形式 バイト **INS ES:(E)DI,DX**

ワード **INS ES:DI,DX**

ダブルワード **INS ES:EDI,DX**

レジスタ DX で指定する I/O ポートからデスティネーション・オペランドにデータを転送します。

ワード転送とダブル・ワード転送は同じオペコードです、どちらになるかはオペランド・サイズ属性によります。

命令の実行後に EDI または DI は、インクリメントかデクリメントされますが、EFLAGS レジスタの DF フラグによります。

DF=0 インクリメント

DF=1 デクリメント

サイズによってバイトが1、ワードが2、ダブル・ワードが4バイト単位で増減します。

この命令は REP プリフィックスを用いてブロック転送にすることが出来ます。

- **INSB/INSW/INSD**

記述形式 バイト **INSB**

ワード **INSW**

ダブルワード **INSD**

INS の短縮形でオペランドはありませんが INS と同じです。

- **OUTS**

Output String to Port

記述形式 バイト **OUTS DX,DS:(E)SI**

ワード **OUTS DX,DS:SI**

ダブルワード **OUTS DX,DS:ESI**

レジスタ DX で指定する I/O ポートにソース・オペランドからデータを転送します。

その他のオペランドについての詳細は INS 命令と同様です。

- **OUTSB/OUTSW/OUTSD**

記述形式 バイト **OUTSB**

ワード **OUTSW**

ダブルワード **OUTSD**

OUTS の短縮形でオペランドはありませんが OUTS と同じです。

- **REP/REPE/REPZ/REPNE/REPNZ**

Repeat while ECX not zero

Repeat while equal / Repeat while zero

Repeat while not equal / Repeat while not zero

記述形式 **REP INS op1,op2**

REP MOVS op1,op2

REP OUTS op1,op2

REP LODS op

REP STOS op

REPE CMPS op1,op2

REPE SCAS op

REPNE CMPS op1,op2

REPNE SCAS op

通常の命令ではなく、命令プリフィックスと言います、ストリング命令と組み合わせて使用されます。

ECX が 0 になるまで、繰り返し実行されます、ECX はストリング命令の実行後にデクリメントされます。

REPE/REPZ と REPNE/REPZ は、上記の条件に加えて ZF もチェックされます。

REPE と REPZ は同じ命令で、ZF=0 も終了条件になります。

REPNE と REPZ は同じ命令で、ZF=1 も終了条件になります。

6.1.9 ループ命令

- **LOOP**

Loop According to ECX Counter

記述形式 **LOOP op**

カウンタをデクリメントして、それが 0 でなければ分岐する、条件分岐命令です。

この命令は EFLAGS レジスタには影響しません。

オペランドには分岐先を 8 ビットの符号付きの即値で指定します。

カウンタは ECX,CX レジスタに決められています。

通常、次のような使い方になります。

```
MOV    ECX,count
L1:    .
      .
      LOOP  L1
```

ECX にループ・カウンタを設定して、L1 と LOOP 命令の間を設定回数だけ繰り返します。

ECX になるか CX になるかは、オペランド・サイズ属性によります。

- **LOOPZ/LOOPE**

記述形式 **LOOPZ op**

LOOPE op

LOOP 命令の条件に加えて、ZF が 1 であるかもチェックされます、どちらかの条件が成立すれば、オペランドには分岐せずに LOOPZ/LOOPE の次の命令を実行します。

この命令のカウント操作は ZF に影響しません。オペランドは LOOP 命令と同じです。

- **LOOPNZ/LOOPNE**

記述形式 **LOOPNE op**

LOOPNZ op

LOOP 命令の条件に加えて、ZF が 0 であるかもチェックされます、どちらかの条件が成立すれば、オペランドには分岐せずに LOOPNE/LOOPNZ の次の命令を実行します。この命令のカウント操作は ZF に影響しません。オペランドは LOOP 命令と同じです。

6.1.10 I/O 命令**• IN**

Input from Port

記述形式 **IN** opd,ops

ソース・オペランドの指定する I/O ポートからデスティネーション・オペランドにデータを転送します。

デスティネーション・オペランドはレジスタを使いますがサイズによって、暗黙的にバイトなら AL、ワードなら AX, ダブル・ワードなら EAX に決められています。

ワードになるかダブル・ワードになるかはオペランド・サイズ属性によります。

ソース・オペランドは 8 ビットの即値か DX レジスタで指定します。

• OUT

Output to Port

記述形式 **OUT** opd,ops

ソース・オペランドからデスティネーション・オペランドの指定する I/O ポートにデータを転送します。

ソース・オペランドはレジスタを使いますがサイズによって、暗黙的にバイトなら AL、ワードなら AX, ダブル・ワードなら EAX に決められています。

ワードになるかダブル・ワードになるかはオペランド・サイズ属性によります。

デスティネーション・オペランドは 8 ビットの即値か DX レジスタで指定します。

6.1.11 フラグ制御命令**• STC**

Set Carry Flag

記述形式 **STC**

EFLAGS レジスタの CF フラグを 1 にします。

• CLC

Clear Carry Flag

記述形式 **CLC**

EFLAGS レジスタの CF フラグを 0 にします。

• CMC

Complement Carry Flag

記述形式 **CMC**

EFLAGS レジスタの CF フラグの補数をとります。

• CLD

Clear Direction Flag

記述形式 **CLD**

EFLAGS レジスタの DF フラグを 0 にします。

• STD

Set Direction Flag

記述形式 **STD**

EFLAGS レジスタの DF フラグを 1 にします。

• LAHF

Load Status Flags into AH Register

記述形式 **LAHF**

EFLAGS レジスタの下位 8 ビットを AH レジスタに転送します。

• SAHF

Store AH info Flags

記述形式 **SAHF**

EFLAGS レジスタの SF,ZF,PF,CF フラグに AH レジスタからビット値を転送します。

それぞれのフラグは EFLAGS でのビット位置で AH レジスタに設定します、上記フラグ以外のビットは EFLAGS レジスタに転送されません。

• PUSHF/PUSHFD

Push EFLAGS Register onto the Stack

記述形式 **PUSHF**

PUSHFD

EFLAGS 又は FLAGS レジスタの値をスタックにプッシュします。

どちらになるかは、オペランド・サイズ属性によります。

• POPF/POPF

Pop Stack into EFLAGS Register

記述形式 **POPF**

POPFD

スタックをポップして EFLAGS 又は FLAGS レジスタに転送します。

どちらになるかは、オペランド・サイズ属性によります。

- **STI**

Set Interrupt Flag

記述形式 **STI**

EFLAGS レジスタの IF フラグを 1 にします。
マスク可能な外部割り込みをイネーブルにします。

- **CLI**

Clear Interrupt Flag

記述形式 **CLI**

EFLAGS レジスタの IF フラグを 0 にします。
マスク可能な外部割り込みをディセーブルにします。

6.1.12 割り込み関連の命令

- **INT**

記述形式 **INT op**

オペランドの指定する割り込み番号の割り込みをソフトウェアで発生させます。

オペランドは 8 ビットの即値です。

- **INTO**

記述形式 **INTO**

EFLAGS レジスタの OF フラグが 1 の場合に、割り込み番号 4 のオーバーフロー割り込みを発生します。

- **INT3**

記述形式 **INT3**

割り込み番号 3 のブレーク・ポイント割り込みを発生します。

- **BOUND**

Check Array Index Against Bounds

記述形式 **BOUND op1,op2**

配列のインデックスが配列の範囲を越えていないかどうかを、チェックします。

範囲内になければ、割り込み番号 5 を発生します。

op1 にインデックス、**op2** に配列の下限と上限を指定します。

op1 はレジスタ、**op2** はメモリを使います。

op2 はワードかダブル・ワードのペアで下限:上限の順番です。

インデックスはワードかダブル・ワードの符号付き整数です、インデックスと範囲の指定がワードとダブル・ワードのどちらになるかはオペランド・サイズ属性によります。

- **IRET/IRETD**

Interrupt Return

記述形式 **IRET**

IRETD

割り込みプログラムを終了して、元の実行位置に戻るために使います。

スタックから

EIP,CS,EFLAGS か IP,CS,FLAGS

に値が戻されます、どちらになるかはオペランド・サイズ属性によります。

6.1.13 その他の命令

- **LEA**

Load Effective Address

記述形式 **LEA opd,ops**

ソース・オペランドの実効アドレスをデスティネーション・オペランドに格納します。

ソース・オペランドはメモリを指定します、デスティネーション・オペランドにはレジスタを指定します。

実効アドレス:セグメント・ベースからのオフセット。

- **NOP**

No Operation

記述形式 **NOP**

インストラクション・ポインタを進める以外は何もしません、1 バイトのオペコードを持ちます。

- **UD2**

Underfined Instruction

記述形式 **UD2**

無効オペコード例外を発生させる命令です、2 バイトのオペコードを持ちます。

- **XLAT/XLATB**

Table Look-up Translation

記述形式 **XLAT op**

記述形式 **XLAT op**

- **CPUID**

CPU Identification

記述形式 **CPUID**

プロセッサの情報を、EAX,EBX,ECX,EDX に格納します。

情報は 3 種類に分けられ、EAX に 0,1,2 の何れかの値を設定して CPUID 命令を実行すると、それぞれの情報がレジスタに格納されます。

情報の詳細は省略します。

6.1.14 セグメント・レジスタ命令

- **LDS/LFS/LGS/LSS**

Load Far Pointer

記述形式 **LDS opd,ops**

LFS opd,ops

LGS opd,ops

LSS opd,ops

ソース・オペランドの far ポインタのセグメント・セクタをニモニックで指定したセグメント・レジスタに、オフセットをデスティネーション・オペランドのレジスタに格納します。far ポインタのサイズはオペランド・サイズ属性によります。

6.1.15 システム命令

- **LGDT**

Load Global Descriptor Table Register

記述形式 **LGDT op**

オペランドの内容を GDTR にロードします。

オペランドはメモリを指定しますが、オペランド・サイズ属性の影響を受けます。

16 ビット属性なら 32 ビット値がロードされ、32 ビット属性なら 48 ビット値がロードされます。

ロードされる値は上位にベース・アドレス、下位にリミットがあります、ベース・アドレスは 6 ビット属性なら 24 ビット、32 ビット属性なら 32 ビットです。

- **SGDT**

Store Global Descriptor Table Register

記述形式 **SGDT op**

GDTR の内容をオペランドに格納します。

オペランドはメモリの 6 バイトを指定します。格納される内容は、オペランド・サイズ属性の影響を受けます、これはベース・アドレスが属性で 24 ビット、32 ビットと異なるためです、よって 16 ビット属性の場合は 6 バイト目のデータには 0 が格納されます。

- **LLDT**

Load Local Descriptor Table Register

記述形式 **LLDT op**

オペランドの内容を LDTR にロードします。

オペランドは 16 ビットのレジスタか、2 バイトのメモリを指定します。

この命令はプロテクト・モードでのみ実行出来ます。

- **SLDT Store Local Descriptor Table Register**

記述形式 **SLDT op**

LDTR の内容をオペランドのレジスタかメモリに格納します。

オペランドはオペランド・サイズ属性の影響を受けます。

この命令はプロテクト・モードでのみ実行出来ます。

- **LTR**

Load Task Register

記述形式 **LTR op**

オペランドの内容を TR にロードします。

オペランドは 16 ビットのレジスタか、2 バイトのメモリを指定します。

TR にロードされたセクタの指す TSS のタスクはビジーにされます、TR にはビジーなタスクをロードすると例外を発生するので、同じセクタを再ロードしないようにします。

この命令はプロテクト・モードでのみ実行出来ます。

- **STR**

Store Task Register

記述形式 **STR op**

タスク・レジスタの内容をオペランドのレジスタか 2 バイトのメモリに格納します。

この命令はプロテクト・モードでのみ実行出来ます。

- **LIDT**

Load Interrupt Descriptor Table Register

記述形式 **LIDT op**

オペランドの値を GDTR にロードします。

オペランドはメモリを指定しますが、オペランド・サイズ属性の影響を受けます。

16 ビット属性なら 32 ビット値がロードされ、32 ビット属性なら 48 ビット値がロードされます。

ロードされる値は上位にベース・アドレス、下位にリミットがあります、ベース・アドレスは 6 ビット属性なら 24 ビット、32 ビット属性なら 32 ビットです。

- **SIDT**

Store Interrupt Descriptor Table Register

記述形式 **SIDT op**

IDTR の内容をオペランドに格納します。

オペランドはの 6 バイトを指定します。

格納される内容は、オペランド・サイズ属性の影響を受けます、これはベース・アドレスが属性で 24 ビット、32 ビットと異なるためです、16 ビット属性の場合は 6 バイト目のデータには 0 が格納されます。

- **MOV**

Move to/from Control Register

コントロール・レジスタは CR0,2~4 です。

記述形式 **MOV CRx,op**

オペランドの内容を、コントロール・レジスタにロードします。

オペランドは 32 ビット汎用レジスタを使います。

記述形式 **MOV op,CRx**

コントロール・レジスタの値をオペランドの指定するメモリの 4 バイトに格納します。

Move to/from Debug Register

記述形式 **MOV DRx,op**

オペランドの内容を、デバッグ・レジスタにロードします。

オペランドは 32 ビット汎用レジスタを指定します。

記述形式 **MOV op,DRx**

デバッグ・レジスタの値をオペランドの指定す

るメモリの 4 バイトに格納します。

デバッグ・レジスタの選択には、CR4 の DE ビットも関係します、このビットが 0 のときは DR4,5 の参照が R6, 7 への参照になります、1 のときは未定義命令の例外が発生します。

- **LMSW**

Load Machine Status Word

記述形式 **LMSW op**

オペランドの内容を CR0 の下位 16 ビットにロードします。

オペランドは 16 ビットのレジスタか、2 バイトのメモリを指定します。

この命令は 80286 との互換性のためにあり、通常は MOV 命令を使います。

- **SMSW**

Store Machine Status Word

記述形式 **SMSW op**

CR0 の下位 16 ビットをオペランドを格納します。

オペランドはレジスタか、2 バイトのメモリを指定します。

この命令は 80286 との互換性のためにあり、通常は MOV 命令を使います。

- **CLTS**

Clear Task-Switched Flag in CR0

記述形式 **CLTS**

CR0 の TS フラグを 0 にします。

- **ARPL**

Adjust RPL Field of Segment Selector

記述形式 **ARPL opd,ops**

2 つのセグメント・セクタの RPL を比較します。

デスティネーション・オペランドの RPL がソース・オペランドの RPL より小さい場合は ZF が 1 にされ、他の場合は 0 にされます。

ZF が 1 にされた場合は、ソース・オペランドの RPL がデスティネーションの RPL にロードされます。

ソース・オペランドは 16 ビット・レジスタか 2 バイトのメモリを指定します。

デスティネーション・オペランドには 16 ビットのレジスタを指定します。

- **LAR**

Load Access Rights Byte

記述形式 **LAR** *opd,ops*

ソース・オペランドの指定するアクセス権をデスティネーション・オペランドに転送して、ZF をセットします。

アクセス権が正常にロードされた場合は ZF を 1 にします、そうでない場合は 0 にします。

ソース・オペランドはディスクリプタのあるメモリ位置を指定します、デスティネーション・オペランドはレジスタを指定します。

割り込みゲートとトラップ・ゲートは参照出来ません。

オペランドとロードされる内容はオペランド・サイズ属性の影響を受けます。

16 ビット属性では、ディスクリプタのバイト 4,5 がロードされて FF00h でマスクされます。

32 ビット属性では、ディスクリプタのバイト 4~7 がロードされて 00FxFF00h でマスクされます。

- **LSL**

この命令はプロテクト・モードでのみ実行出来ます。

Load Segment Limit

記述形式 **LSL** *opd,ops*

ソース・オペランドの指定するリミットをデスティネーション・オペランドに転送して、ZF をセットします。

リミットが正常にロードされた場合は ZF を 1 にします、そうでない場合は 0 にします。

ソース・オペランドはディスクリプタのあるメモリを指定します、デスティネーション・オペランドはレジスタを指定します。

ゲート・ディスクリプタは参照出来ません。

オペランドとロードされる内容はオペランド・サイズ属性の影響を受けます。

- 16 ビット属性

16 ビットのリミット値がロードされます。

- 32 ビット属性

20 ビットのリミット値がロードされます。

また、ディスクリプタの G ビットが 1 のときは、上記の値を 12 ビット左シフトし

て、下位の 12 ビットに 1 を充填します。

この命令はプロテクト・モードでのみ実行出来ます。

- **VERR/VERW**

Verify a Segment for Reading or Writing

記述形式 **VERR** *op*

VERW *op*

オペランドで指定されるセグメント・セクタが現行特権レベルから、読みとり可能か (VERR)、または、書き込み可能か (VERW) をチェックして、結果を ZF にセットします。

結果は、読みとり、又は、書き込みが可能であれば ZF が 1 にされ、不可能であれば 0 にされます。

- **INVD**

Invalidate Internal Caches

記述形式 **INVD**

プロセッサの内部キャッシュを無効にし、外部キャッシュにも無効を指示する、特殊バス・サイクルを発行します。

- **WBINVD**

Write Back and Invalidate Cache

記述形式 **WBINVD**

プロセッサの内部キャッシュをライト・バックしてから、プロセッサの内部キャッシュを無効にし、外部キャッシュにも無効を指示する、特殊バス・サイクルを発行します。

- **INVLPG**

Invalidate TLB Entry

記述形式 **INVLPG** *op*

ソース・オペランドの指定するメモリの含まれるページの TLB を無効にします。

- **LOCK**

Assert LOCK # Signal Prefix

記述形式 **LOCK**

LOCK プリフィックスが使われる命令の実行中に、プロセッサの LOCK#信号をアサートします。

LOCK プリフィックスは次の命令に適用出来ます。

ADD,ADC,AND,BTC,BTR,BTS,CMPXCHG,DEC,INC,NEG,NOT,OR,SBB,SUB,XOR,XADD,XCHG

- **HLT**
Halt
記述形式 **HLT**
命令の実行を停止して、プロセッサを HALT 状態にします。
- **RSM**
Resume from System Management Mode
記述形式 **RSM**
システム・マネジメント・モード (SMM) から復帰します。
- **RDMSR**
Read from Model Specific Register
記述形式 **RDMSR**
ECX レジスタで指定するモデル固有レジスタ (MSR) の内容を、レジスタ EDX:EAX にロードします。
- **WRMSR**
Write to Model Specific Register
記述形式 **WRMSR**
レジスタ EDX:EAX の内容を ECX レジスタで指定されたモデル固有レジスタ (MSR) に書き込みます。
- **RDPMSR**
Read Performance-Monitoring Counters
記述形式 **RDPMSR**
ECX レジスタで指定する性能モニタリング・カウンタの内容を、レジスタ EDX:EAX にロードします。
- **RDTSC**
Read Time-Stamp Counter
記述形式 **RDTSC**
プロセッサのタイム・スタンプ・カウンタの内容をレジスタ EDX:EAX にロードします。

6.2 浮動小数点命令

6.2.1 データ転送命令

- **FLD**
Load Real
記述形式 **FLD op**
FPU レジスタ・スタックにオペランドの内容をプッシュします。

オペランドはメモリか FPU レジスタ・スタックを指定します。

- **FST**
Store Real
記述形式 **FST op**
ST(0) の内容をオペランドに転送します。
オペランドはメモリか FPU レジスタ・スタックを指定します。
拡張実数の転送は出来ません。
- **FSTP**
Store Real and pop register stack
記述形式 **FSTP op**
FST 命令と同じ命令を実行した後に、FPU レジスタ・スタックをポップします。
拡張実数の転送が出来ます。
- **FILD**
Load Integer
記述形式 **FILD op**
符号付き整数のオペランドを、FPU レジスタ・スタックにプッシュします。
- **FIST**
Store Integer
記述形式 **FIST op**
ST(0) の内容を符号付き整数に変換して、オペランドに格納します。
- **FISTP**
Store Integer and pop register
記述形式 **FISTP op**
FIST 命令と同じ命令を実行した後に、FPU レジスタ・スタックをポップします。
- **FBLD**
Load Binary Coded Decimal
記述形式 **FBLD op**
オペランドの BCD 値を、FPU レジスタ・スタックにプッシュします。
- **FBSTP**
Store BCD Integer and Pop
記述形式 **FBSTP op**
ST(0) レジスタの内容を、18 桁のパック形式 BCD 整数に変換して、その結果をオペランドに格納します。
その後に、FPU レジスタ・スタックをポップし

表 6.3 FCMOVcc 命令

命令 (cc)	条件フラグ
B	CF=1
E	ZF=1
BE	CF=1 or ZF=1
U	PF=1
NB	CF=0
NE	ZF=0
NBE	CF =0 and ZF=0
NU	PF=0

ます。

• FXCH

Exchange Register Contents

記述形式 **FXCH ST(i)**
FXCH

ST(0) とオペランドで指定する ST(i) の内容を交換します。

オペランドが無い場合は ST(0) と ST(1) の内容が交換されます。

• FCMOVcc

Floating-Point Conditional Move

記述形式 **FCMOVcc ST(0),ST(i)**

条件付きのデータ転送命令です、EFLAGS レジスタのフラグをチェックして、条件が満たされれば命令は実行され、満たされなければ次の命令に移ります。

この命令は PentiumPro からサポートされた命令ですが、使用する場合は CPUID 命令でサポートを確認して下さい。

条件により表 6.3 の種類があります。

6.2.2 基本算術命令

• FADD

記述形式 **FADD op**
FADD ST(0),ST(i)
FADD ST(i),ST(0)

オペランドが 1 個の場合は、オペランドと ST(0) を加算して、その結果を ST(0) に格納します。

オペランドには、倍精度実数か単精度実数のあ

るメモリを指定します。

オペランドが 2 個の場合は、デスティネーション・オペランド (1 番目) からソース・オペランド (2 番目) を減算して、デスティネーション・オペランドに格納します。

• FADDP

記述形式 **FADDP ST(i),ST(0)**
FADDP

オペランドが無い場合は、ST(0) と ST(1) を加算して、その結果を、ST(0) に格納します。

オペランドが 2 個の場合は、ST(i) と ST(0) を加算して、その結果を ST(i) に格納します。

その後、FPU レジスタ・スタックをポップします。

• FIADD

記述形式 **FIADD op**

オペランドの指定する、整数のあるメモリの内容を ST(0) と加算して、その結果を ST(0) に格納します。

• FSUB

記述形式 **FSUB op**
FSUB ST(0),ST(i)
FSUB ST(i),ST(0)

オペランドが 1 個の場合は、ST(0) からオペランドの内容を減算して、その結果を ST(0) に格納します。

オペランドには、倍精度実数か単精度実数のあるメモリを指定します。

オペランドが 2 個の場合は、デスティネーション・オペランド (1 番目) からソース・オペランド (2 番目) を減算して、デスティネーション・オペランドに格納します。

• FSUBP

記述形式 **FSUBP ST(i),ST(0)**
FSUBP

オペランドが無い場合は、ST(1) から ST(0) を減算して、その結果を、ST(1) に格納します。

オペランドが 2 個の場合は、ST(i) から ST(0) を減算して、その結果を ST(i) に格納します。

その後、FPU レジスタ・スタックをポップします。

• FISUB

記述形式 **FISUB op**

オペランドの指定する、整数のあるメモリの内容を ST(0) から減算して、その結果を ST(0) に格納します。

- **FSUBR/FUBRP/FISUBR**

FSUBR,FSUBRP,FISUBR は、それぞれ FSUB,FSUBP,FISUB の減算の対象が逆になっています。

例を上げると

FSUB ST(i),ST(0) は

FSUBR ST(0),ST(i) と同じ操作を実行します。

- **FMUL**

記述形式 **FMUL op**

FMUL ST(0),ST(i)

FMUL ST(i),ST(0)

オペランドが 1 個の場合は、オペランドと ST(0) を乗算して、その結果を ST(0) に格納します。

オペランドには、倍精度実数か単精度実数のあるメモリを指定します。

オペランドが 2 個の場合は、デスティネーション・オペランド (1 番目) とソース・オペランド (2 番目) を乗算して、デスティネーション・オペランドに格納します。

- **FMULP**

記述形式 **FMULP ST(i),ST(0)**

FMULP

オペランドが無い場合は、ST(1) と ST(0) を乗算して、その結果を、ST(1) に格納します。

オペランドが 2 個の場合は、ST(i) と ST(0) を乗算して、その結果を ST(i) に格納します。

その後、FPU レジスタ・スタックをポップします。

- **FIMUL**

記述形式 **FIMUL op**

オペランドの指定する、整数のあるメモリの内容を ST(0) と乗算して、その結果を ST(0) に格納します。

- **FDIV**

記述形式 **FDIV op**

FDIV ST(0),ST(i)

FDIV ST(i),ST(0)

オペランドが 1 個の場合は、オペランドを ST(0) で除算して、その結果を ST(0) に格納します。

オペランドには、倍精度実数か単精度実数のあるメモリを指定します。

オペランドが 2 個の場合は、デスティネーション・オペランド (1 番目) をソース・オペランド (2 番目) で除算して、デスティネーション・オペランドに格納します。

- **FDIVP**

記述形式 **FDIVP ST(i),ST(0)**

FDIVP

オペランドが無い場合は、ST(1) を ST(0) で除算して、その結果を、ST(1) に格納します。

オペランドが 2 個の場合は、ST(i) を ST(0) で除算して、その結果を ST(i) に格納します。

その後、FPU レジスタ・スタックをポップします。

- **FIDIV**

記述形式 **FIDIV op**

オペランドの指定する、整数のあるメモリの内容で ST(0) を除算して、その結果を ST(0) に格納します。

- **FIDIVR/FDIVRP/FIDIVR**

FDIVR,FDIVRP,FIDIVR は、それぞれ

FDIV,FDIVP,FIDIV の除算の対象が逆になっています。

例を上げると

FDIV ST(i),ST(0) は

FDIVR ST(0),ST(i) と同じ操作を実行します。

- **FPREM**

Partial Remainder

記述形式 **FPREM**

ST(0) の内容を ST(1) で除算して、その余りを ST(0) に格納します、余りは次の式によります。

$$\text{余り} = ST(0) - (N \times ST(1))$$

N は、 $ST(0)/ST(1)$ の実数の商をゼロ方向に切り捨てて得られた整数値です。

符号は被除数の符号と同じになります。

8087 及び 80287 と互換性のある剰余を計算します。

- **FPREM1**

記述形式 **FPREM1**

ST(0) の内容を ST(1) で除算して、その余りを ST(0) に格納します、余りは次の式によります。

$$\text{余り} = ST(0) - (N \times ST(1))$$

N は、 $ST(0)/ST(1)$ の実数の商を最近の整数値に丸めることによって得られた整数値です。

符号は被除数の符号と同じになります。

IEEE754 規格と互換性のある剰余を計算します。

- **FABS**

Absolute Value

記述形式 **FABS**

ST(0) の符号ビットを 0 にして、オペランドの絶対値を与えます。

- **FCHS**

Change Sign

記述形式 **FCHS**

ST(0) の符号ビットを反転します。

- **FRNDINT**

Round to Integer

記述形式 **FRNDINT**

現在の丸めモードにしたがって、ST(0) の内容を整数の最近値に丸めて、ST(0) に格納します。

- **FSCALE**

記述形式 **FSCALE**

ST(1) の 2 のべき乗を ST(0) と乗算して、その結果を ST(0) に格納します。

- **FSQRT**

Square Root

記述形式 **FSQRT**

ST(0) の平方根を計算して、ST(0) に格納します。

- **FXTRACT**

Extract Exponent and Significand

記述形式 **FXTRACT**

ST(0) の指数部と仮数部を分離して、指数部を ST(0) に格納し、仮数部を FPU レジスタ・スタックにプッシュします。

この操作の後には、ST(0) に仮数部、ST(1) に指数部が残っています。

6.2.3 比較命令

- **FCOM**

Compare Real

記述形式 **FCOM**

記述形式 **FCOM op**

記述形式 **FCOM ST(i)**

ST(0) をソースと比較して、結果を FPU ステータス・レジスタの C3,2,0 に設定します。

ソースはオペランドで指定します、オペランドが無い場合は ST(1) が比較されます。

比較結果は次の表を見て下さい。

条件	C3	C2	C0
ST(0)>op	0	0	0
ST(0)<op	0	0	1
ST(0)=op	1	0	0
無効	1	1	1

- **FCOMP**

Compare Real and pop register stack

記述形式 **FCOMP**

記述形式 **FCOMP op**

記述形式 **FCOMP ST(i)**

FCOM 命令と同じ操作を実施した後、FPU レジスタ・スタックをポップします。

- **FCOMPP**

Compare Real and pop register stack twice

記述形式 **FCOMPP**

FCOM 命令と同じ操作を実施した後、FPU レジスタ・スタックを 2 回ポップします。

- **FUCOM**

Unordered Compare Real

記述形式 **FUCOM**

記述形式 **FUCOM ST(i)**

FCOM 命令と同じですが、無効演算例外の扱いが異なります。

- **FUCOMP**

記述形式 **FUCOMP**

記述形式 **FUCOMP op**

記述形式 **FUCOMP ST(i)**

FCOMP 命令と同じですが、無効演算例外の扱いが異なります。

- **FUCOMPP**

記述形式 **FUCOMPP**

FCOMPP 命令と同じですが、無効演算例外の扱いが異なります。

- **FICOM**

Compare Integer

記述形式 **FICOM op**

ST(0) をオペランドの指定する整数と比較して、結果を FPU ステータス・レジスタの C3,2,0 に設定します。

比較結果は FCOM 命令の表を参照して下さい。

- **FICOMP**

Compare Integer and pop stack register

記述形式 **FICOMP op**

FICOM 命令と同じ操作を実施した後、FPU レジスタ・スタックをポップします。

- **FCOMI**

Compare Real and Set EFLAGS

記述形式 **FCOMI ST(0),ST(i)**

ST(0) と ST(i) の内容と比較して、その結果を EFALGS レジスタの ZF,PF,CF に設定します。比較結果は次の表を見て下さい。

条件	ZF	PF	CF
ST(0)>ST(i)	0	0	0
ST(0)<ST(i)	0	0	1
ST(0)=ST(i)	1	0	0
無効	1	1	1

- **FCOMIP**

Compare Real and Set EFLAGS, and pop register stack

記述形式 **FCOMIP ST(0),ST(i)**

FCOMI 命令と同じ操作を実施した後、FPU レジスタ・スタックをポップします。

- **FUCOMI**

記述形式 **FUCOMI ST(0),ST(i)**

FCOMI 命令と同じですが、無効演算例外の扱いが異なります。

- **FUCOMIP**

記述形式 **FUCOMIP ST(0),ST(i)**

FCOMIP 命令と同じですが、無効演算例外の扱いが異なります。

- **FTST**

記述形式 **FTST**

ST(0) を 0.0 と比較して、その結果を EFALGS レジスタの C3,2,0 に設定します。

比較結果は FCOM 命令の表を見て下さい。

- **FXAM**

Examine

記述形式 **FXAM**

ST(0) を調べて、FPU ステータス・レジスタの C3,2,0 に設定します。

比較結果は次の表を見て下さい。

クラス	C3	C2	C0
非サポート	0	0	0
NaN	0	0	1
ノーマル有限数	0	1	0
無限大	0	1	1
ゼロ	1	0	0
空	1	0	1
デノーマル数	1	1	0

6.2.4 超越関数命令

- **FSIN**

記述形式 **FSIN**

ST(1) の正弦を計算して、その結果を ST(0) に格納します。

ST(1) にはラジアン値を指定します。

- **FCOS**

記述形式 **FCOS**

ST(1) の余弦を計算して、その結果を ST(0) に格納します。

ST(1) にはラジアン値を指定します。

- **FSINCOS**

記述形式 **FSINCOS**

ST(0) の正弦と余弦を計算して、正弦を ST(0) に格納して、余弦を FPU のスタック・レジスタにプッシュします。

ST(0) にはラジアン値を指定します。

命令の実行後には ST(0) には余弦が、ST(1) には正弦が残っています。

- **FPTAN**

記述形式 **FPTAN**

ST(0) の正接を計算して、その結果を ST(0) に格納して、1.0 を FPU レジスタ・スタックにプッシュします。

ST(0) にはラジアン値を指定します。

命令の実行後には ST(0) には 1.0 が、ST(1) には正接が残っています。

- **FPATAN**

記述形式 **FPATAN**

ST(1) を ST(0) で除算して、その結果の逆正接を計算して、ST(1) に格納します。

その後、FPU レジスタ・スタックをポップします。

命令の実行後には、ST(0) に逆正接が残っています。

- **F2XM1**

Compute $2^x - 1$

記述形式 **F2XM1**

2 の ST(0) 乗の指数演算を行い 1 を引きます、結果は ST(0) に格納されます。

- **FYL2X**

Compute $y \log_2 x$

記述形式 **FYL2X**

ST(1) $\times \log_2$ (ST(0)) を計算して、その結果を ST(1) に格納します。

その後、FPU レジスタ・スタックをポップします。

命令の実行後には、ST(0) に計算結果が残っています。

- **FYL2XP1**

Compute $y \log_2(x + 1)$

記述形式 **FYL2XP1**

ST(1) $\times \log_2$ (ST(0) + 1.0)) を計算して、その結果をレジスタ ST(1) に格納します。

その後、FPU レジスタ・スタックをポップします。

命令の実行後には、ST(0) に計算結果が残っています。

6.2.5 定数ロード命令

- **FLD1**

記述形式 **FLD1**

+1.0 を FPU レジスタ・スタックにプッシュします。

- **FLDZ**

記述形式 **FLDZ**

+0.0 を FPU レジスタ・スタックにプッシュします。

- **FLDPI**

記述形式 **FLDPI**

π を FPU レジスタ・スタックにプッシュします。

- **FLDL2E**

記述形式 **FLDL2E**

$\log_2 e$ を FPU レジスタ・スタックにプッシュします。

- **FLDLN2**

記述形式 **FLDLN2**

$\log_e 2$ を FPU レジスタ・スタックにプッシュします。

- **FLDL2T**

記述形式 **FLDL2T**

$\log_2 10$ を FPU レジスタ・スタックにプッシュします。

- **FLDLG2**

記述形式 **FLDLG2**

$\log_{10} 2$ を FPU レジスタ・スタックにプッシュします。

6.2.6 FPU 制御命令

- **FINCSTP**

Increment Stack-Top Pointer

記述形式 **FINCSTP**

スタック・トップ・ポインタ (FPU ステータス・レジスタの TOP フィールド) をインクリメントします。

- **FDECSTP**

Decrement Stack-Top Pointer

記述形式 **FDECSTP**

スタック・トップ・ポインタ (FPU ステータス・レジスタの TOP フィールド) をデクリメントします。

- **FFREE**

Free Floating-Point Register

記述形式 **FFREE ST(i)**

ST(i) に関連付けられている FPU タグ・レジスタのタグを空 (11b) にします。

- **FINIT**

Initialized Floating-Point Unit

記述形式 **FINIT**

FPU を初期状態にします。

初期化の前にマスクされていない、保留中の浮動小数点例外があれば実行します。

- **FNINIT**

記述形式 **FNINIT**

FPU を初期状態にします。

- **FCLEX**

Clear Exception

記述形式 **FCLEX**

FPU ステータス・レジスタの浮動小数点例外フラグ (PE,UE,OE,ZE,DE,IE)、ES、SF、B をクリアします。

クリアの前にマスクされていない、保留中の浮動小数点例外があれば実行します。

- **FNCLEX**

記述形式 **FNCLEX**

FPU ステータス・レジスタの浮動小数点例外フラグ (PE,UE,OE,ZE,DE,IE)、ES、SF、B をクリアします。

- **FSTCW**

Store Control World

記述形式 **FSTCW op**

FPU コントロール・レジスタの現在値を、オペランドの指定するメモリに格納します。

上記操作の前にマスクされていない、保留中の浮動小数点例外があれば実行します。

- **FNSTCW**

記述形式 **FNSTCW op**

FPU コントロール・レジスタの現在値を、オペランドの指定するメモリに格納します。

- **FLDCW**

Load Control Word

記述形式 **FLDCW op**

オペランドの指定する 2 バイトのメモリから、FPU コントロール・ワードにロードします。

- **FSTENV**

Store FPU Environment

記述形式 **FSTENV op**

現在の FPU の動作環境を、オペランドの指定するメモリにセーブします。

その後、すべての浮動小数点例外をマスクします。

セーブされる内容のメモリ内レイアウトは、プロセッサの動作モードやオペランド・サイズ属性で異なります。

セーブする前にマスクされていない、保留中の浮動小数点例外があれば実行します。

- **FNSTENV**

記述形式 **FNSTENV op**

現在の FPU の動作環境を、オペランドの指定するメモリにセーブします。

その後、すべての浮動小数点例外をマスクします。

セーブされる内容のメモリ内レイアウトは、プロセッサの動作モードやオペランド・サイズ属性で異なります。

- **FLDENV**

Load FPU Environment

記述形式 **FLDENV op**

完全な FPU 動作環境をメモリから FPU にロードします。

通常この命令は同じ動作モードとオペランド・サイズ属性で実行された FSTENV か FNSTENV と対で使用します。

- **FSAVE**

Store FPU State

記述形式 **FSAVE op**

現在の FPU の状態を、オペランドの指定するメモリにセーブして、FPU を再初期化します。セーブの前にマスクされていない、保留中の浮動小数点例外があれば実行します。

- **FNSAVE**

記述形式 **FNSAVE op**

現在の FPU の状態を、オペランドの指定するメモリにセーブして、FPU を再初期化します。

- **FRSTOR**

Restore FPU State

FPU 状態をオペランドで指定したメモリからロードします

記述形式 **FRSTOR op**

通常この命令は同じ動作モードとオペランド・サイズ属性で実行された FSAVE か FNSAVE

と対で使用します。

- **FSTSW**

Store Status Word

記述形式 **FSTSW op**

FPU ステータス・レジスタをオペランドに格納します。

オペランドは AX か 2 バイトのメモリです。

格納する前にマスクされていない、保留中の浮動小数点例外があれば実行します。

- **FNSTSW**

記述形式 **FNSTSW op**

FPU ステータス・レジスタをオペランドに格納します。

オペランドは AX か 2 バイトのメモリです。

- **WAIT/FWAIT**

記述形式 **WAIT**

FWAIT

プロセッサが実行を続行をする前にマスクされていない、保留中の浮動小数点例外があれば実行します。

- **FNOP**

No Operation

記述形式 **FNOP**

インストラクション・ポインタを進める以外は何もしません、2 バイトのオペコードを持ちます。

6.3 アドレッシング・モード

6.3.1 メモリ

アドレッシングの要素を次に示します。

- **base**
汎用レジスタで指定します。
- **index**
ESP を除いた汎用レジスタで指定します。
- **scale**
index に即値を掛け合わせます、1,2,3,4 を指定します。
- **displacement**
即値で指定します、サイズは 8,16,32 ビットがあります。

次の構成でメモリを指定します。

- *base*
- *base + displacement*
- *(index × scale) + displacement*
- *base + index + displacement*
- *base + (index × scale) + displacement*

さらに前部に、セグメント・レジスタを指定 (セグメント・オーバーライド・プリフィックス) することも出来ます、指定が無い場合は DS が使われます、次に一例を示します。

ES:EAX[EBX,ECX*4,123]

6.3.2 I/O

I/O アドレスは 8 ビットの即値か DX で指定します。

6.4 命令コード・フォーマット

6.4.1 基本形式

図 6.2 が命令の基本的なフォーマットです、全ての命令は、これの全部または部分を使って表現します。

6.4.2 プリフィックス

指定されたオペコードの機能や、オペランドのサイズを拡張したり、変更するために、オペコードの前に次の 4 種類を、1 バイトのコードで指定します、重ねて指定することが可能です。

命令プリフィックス

特定の命令の機能を拡張するときに使います。

- REP/REPE/REPZ/REPNE/REPNZ

116 頁の 6.1.8 を参照して下さい。

- LOCK

121 頁の 6.1.15 を参照して下さい。

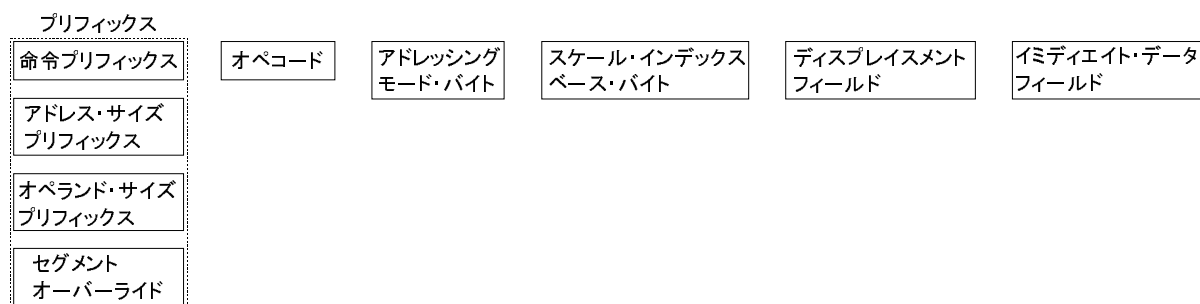
アドレス・サイズ・プリフィックス 67h

実効アドレスを変更します。

実効アドレスは 16 ビットと 32 ビットで指定出来ますが、リアルモードでは 64k バイトまでの範囲に限られます、プロテクト・モードではセグメント・ディスクリプタの D ビットで指定します。

このプリフィックスは、セグメント・ディスクリプタの指定が 16 ビットなら 32 ビットに、32 ビットなら 16 ビットに変更します。

図 6.2 命令の基本的なフォーマット

**オペランド・サイズ・プリフィックス 66h**

8086 でオペランドのレジスタやメモリのデータのサイズを指定するときに、8 ビットと 16 ビットの選択をひとつのビットで指定しました。

80386 以降のプロセッサでは、セグメント・ディスクリプタの D ビットが 1 なら、8/16 ビットの選択を 8/32 ビットの選択に変更します。

このプリフィックスは 8/16 ビットの選択を 8/32 ビットの選択に、8/32 ビットの選択を 8/16 ビットの選択に変更します。

セグメント・オーバーライド・プリフィックス

デフォルトのセグメント・レジスタを変更する場合に指定します。

```

CS    2Eh
DS    3Eh
ES    26h
FS    64h
GS    65h
SS    36h

```

セグメント・オーバーライド・プリフィックスの適用の例です。

使用例

リスト 19 を参照して下さい。

SEGMENT の USE16 は 16 ビット・セグメント、USE32 は 32 ビットのセグメントを指定しています。

- TEST_A/TEST_B
アドレス・サイズ・プリフィックスの適用の例です。
- TEST_C/TEST_D
オペランド・サイズ・プリフィックスの適用の例です。
- TEST_E

リスト19 プリフィックス使用例

Turbo Assembler Version 3.5 98/08/04 14:33:56 Page 1

TEST.ASM

```

1          .486P
2
3      00000000      .model      small
4      00000000      .code
5
6          0000      SEGMENT TEST_A      WORD USE16
7
8          0000 8B 07      MOV      AX,WORD PTR [BX]
9          0002 67| 8B 03      MOV      AX,WORD PTR [EBX]
10
11         0005      ENDS TEST_A
12
13      00000000      SEGMENT TEST_B      WORD USE32
14
15      00000000 66| 67| 8B 07 MOV      AX,WORD PTR [BX]
16      00000004 66| 8B 03      MOV      AX,WORD PTR [EBX]
17
18      00000007      ENDS TEST_B
19
20         0000      SEGMENT TEST_C      WORD USE16
21
22         0000 8B C3      MOV      AX,BX
23         0002 66| 8B C3      MOV      EAX,EBX
24
25         0005      ENDS TEST_C
26
27      00000000      SEGMENT TEST_D      WORD USE32
28
29      00000000 66| 8B C3      MOV      AX,BX
30      00000003 8B C3      MOV      EAX,EBX
31
32      00000005      ENDS TEST_D
33
34         0000      SEGMENT TEST_E      WORD USE16
35
36         0000 26: 8B 07      MOV      AX,WORD PTR ES:[BX]
37         0003 64: 67| 8B 03 MOV      AX,WORD PTR FS:[EBX]
38
39         0007      ENDS TEST_E
40
41      end

```

図 6.3 アドレッシング・モード・バイト

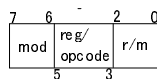
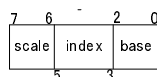


図 6.4 スケール・インデックス・ベース・バイト



6.4.3 オペコード

オペコードは 1,2 バイトからなります、またアドレッシング・モード・バイトに定義ビットを持つ場合もあります。

オペコードは個々の命令を区別しますが、オペランド・サイズやデータの転送方向を定義するビットを含む場合があります。

表 6.9 から表 6.14 に整数命令のオペコード表を

表 6.15 から表 6.16 に FPU 命令のオペコード表を示します。

6.4.4 アドレッシング・モード・バイト

オペランドをコード化するとき、主な役割を担います、図 6.3 を参照して下さい。

mod と r/m の 5 ビットで 32 個の組み合わせを定義します。

reg の 3 ビットで 8 個の組み合わせを定義します。

どちらがソースになるか、デスティネーションになるかはオペコードにより決定されています。

基本的にレジスタ間とメモリ ⇄ レジスタ間のデータ転送を定義します。

オペランド・サイズ属性で 16/32 ビットのモードが選択されますが、組み合わせの結果のアドレッシングが異なります。

組み合わせは表 6.4 と表 6.5 と表 6.6 を参照して下さい。

6.4.5 スケール・インデックス・ベース・バイト

オペランド・サイズ属性が 32 ビットでアドレッシング・モード・バイトが [-][-] の場合は (表 6.5 参照) スケール・インデックス・ベース・バイトが続きます。

これで、ベース・レジスタ、インデックス・レジスタ、スケーリングを定義します。

表 6.7 と表 6.8 を参照して下さい。

表 6.4 アドレッシング・モード・バイト 16 ビット

実効アドレス	mod	r/m
[BX+SI]	00	000
[BX+DI]		001
[BP+SI]		010
[BP+DI]		011
[SI]		100
[DI]		101
disp16		110
[BX]		111
[BX+SI]+disp8	01	000
[BX+DI]+disp8		001
[BP+SI]+disp8		010
[BP+DI]+disp8		011
[SI]+disp8		100
[DI]+disp8		101
[BP]+disp8		110
[BX]+disp8		111
[BX+SI]+disp16	10	000
[BX+DI]+disp16		001
[BP+SI]+disp16		010
[BP+DI]+disp16		011
[SI]+disp16		100
[DI]+disp16		101
[BP]+disp16		110
[BX]+disp16		111
EAX/AX/AL	11	000
ECX/CX/CL		001
EDX/DX/DL		010
EBX/BX/BL		011
ESP/SP/AH		100
EBP/BP/CH		101
ESI/SI/DH		110
EDI/DI/BH		111

表 6.5 アドレッシング・モード・バイト 32 ビット

実効アドレス	mod	r/m
[EAX]	00	000
[ECX]		001
[EDX]		010
[EBX]		011
[-][-]		100
disp32		101
[ESI]		110
[EDI]		111
disp8[EAX]	01	000
disp8[ECX]		001
disp8[EDX]		010
disp8[EBX]		011
disp8[-][-]		100
disp8[EBP]		101
disp8[ESI]		110
disp8[EDI]		111
disp32[EAX]	10	000
disp32[ECX]		001
disp32[EDX]		010
disp32[EBX]		011
disp32[-][-]		100
disp32[EBP]		101
disp32[ESI]		110
disp32[EDI]		111
EAX/AX/AL	11	000
ECX/CX/CL		001
EDX/DX/DL		010
EBX/BX/BL		011
ESP/SP/AH		100
EBP/BP/CH		101
ESI/SI/DH		110
EDI/DI/BH		111

表 6.6 アドレッシング・モード・バイト 32 ビット

reg	8 ビット	16 ビット	32 ビット	opcode
000	AL	AX	EAX	0
001	CL	CX	ECX	1
010	DL	DX	EDX	2
011	BL	BX	EBX	3
100	AH	SP	ESP	4
101	CH	BP	EBP	5
110	DH	SI	ESI	6
111	BH	DI	EDI	7

表 6.7 スケール・インデックス・ベース・バイト (ベース)

ベース レジスタ	base
EAX	000
ECX	001
EDX	010
EBX	011
ESP	100
EBP *	101
ESI	110
EDI	111

* mod=000 の場合はベース無しになります。

disp[index]

6.4.6 ディスプレズメント・フィールド

アドレッシング・モード・バイトでディスプレズメント (disp8,16,32) を必要とする場合に設定します。

6.4.7 イミディエイト・データ・フィールド

オペランドで即値 (imm8,16,32) が指定された場合に設定します。

6.5 記号の説明

- **rel8**

符号付き 8 ビット整数で ± 127 の相対アドレスを示します。

- **rel16/rel32**

それぞれ、符号付き整数の相対アドレスを示します。

- **ptr16:16/ptr16:32**

表 6.8 スケール・インデックス・ベース・バイト (インデックス)

インデックス レジスタ	scale	index
[EAX]	00	000
[ECX]		001
[EDX]		010
[EBX]		011
—		100
[EBP]		101
[ESI]		110
[EDI]		111
[EAX*2]	01	000
[ECX*2]		001
[EDX*2]		010
[EBX*2]		011
—		100
[EBP*2]		101
[ESI*2]		110
[EDI*2]		111
[EAX*4]	10	000
[ECX*4]		001
[EDX*4]		010
[EBX*4]		011
—		100
[EBP*4]		101
[ESI*4]		110
[EDI*4]		111
[EAX*8]	11	000
[ECX*8]		001
[EDX*8]		010
[EBX*8]		011
—		100
[EBP*8]		101
[ESI*8]		110
[EDI*8]		111

far ポインタを示します。

二つの部分からなり、ptr16:16 の前部の ptr16 はセグメント・レジスタに設定する 16 ビットの値です。

後部の 16 はセグメント内の 16 ビットのオフセットです。

16 ビット属性では ptr16:16、32 ビット属性では ptr16:32 を使います。

- **r8**

8 ビットのレジスタです。

AL,CL,DL,BL,AH,CH,DH,BH があります。

- **r16**

16 ビットのレジスタです。

AX,CX,DX,BX,SP,BP,SI,DI があります。

- **r32**

32 ビットのレジスタです。

EAX,ECX,EDX,EBX,ESP,EBP,ESI,EDI があります。

- **imm8**

8 ビットの符号付き整数で、即値を示します。

- **imm16**

16 ビットの符号付き整数で、即値を示します。

- **imm32**

32 ビットの符号付き整数で、即値を示します。

- **r/m8**

8 ビットのレジスタ (r8 と同じ) か、メモリ中の 8 ビット・データを示します。

- **r/m16**

16 ビットのレジスタ (r16 と同じ) か、メモリ中の 16 ビット・データを示します。

- **r/m32**

32 ビットのレジスタ (r32 と同じ) か、メモリ中の 32 ビット・データを示します。

- **m**

16 ビットあるいは 32 ビットのメモリ・オペランド。

- **m8**

DS:(E)SI あるいは ES:(E)DI によってアドレス指定されるメモリの 1 バイト (ストリング命令でのみ使用されます)。

- **m16**

DS:(E)SI あるいは ES:(E)DI によってアドレス

指定されるメモリの 1 ワード (ストリング命令でのみ使用されます)。

- **m32**
DS:(E)SI あるいは ES:(E)DI によってアドレス指定されるメモリの 1 ダブル・ワード (ストリング命令でのみ使用されます)。
- **m64**
メモリの 1 クワッド・ワード (CMPXCHG8B 命令でのみ使用されます)。
- **m16:16,m16:32**
far ポインタのあるメモリ・オペランドを示します。
16 ビット属性では m16:16、32 ビット属性では m16:32 を使います。
- **sreg**
セグメント・レジスタを示します。
CS,DS,SS,ES,FS,GS があります。
- **addr8,16,32**
バイト、ワード、ダブルワードのあるメモリの、セグメント内オフセットを示します。
このオペランドではアドレッシング・モード・バイトを使いません、MOV 命令でのみ使用されます。
- **m32real,m64real,m80real**
浮動小数点オペランドで、単精度実数、倍精度実数、拡張実数を示します。
- **m16int,m32int,m64int**
浮動小数点オペランドで、ワード整数、ショート整数、ロング整数を示します。
- **ST(0)**
FPU レジスタのスタック・トップを示します。
- **ST(i)**
FPU スタック・レジスタのトップから i 番目の要素 (i=0..7) を示します。
6 頁の [図 1.9](#) を参照して下さい。

6.6 フラグ

EFLAGS レジスタのフラグに対する命令の作用を [表 6.17](#)、[表 6.18](#)、[表 6.19](#) に示します。

6.7 プロセッサの変遷による命令の追加

6.7.1 8086

ファミリの原典のプロセッサです、この命令群を基本に追加と拡張、改善がなされています。

6.7.2 80186

次の命令が追加されました。

- PUSH imm8,16,32
- PUSHA
- POPA
- IMUL
オペランドに即値が使える IMUL 命令が追加されています。
- SAR/SHR/SAL/SHL/ROR/ROL/RCR/RCL
8 ビットの即値が使える命令が追加されています。
- INS/OUTS
- BOUND
- ENTER
- LEAVE

6.7.3 80286

このプロセッサからプロテクト・モードが導入されたので、その関係の命令が追加されています。

- LGDT
- SGDT
- LIDT
- SIDT
- LLDT
- SLDT
- LTR
- STR
- LMSW
- SMSW
- CLTS
- LAR
- LSL
- ARPL
- VERR
- VERW

表 6.9 オペコード (1/6)

命令	コード	命令	コード	命令	コード
MOV r/m8,r8	88	CMOVNA r16,r/m16	0F 46	XCHG AX,r16	90
MOV r/m16,r16	89	CMOVNA r32,r/m32	0F 46	XCHG r16,AX	90
MOV r/m32,r32	89	CMOVNAE r16,r/m16	0F 42	XCHG EAX,r32	90
MOV r8,r/m8	8A	CMOVNAE r32,r/m32	0F 42	XCHG r32,EAX	90
MOV r16,r/m16	8B	CMOVNB r16,r/m16	0F 43	XCHG r/m8,r8	86
MOV r32,r/m32	8B	CMOVNB r32,r/m32	0F 43	XCHG r8,r/m8	86
MOV r/m16,sreg	8C	CMOVNBE r16,r/m16	0F 47	XCHG r/m16,r16	87
MOV sreg,r/m16	8E	CMOVNBE r32,r/m32	0F 47	XCHG r16,r/m16	87
MOV AL,addr8	A0	CMOVNC r16,r/m16	0F 43	XCHG r/m32,r32	87
MOV AX,addr16	A1	CMOVNC r32,r/m32	0F 43	XCHG r32,r/m32	87
MOV EAX,addr32	A1	CMOVNE r16,r/m16	0F 45	BSWAP r32	0F C8
MOV addr8,AL	A2	CMOVNE r32,r/m32	0F 45	XADD r/m8,r8	0F C0
MOV addr16,AX	A3	CMOVNG r16,r/m16	0F 4E	XADD r/m16,r16	0F C1
MOV addr32,EAX	A3	CMOVNG r32,r/m32	0F 4E	XADD r/m32,r32	0F C1
MOV r8,imm8	B0	CMOVNGE r16,r/m16	0F 4C	CMPXCHG r/m8,r8	0F B0
MOV r16,imm16	B8	CMOVNGE r32,r/m32	0F 4C	CMPXCHG r/m16,r16	0F B1
MOV r32,imm32	B8	CMOVNL r16,r/m16	0F 4D	CMPXCHG r/m32,r32	0F B1
MOV r/m8,imm8	C6	CMOVNL r32,r/m32	0F 4D	CMPXCHG8B m64	0F C7 [..00 1...]
MOV r/m16,imm16	C7	CMOVNLE r16,r/m16	0F 4F	PUSH r/m16	FF [..11 0...]
MOV r/m32,imm32	C7	CMOVNLE r32,r/m32	0F 4F	PUSH r/m32	FF [..11 0...]
CMOVA r16,r/m16	0F 47	CMOVNO r16,r/m16	0F 41	PUSH r16	50
CMOVA r32,r/m32	0F 47	CMOVNO r32,r/m32	0F 41	PUSH r32	50
CMOVAE r16,r/m16	0F 43	CMOVNP r16,r/m16	0F 4B	PUSH imm8	6A
CMOVAE r32,r/m32	0F 43	CMOVNP r32,r/m32	0F 4B	PUSH imm16	68
CMOVB r16,r/m16	0F 42	CMOVNS r16,r/m16	0F 49	PUSH imm32	68
CMOVB r32,r/m32	0F 42	CMOVNS r32,r/m32	0F 49	PUSH CS	0E
CMOVBE r16,r/m16	0F 46	CMOVNZ r16,r/m16	0F 45	PUSH SS	16
CMOVBE r32,r/m32	0F 46	CMOVNZ r32,r/m32	0F 45	PUSH DS	1E
CMOVC r16,r/m16	0F 42	CMOVO r16,r/m16	0F 40	PUSH ES	06
CMOVC r32,r/m32	0F 42	CMOVO r32,r/m32	0F 40	PUSH FS	0F A0
CMOVE r16,r/m16	0F 44	CMOVP r16,r/m16	0F 4A	PUSH GS	0F A8
CMOVE r32,r/m32	0F 44	CMOVP r32,r/m32	0F 4A	POP m16	8F [..00 0...]
CMOVG r16,r/m16	0F 4F	CMOVPE r16,r/m16	0F 4A	POP m32	8F [..00 0...]
CMOVG r32,r/m32	0F 4F	CMOVPE r32,r/m32	0F 4A	POP r16	58
CMOVGE r16,r/m16	0F 4D	CMOVPO r16,r/m16	0F 4B	POP r32	58
CMOVGE r32,r/m32	0F 4D	CMOVPO r32,r/m32	0F 4B	POP DS	1F
CMOVL r16,r/m16	0F 4C	CMOVS r16,r/m16	0F 48	POP ES	07
CMOVL r32,r/m32	0F 4C	CMOVS r32,r/m32	0F 48	POP SS	17
CMOVLE r16,r/m16	0F 4E	CMOVZ r16,r/m16	0F 44	POP FS	0F A1
CMOVLE r32,r/m32	0F 4E	CMOVZ r16,r/m16	0F 44	POP GS	0F A9

表 6.10 オペコード (2/6)

命令	コード	命令	コード	命令	コード
PUSHA	60	XOR r/m16,imm16	81 [..11 0...]	SHL r/m8,1	D0 [..10 0...]
PUSHAD	60	XOR r/m32,imm32	81 [..11 0...]	SHL r/m8,CL	D2 [..10 0...]
POPA	61	XOR r/m16,imm8	83 [..11 0...]	SHL r/m8,imm8	C0 [..10 0...]
POPAD	61	XOR r/m32,imm8	83 [..11 0...]	SHL r/m16,1	D1 [..10 0...]
ENTER imm16,0	C8	XOR r/m8,r8	30	SHL r/m16,CL	D3 [..10 0...]
ENTER imm16,1	C8	XOR r/m16,r16	31	SHL r/m16,imm8	C1 [..10 0...]
ENTER imm16,imm8	C8	XOR r/m32,r32	31	SHL r/m32,1	D1 [..10 0...]
LEAVE	C9	XOR r8,r/m8	32	SHL r/m32,CL	D3 [..10 0...]
AND AL,imm8	24	XOR r16,r/m16	33	SHL r/m32,imm8	C1 [..10 0...]
AND AX,imm16	25	XOR r32,r/m32	33	SHRD r/m16,r16,imm8	0F AC
AND EAX,imm32	25	NOT r/m8	F6 [..01 0...]	SHRD r/m16,r16,CL	0F AD
AND r/m8,imm8	80 [..10 0...]	NOT r/m16	F7 [..01 0...]	SHRD r/m32,r32,imm8	0F AC
AND r/m16,imm16	81 [..10 0...]	NOT r/m32	F7 [..01 0...]	SHRD r/m32,r32,CL	0F AD
AND r/m32,imm32	81 [..10 0...]	SAR r/m8,1	D0 [..11 1...]	SHLD r/m16,r16,imm8	0F A4
AND r/m16,imm8	83 [..10 0...]	SAR r/m8,CL	D2 [..11 1...]	SHLD r/m16,r16,CL	0F A5
AND r/m32,imm8	83 [..10 0...]	SAR r/m8,imm8	C0 [..11 1...]	SHLD r/m32,r32,imm8	0F A4
AND r/m8,r8	20	SAR r/m16,1	D1 [..11 1...]	SHLD r/m32,r32,CL	0F A5
AND r/m16,r16	21	SAR r/m16,CL	D3 [..11 1...]	ROR r/m8,1	D0 [..00 1...]
AND r/m32,r32	21	SAR r/m16,imm8	C1 [..11 1...]	ROR r/m8,CL	D2 [..00 1...]
AND r8,r/m8	22	SAR r/m32,1	D1 [..11 1...]	ROR r/m8,imm8	C0 [..00 1...]
AND r16,r/m16	23	SAR r/m32,CL	D3 [..11 1...]	ROR r/m16,1	D1 [..00 1...]
AND r32,r/m32	23	SAR r/m32,imm8	C1 [..11 1...]	ROR r/m16,CL	D3 [..00 1...]
OR AL,imm8	0C	SHR r/m8,1	D0 [..10 1...]	ROR r/m16,imm8	C1 [..00 1...]
OR AX,imm16	0D	SHR r/m8,CL	D2 [..10 1...]	ROR r/m32,1	D1 [..00 1...]
OR EAX,imm32	0D	SHR r/m8,imm8	C0 [..10 1...]	ROR r/m32,CL	D3 [..00 1...]
OR r/m8,imm8	80 [..00 1...]	SHR r/m16,1	D1 [..10 1...]	ROR r/m32,imm8	C1 [..00 1...]
OR r/m16,imm16	81 [..00 1...]	SHR r/m16,CL	D3 [..10 1...]	ROL r/m8,1	D0 [..00 0...]
OR r/m32,imm32	81 [..00 1...]	SHR r/m16,imm8	C1 [..10 1...]	ROL r/m8,CL	D2 [..00 0...]
OR r/m16,imm8	83 [..00 1...]	SHR r/m32,1	D1 [..10 1...]	ROL r/m8,imm8	C0 [..00 0...]
OR r/m32,imm8	83 [..00 1...]	SHR r/m32,CL	D3 [..10 1...]	ROL r/m16,1	D1 [..00 0...]
OR r/m8,r8	08	SHR r/m32,imm8	C1 [..10 1...]	ROL r/m16,CL	D3 [..00 0...]
OR r/m16,r16	09	SAL r/m8,1	D0 [..10 0...]	ROL r/m16,imm8	C1 [..00 0...]
OR r/m32,r32	09	SAL r/m8,CL	D2 [..10 0...]	ROL r/m32,1	D1 [..00 0...]
OR r8,r/m8	0A	SAL r/m8,imm8	C0 [..10 0...]	ROL r/m32,CL	D3 [..00 0...]
OR r16,r/m16	0B	SAL r/m16,1	D1 [..10 0...]	ROL r/m32,imm8	C1 [..00 0...]
OR r32,r/m32	0B	SAL r/m16,CL	D3 [..10 0...]	RCR r/m8,1	D0 [..01 1...]
XOR AL,imm8	34	SAL r/m16,imm8	C1 [..10 0...]	RCR r/m8,CL	D2 [..01 1...]
XOR AX,imm16	35	SAL r/m32,1	D1 [..10 0...]	RCR r/m8,imm8	C0 [..01 1...]
XOR EAX,imm32	35	SAL r/m32,CL	D3 [..10 0...]	RCR r/m16,1	D1 [..01 1...]
XOR r/m8,imm8	80 [..11 0...]	SAL r/m32,imm8	C1 [..10 0...]	RCR r/m16,CL	D3 [..01 1...]

表 6.11 オペコード (3/6)

命令	コード	命令	コード	命令	コード
RCR r/m16,imm8	C1 [..01 1...]	ADC r32,r/m32	13	MUL r/m16	F7 [..10 0...]
RCR r/m32,1	D1 [..01 1...]	SUB AL,imm8	2C	MUL r/m32	F7 [..10 0...]
RCR r/m32,CL	D3 [..01 1...]	SUB AX,imm16	2D	IMUL r/m8	F6 [..10 1...]
RCR r/m32,imm8	C1 [..01 1...]	SUB EAX,imm32	2D	IMUL r/m16	F7 [..10 1...]
RCL r/m8,1	D0 [..01 0...]	SUB r/m8,imm8	80 [..10 1...]	IMUL r/m32	F7 [..10 1...]
RCL r/m8,CL	D2 [..01 0...]	SUB r/m16,imm16	81 [..10 1...]	IMUL r16,r/m16	0F AF
RCL r/m8,imm8	C0 [..01 0...]	SUB r/m32,imm32	81 [..10 1...]	IMUL r32,r/m32	0F AF
RCL r/m16,1	D1 [..01 0...]	SUB r/m16,imm8	83 [..10 1...]	IMUL r16,r/m16,imm8	6B
RCL r/m16,CL	D3 [..01 0...]	SUB r/m32,imm8	83 [..10 1...]	IMUL r32,r/m32,imm8	6B
RCL r/m16,imm8	C1 [..01 0...]	SUB r/m8,r8	28	IMUL r16,imm8	6B
RCL r/m32,1	D1 [..01 0...]	SUB r/m16,r16	29	IMUL r32,imm8	6B
RCL r/m32,CL	D3 [..01 0...]	SUB r/m32,r32	29	IMUL r16,r/m16,imm16	69
RCL r/m32,imm8	C1 [..01 0...]	SUB r8,r/m8	2A	IMUL r32,r/m32,imm32	69
ADD AL,imm8	04	SUB r16,r/m16	2B	IMUL r16,imm16	69
ADD AX,imm16	05	SUB r32,r/m32	2B	IMUL r32,imm32	69
ADD EAX,imm32	05	SBB AL,imm8	1C	DIV r/m8	F6 [..11 0...]
ADD r/m8,imm8	80 [..00 0...]	SBB AX,imm16	1D	DIV r/m16	F7 [..11 0...]
ADD r/m16,imm16	81 [..00 0...]	SBB EAX,imm32	1D	DIV r/m32	F7 [..11 0...]
ADD r/m32,imm32	81 [..00 0...]	SBB r/m8,imm8	80 [..01 1...]	IDIV r/m8	F6 [..11 1...]
ADD r/m16,imm8	83 [..00 0...]	SBB r/m16,imm16	81 [..01 1...]	IDIV r/m16	F7 [..11 1...]
ADD r/m32,imm8	83 [..00 0...]	SBB r/m32,imm32	81 [..01 1...]	IDIV r/m32	F7 [..11 1...]
ADD r/m8,r8	00	SBB r/m16,imm8	83 [..01 1...]	DAA	27
ADD r/m16,r16	01	SBB r/m32,imm8	83 [..01 1...]	DAS	2F
ADD r/m32,r32	01	SBB r/m8,r8	18	AAA	37
ADD r8,r/m8	02	SBB r/m16,r16	19	AAS	3F
ADD r16,r/m16	03	SBB r/m32,r32	19	AAM	D4 0A
ADD r32,r/m32	03	SBB r8,r/m8	1A	AAD	D5 0A
ADC AL,imm8	14	SBB r16,r/m16	1B	CMP AL,imm8	3C
ADC AX,imm16	15	SBB r32,r/m32	1B	CMP AX,imm16	3D
ADC EAX,imm32	15	INC r/m8	FE [..00 0...]	CMP EAX,imm32	3D
ADC r/m8,imm8	80 [..01 0...]	INC r/m16	FF [..00 0...]	CMP r/m8,imm8	80 [..11 1...]
ADC r/m16,imm16	81 [..01 0...]	INC r/m32	FF [..00 0...]	CMP r/m16,imm16	81 [..11 1...]
ADC r/m32,imm32	81 [..01 0...]	INC r16	[0100 0...]	CMP r/m32,imm32	81 [..11 1...]
ADC r/m16,imm8	83 [..01 0...]	INC r32	[0100 0...]	CMP r/m16,imm8	83 [..11 1...]
ADC r/m32,imm8	83 [..01 0...]	DEC r/m8	FE [..00 1...]	CMP r/m32,imm8	83 [..11 1...]
ADC r/m8,r8	10	DEC r/m16	FF [..00 1...]	CMP r/m8,r8	38
ADC r/m16,r16	11	DEC r/m32	FF [..00 1...]	CMP r/m16,r16	39
ADC r/m32,r32	11	DEC r16	[0100 1...]	CMP r/m32,r32	39
ADC r8,r/m8	12	DEC r32	[0100 1...]	CMP r8,r/m8	3A
ADC r16,r/m16	13	MUL r/m8	F6 [..10 0...]	CMP r16,r/m16	3B

表 6.12 オペコード (4/6)

命令	コード	命令	コード	命令	コード
CMP r32,r/m32	3B	JS rel8	78	CALL m16:32	FF[..01 1...]
JMP rel8	EB	JZ rel8	74	RET	C3
JMP rel16	E9	JA rel16/32	0F 87	RET(F)	CB
JMP rel32	E9	JAE rel16/32	0F 83	RET imm16	C2
JMP r/m16	FF[..10 0...]	JB rel16/32	0F 82	RET(F) imm16	CA
JMP r/m32	FF[..10 0...]	JBE rel16/32	0F 86	BT r/m16,r16	0F A3
JMP ptr16:16	EA	JC rel16/32	0F 82	BT r/m32,r32	0F A3
JMP ptr16:32	EA	JE rel16/32	0F 84	BT r/m16,imm8	0F BA[..10 0...]
JMP m16:16	FF[..10 1...]	JZ rel16/32	0F 84	BT r/m32,imm8	0F BA[..10 0...]
JMP m16:32	FF[..10 1...]	JG rel16/32	0F 8F	BTS r/m16,r16	0F AB
JA rel8	77	JGE rel16/32	0F 8D	BTS r/m32,r32	0F AB
JAE rel8	73	JL rel16/32	0F 8C	BTS r/m16,imm8	0F BA[..10 1...]
JB rel8	72	JLE rel16/32	0F 8E	BTS r/m32,imm8	0F BA[..10 1...]
JBE rel8	76	JNA rel16/32	0F 86	BTR r/m16,r16	0F B3
JC rel8	72	JNAE rel16/32	0F 82	BTR r/m32,r32	0F B3
JCXZ rel8	E3	JNB rel16/32	0F 83	BTR r/m16,imm8	0F BA[..11 0...]
JECXZ rel8	E3	JNBE rel16/32	0F 87	BTR r/m32,imm8	0F BA[..11 0...]
JE rel8	74	JNC rel16/32	0F 83	BTC r/m16,r16	0F BB
JG rel8	7F	JNE rel16/32	0F 85	BTC r/m32,r32	0F BB
JGE rel8	7D	JNG rel16/32	0F 8E	BTC r/m16,imm8	0F BA[..11 1...]
JL rel8	7C	JNGE rel16/32	0F 8C	BTC r/m32,imm8	0F BA[..11 1...]
JLE rel8	7E	JNL rel16/32	0F 8D	BSF r16,r/m16	0F BC
JNA rel8	76	JNLE rel16/32	0F 8F	BSF r32,r/m32	0F BC
JNAE rel8	72	JNO rel16/32	0F 81	BSR r16,r/m16	0F BD
JNB rel8	73	JNP rel16/32	0F 8B	BSR r32,r/m32	0F BD
JNBE rel8	77	JNS rel16/32	0F 89	SETA r/m8	0F 97
JNC rel8	73	JNZ rel16/32	0F 85	SETAE r/m8	0F 93
JNE rel8	75	JO rel16/32	0F 80	SETB r/m8	0F 92
JNG rel8	7E	JP rel16/32	0F 8A	SETBE r/m8	0F 96
JNGE rel8	7C	JPE rel16/32	0F 8A	SETC r/m8	0F 92
JNL rel8	7D	JPO rel16/32	0F 8B	SETE r/m8	0F 94
JNLE rel8	7F	JS rel16/32	0F 88	SETG r/m8	0F 9F
JNO rel8	71	JZ rel16/32	0F 84	SETGE r/m8	0F 9D
JNP rel8	7B	CALL rel16	E8	SETL r/m8	0F 9C
JNS rel8	79	CALL rel32	E8	SETLE r/m8	0F 9E
JNZ rel8	75	CALL r/m16	FF[..01 0...]	SETNA r/m8	0F 96
JO rel8	70	CALL r/m32	FF[..01 0...]	SETNAE r/m8	0F 92
JP rel8	7A	CALL ptr16:16	9A	SETNB r/m8	0F 93
JPE rel8	7A	CALL ptr16:32	9A	SETNBE r/m8	0F 97
JPO rel8	7B	CALL m16:16	FF[..01 1...]	SETNC r/m8	0F 93

表 6.13 オペコード (5/6)

命令	コード	命令	コード	命令	コード
SETNE r/m8	0F 95	SCASB	AE	REP STOS m16	F3 AB
SETNG r/m8	0F 9E	SCASW	AF	REP STOS m32	F3 AB
SETNGE r/m8	0F 9C	SCASD	AF	REPE CMPS m8,m8	F3 A6
		LODS DS:(E)SI	AC	REPE CMPS m16,m16	F3 A7
SETNL r/m8	0F 9D	LODS DS:SI	AD	REPE CMPS m32,m32	F3 A7
SETNLE r/m8	0F 9F	LODS DS:ESI	AD	REPE SCAS m8	F3 AE
SETNO r/m8	0F 91	LODSB	AC	REPE SCAS m16	F3 AF
SETNP r/m8	0F 9B	LODSW	AD	REPE SCAS m32	F3 AF
SETNS r/m8	0F 99	LODSD	AD	REPNE CMPS m8,m8	F2 A6
SETNZ r/m8	0F 95	STOS ES:(E)DI	AA	REPNE CMPS m16,m16	F2 A7
SETO r/m8	0F 90	STOS ES:DI	AB	REPNE CMPS m32,m32	F2 A7
SETP r/m8	0F 9A	STOS ES:EDI	AB	REPNE SCAS m8	F2 AE
SETPE r/m8	0F 9A	STOSB	AA	REPNE SCAS m16	F2 AF
SETPO r/m8	0F 9B	STOSW	AB	REPNE SCAS m32	F2 AF
SETS r/m8	0F 98	STOSD	AB	LOOP rel8	E2
SETZ r/m8	0F 94	INS ES:(E)DI,DX	6C	LOOPE rel8	E1
TEST AL,imm8	A8	INS ES:DI,DX	6D	LOOPZ rel8	E1
TEST AX,imm16	A9	INS ES:EDI,DX	6D	LOOPNE rel8	E0
TEST EAX,imm32	A9	INSB	6C	LOOPNZ rel8	E0
TEST r/m8,imm8	F6 [..00 0...]	INSW	6D	IN AL,imm8	E4
TEST r/m16,imm16	F7 [..00 0...]	INSD	6D	IN AX,imm8	E5
TEST r/m32,imm32	F7 [..00 0...]	OUTS DX,DS:(E)SI	6E	IN EAX,imm8	E5
TEST r/m8,r8	84	OUTS DX,DS:SI	6F	IN AL,DX	EC
TEST r/m16,r16	85	OUTS DX,DS:ESI	6F	IN AX,DX	ED
TEST r/m32,r32	85	OUTSB	6E	IN EAX,DX	ED
MOVS ES:(E)DI,DS:(E)SI	A4	OUTSW	6F	OUT imm8,AL	E6
MOVS ES:DI,DS:SI	A5	OUTSD	6F	OUT imm8,AX	E7
MOVS ES:EDI,DS:ESI	A5	REP INS r/m8,DX	F3 6C	OUT imm8,EAX	E7
MOVSB	A4	REP INS r/m16,DX	F3 6D	OUT DX,AL	EE
MOVSW	A5	REP INS r/m32,DX	F3 6D	OUT DX,AX	EF
MOVSD	A5	REP MOVS m8,m8	F3 A4	OUT DX,EAX	EF
CMPS DS:(E)SI,ES:(E)DI	A6	REP MOVS m16,m16	F3 A5	STC	F9
CMPS DS:SI,ES:DI	A7	REP MOVS m32,m32	F3 A5	CLC	F8
CMPS DS:ESI,ES:EDI	A7	REP OUTS DX,r/m8	F3 6E	CMC	F5
CMPSB	A6	REP OUTS DX,r/m16	F3 6F	CLD	FC
CMPSW	A7	REP OUTS DX,r/m32	F3 6F	STD	FD
CMPSD	A7	REP LODS AL	F3 AC	LAHF	9F
SCAS ES:(E)DI	AE	REP LODS AX	F3 AD	SAHF	9E
SCAS ES:DI	AF	REP LODS EAX	F3 AD	PUSHF	9C
SCAS ES:EDI	AF	REP STOS m8	F3 AA	PUSHFD	9C

表 6.14 オペコード (6/6)

命令	コード	命令	コード	命令	コード
STI	FB	MOV r32,CR2	0F 20 [..01 0...]	MOVSBX r16,r/m8	0F BE
CLI	FA	MOV r32,CR3	0F 20 [..01 1...]	MOVSBX r32,r/m8	0F BE
INT3	CC	MOV r32,CR4	0F 20 [..10 0...]	MOVSBX r32,r/m16	0F BF
INT imm8	CD	MOV r32,DR0	0F 21 [..00 0...]	MOVZX r16,r/m8	0F B6
INTO	CE	MOV r32,DR1	0F 21 [..00 1...]	MOVZX r32,r/m8	0F B6
BOUND r16,m16&16	62	MOV r32,DR2	0F 21 [..01 0...]	MOVZX r32,r/m16	0F B7
BOUND r32,m32&32	62	MOV r32,DR3	0F 21 [..01 1...]		
IRET	CF	MOV r32,DR4	0F 21 [..10 0...]		
IRETD	CF	MOV r32,DR5	0F 21 [..10 1...]		
LEA r16,m	8D	MOV r32,DR6	0F 21 [..11 0...]		
LEA r32,m	8D	MOV r32,DR7	0F 21 [..11 1...]		
NOP	90	MOV DR0,r32	0F 23 [..00 0...]		
UD2	0F 0B	MOV DR1,r32	0F 23 [..00 1...]		
XLAT m8	D7	MOV DR2,r32	0F 23 [..01 0...]		
XLATB	D7	MOV DR3,r32	0F 23 [..01 1...]		
CPUID	0F A2	MOV DR4,r32	0F 23 [..10 0...]		
LDS r16,m16:16	C5	MOV DR5,r32	0F 23 [..10 1...]		
LDS r32,m16:32	C5	MOV DR6,r32	0F 23 [..11 0...]		
LSS r16,m16:16	0F B2	MOV DR7,r32	0F 23 [..11 1...]		
LSS r32,m16:32	0F B2	LMSW r/m16	0F 01 [..11 0...]		
LES r16,m16:16	C4	SMSW r/m16	0F 01 [..10 0...]		
LES r32,m16:32	C4	SMSW r32/m16	0F 01 [..10 0...]		
LFS r16,m16:16	0F B4	CLTS	0F 06		
LFS r32,m16:32	0F B4	ARPL r/m16,r16	63		
LGS r16,m16:16	0F B5	LAR r16,r/m16	0F 02		
LGS r32,m16:32	0F B5	LAR r32,r/m32	0F 02		
LGDT m16&32	0F 01 [..01 0...]	LSL r16,r/m16	0F 03		
LIDT m16&32	0F 01 [..01 1...]	LSL r32,r/m32	0F 03		
SGDT m	0F 01 [..00 0...]	VERR r/m16	0F 00 [..10 0...]		
SIDT m	0F 01 [..00 1...]	VERW r/m16	0F 00 [..10 1...]		
LLDT r/m16	0F 00 [..01 0...]	INVD	0F 08		
SLDT r/m16	0F 00 [..00 0...]	WBINVD	0F 09		
SLDT r/m32	0F 00 [..00 0...]	INVLPG m	0F 01 [..11 1...]		
LTR r/m16	0F 00 [..01 1...]	LOCK	F0		
STR r/m16	0F 00 [..00 1...]	HLT	F4		
MOV CR0,r32	0F 22 [..00 0...]	RSM	0F AA		
MOV CR2,r32	0F 22 [..01 0...]	RDMSR	0F 32		
MOV CR3,r32	0F 22 [..01 1...]	WRMSR	0F 30		
MOV CR4,r32	0F 22 [..10 0...]	RDPMSR	0F 33		
MOV r32,CR0	0F 20 [..00 0...]	RDTSC	0F 31		

表 6.15 FPU オペコード (1/2)

命令	コード	命令	コード
FLD m32real	D9 [..00 0...]	FSUB m64real	DC [..10 0...]
FLD m64real	DD [..00 0...]	FSUB ST(0),ST(i)	D8 [1110 0 ST(i)]
FLD m80real	DB [..10 1...]	FSUB ST(i),ST(0)	DC [1110 1 ST(i)]
FLD ST(i)	D9 [1100 0 ST(i)]	FSUBP ST(i),ST(0)	DE [1110 1 ST(i)]
FST m32real	D9 [..01 0...]	FSUBP	DE E9
FST m64real	DD [..01 0...]	FISUB m32int	DA [..10 0...]
FST ST(i)	DD [1101 0 ST(i)]	FISUB m16int	DE [..10 0...]
FSTP m32real	D9 [..01 1...]	FSUBR m32real	D8 [..10 1...]
FSTP m64real	DD [..01 1...]	FSUBR m64real	DC [..10 1...]
FSTP m80real	DB [..11 1...]	FSUBR ST(0),ST(i)	D8 [1110 1 ST(i)]
FSTP ST(i)	DD [1101 1 ST(i)]	FSUBR ST(i),ST(0)	DC [1110 0 ST(i)]
FILD m16int	DF [..00 0...]	FSUBRP ST(i),ST(0)	DE [1110 0 ST(i)]
FILD m32int	DB [..00 0...]	FSUBRP	DE E1
FILD m64int	DF [..10 1...]	FISUBR m32int	DA [..10 1...]
FIST m16int	DF [..01 0...]	FISUBR m16int	DE [..10 1...]
FIST m32int	DB [..01 0...]	FMUL m32real	D8 [..00 1...]
FISTP m16int	DF [..01 1...]	FMUL m64real	DC [..00 1...]
FISTP m32int	DB [..01 1...]	FMUL ST(0),ST(i)	D8 [1100 1 ST(i)]
FISTP m64int	DF [..11 1...]	FMUL ST(i),ST(0)	DC [1100 1 ST(i)]
FBLD m80dec	DF [..10 0...]	FMULP ST(i),ST(0)	DE [1100 1 ST(i)]
FBSTP m80bcd	DF [..11 0...]	FMULP	DE C9
FXCH ST(i)	D9 [1100 1...]	FIMUL m32int	DA [..00 1...]
FXCH	D9 C9	FIMUL m16int	DE [..00 1...]
FCMOVB ST(0),ST(i)	DA [1100 0 ST(i)]	FDIV m32real	D8 [..11 0...]
FCMOVE ST(0),ST(i)	DA [1100 1 ST(i)]	FDIV m64real	DC [..11 0...]
FCMOVBE ST(0),ST(i)	DA [1101 0 ST(i)]	FDIV ST(0),ST(i)	D8 [1111 0 ST(i)]
FCMOVU ST(0),ST(i)	DA [1101 1 ST(i)]	FDIV ST(i),ST(0)	DC [1111 1 ST(i)]
FCMOVNB ST(0),ST(i)	DB [1100 0 ST(i)]	FDIVP ST(i),ST(0)	DE [1111 1 ST(i)]
FCMOVNE ST(0),ST(i)	DB [1100 1 ST(i)]	FDIVP	DE F9
FCMOVNBE ST(0),ST(i)	DB [1101 0 ST(i)]	FIDIV m32int	DA [..11 0...]
FCMOVNU ST(0),ST(i)	DB [1101 1 ST(i)]	FIDIV m16int	DE [..11 0...]
FADD m32real	D8 [..00 0...]	FDIVR m32real	D8 [..11 1...]
FADD m64real	DC [..00 0...]	FDIVR m64real	DC [..11 1...]
FADD ST(0),ST(i)	D8 [1100 0 ST(i)]	FDIVR ST(0),ST(i)	D8 [1111 1 ST(i)]
FADD ST(i),ST(0)	DC [1100 0 ST(i)]	FDIVR ST(i),ST(0)	DC [1111 0 ST(i)]
FADDP ST(i),ST(0)	DE [1100 0 ST(i)]	FDIVRP ST(i),ST(0)	DE [1111 0 ST(i)]
FADDP	DE C1	FIDIVR m32int	DA [..11 1...]
FIADD m32int	DA [..00 0...]	FIDIVR m16int	DE [..11 1...]
FIADD m16int	DE [..00 0...]	FPPREM	D9 F8
FSUB m32real	D8 [..10 0...]	FPPREM1	D9 F5

表 6.16 FPU オペコード (2/2)

命令	コード	命令	コード
FABS	D9 E1	FLDL2E	D9 EA
FCHS	D9 E0	FLDPI	D9 EB
FRNDINT	D9 FC	FLDLG2	D9 EC
FSCALE	D9 FD	FLDLN2	D9 ED
FSQRT	D9 FA	FLDZ	D9 EE
EXTRACT	D9 F4	FINCSTP	D9 F7
FCOM m32real	D8 [..01 0...]	FDECSTP	D9 F6
FCOM m64real	DC [..01 0...]	FFREE ST(i)	DD [1100 0 ST(i)]
FCOM ST(i)	D8 [1101 0 ST(i)]	FINIT	9B DB E3
FCOM	D8 D1	FNINIT	DB E3
FCOMP m32real	D8 [..01 1...]	FCLEX	9B DB E2
FCOMP m64real	DC [..01 1...]	FNCLEX	DB E2
FCOMP ST(i)	D8 [1101 1 ST(i)]	FSTCW m2byte	9B D9 [..11 1...]
FCOMP	D8 D9	FNSTCW m2byte	D9 [..11 1...]
FCOMPP	DE D9	FLDCW m2byte	D9 [..10 1...]
FUCOM ST(i)	DD [1110 0 ST(i)]	FSTENV m14/28byte	9B D9 [..11 0...]
FUCOM	DD E1	FNSTENV m14/28byte	D9 [..11 0...]
FUCOMP ST(i)	DD [1110 1 ST(i)]	FLDENV m14/28byte	D9 [..10 0...]
FUCOMP	DD E9	FSAVE m94/108byte	9B DD [..11 0...]
FUCOMPP	DA E9	FNSAVE m94/108byte	DD [..11 0...]
FICOM m16int	DE [..01 0...]	FRSTOR m94/108byte	DD [..10 0...]
FICOM m32int	DA [..01 0...]	FSTSW m2byte	9B DD [..11 1...]
FICOMP m16int	DE [..01 1...]	FSTSW AX	9B DF E0
FICOMP m32int	DA [..01 1...]	FNSTSW m2byte	DD [..11 1...]
FCOMI ST(0),ST(i)	DB [1111 0 ST(i)]	FNSTSW AX	DF E0
FCOMIP ST(0),ST(i)	DF [1111 0 ST(i)]	FWAIT	9B
FUCOMI ST(0),ST(i)	DB [1110 1 ST(i)]	WAIT	9B
FUCOMIP ST(0),ST(i)	DF [1110 1 ST(i)]	FNOP	D9 D0
FTST	D9 E4		
FXAM	D9 E5		
FSIN	D9 FE		
FCOS	D9 FF		
FSINCOS	D9 FB		
FPTAN	D9 F2		
FPATAN	D9 F3		
F2XM1	D9 F0		
FYL2X	D9 F1		
FYL2XP1	D9 F9		
FLD1	D9 E8		
FLDL2T	D9 E9		

表 6.17 EFLAGS クロス・リファレンス (1/3)

命令	OF	SF	ZF	AF	PF	CF	TF	IF	DF	NT	RF
AAA AAD AAM AAS	- - - -	- M M -	- M M -	TM - - TM	- M M -	M - - M					
ADC ADD AND ARPL	M M 0 	M M M 	M M M M	M M - 	M M M 	TM M 0 					
BOUND BSF/BSR BSWAP BT/BTS/BTR/BTC	 - -	 - -	 M -	 - -	 - -	 - M					
CALL CBW CLC CLD	 	 	 	 	 	 0 	 	 0	 	 	
CLI CLTS CMC CMOVcc	 T	 T	 T	 	 T	 M T	 	0 	 	 	
CMP CMPS CMPXCHG CMPXCHG8B	M M M 	M M M 	M M M M	M M M 	M M M 	M M M 	 	 	T 	 	
CPUID CWD DAA DAS	 - -	 M M	 M M	 TM TM	 M M	 TM TM	 	 	 	 	
DEC DIV ENTER ESC	M - 	M - 	M - 	M - 	M - 	 - 	 	 	 	 	
FCMOVcc FCOMI,FCOMIP,FUCOMI FUCOMIP HLT IDIV	 - 	 - 	T M - 	 - 	T M - 	T M - 	 	 	 	 	
INT INTO INVD INVLPG	 T 	 	 	 	 	 	0 0 	 	 	0 0 	

表 6.18 EFLAGS クロス・リファレンス (2/3)

命令	OF	SF	ZF	AF	PF	CF	TF	IF	DF	NT	RF
IRET Jcc JCXZ JMP	R T	R T	R T	R	R T	R T	R	R	R	R	
LAHF LAR LDS/LES/LSS/LFS/LGS LEA			M								
LEAVE LGDT/LIDT/LLDT/LMSW LOCK LODS									T		
LOOP LOOPE/LOOPNE LSL LTR			T M								
MOV MOV control,debug,test MOVS MOVSX/MOVZX	–	–	–	–	–	–			T		
MUL NEG NOP NOT	M M	– M	– M	– M	– M	M M					
OR OUT OUTS POP/POPA	0	M	M	–	M	0			T		
POPF PUSH/PUSHA/PUSHF RCL/RCR 1 RCL/RCR count	R M –	R	R	R	R	R TM TM	R	R	R	R	
RDMSR RDPMC RDTSC REP/REPE/REPNE											
RET ROL/ROR 1 ROL/ROR count RSM	M – M					M M M					M

表 6.19 EFLAGS クロス・リファレンス (3/3)

命令	OF	SF	ZF	AF	PF	CF	TF	IF	DF	NT	RF
SAHF		R	R	R	R	R					
SAL/SAR/SHL/SHR 1	M	M	M	–	M	M					
SAL/SAR/SHL/SHR count	–	M	M	–	M	M					
SBB	M	M	M	M	M	TM					
SCAS	M	M	M	M	M	M			T		
SETcc	T	T	T		T	T					
SGDT/SIDT/SLDT/SMSW											
SHLD/SHRD	–	M	M	–	M	M					
STC						1					
STD									1		
STI								1			
STOS									T		
STR											
SUB	M	M	M	M	M	M					
TEST	0	M	M	–	M	0					
UD2											
VERR/VERW			M								
WAIT											
WBINVD											
WRMSR											
XADD	M	M	M	M	M	M					
XCHG											
XLAT											
XOR	0	M	M	–	M	0					
IMUL	M	–	–	–	–	M					
IN											
INC	M	M	M	M	M						
INS									T		

T 命令はフラグをテストする。

M 命令はフラグを変更する。

0 命令はフラグを 0 にする。

1 命令はフラグを 1 にする。

– フラグに対する命令の影響は未定義。

R 命令はフラグの元の値に戻す。

6.7.4 80386

ビット操作命令が多数追加されています。

- LSS/LFS/LGS
- BT/BTC/BTR/BTS
- BSF/BSR
- SHRD/SHLD
- SETcc
- MOVSX/MOVZX
- IMUL r,r/m

6.7.5 80486

このプロセッサではキャッシュが内蔵されたので、その関係の命令などが追加されています。

- CMPXCHG
- XADD
- BSWAP
- WBINVD/INVD/INVLPG

6.7.6 Pentium

- CMPXCHG8B
- CPUID
- RDTSC
- RDMSR
- WRMSR
- RSM

6.7.7 PentiumPro

- CMOVcc
- RDPMC
- FCMOVcc
- FCOMI/FCOMIP/FUCOMI/FUCOMIP

6.8 32 ビット命令の拡張

既存の命令で、AX などの 16 ビットではなく、EAX などの 32 ビットのレジスタやダブル・ワードのメモリをオペランドに指定する場合は、新しいオペコードを割り当てずに、セグメントに 16/32 ビットの属性を指定することで AX が選択されたり、EAX が選択されたりします。16 ビットのセグメントで一時的に 32 ビットのレジスタやダブル・ワードのメモリを指定したい場合は、その命令の前にオペランド・サイズ・プリフィックス (66H) をつけて、その命令のみ 32 ビッ

ト化します。

また、オペランドの指定するメモリのオフセットが 64k を越える場合はアドレス・サイズ・プリフィックス (67H) も必要です。

リアル・モードではセグメントは 16 ビットですから、プリフィックスを使う場合のみ 32 ビット命令が使えます。

アセンブラを使う場合は、SEGMENT 疑似命令で 16/32 ビットの属性を指定します。

第Ⅳ部

文献

参考文献

- [1] 86 系 ROM 化支援ソフトウェアの製作
中島 信行 著 CQ 出版社
資料番号 240950J-004
- [2] はじめて読む 486
蒲地 輝尚 著 アスキー出版局
インテルジャパン CQ 出版社
- [3] 80286 ハンドブック
大貫 広幸
田中 恵介
葦原 隆 共著 アスキー出版局
[14] i486DX マイクロプロセッサ・ハードウェア・リ
ファレンス・マニュアル
資料番号 240552J-001
インテルジャパン CQ 出版社
- [4] 80x86 / 80x87 ファミリー・テクニカルハンド
ブック
Robert L.Hummel 著
樋田 浩一 訳 技術評論社
[15] 数値演算入門
8086 による浮動小数点演算の実際と 8087 の使
い方.
大貫広幸著 CQ 出版社
- [5] 80286 の使い方
本岡 善綱 著 オーム社
[16] 別冊インターフェース 数値演算プロセッサ
浮動小数点演算入門から高速演算プログラミング
まで.
- [6] 80386 の使い方
W.B. スルヤント 著 オーム社
[17] 80x86/80x87 ファミリー・テクニカルハンドブ
ック
Robert L.Hummel 著
樋田 浩一 訳 技術評論社
- [7] 80386 システムプログラム
吉田 俊郎 著 オーム社
[18] 80387 プログラマーズ・リファレンス・マニユ
アル
資料番号 231917-001
インテルジャパン株式会社 CQ 出
版社
- [8] 8086 ファミリー・ハンドブック
相沢一石 著 CQ 出版社
[19] Pentium ファミリーディベロッパーズ・マニユ
アル 上巻
データブック
資料番号 241428J-004
インテルジャパン株式会社 CQ 出
版社
- [9] 80286 / 8086 プログラミング技法
村山仁郎 著 工学社
[20] Pentium ファミリーディベロッパーズ・マニユ
アル 下巻
アーキテクチャーとプログラミング・マニュアル
資料番号 241430J-004
インテルジャパン株式会社 CQ 出
版社
- [10] 80386 ハードウェア・リファレンス・マニュアル
資料番号 231732J-002
インテルジャパン CQ 出版社
[21] Pentium ファミリーディベロッパーズ・マニユ
アル
- [11] 80387 プログラマーズ・リファレンス・マニユ
アル
資料番号 231917-001
インテルジャパン CQ 出版社
- [12] インテル 486DX マイクロプロセッサデータブ
ック
資料番号 240440J-004
インテルジャパン CQ 出版社
- [13] インテル 486SX マイクロプロセッサデータブ
ック

- ル 下巻
アーキテクチャーとプログラミング・マニュアル
資料番号 241430J-004
インテルジャパン株式会社 CQ 出版社
- [22] PentiumPro ファミリーデベロッパーズ・マニュアル 上巻
仕様
資料番号 242690J-001
インテルジャパン株式会社 CQ 出版社
- [23] PentiumPro ファミリーデベロッパーズ・マニュアル 中巻
プログラマーズリファレンスマニュアル
資料番号 242691J-001
インテルジャパン株式会社 CQ 出版社
- [24] PentiumPro ファミリーデベロッパーズ・マニュアル 下巻
オペレーティングシステムライターズマニュアル
資料番号 242692J-001
インテルジャパン株式会社 CQ 出版社
- [25] 情報処理技術者必携
コンピュータの数 2進数
中楯文男著 日刊工業新聞社
- [26] 入門特集 8086 アセンブリ・プログラミング
マイコンピュータ No.14
CQ 出版社